

使用手册

1. 关于本手册

本手册面向于 NJET 应用引擎的非管理员用户，即属于普通角色的用户，主要包括各个功能模块的操作指南。

2. NJET 应用引擎介绍

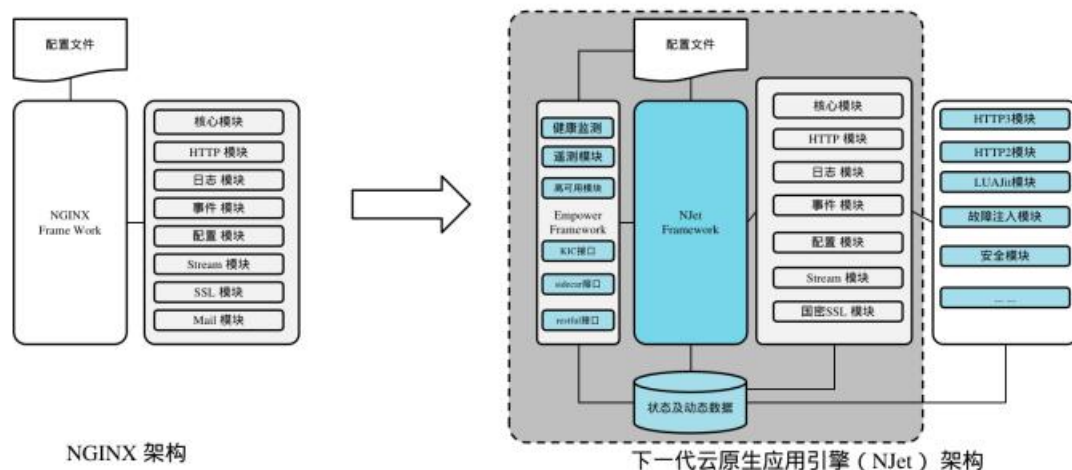
2.1 应用引擎技术介绍

应用引擎是面向互联网和云原生应用提供的运行时组态服务程序。具备环境感知、安全控制、加速优化等能力，一般呈现为 Web 服务、流媒体服务、代理(Proxy)、应用中间件、API 网关、消息队列等产品形态。

应用引擎在云原生架构中，除了提供南北向通信网关的功能以外，因为提供了服务网格中东西向通信、透明流量劫持、熔断、遥测与故障注入等新功能特性，其地位和作用在云原生架构中变得愈发重要。

NJet 最早是基于 NGINX1.19 基础，fork 并独立演进的开源应用引擎，并随着 NGINX 版本迭代，吸收上游 NGINX 的更新，已经同步更新到 NGINX1.23.1 版本。NJet 的目标在于适应国内特定的技术规范及标准，如国密算法套件支持，并构建安全可控的云原生数据面，支撑我国云原生产业生态。作为底层引擎，NJet 利用动态加载机制可以实现不同的产品形态，如 API 网关、消息代理、出入向代理，负载均衡，WAF 等等

2.2 系统架构介绍



相比市面其他类型的 API 网关，高性能是 NGINX 主要的优点，但动态配置能力的缺乏一直受到业界的诟病。NJet 在 NGINX 的架构上进行了扩充，对其框架进行了改写，增加了 copilots framework 及可持久化的动态存储能力，解决了指令配置变更动态生效的关键问题，扩展了 NJet 的应用场景。此外，业界对应用引擎可观测性的需求，需要应用引擎持续不断的采集性能指标、日志数据以及注入跟踪信息，但这对应应用引擎的性能造成了不可忽视的影响，NJet 利用 copilots framework 隔离了业务处理及配置变更和指标采集，避免了遥测对性能的影响。作为云原生的应用引擎，NJet 需要支持业界流行的 Ingress 及 sidecar 的 api 规范，基于动态配置+ copilots framework 架构，NJet 可以通过不断更新独立的相关 copilots module，实现对响应标准规范的及时支持

2.3 njet 编译与安装

2.3.1 编译环境

- 准备一台能够访问外网的
- CentOS Linux release 7.9.2009 (Core)

2.3.2 编译环境配置

2.3.2.1 配置 yum 源

- 执行以下指令：`sudo yum --enablerepo=extras install -q -y epel-release centos-release-scl-rh` <https://repo.ius.io/ius-release-el7.rpm>
- `sudo curl -o /etc/yum.repos.d/mercurial.repo https://www.mercurial-scm.org/release/centos7/mercurial.repo`

- c. 上面步骤完成后，文件系统的目录/etc/yum.repos.d 将生成对应的 repo 文件。
- d. [root@CDN102 home]# ls -al /etc/yum.repos.d/mercurial.repo
- e. -rw-r--r--. 1 root root 267 3 月 7 11:26
/etc/yum.repos.d/mercurial.repo

2.3.2.2 yum 安装软件包

1. sudo yum install -y devtoolset-8-make devtoolset-8-toolchain ca-certificates mercurial zlib-devel cmake3 ninja-build libunwind-devel pcre-devel openssl-devel libtool libtool-ltdl

2.3.2.3 创建符号连接

- a. sudo ln -s /opt/rh/devtoolset-8/root/usr/bin/gcc /usr/local/bin/gcc
- b. sudo ln -s /opt/rh/devtoolset-8/root/usr/bin/c++ /usr/local/bin/c++
- c. sudo ln -s /opt/rh/devtoolset-8/root/usr/bin/cc /usr/local/bin/cc
- d. sudo ln -s /opt/rh/devtoolset-8/root/usr/bin/make /usr/local/bin/make

2.3.2.4 修改 ld.so.conf 配置

执行以下命令：

```
sudo bash -c 'echo "/usr/local/lib" >> /etc/ld.so.conf'
sudo ldconfig
```

2.3.3 编译代码

把 njet1.0.zip 包上传到 /home 目录下。并解压 unzip njet1.0.zip，如图：

```
[root@CDN102 home]# ls
njet1.0  njet1.0.zip
[root@CDN102 home]# cd njet1.0/
[root@CDN102 njet1.0]# ls
3rd_lib  build  build_k8s.sh  CHANGES.rst  configure  html  luajit  man  mosquito  njet_sidecar.conf  objs  README  README.md  src  telemetry
auto  build_cc.sh  CHANGES  conf  contrib  LICENSE  lua1ib  modules  njet_sidecar.conf  openapi
```

2.3.3.1 编译：njet

2.3.3.1.1 执行： sh build_cc.sh conf

```

+ njet_http_kv_module was configured
adding module in ./modules/njet-http-sendmsg-module
+ njet_http_sendmsg_module was configured
adding module in ./modules/njet-http-split-clients-2-module
+ njet_http_split_clients_2_module was configured
adding module in ./modules/njet-health-check-helper
checking for njet_health_check_helper ... found
+ njet_health_check_helper was configured
checking for zlib library ... found
creating objs/Makefile

Configuration summary
+ using PCRE library: auto/lib/pcre-8.45
+ using OpenSSL library: auto/lib/tassl
+ using system zlib library

njet path prefix: "/etc/njet"
njet binary file: "/usr/sbin/njet"
njet modules path: "/usr/lib/njet/modules"
njet configuration prefix: "/etc/njet"
njet configuration file: "/etc/njet/njet.conf"
njet pid file: "/etc/njet/logs/njet.pid"
njet error log file: "/etc/njet/logs/error.log"
njet http access log file: "/etc/njet/logs/access.log"
njet http client request body temporary files: "client_body_temp"
njet http proxy temporary files: "proxy_temp"
njet http fastcgi temporary files: "fastcgi_temp"
njet http uwsgi temporary files: "uwsgi_temp"
njet http scgi temporary files: "scgi_temp"

[root@CDN102 njet1.0]#

```

2.3.3.1.2 执行: make

如果 make 后, 有如下提示: 则继续 执行 make 指令。

```

Using OS-specific seed configuration
Creating configdata.pm
Creating Makefile

*****
***
***      openssl has been successfully configured      ***
***
***      If you encounter a problem while building, please open an ***
***      issue on GitHub <https://github.com/openssl/openssl/issues> ***
***      and include the output from the following command: ***
***
***      perl configdata.pm --dump ***
***
***      (If you are new to openssl, you might want to consult the ***
***      'Troubleshooting' section in the INSTALL file first) ***
***
*****
*****
***      Please run the same make command again ***
***
*****
make[2]: *** [Makefile:690: configdata.pm] 错误 1
make[2]: 离开目录"/home/njet1.0/auto/lib/tassl"

```

2.3.3.1.3 正确编译完成: 如图

```

/usr/bin/cc -c -fPIC -I/etc/njet/luajit/include/luajit-2.1 -I/etc/njet/luajit/include/luajit-2.1 -pipe -O -w -Wall -Wpointer-arith -Wno-unu-
ter -Werror -g -O0 -ggdb -DNDK_SET_VAR -I src/core -I src/event -I src/event/modules -I src/os/unix -I ./modules/njet-stream-proto-module -I :
a/kit/objs -I objs/addon/ndk -I src/ext/luajit/http/src/api -I src/ext/luajit/stream/src/api -I ./modules/njet-stream-upstream-dynamic-servers-modu-
les/njet-http-upstream-dynamic-servers-module -I ./modules/njet-http-match-module -I ./modules/njet-http-health-check-module -I ./modules/nj-
tick-module -I ./modules/njet-mqconf-module/ -I ./modules/njet-http-location-module/src -I ./modules/njet-http-kv-module/src -I ./modules/nj-
ndmsg-module/src -I auto/lib/pcre-8.45 -I auto/lib/tassl/openssl/include -I objs -I src/http -I src/http/modules -I src/http/v2 -I src/ext/lj-
-I src/ext/luajit/src -I src/ext/luajit/kit/objs -I objs/addon/ndk -I objs/njet/luajit/include/luajit-2.1 -I src/mail -I src/stream -I /etc/nj-
include/luajit-2.1 -I ./modules/njet-stream-health-check-module/src \
-o objs/njt_helper_health_check_agent_modules.o \
objs/njt_helper_health_check_agent_modules.c
/usr/bin/cc -o objs/njt_helper_health_check_agent.so \
objs/addon/src/njt_helper_health_check_agent.o \
objs/addon/src/njt_common_health_check.o \
objs/njt_helper_health_check_agent_modules.o \
-L/etc/njet/luajit/lib -L/etc/njet/luajit/lib -L./3rd.lib -ldl \
-shared
sed -e "s|%%PREFIX%%|/etc/njet|" \
-e "s|%%PID_PATH%%|/etc/njet/logs/njet.pid|" \
-e "s|%%CONF_PATH%%|/etc/njet/njet.conf|" \
-e "s|%%ERROR_LOG_PATH%%|/etc/njet/logs/error.log|" \
< man/njet.8 > objs/njet.8
make[1]: 离开目录"/home/njet1.0"
[root@CDN102 njet1.0]#

```

2.3.3.1.4 最后执行: make install

3. 系统功能

1.0 目标是实现基本动态框架，为动态配置打下基础，全面改造 SSL 支持，从而支持国密，DTLS.初步实现 sidecar 的 xDS 接口，可作为边车应用在服务网格中。共涉及引擎内核、边车、出入口网关、web 引擎、应用代理、安全、监控、可编程控制、api 网关等 4 大类 38 项功能。在下述功能清单中，着重描述扩展、定制的功能，原生 NGINX 的能力可以参考 <https://nginx.org/en/docs>

3.1 基本使用

3.1.1 目录结构及功能说明

```
Bash
├── 3rd_lib          依赖的第三方动态库，比如
libmosquitto_emb.so
├── build/rpm        rpm 包编译脚本
├── auto             自动检测系统环境以及编译相关的脚本
│   ├── cc          关于编译器相关的编译选项的检测脚本
│   ├── lib         njet 编译所需要的一些库的检测脚本
│   └── os          与平台相关的一些系统参数与系统调用相关的检测
├── types           与数据类型相关的一些辅助脚本
├── conf            存放默认配置文件，在 make install 后，会
拷贝到安装目录中去
├── contrib         存放一些实用工具，如 geo 配置生成工具
                    (geo2njet.pl)
├── html            存放默认的网页文件，在 make install 后，
会拷贝到安装目录中去
├── repos           存放 yum 数据源
├── doc             njet 的 api 文档
│   ├── swagger    openapi 接口网页文档
│   ├── gui        前端展示页面文档
│   └── manual      njet 文档手册
├── luajit          luajit
├── lualib          lualib
├── modules         njet 动态模块以及 util 模块
├── openapi         openapi 定义文件
├── src             存放 njet 的源代码
│   ├── core       njet 的核心源代码，包括常用数据结构的定义，以及 njet 初始化运行的核心代码如 main 函数
│   └── event      对系统事件处理机制的封装，以及定时器的实
```

现相关代码

- | └─ modules 不同事件处理方式的模块化，如 `select`、`poll`、`epoll`、`kqueue` 等
- | └─ http njet 作为 http 服务器相关的代码
- | └─ modules 包含 http 的各种功能模块
- | └─ ext/lua lua 模块
- | └─ mail njet 作为邮件代理服务器相关的代码
- | └─ stream tcp/ udp 四层网络代理服务器相关的代码
- | └─ misc 一些辅助代码，测试 `c++` 头的兼容性，以及对 `google_perftools` 的支持
- | └─ os 主要是对各种不同体系系统结构所提供的系统函数的封装，对外提供统一的系统调用接口

3.1.2 使用命令

3.1.2.1 显示帮助信息

```
njet -h
```

3.1.2.2 启动

```
njet -p /tmp/njet/ -c conf/njet.conf
```

常见启动参数：

- p 指定 prefix 配置文件路径，不指定，默认 `/etc/njet`
- c 指定配置文件，不指定，默认 `njet.conf`
- e 指定 error 日志文件

3.1.2.3 测试配置信息是否有错误

```
njet -t
```

3.1.2.4 显示版本

```
njet -v
```

3.1.2.5 显示编译阶段的参数

```
njet -V
```

3.1.2.6 快速停止

njet -s stop 或者 kill -TERM {进程 id}

3.1.2.7 优雅停止服务

njet -s quit 或者 kill -QUIT {进程 id}

3.1.2.8 重新加载配置

njet -s reload 或者 kill -HUP {进程 id}

3.2 CoPilot

copilot 类似于 NGINX 中的 helper，但通过配置 helper 指令，实现装载不同的 helper 实现。从而实现不同的功能。目前的 NJet 的事件总线、控制平面集成接口都是通过其设置

copilot 框架依赖 mqconf 模块提供的 helper 指令：

指令说明：

Syntax	helper tag so_file conf_file;
Default	---
Context	main

参数说明：

参数名称	是否必须	参数说明
tag	是	指定一个名字标签
so_file	是	so 文件
conf_file	是	配置文件

3.2.1 copilot:ctrl

该模块是一个基础模块，给 http-sendmsg 和 http_health_check 等模块提供运行环境。

为了运行该模块，需要在主配置文件中配置：

下面为示例配置：

```
helper ctrl objs/njt_helper_ctrl_module.so njt_ctrl.conf;
```

该模块自身的配置，即上面主配置文件中 `helper` 指令中指定的配置文件的内容。

配置的指令分为两类：

1) Njet 中标准的指令；

1. **需要注意：**用 `error_log` 和 `access_log` 指令指定的 log 文件要不同于主配置文件中的 log 文件。

2) 通过 `load_module` 指令加载的动态模块中扩展出来的指令

配置示例如下：

```
C++
load_module
/home/njet/modules/njt_http_sendmsg_module.so;
load_module
/home/njet/modules/njt_http_health_check_helper.so;
user njet njet;

error_log logs/error_ctrl.log info;

events {
    worker_connections 1024;
}

http {
    dyn_sendmsg_conf conf/iot_msg.conf;
    access_log logs/access_ctrl.log combined;

    server {
        listen      8081;

        location / {
            return 200 "NJet control_plane stub\n";
        }
        location /hc {
            health_check_api;
        }
    }
}
```

```
}

```

3.2.2 copilot:broker

该模块提供消息服务端功能，要开启该功能在 njet.conf 的 main block 中添加如下指令：

```
helper broker /home/njet/modules/njt_helper_broker_module.so
conf/mqtt.conf;
```

其中配置文件 conf/mqtt.conf，路径是相对于 prefix 的路径，prefix 可以通过命令行参数 -p 指定，未指定时，将使用编译时的默认 prefix 路径。

mqtt.conf Message Broker 配置文件

配置项	必须修改	配置说明
log_dest	否	日志输出方法，一般配置为文件输出，配置值为 file 文件名（绝对路径） 默认： \$PREFIX/logs/mosquitto.log
log_type	否	日志级别： debug, error, warning, notice, information, mqtt 库中的日志输出方式和 java 中的不一样，配置了某个日志级别，只有该日志级别的消息才会输出到日志中， 因此需要为需要的日志级别单独配置一行。默认： error
listener	否	监听端口及地址，默认： 0 \$PREFIX/data/mosquitto.sock
allow_anonymous	否	是否允许匿名连接，目前配置为 true
persistence	否	是否开启消息持久化 (true, false)，默认： true
autosave_on_changes	否	是否有消息时进行自动保存 默认： true
autosave_interval	否	当自动保存开启时，当新消息个数大于这个配

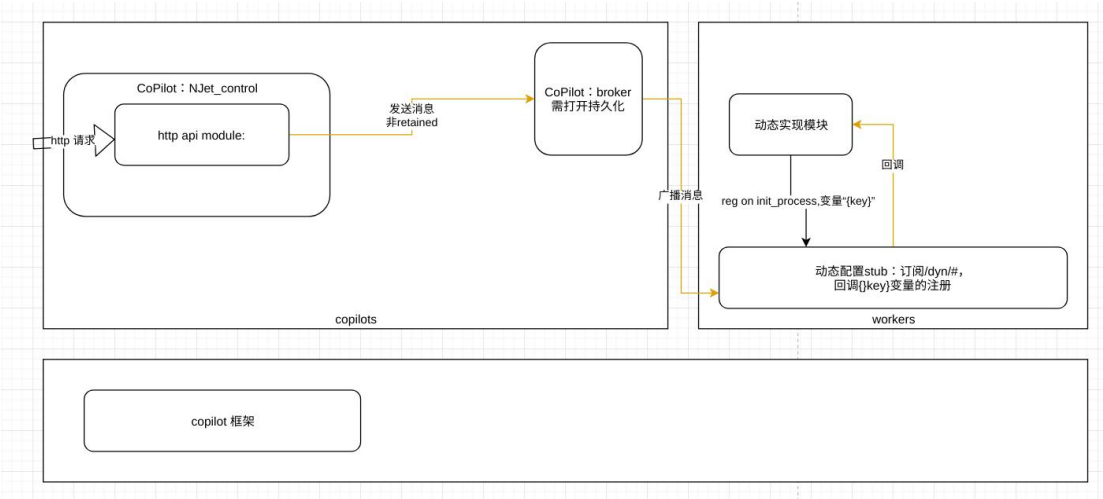
al		置值时，将触发保存操作 默认： 1
persistence_location	否	持久化文件保存路径，将在该路径下创建 mosquito.db，默认： \$PREFIX/data

配置都会有默认值，建议只配置日志级别。

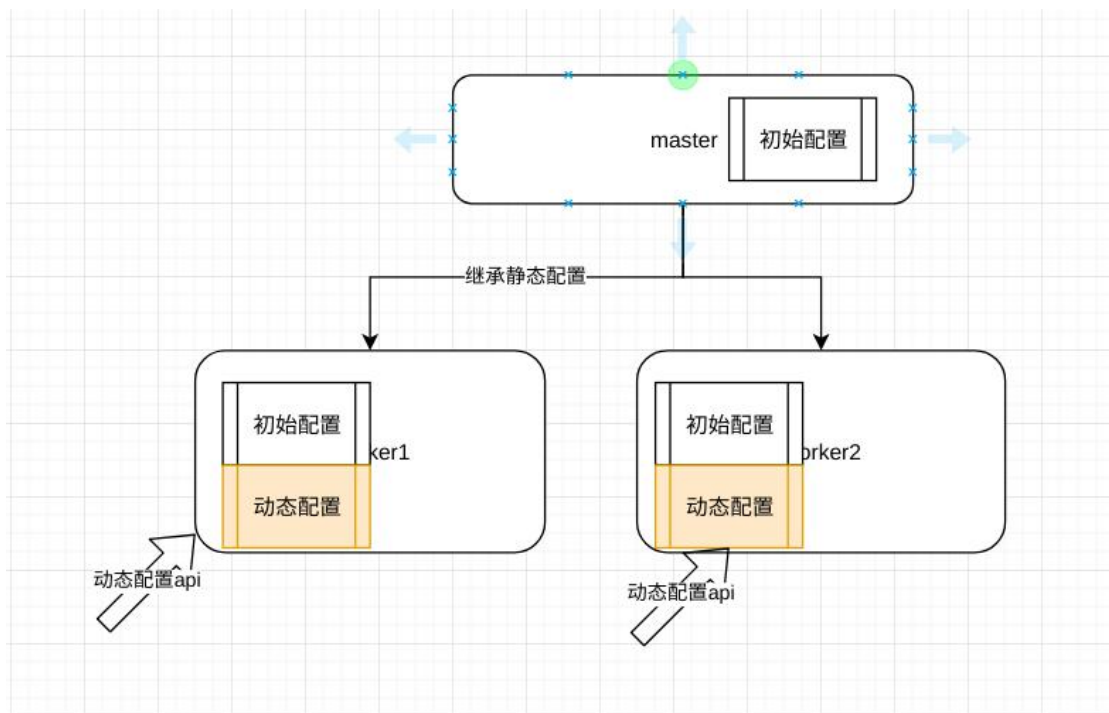
```
C++
log_type error
```

3.3 动态配置框架

njet 提供基于 eventbus 构建的消息总线，其包括 copilot: broker，动态配置 stub 模块 njet_http_kv, sendmsg 模块 njet_http_sendmsg 模块， 及 HTTP 配置 API。



如上图所示意， NJet 的动态配置一般由控制面发送配置消息，不同的 worker 通过加载动态配置 stub，监听不同的消息，并回调不同的处理函数。



3.3.1 kv 模块配置

该模块处于数据平面中，已经静态编译到 Njet 可执行文件， 职责是提供注册接口， 各动态模块将配置查询及配置更新的回调函数通过 kv 模块提供的接口进行注册。 当 kv 模块收到对应/dyn/# 及 /rpc/# 主题下的消息时， 会调用对应模块的回调函数。

在 http block 下， 指定该模块的配置文件：

```
C++
http {
    dyn_kv_conf conf/iot.conf;
}
```

iot.conf MQTT 客户端配置

配置项	必须修改	配置说明
broker_addr	否	MQ broker 地址。默认： unix:\$PREFIX/data/mosquitto.sock
log_type	否	日志级别： debug, error, warning, notice, information, 不同的日志级别需要单独配置一行。默认： error

topic	否	默认订阅的主题 /cluster/+kv_set/# 用于 kvstore 值的设置 /dyn/# 用于动态 location 更新 /rpc/# 用于发送 rpc 消息
keepalive	否	给服务端发送 PING 命令的间隔时间。默认：30
kv_store_dir	否	Key value store 持久化文件路径。默认：\$PREFIX/data
protocol_version	否	使用的协议版本，要支持 request response 消息， 必须填 5 。默认：5

配置都会有默认值，建议只配置日志级别

```
C++
log_type error
```

3.3.2 Sendmsg

该模块处于控制平面中，职责是提供消息发送接口，消息类型有普通消息及 RPC 消息，HTTP 配置 API，调用 sendmsg 模块的函数，将消息发送到 broker，如果是 RPC 类型的消息，数据面中的 KV 模块在接收到消息并调用对应模块注册的回调函数后，会将消息返回给 HTTP 配置 API。

要开启该功能，需在 njet_ctrl.conf 的 main block 中加载该模块：

```
load_module /home/njet/modules/njt_http_sendmsg_module.so;
```

并在 http block 下，指定该模块的配置文件：

```
C++
http {
    dyn_sendmsg_conf conf/iot_ctrl.conf;
```

```

server {
    ...
    location /kv {
        dyn_sendmsg_kv;
    }
    ...
}

```

dyn_sendmsg_kv 配置不是必须的，这个是对外提供了 kv 值设置及查询的 http API 接口。

iot_ctrl.conf MQTT 客户端配置

配置项	必须修改	配置说明
broker_addr	否	MQ broker 地址. 默认: unix:\$PREFIX/data/mosquitto.sock
log_type	否	日志级别: debug, error, warning, notice, information, 不同的日志级别需要单独配置一行。默认: error
keepalive	否	给服务端发送 PING 命令的间隔时间. 默认: 30
kv_store_dir	否	Key value store 持久化文件路径. 默认: \$PREFIX/data
protocol_version	否	使用的协议版本, 要支持 request response 消息, 必须填 5. 默认: 5

配置都会有默认值, 建议只配置日志级别

```

C++
log_type error

```

3.4 国密支持

3.4.1 功能说明

使用国密版 OpenSSL 库来使用 SM2、SM3、SM4 算法（以下简称为 SM234 算法）。

SM234 算法使用双证书，包括签名证书和加密证书，这是国密支持在证书上的一个特点。

同时，还要求使用的国密版 OpenSSL 兼容标准 OpenSSL。

根据 NJet 的使用场景，国密支持的场景也细分为 Server 和反向代理两种场景。

3.4.2 配置说明

3.4.2.1 NJet 作为国密单向 Server 支持时的配置

```
Nginx
server {
    listen      443 ssl;
    server_name localhost;

    # 国密支持添加的证书
    ssl_certificate
/home/certs/certgm/serversign.crt;
    ssl_certificate_key
/home/certs/certgm/serversign.key;
    ssl_certificate
/home/certs/certgm/serverenc.crt;
    ssl_certificate_key
/home/certs/certgm/serverenc.key;

    location / {
        root    html;
        index  index.html index.htm;
    }
}
```

3.4.2.2 NJet 作为国密双向 Server 支持时的配置

```
Nginx
server {
    listen      443 ssl;
```

```

server_name localhost;
#启用客户端证书验证
ssl_verify_client on;
#指定客户端证书的 CA 证书路径
ssl_client_certificate
/home/certs/certgm/sm2.ca.pem;
# 国密支持添加的证书
ssl_certificate
/home/certs/certgm/serversign.crt;
ssl_certificate_key
/home/certs/certgm/serversign.key;
ssl_certificate
/home/certs/certgm/serverenc.crt;
ssl_certificate_key
/home/certs/certgm/serverenc.key;

location / {
    root    html;
    index  index.html index.htm;
}
}

```

3.4.2.3 NJet 作为国密和标准 ssl 自适应 Server 时的配置

```

Nginx
server {
    listen      443 ssl;
    server_name localhost;

    # 原 RSA 证书
    ssl_certificate      /home/certs/cert/sslserver.crt;
    ssl_certificate_key  /home/certs/cert/sslserver.key;

    # 国密支持添加的证书
    ssl_certificate
/home/certs/certgm/serversign.crt;
    ssl_certificate_key
/home/certs/certgm/serversign.key;
    ssl_certificate
/home/certs/certgm/serverenc.crt;
    ssl_certificate_key
/home/certs/certgm/serverenc.key;
}

```

```

        location / {
            root    html;
            index   index.html index.htm;
        }
    }
}

```

3.4.2.4 NJet 作为反向代理时，增加以下三个指令

`proxy_ssl_gm` 指令，用于 NJet 连接后端服务器时，指定是否使用国密。

Plain Text

Syntax: `proxy_ssl_gm on | off;`
 Default: `proxy_ssl_gm off`
 Context: `http, server, location`

`proxy_ssl_enc_certificate` 指令，用于 NJet 连接后端服务器(双向国密服务)时，指定加密证书文件。

Plain Text

Syntax: `proxy_ssl_enc_certificate file;`
 Default: `-`
 Context: `http, server, location`

`proxy_ssl_enc_certificate_key` 指令，用于 NJet 连接后端服务器(双向国密服务)时，指定加密密钥文件。

Plain Text

Syntax: `proxy_ssl_enc_certificate_key file;`
 Default: `-`
 Context: `http, server, location`

配置示例：

Plain Text

```

server {
    listen      8000;
    server_name localhost;

    proxy_ssl on;
    proxy_ssl_gm on;
    proxy_ssl_ciphers ECDHE-SM4-SM3; #也可以指定 ECC-SM4-SM3 国密套件
}

```

```
proxy_ssl_certificate      /home/certs/CS.crt;
proxy_ssl_certificate_key  /home/certs/CS.key;
proxy_ssl_enc_certificate  /home/certs/CE.crt;
proxy_ssl_enc_certificate_key /home/certs/CE.key;

# 其他配置，例如：
location / {
    proxy_pass https://192.168.183.111:443;
}
}
```

3.4.3 功能验证

3.4.3.1 验证服务器对国密的支持

验证工具为分别为密信浏览器和 gmcurl。

密信浏览器：



使用密信浏览器访问 server:



Welcome to njet!

If you see this page, the njet web server is successfully installed and working. Further configuration is required.

Thank you for using njet.

使用 gmcurl 访问 server:

```
[root@112 ~]# ./gmcurl --gmssl -k https://192.168.183.111:443
GM Version: 1.0.0 Ported by www.gmssl.cn
GM options:
--gmssl, use TLCP protocol
--cert,  use sm2 sig pem cert
--key,   use sm2 sig pem key
--cert2, use sm2 enc pem cert
--key2,  use sm2 enc pem key
<!DOCTYPE html>
<html>
<head>
<title>Welcome to njet!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to njet!</h1>
<p>If you see this page, the njet web server is successfully installed and
working. Further configuration is required.</p>

<p><em>Thank you for using njet.</em></p>
</body>
</html>
```

3.4.3.2 验证服务器对 RSA 证书的支持

上节的 server 的配置中分别指定了 RSA 证书和国密证书，再验证一下服务器对 RSA 证书的支持。验证工具为分别为 Chrome 浏览器和 curl。

使用 Chrome 浏览器访问 server:



Welcome to njet!

If you see this page, the njet web server is successfully installed and working. Further configuration is required.

Thank you for using njet.

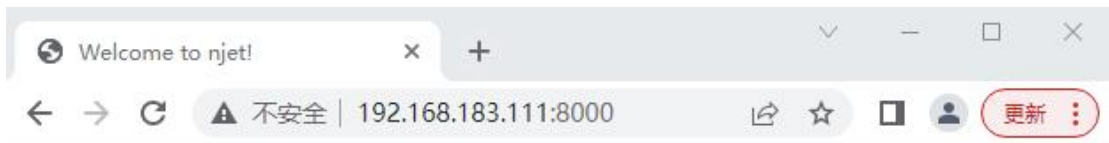
使用 curl 访问 server:

```
[root@112 ~]# curl -k https://192.168.183.111:443
<!DOCTYPE html>
<html>
<head>
<title>Welcome to njet!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to njet!</h1>
<p>If you see this page, the njet web server is successfully installed and
working. Further configuration is required.</p>

<p><em>Thank you for using njet.</em></p>
</body>
</html>
```

3.4.3.3 后端 upstream 使用国密验证

NJet 作反向代理访问后端国密 upstream 服务器。验证工具为分别为 Chrome 浏览器和 curl。



Welcome to njet!

If you see this page, the njet web server is successfully installed and working. Further configuration is required.

Thank you for using njet.

```
[root@112 ~]# curl -k 192.168.183.111:8000
<!DOCTYPE html>
<html>
<head>
<title>Welcome to njet!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to njet!</h1>
<p>If you see this page, the njet web server is successfully installed and
working. Further configuration is required.</p>

<p><em>Thank you for using njet.</em></p>
</body>
</html>
```

3.5 动态 Location

3.5.1 功能说明

本系统支持对 location 模块的动态添加、删除，可以对 locaiton 进行便捷的配置和所需指令功能的添加使用。

3.5.2 配置说明

请参考标准配置文件章节配置，全量配置即可配置上此功能，但请注意一定要包含以下指令和块：

njet.conf 配置文件中务必有此模块

Bash

```
load_module /home/njet/modules/njt_http_location_module.so;
```

ctrl.conf 配置文件中务必包含这些模块

Bash

```
load_module /home/njet/modules/njt_http_sendmsg_module.so;
load_module /home/njet/modules/njt_http_location_api_module.so;
```

```
http {
    server {
        listen      8081;
        location /dyn_location {
            dyn_location_api;
        }
    }
}
```

3.5.3 API 说明

动态 location 增加说明:

配置项	必填	配置说明
type	是	"add" 添加location
addr_port	是	添加的主机的, port 端口。 例如: "192.168.40.203: 8000", 或 "8000"
server_name	是	主机的server_name, 例如: " cluster.tmlake.com "
locations	是	List 对象列表。 对象字段: location_rule, location_name, location_body, proxy_pass [{"location_rule": "=", "location_name": "/test", "location_body": "return 200 1", "proxy_pass": "http://http2"}]

动态 location 删除说明:

配置项	必填	配置说明
type	是	"del" 删除location
addr_port	是	添加的主机的, port 端口。 例如: "192.168.40.203: 8000", 或 "8000"
server_name	是	主机的server_name, 例如: "cluster.tmlake.com"
location_rule	否	Location 的表达式, 例如: "=" 或 其他的正则式
location_name	是	location的名字, 也就是表达式后面的。 例如: "/" 或 "/test"
location_body	否	Location {} 中的内容。 例如"return 200 123", 证书等信息, 和Nginx 官方一致。 location_body , proxy_pass 必须有一个。
proxy_pass	否	proxy_pass 信息, 如果后面是 upstream 名字, 则需要该upstream 在njet.conf 文件中预先定义 location_body , proxy_pass 必须有一个。

3.5.4 调用样例

3.5.4.1 post 添加

回显 Success 即为添加成功, 但请注意, 回显 Success 只代表 json 格式无误, 如果 location 添加时逻辑有错误, 还是无法添加成功的。

Bash

```
curl -v -s -X POST http://127.0.0.1:8081/dyn_location -d
'{"type":"add","addr_port":"0.0.0.0:8899","server_name":"cluster1","locations":[{"location_rule":"","location_name":"/","location_body":"","proxy_pass":"http://http1"}]}'
* About to connect() to 127.0.0.1 port 8081 (#0)
*   Trying 127.0.0.1...
* Connected to 127.0.0.1 (127.0.0.1) port 8081 (#0)
> POST /dyn_location HTTP/1.1
> User-Agent: curl/7.29.0
> Host: 127.0.0.1:8081
> Accept: */*
> Content-Length: 149
> Content-Type: application/x-www-form-urlencoded
>
* upload completely sent off: 149 out of 149 bytes
< HTTP/1.1 200 OK
< Server: njet/1.23.1
< Date: Fri, 31 Mar 2023 03:09:09 GMT
< Content-Type: text/plain
< Content-Length: 7
```

```
< Connection: keep-alive
<
* Connection #0 to host 127.0.0.1 left intact
Success
```

3.5.4.2 查询

添加成功后，可以对所添加的 `location` 对应的上游资源进行请求，已验证是否可以正常对资源进行请求查询。

```
Bash
curl -v -s http://127.0.0.1:8899/1.html
* About to connect() to 127.0.0.1 port 8899 (#0)
*   Trying 127.0.0.1...
* Connected to 127.0.0.1 (127.0.0.1) port 8899 (#0)
> GET /1.html HTTP/1.1
> User-Agent: curl/7.29.0
> Host: 127.0.0.1:8899
> Accept: */*
>
< HTTP/1.1 200 OK
< Server: njet/1.23.1
< Date: Fri, 31 Mar 2023 03:34:00 GMT
< Content-Type: text/html
< Content-Length: 49
< Connection: keep-alive
< ETag: W/"49-1641534167000"
< Last-Modified: Fri, 07 Jan 2022 05:42:47 GMT
< Vary: Accept-Encoding
< X-Cache-Status: STALE
< Accept-Ranges: bytes
<
<html>
<head></head>
* Connection #0 to host 127.0.0.1 left intact
<body>111111</body></html>
```

3.5.4.3 put 禁用删除

动态 `location` 功能也支持动态进行删除操作，以下将展示如何进行验证：

进行动态删除 `location` 操作：

```
Shell
```

```
curl -v -s -X PUT http://127.0.0.1:8081/dyn_location -d
'{"type":"del","addr_port":"0.0.0.0:8899","server_name":"cluster1","location_rule":"","location_name":"/"}'
* About to connect() to 127.0.0.1 port 8081 (#0)
*   Trying 127.0.0.1...
* Connected to 127.0.0.1 (127.0.0.1) port 8081 (#0)
> PUT /dyn_location HTTP/1.1
> User-Agent: curl/7.29.0
> Host: 127.0.0.1:8081
> Accept: */*
> Content-Length: 105
> Content-Type: application/x-www-form-urlencoded
>
* upload completely sent off: 105 out of 105 bytes
< HTTP/1.1 200 OK
< Server: njet/1.23.1
< Date: Fri, 31 Mar 2023 03:39:10 GMT
< Content-Type: text/plain
< Content-Length: 7
< Connection: keep-alive
<
* Connection #0 to host 127.0.0.1 left intact
Success
```

再次请求之前的上游资源，可以发现已经请求失败，删除成功：

```
Bash
curl -v -s http://127.0.0.1:8899/1.html
* About to connect() to 127.0.0.1 port 8899 (#0)
*   Trying 127.0.0.1...
* Connected to 127.0.0.1 (127.0.0.1) port 8899 (#0)
> GET /1.html HTTP/1.1
> User-Agent: curl/7.29.0
> Host: 127.0.0.1:8899
> Accept: */*
>
< HTTP/1.1 404 Not Found
< Server: njet/1.23.1
< Date: Fri, 31 Mar 2023 03:40:05 GMT
< Content-Type: text/html
< Content-Length: 152
< Connection: keep-alive
<
<html>
```

```
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>njet/1.23.1</center>
</body>
</html>
* Connection #0 to host 127.0.0.1 left intact
```

3.6 主动健康检查

3.6.1 简介

主动健康检查与被动健康检查：主动健康检查由 Njet 主动发起请求测试上游服务是否正常，如不正常则及时剔除对应服务；被动健康检查由 Njet 检测上游服务的异常返回发现服务异常，不能主动感知上游服务状态。

helper 进程主动健康检查：可以实现健康检查由独立进程完成，对 work 进程无影响，完成后修改共享内存完成 upstream 状态回写。

3.6.2 指令说明

health_check_api

Plaintext

语法： health_check_api ;

默认值： -

允许配置位置： location

在当前 location 开启健康检查 API。

API 说明

参见下文 openapi 链接

<http://192.168.40.119:8081/doc/swagger>

部分 api 字段补充

健康检查 HTTP > status 配置方式

配置内容	判定为健康的描述
200	http 返回状态码为 200

! 500	http 返回状态码不是 500
200 204	http 返回状态码是 200 或者 204
! 301 302	http 返回状态码不是 301 且不是 302
200-399	http 返回状态码是 200 到 399 的，包含 200 和 399
! 400-599	http 返回状态码不是 400 到 599 的，包含 400 和 599
301-303 307	http 返回状态码是 301 到 303 的，包含 301 和 303，或者 307

健康检查 http > header 配置方式

配置内容	判定为健康的描述
Content-Type = text/html	返回 header 中存在"Content-Type" header，且 header 内容为"text/html"
Content-Type != text/html	返回 header 中存在"Content-Type" header，且值为除"text/html"外任意内容。
Connection ~ close	返回 header 中存在 Connection，且值可以匹配表达式"~ close"
Connection !~ close	返回 header 中存在 Connection，且值可以不匹配表达式"~ close"
Host	返回 header 中存在 Host
!X-Accel-Redirect	返回 header 中不存在 X-Accel-Redirect

健康检查 http > body 配置方式

配置内容	判定为健康的描述
~ Welcome	返回 body 内容匹配表达式"~ Welcome"
!~ Welcome	返回 body 内容不匹配表达式"!~ Welcome"

3.6.3 请求示例

增加新的健康检查

请求

```
HTTP
POST http://127.0.0.1:8081/hc/1/hc/http/backend1
Content-Type: application/json

{
  "interval": "3s",
  "jitter": "1s",
  "timeout": "10s",
  "passes": 2,
  "fails": 1,
  "http": {
    "header": [
      "Host"
    ],
    "uri": "/test",
    "body": "~ ok",
    "status": "200"
  },
  "ssl": {
    "enable": false
  }
}
```

返回值

```
HTTP
HTTP/1.1 200 OK
Server: njet/1.23.1
Date: Fri, 10 Feb 2023 13:06:24 GMT
Content-Type: application/json
Content-Length: 37
Connection: keep-alive

{
  "code": 0,
  "msg": "success"
}
```

删除健康检查

请求

```
HTTP
DELETE http://127.0.0.1:8081/hc/1/hc/http/backend1
```

返回值

```
HTTP
HTTP/1.1 200 OK
Server: njet/1.23.1
Date: Fri, 10 Feb 2023 13:06:21 GMT
Content-Type: application/json
Content-Length: 37
Connection: keep-alive

{
  "code": 0,
  "msg": "success"
}
```

[查看健康检查列表](#)

请求

```
Nginx
GET http://127.0.0.1:8081/hc/1/hc
```

返回值

```
HTTP
HTTP/1.1 200 OK
Server: njet/1.23.1
Date: Fri, 10 Feb 2023 13:07:12 GMT
Content-Type: application/json
Content-Length: 44
Connection: keep-alive

[
  {
    "upstream": "backend1",
    "type": "http"
  }
]
```

[查看健康检查配置详情](#)

请求

HTTP

GET http://127.0.0.1:8081/hc/1/hc/http/backend1

返回值

HTTP

HTTP/1.1 200 OK

Server: njet/1.23.1

Date: Fri, 10 Feb 2023 13:07:14 GMT

Content-Type: application/json

Content-Length: 157

Connection: keep-alive

```
{
  "interval": "3s",
  "jitter": "1s",
  "timeout": "10s",
  "passes": 2,
  "fails": 1,
  "http": {
    "uri": "/test",
    "status": "200",
    "body": "~ ok",
    "header": [
      "Host"
    ]
  }
}
```

健康检查开启国密

请求

HTTP

POST http://127.0.0.1:8081/api/1/hc/http/demos

Content-Type: application/json

```
{
  "interval": "3s",
  "jitter": "1s",
  "timeout": "10s",
  "passes": 2,
  "fails": 1,
```

```
"http": {
  "uri": "/test",
  "status": "200"
},
"ssl": {
  "enable": true,
  "ntls": true
}
}
```

返回

```
HTTP
HTTP/1.1 200 OK
Server: njet/1.23.1
Date: Mon, 13 Feb 2023 07:35:27 GMT
Content-Type: application/json
Content-Length: 37
Connection: keep-alive

{
  "code": 0,
  "msg": "success"
}
```

3.7 metrics

3.7.1 功能概述

`metrics` 模块提供对虚拟主机状态信息的访问，包含了 `servers`, `upstreams`, `caches` 的当前状态信息。

3.7.2 具体功能

首先，指令 `vhost_traffic_status_zone` 是必需的，然后如果指令 `vhost_traffic_status_display` 设置后，可按如下方式访问：

- `/metrics/format/json`

如果您要求 `/metrics/format/json`，将使用包含当前活动数据的 JSON 文档进行响应，以便在实时仪表板和第三方监视工具中使用。

- `/metrics/format/html`

如果您要求 `/metrics/format/html`，将使用 HTML 中的内置实时仪表板进行响

应，该仪表板在内部请求 `/metrics/format/json`。

- `/metrics/format/jsonp`

如果您要求 `/metrics/format/jsonp`，将使用包含当前活动数据的 JSONP 回调函数进行响应，以便在实时仪表板和第三方监视工具中使用。

- `/metrics/format/prometheus`

如果您要求 `/metrics/format/prometheus`，将响应 `prometheus` 格式，包含当前活动数据的文档。

3.7.2.1 JSON 数据示例

```
JSON
{
  "hostName": ...,
  "moduleVersion": ...,
  "nginxVersion": ...,
  "loadMsec": ...,
  "nowMsec": ...,
  "connections": {
    "active":...,
    "reading":...,
    "writing":...,
    "waiting":...,
    "accepted":...,
    "handled":...,
    "requests":...
  },
  "sharedZones": {
    "name":...,
    "maxSize":...,
    "usedSize":...,
    "usedNode":...
  },
  "serverZones": {
    "...":{
      "requestCounter":...,
      "inBytes":...,
      "outBytes":...,
      "responses":{
        "1xx":...,
        "2xx":...,
        "3xx":...,
        "4xx":...,

```

```

        "5xx":...,
        "miss":...,
        "bypass":...,
        "expired":...,
        "stale":...,
        "updating":...,
        "revalidated":...,
        "hit":...,
        "scarce":...
    },
    "requestMsecCounter":...,
    "requestMsec":...,
    "requestMsecs":{
        "times":[...],
        "msecs":[...]
    },
    "requestBuckets":{
        "msecs":[...],
        "counters":[...]
    },
}
...
},
"filterZones": {
    "...":{
        "...":{
            "requestCounter":...,
            "inBytes":...,
            "outBytes":...,
            "responses":{
                "1xx":...,
                "2xx":...,
                "3xx":...,
                "4xx":...,
                "5xx":...,
                "miss":...,
                "bypass":...,
                "expired":...,
                "stale":...,
                "updating":...,
                "revalidated":...,
                "hit":...,
                "scarce":...
            },

```

```

        "requestMsecCounter":...,
        "requestMsec":...,
        "requestMsecs":{
            "times":[...],
            "msecs":[...]
        },
        "requestBuckets":{
            "msecs":[...],
            "counters":[...]
        },
    },
    ...
},
...
},
"upstreamZones": {
    "...":[
        {
            "server":...,
            "requestCounter":...,
            "inBytes":...,
            "outBytes":...,
            "responses":{
                "1xx":...,
                "2xx":...,
                "3xx":...,
                "4xx":...,
                "5xx":...
            },
            "requestMsecCounter":...,
            "requestMsec":...,
            "requestMsecs":{
                "times":[...],
                "msecs":[...]
            },
            "requestBuckets":{
                "msecs":[...],
                "counters":[...]
            },
            "responseMsecCounter":...,
            "responseMsec":...,
            "responseMsecs":{
                "times":[...],
                "msecs":[...]
            }
        }
    ]
}

```

```

        },
        "responseBuckets":{
            "msecs":[...],
            "counters":[...]
        },
        "weight":...,
        "maxFails":...,
        "failTimeout":...,
        "backup":...,
        "down":...
    }
    ...
],
...
}
"cacheZones": {
    "...":{
        "maxSize":...,
        "usedSize":...,
        "inBytes":...,
        "outBytes":...,
        "responses":{
            "miss":...,
            "bypass":...,
            "expired":...,
            "stale":...,
            "updating":...,
            "revalidated":...,
            "hit":...,
            "scarce":...
        }
    },
    ...
}
}

```

3.7.2.2 JSON 字段含义

以下含义以 JSON 示例顺序提供：

Bash

hostName: 主机名

moduleVersion: {Version}(|.dev.{commit})格式的模块版本

njetVersion: njet 版本

loadMsec: 加载过程时间（以毫秒为单位）

nowMsec: 当前时间（毫秒）

connections

- active:** 当前活动客户端连接数
- reading:** 读取客户端连接的总数
- writing:** 写入客户端连接的总数
- waiting:** 等待客户端连接的总数
- accepted:** 接受的客户端连接总数
- handled:** 已处理的客户端连接总数。
- requests:** 请求的客户端连接总数。

sharedZones

- name:** 在配置中指定的共享内存的名称。（默认值：**vhost_traffic_status**）
- maxSize:** 在配置中指定的共享内存的最大大小的限制
- usedSize:** 共享内存的当前大小
- usedNode:** 共享内存中使用的当前节点数。它可以通过以下公式获得一个节点的大致大小： $(usedSize / usedNode)$

serverZones

- requestCounter:** 从客户端收到的客户端请求总数
- inBytes:** 从客户端接收的总字节数
- outBytes:** 发送到客户端的总字节数

responses

- 1xx,2xx,3xx,4xx,5xx:** 状态代码为 1xx、2xx、3xx、4xx 和 5xx 的响应数
- miss:** cache miss 的个数
- bypass:** 缓存绕过的数量
- expired:** 缓存过期的数量
- stale:** 缓存过时的数量
- updating:** 缓存更新的次数
- revalidated:** 重新验证的缓存数
- hit:** 缓存命中数
- scarce:** 缓存不足的数量

requestMsecCounter: 累积请求处理时间的次数（以毫秒为单位）

requestMsec: 请求处理时间的平均时间（以毫秒为单位）

requestMsecs

- times:** 请求处理时间以毫秒为单位
- msecs:** 请求处理时间（包括上游），以毫秒为单位

requestBuckets

- msecs:** 由 **vhost_traffic_status_histogram_buckets** 指令设置的柱状图的桶值
- counters:** 每个存储桶值大于或等于请求处理时间的累积值

filterZones: 提供了与 **serverzone** 相同的字段

upstreamZones

- server:** 服务器的地址
- requestCounter:** 转发到此服务器的客户端连接总数
- inBytes:** 从此服务器接收的总字节数
- outBytes:** 发送到此服务器的总字节数
- responses**
 - 1xx,2xx,3xx,4xx,5xx:** 状态代码为 1xx、2xx、3xx、4xx 和 5xx 的响应数
- requestMsecCounter:** 包括上游在内的累计请求处理时间数（以毫秒为单位）
- requestMsec:** 请求处理时间（包括上游）的平均时间（以毫秒为单位）
- requestMsecs**
 - times:** 请求处理时间以毫秒为单位
 - msecs:** 请求处理时间（包括上游），以毫秒为单位
- requestBuckets**
 - msecs:** 由 **vhost_traffic_status_histogram_buckets** 指令设置的柱状图的桶值
 - counters:** 每个存储桶值大于或等于请求处理时间（包括上游）的累积值
- responseMsecCounter:** 累积的仅上游响应处理时间的数量，以毫秒为单位
- responseMsec:** 仅上游响应处理时间的平均值（以毫秒为单位）
- responseMsecs**
 - times:** 请求处理时间以毫秒为单位
 - msecs:** 唯一的上游响应处理时间（以毫秒为单位）
- responseBuckets**
 - msecs:** 由 **vhost_traffic_status_histogram_buckets** 指令设置的柱状图的桶值
 - counters:** 每个存储桶值大于或等于唯一上游响应处理时间的累积值
- weight:** 当前服务器设置的权重
- maxFails:** 服务器当前 **max_fails** 设置
- failTimeout:** 服务器当前 **fail_timeout** 设置
- backup:** 服务器当前备份设置
- down:** 服务器当前 **down** 设置。基本上，这只是一个标记

ngx_http_upstream_module 的服务器关闭(例如: 服务器 **backend3.example.com** 关闭), 而不是实际的上游服务器状态。如果你启用了 **upstream zone** 指令, 它将改变为实际状态

cacheZones

- maxSize:** 在配置中指定的缓存的最大大小的限制。如果 **proxy_cache_path** 指令中的 **max_size** 未指定, 则默认分配系统依赖值 **NGX_MAX_OFF_T_VALUE**
- usedSize:** 缓存的当前大小。这个值取自 **njet**, 就像上面的 **maxSize** 值一样。

inBytes: 从缓存接收的总字节数。
outBytes: 从缓存发送的总字节数。
responses
 miss: 缓存未命中数。
 bypass: 缓存绕过的数量。
 expired: 缓存过期的数量。
 stale: 缓存过时的数量。
 updating: 缓存更新的次数。
 revalidated: 重新验证的缓存数。
 hit: 缓存命中数。
 scarce: 缓存不足的数量。

3.7.3 指令介绍

3.7.3.1 vhost_traffic_status_zone

SQL
Syntax: `vhost_traffic_status_zone [shared:name:size]`
Default: `shared:vhost_traffic_status:1m`
Context: `http`

描述: 数据面配置。用于定义一个共享内存区域来存储虚拟主机流量统计信息。该指令必须在 `http` 配置块中使用, 定义的共享内存区域会在所有 `server` 块中共享, 以用于存储所有虚拟主机的统计信息。

3.7.3.2 vhost_traffic_status_filter_by_set_key

SQL
Syntax: `vhost_traffic_status_filter_by_set_key key [name]`
Default: `-`
Context: `http`

描述: 数据面配置。该指令必须在 `http` 配置块中使用, 通过用户定义的变量将具有相同键值的访问信息合并统计, 方便按照特定条件进行统计和分析。`key` 和 `name` 可以使用变量, 如 `$status`、`$server_name` 等。

示例如下:

```
Bash
http{
    vhost_traffic_status_filter_by_set_key $status $server_name;
```

```
...  
}
```

3.7.3.3 vhost_traffic_status_display

SQL

Syntax: vhost_traffic_status_display

Default: -

Context: location

描述：控制配置。启用或禁用控制面显示处理程序。

3.7.3.4 vhost_traffic_status_display_format

SQL

Syntax: vhost_traffic_status_display_format

<json|html|jsonp|prometheus>

Default: -

Context: location

描述：控制面配置，该指令的作用是设置展示统计结果的格式

- 如果设置 json，以 JSON 格式展示统计结果；
- 如果设置 html，以 HTML 格式展示统计结果；JSONP
- 如果设置 jsonp，以 JSONP 格式展示统计结果；
- 如果设置 prometheus，以 Prometheus 格式展示统计结果。

控制面示例如下：

```
Nginx  
location /metrics{  
    vhost_traffic_status_display;  
    vhost_traffic_status_display_format html;  
}
```

3.7.4 动态开关

metrics 模块可以通过动态配置接口实现对是否统计及统计指标的动态配置，详见：
动态开关功能 metric

3.8 Http 会话保持

3.8.1 功能概述

该模块可以识别用户 Session，从而能够鉴别不同的客户端，并且可以将来自同一个客户端的请求发往同一个上游服务器。这在当应用保存了用户状态的情况下非常有用，可以避免负载均衡器按照某个算法将请求发到别的服务器上去。另外，在共享用户信息的集群服务器这种方式也非常有用。

该模块按照如下的步骤进行工作：

- 如果 request 匹配某个 Session 一致性的规则，那么根据这个规则选取上游服务器；
- 如果没有匹配上或者匹配的服务器无法使用，那么使用负载均衡算法选择上游服务器；

具体实现方式，有三种：sticky cookie，sticky learn 和 sticky route。

sticky cookie:

由负载均衡器，为其分配上游服务器，并记录到客户端 cookie 中，默认轮询算法

sticky route:

由上游服务器分配，负载均衡器配合配置，并记录到客户端 cookie 中，默认轮询算法

sticky learn:

由上游服务器分配，负载均衡器自动学习，并记录到客户端 cookie 中，默认轮询算法

3.8.2 sticky cookie

3.8.2.1 配置示例

```
Plain Text
upstream backend {
    server backend1.example.com;
    server backend2.example.com;
    sticky cookie srv_id expires=1h domain=.example.com
    path=/;
}
```

3.8.2.2 指令介绍

Bash

Syntax: `sticky cookie name [expires=time] [domain=domain] [httponly] [samesite=strict|lax|none|$variable] [secure] [path=path]`

Default:—

Context:upstream

JSON

`expires=time`

当设置的过期时间是特殊值 "max" 时，这意味着该 `cookie` 将在 2037 年 12 月 31 日 23:55:55 GMT 过期。如果没有设置过期时间，那么 `cookie` 将在浏览器会话结束时过期，即用户关闭浏览器时。如果显式地将过期时间设置为某个时间点，则该 `cookie` 将在该时间点过期。

`domain=domain`

指定 `cookie` 可以被哪些域名访问。如果它与请求中的域名相同或是它的后缀为请求中域名的后缀，则请求中的域名可以访问该 `cookie`。如果指定的是顶级域名，例如 `.example.com`，则该域名下的所有子域名都可以访问该 `cookie`。

`httponly`

是一个 `cookie` 标志属性，用于指定某个 `cookie` 是否应该仅通过 HTTP 协议来访问，而不能被客户端 JavaScript 脚本访问。这是一个用于增强 Web 应用程序安全性的措施，可以有效防止跨站脚本攻击（XSS）。

如果 `HttpOnly` 属性设置为 `true`，浏览器将只在 HTTP 请求头中发送该 `cookie`，而不会在客户端 JavaScript 中暴露该 `cookie`。这意味着攻击者无法使用 JavaScript 访问该 `cookie`，因此无法窃取与该 `cookie` 相关的敏感信息。

`samesite=strict| lax| none|$variable`

`samesite` 是用来设置 `SameSite` 属性的，`SameSite` 属性是用于防止 CSRF 攻击和用户追踪的一种安全机制。它的取值可以是 `strict`、`lax`、`none` 或者变量。

strict: 表示同站点且仅在 `https` 协议下可以发送 `cookie`。这样可以防止跨站点攻击，保证用户的隐私安全，但是这样可能会影响用户的使用体验。

lax: 也只在同站点且仅在 `https` 协议下可以发送 `cookie`，但是除了从第三方站点导航到目标站点的 GET 请求外，还允许从其他站点的链接导航到目标站点。这样会有一定的跨站点攻击风险，但相对于 `strict` 模式用户的使用体验更好。

none: 没有任何限制，无论是同站点还是跨站点，无论 `http` 还是 `https` 协议，都可以发送 `cookie`。不过需要注意，设置 `none` 模式可能会

带来安全风险。

\$variable: 表示可以从变量中获取 **SameSite** 属性的值。该变量可是在 **server** 块或 **location** 块中定义的任意变量，但是必须是字符串类型的。该方式可以提高灵活性。

secure

secure 是一个 **cookie** 属性，表示只有在安全的 **HTTPS** 连接上才能传输 **cookie**。如果将 **secure** 设置为 **true**，则 **cookie** 将只在 **HTTPS** 连接中发送到服务器，否则 **cookie** 将在任何情况下都被发送到服务器。当一个 **cookie** 的 **secure** 属性被设置为 **true** 时，这个 **cookie** 只有在 **HTTPS** 连接中被传输，并且不能被恶意攻击者所窃听和窃取。

path=path

path=path 是在设置 **Cookie** 时可选的参数，它指定了客户端浏览器所请求的 **URL** 路径必须匹配 **Cookie** 的路径才能发送 **Cookie**。该参数的值必须以正斜杠 **/** 开头，表示在该路径及其子路径下的请求都将发送该 **Cookie**。

例如，当设置 **path=/example** 时，**Cookie** 将被发送到以 **/example** 开头的 **URL**。如果客户端请求的 **URL** 不是以 **/example** 开头的，那么就不会发送该 **Cookie**。

需要注意的是，**path** 参数的默认值是当前请求的 **URL** 路径的最左侧目录，例如，如果请求的 **URL** 是 **http://example.com/foo/bar**，那么默认的 **path** 值将是 **/foo**。如果想将 **Cookie** 设置为整个域名下都可用，需要将 **path** 设置为根目录 **/**。

备注：如果设置 **Cookie** 时缺少某些参数，那么对应的 **Cookie** 字段就不会被设置

3.8.3 sticky route

3.8.3.1 配置示例

Bash

```
map $cookie_jsessionid $route_cookie {
    ~(?P<route>\w+)$ $route;
}

map $request_uri $route_uri {
    ~jsessionid=(?P<route>\w+)$ $route;
}

upstream backend {
    server backend1.example.com route=a;
```

```
server backend2.example.com route=b;
sticky route $route_cookie $route_uri;
}
```

3.8.3.2 指令介绍

Bash
Syntax: sticky route \$variable ...
Default: -
Context: upstream

Plain Text

sticky route

定义 **route** 模式使用的变量名。一个或 2 个。两个时按顺序优先使用 例如: sticky route \$route_cookie \$route_uri;

route

设置 **server** 的 **route** 值。例如: server backend1.example.com route=a;

此处, **route** 取自 **cookie** 中的 "jsessionid" (如果存在于请求中)。否则, 使用来自 **URI** 中的 **jsessionid**

3.8.4 sticky learn

3.8.4.1 配置示例

```
Bash
upstream backend {
    server backend1.example.com;
    server backend2.example.com;
    sticky learn
        create=$upstream_cookie_examplecookie
        lookup=$cookie_examplecookie
        zone=client_sessions:1m
        timeout=1h
        header;
}
```

3.8.4.2 指令介绍

Bash

```
Syntax:sticky learn create=$variable lookup=$variable
zone=name:size [timeout=time] [header] [sync]
Default:–
Context:upstream
```

Plain Text

create

用于指定上游服务器的 **cookie** 名称，可以是现有的 **cookie** 或新 **cookie** 的名称。在这个例子中，它被设置为 **\$upstream_cookie_examplecookie**，表示使用上游服务器返回的名为 **examplecookie** 的 **cookie**。

lookup

用于指定将要查找和匹配的 HTTP 请求的 **cookie** 名称。在这个例子中，它被设置为 **\$cookie_examplecookie**，表示将查找名为 **examplecookie** 的请求 **cookie**。

zone

用于指定存储客户端会话状态信息的共享内存区域的名称和大小。

timeout

用于指定客户端会话状态信息的过期时间。如果客户端的请求在过期时间之后到达，则会将其发送到上游服务器，以确保会话状态信息是最新的。

header

header 是一个可选参数，表示在学习和查找 **cookie** 时，使用 HTTP 请求头部中指定的 **cookie** 值。如果没有指定该参数，则使用默认的 **cookie** 参数

3.9 Fastcgi

3.9.1 功能概述

FastCGI 是一种用于实现 Web 服务器与应用程序之间的协议的方法，它可以提供高性能和灵活性，使得 Web 服务器可以与各种不同类型的应用程序进行通信。FastCGI 是一种比较新的 CGI 协议，相较于旧版 CGI 协议，它更加稳定，也更容易扩展和管理。在 Njet 中，FastCGI 可以与 PHP、Ruby、Python、Java 等多种编程语言和应用程序进行通信，实现动态 Web 应用程序的运行。FastCGI 协议通过套接字(socket)连接，将 Web 服务器和应用程序分离，从而提高了安全性和性能，并且使得应用程序可以独立地运行在 Web 服务器之外。此外，NjetFast CGI 还支持各种负载均衡和缓存技术，以提高性能和可用性。

3.9.2 配置示例

njet.conf

```
C
http {

    ....
    ....

    server {
        listen      80;
        server_name localhost;
        location ~ /\.php$ {
            root /usr/share/njet/html;
            fastcgi_pass 127.0.0.1:9000;
            fastcgi_index index.php;
            include      fastcgi.conf;
        }
    }
}
```

fastcgi.conf

```
C
fastcgi_param SCRIPT_FILENAME
    $document_root$fastcgi_script_name;
fastcgi_param QUERY_STRING      $query_string;
fastcgi_param REQUEST_METHOD    $request_method;
fastcgi_param CONTENT_TYPE      $content_type;
fastcgi_param CONTENT_LENGTH    $content_length;

fastcgi_param SCRIPT_NAME        $fastcgi_script_name;
fastcgi_param REQUEST_URI        $request_uri;
fastcgi_param DOCUMENT_URI       $document_uri;
fastcgi_param DOCUMENT_ROOT      $document_root;
fastcgi_param SERVER_PROTOCOL    $server_protocol;
fastcgi_param REQUEST_SCHEME     $scheme;
fastcgi_param HTTPS              $https if_not_empty;

fastcgi_param GATEWAY_INTERFACE  CGI/1.1;
fastcgi_param SERVER_SOFTWARE    njet/$njet_version;

fastcgi_param REMOTE_ADDR        $remote_addr;
fastcgi_param REMOTE_PORT        $remote_port;
```

```
fastcgi_param  SERVER_ADDR      $server_addr;
fastcgi_param  SERVER_PORT      $server_port;
fastcgi_param  SERVER_NAME      $server_name;

# PHP only, required if PHP was built with --enable-force-
cgi-redirect
fastcgi_param  REDIRECT_STATUS  200;
```

3.9.1 指令介绍

3.9.1.1 fastcgi_cache

```
SQL
Syntax: fastcgi_cache zone | off;
Default: fastcgi_cache off;
Context: http, server, location
```

用于开启或关闭 FastCGI 缓存，并指定缓存使用的共享内存区域

参数：

- **zone**：指定 FastCGI 缓存使用的共享内存区域，由 **fastcgi_cache_path** 指令定义。多个 **location** 可以共享同一个缓存区域。
- **off**：禁用从上级配置级别继承的缓存配置。

3.9.1.2 fastcgi_cache_key

```
SQL
Syntax: fastcgi_cache_key string;
Default: -
Context: http, server, location
```

用于设置 FastCGI 缓存键值的生成规则，以便在缓存中查找和存储响应内容

3.9.1.3 fastcgi_cache_bypass

```
Plaintext
Syntax: fastcgi_cache_bypass string ...;
Default: -
Context: http, server, location
```

用于定义不从缓存中获取响应的条件。如果至少一个字符串参数的值不为空且不等于“0”，则响应不会从缓存中获取：

```
Bash
fastcgi_cache_bypass $cookie_nocache
$args_nocache$args_comment;
fastcgi_cache_bypass $http_pragma    $http_authorization;
```

可以和 `fastcgi_no_cache` 指令一起使用。

3.9.1.4 fastcgi_no_cache

```
Plaintext
Syntax: fastcgi_no_cache string ...;
Default: -
Context: http, server, location
```

用于定义不将响应保存到缓存的条件。如果至少有一个字符串参数中的值不为空且不等于"0"，则响应将不会被保存：

```
Bash
fastcgi_no_cache $cookie_nocache $args_nocache$args_comment;
fastcgi_no_cache $http_pragma    $http_authorization;
```

可以与 `fastcgi_cache_bypass` 指令一起使用。

3.9.1.5 fastcgi_cache_methods

```
SQL
Syntax: fastcgi_cache_methods GET | HEAD | POST ...;
Default: fastcgi_cache_methods GET HEAD;
Context: http, server, location
```

用于配置哪些 HTTP 请求方法需要缓存，虽然 `GET` 和 `HEAD` 方法总是在该列表中，但我们还是建议您明确指定它们。另请参阅 `fastcgi_no_cache` 指令。

3.9.1.6 fastcgi_cache_min_uses

```
SQL
Syntax: fastcgi_cache_min_uses number;
Default: fastcgi_cache_min_uses 1;
Context: http, server, location
```

用于设置在将响应从缓存中提供给客户端之前，该响应必须在同一位置上出现的次数

3.9.1.7 fastcgi_cache_use_stale

SQL

```
Syntax: fastcgi_cache_use_stale error | timeout |  
invalid_header | updating | http_500 | http_503 | http_403  
| http_404 | http_429 | off ...;  
Default: fastcgi_cache_use_stale off;  
Context: http, server, location
```

当在与 FastCGI 服务器通信期间发生错误时可以使用陈旧的缓存响应。该指令的参数与 `fastcgi_next_upstream` 指令的参数相匹配。

如果无法选择使用 FastCGI 服务器处理请求，则 `error` 参数还允许使用陈旧的缓存响应。

此外，如果它当前正在更新，`updating` 参数允许使用陈旧的缓存响应。这样可以在更新缓存数据时最大限度地减少对 FastCGI 服务器的访问次数。

也可以在响应头中直接启用在响应变为陈旧的指定秒数后使用陈旧的缓存响应。这比使用指令参数的优先级低。

- `Cache-Control` 头字段的 `stale-while-revalidate` 扩展允许使用陈旧的缓存响应当它正在更新。
- `Cache-Control` 头字段的 `stale-if-error` 扩展允许在发生错误时使用陈旧的缓存响应。

为了最大限度地减少填充新缓存元素时对 FastCGI 服务器的访问次数，可以使用 `fastcgi_cache_lock` 指令。

3.9.1.8 fastcgi_cache_valid

SQL

```
Syntax: fastcgi_cache_valid [code ...] time;  
Default: -  
Context: http, server, location
```

为不同的响应码设置缓存时间。例如：

Nginx

```
fastcgi_cache_valid 200 302 10m;  
fastcgi_cache_valid 404 1m;
```

对响应码为 200 和 302 的响应设置 10 分钟缓存，对响应码为 404 的响应设置为 1 分钟。

如果只指定缓存时间 (**time**) :

```
Nginx
fastcgi_cache_valid 5m;
```

那么只缓存 200 、 301 和 302 响应。

另外, 可以指定 **any** 参数来缓存任何响应:

```
Nginx
fastcgi_cache_valid 200 302 10m;
fastcgi_cache_valid 301      1h;
fastcgi_cache_valid any      1m;
```

缓存参数也可以直接在响应头中设置。这比使用指令设置缓存时间具有更高的优先级。

- **X-Accel-Expires** 头字段以秒为单位设置响应的缓存时间。零值会禁用响应缓存。如果该值以 **@** 前缀开头, 则它会设置自 Epoch 以来的绝对时间 (以秒为单位), 最多可以缓存该时间段内的响应。
- 如果头中不包含 **X-Accel-Expires** 字段, 则可以在头字段 **Expires** 或 **Cache-Control** 中设置缓存参数。
- 如果头中包含 **Set-Cookie** 字段, 则不会缓存此类响应。
- 如果头中包含具有特殊值 ***** 的 **Vary** 字段, 则这种响应不会被缓存 (1.7.7)。如果头中包含带有另一个值的 **Vary** 字段, 考虑到相应的请求头字段 (1.7.7), 这样的响应将被缓存。

使用 **fastcgi_ignore_headers** 指令可以禁用一个或多个响应头字段的处理。

3.9.1.9 fastcgi_cache_path

```
SQL
Syntax: fastcgi_cache_path path [levels=levels]
[use_temp_path=on|off] keys_zone=name:size [inactive=time]
[max_size=size] [min_free=size] [manager_files=number]
[manager_sleep=time] [manager_threshold=time]
[loader_files=number] [loader_sleep=time]
[loader_threshold=time] [purger=on|off]
[purger_files=number] [purger_sleep=time]
[purger_threshold=time];
Default: -
Context: http
```

设置缓存的路径和其他参数。缓存数据存储在文件中。缓存中的 **key** 和文件名是代理 URL 经过 MD5 函数处理后得到的值。**levels** 参数定义缓存的层次结构级别：范围从 **1** 到 **3**，每个级别可接受值为 **1** 或 **2**。例如，在以下配置中

```
Nginx
fastcgi_cache_path /data/njet/cache levels=1:2
keys_zone=one:10m;
```

缓存中的文件名如下所示：

```
Plaintext
/data/njet/cache/c/29/b7f54b2df7773722d382f4809d65029c
```

首先将缓存的响应写入临时文件，然后重命名该文件。建议，对于任何给定的位置，缓存和保存临时文件的目录都应该放在同一个文件系统上。临时文件的目录根据 **use_temp_path** 参数设置。如果忽略此参数或将其设置为 **on**，则将使用由 **fastcgi_temp_path** 指令设置的目录。如果该值设置为 **off**，临时文件将直接放在缓存目录中。

另外，所有活跃的 **key** 和有关数据的信息都存储在共享内存区中，其名称和大小由 **keys_zone** 参数配置。一个兆字节的区域可以存储大约 8 千个 **key**。

在 **inactive** 参数指定的时间内未被访问的缓存数据将从缓存中删除。默认情况下，**inactive** 设置为 10 分钟。

“缓存管理器”（cache manager）进程监视的最大缓存大小由 **max_size** 参数设置。当超过此大小时，它将删除最近最少使用的数据。数据在由 **manager_files**、**manager_threshold** 和 **manager_sleep** 参数配置下进行迭代删除。在一次迭代中，不会超过 **manager_files** 项被删除（默认为 100）。一次迭代的持续时间受到 **manager_threshold** 参数（默认为 200 毫秒）的限制。在每次迭代之间存在间隔时间，由 **manager_sleep** 参数（默认为 50 毫秒）配置。

开始后一分钟，“缓存加载器”（cache loader）进程被激活。它将先前存储在文件系统中的缓存数据的有关信息加载到缓存区中。加载也是在迭代中完成。在每一次迭代中，不会加载 **loader_files** 个项（默认情况下为 100）。此外，每一次迭代的持续时间受到 **loader_threshold** 参数的限制（默认情况下为 200 毫秒）。在迭代之间存在间隔时间，由 **loader_sleep** 参数（默认为 50 毫秒）配置。

3.9.1.10 fastcgi_index

```
SQL
Syntax: fastcgi_index name;
Default: -
Context: http, server, location
```

用于设置在 FastCGI 请求中使用的默认的 FastCGI 应用程序的索引文件名。

当使用 FastCGI 处理程序时，如果 FastCGI 响应中没有指定文件名，则会使用 `fastcgi_index` 指令所设置的文件名

3.9.1.11 fastcgi_pass

```
SQL
Syntax: fastcgi_pass address;
Default: -
Context: location, if in location
```

设置 FastCGI 服务器的地址。该地址可以指定为域名或 IP 地址，以及端口：

```
Nginx
fastcgi_pass localhost:9000;
```

或者作为 UNIX 域套接字路径：

```
Nginx
fastcgi_pass unix:/tmp/fastcgi.socket;
```

如果域名解析为多个地址，则所有这些地址都将以循环方式使用。另外，地址可以被指定为服务器组。

参数值可以包含变量。在这种情况下，如果地址被指定为域名，则在所描述的服务器组中搜索名称，如果未找到，则使用解析器来确定。

3.9.1.12 fastcgi_pass_request_body

```
SQL
Syntax: fastcgi_pass_request_body on | off;
Default: fastcgi_pass_request_body on;
Context: http, server, location
```

用于指定是否传递 HTTP 请求主体到 FastCGI 服务器。另请参阅 `fastcgi_pass_request_headers` 指令。

3.9.1.13 fastcgi_pass_request_headers

SQL

Syntax: `fastcgi_pass_request_headers on | off;`

Default: `fastcgi_pass_request_headers on;`

Context: `http, server, location`

用于控制是否将客户端请求的 HTTP 头部信息传递给 FastCGI 后端服务。另请参阅 `fastcgi_pass_request_body` 指令。

3.9.1.14 `fastcgi_hide_header`

SQL

Syntax: `fastcgi_hide_header field;`

Default: `-`

Context: `http, server, location`

默认情况下, nginx 不会将 FastCGI 服务器响应中的头字段 `Status` 和 `X-Accel-...` 传递给客户端。`fastcgi_hide_header` 指令设置不会被传递的附加字段。但是, 如果需要允许传递字段, 则可以使用 `fastcgi_pass_header` 指令。

例如, 以下配置将隐藏 FastCGI 响应中的 `X-Powered-By` 和 `Server` 响应头信息:

Plain Text

```
fastcgi_hide_header X-Powered-By;
```

```
fastcgi_hide_header Server;
```

3.9.1.15 `fastcgi_pass_header`

SQL

Syntax: `fastcgi_pass_header field;`

Default: `-`

Context: `http, server, location`

允许从 FastCGI 服务器向客户端传递隐藏禁用的头字段。

3.9.1.16 `fastcgi_ignore_headers`

SQL

Syntax: `fastcgi_ignore_headers field ...;`

Default: `-`

Context: `http, server, location`

禁止处理来自 FastCGI 服务器的某些响应头字段。以下字段将被忽略: `X-`

`Accel-Redirect`、`X-Accel-Expires`、`X-Accel-Limit-Rate`、`X-Accel-Buffering`、`X-Accel-Charset`、`Expires`、`Cache-Control`、`Set-Cookie` 和 `Vary`

如果未禁用，则处理这些头字段产生以下效果：

- `X-Accel-Expires`、`Expires`、`Cache-Control`、`Set-Cookie` 和 `Vary` 设置响应缓存的参数
- `X-Accel-Redirect` 执行内部重定向到指定的 URI
- `X-Accel-Limit-Rate` 设置响应的传送速率限制
- `X-Accel-Buffering` 启用或禁用缓冲响应
- `X-Accel-Charset` 设置所需的响应字符集

3.9.1.17 `fastcgi_connect_timeout`

SQL

Syntax: `fastcgi_connect_timeout time;`

Default: `fastcgi_connect_timeout 60s;`

Context: `http, server, location`

设置与 FastCGI 服务器建立连接的超时时间。需要注意的是，这个超时通常不能超过 75 秒。

3.9.1.18 `fastcgi_ignore_client_abort`

Plaintext

Syntax: `fastcgi_ignore_client_abort on | off;`

Default: `fastcgi_ignore_client_abort off;`

Context: `http, server, location`

确定当客户端关闭连接而不等待响应时是否关闭与 FastCGI 服务器的连接。

3.9.1.19 `fastcgi_read_timeout`

SQL

Syntax: `fastcgi_read_timeout time;`

Default: `fastcgi_read_timeout 60s;`

Context: `http, server, location`

定义从 FastCGI 服务器读取响应的超时时间。超时设置在两次连续的读操作之间，而不是传输整个响应的过程。如果 FastCGI 服务器在此时间内没有发送任何内容，则连接将被关闭。

3.9.1.20 fastcgi_intercept_errors

SQL

Syntax: `fastcgi_intercept_errors on | off;`

Default: `fastcgi_intercept_errors off;`

Context: `http, server, location`

确定当 FastCGI 服务器响应码大于或等于 300 时是否应传递给客户端，或者拦截并重定向到 `nginx` 以便使用 `error_page` 指令进行处理。

3.9.1.21 fastcgi_next_upstream

Go

Syntax: `fastcgi_next_upstream error | timeout |
invalid_header | http_500 | http_503 | http_403 | http_404
| http_429 | non_idempotent | off ...;`

Default: `fastcgi_next_upstream error timeout;`

Context: `http, server, location`

指定在哪些情况下请求应传递给下一台服务器：

- `errorr`
在与服务器建立连接、传递请求或读取响应头时发生错误
- `timeout`
在与服务器建立连接、传递请求或读取响应头时发生超时
- `invalid_header`
服务器返回了空的或无效的响应
- `http_500`
服务器返回 500 响应码
- `http_503`
服务器返回 503 响应码
- `http_403`
服务器返回 403 响应码
- `http_404`
服务器返回 404 响应码
- `http_429`
服务器返回 429 响应码 (1.11.13)

- `non_idempotent`

通常，如果请求已发送到上游服务器，则具有非幂等方法（POST、LOCK、PATCH）的请求不会传递到下一个服务器，使这个选项明确允许重试这样的请求

- `off`

禁用将请求传递给下一个服务器

该指令还定义了与服务器进行通信的不成功尝试。`errorr`、`timeout` 和 `invalid_header` 的情况总是被认为是失败的尝试，即使它们没有在指令中指定。只有在指令中指定了 `http_500`、`http_503` 和 `http_429` 的情况下，它们才被视为不成功尝试。`http_403` 和 `http_404` 的情况永远不会被视为不成功尝试。

将请求传递给下一台服务器可能受到尝试次数和时间的限制。

3.9.1.22 `fastcgi_next_upstream_timeout`

SQL

Syntax: `fastcgi_next_upstream_timeout time;`

Default: `fastcgi_next_upstream_timeout 0;`

Context: http, server, location

限制请求可以传递到下一个服务器的时间。`0` 值关闭此限制。

3.9.1.23 `fastcgi_split_path_info`

SQL

Syntax: `fastcgi_split_path_info regex;`

Default: `-`

Context: location

定义一个捕获 `$fastcgi_path_info` 变量值的正则表达式。正则表达式应该有两个捕获：第一个为 `$fastcgi_script_name` 变量的值，第二个为 `$fastcgi_path_info` 变量的值。例如以下设置

Nginx

```
location ~ ^(.+\.php)(.*)$ {
    fastcgi_split_path_info      ^(.+\.php)(.*)$;
    fastcgi_param SCRIPT_FILENAME
    /path/to/php$fastcgi_script_name;
    fastcgi_param PATH_INFO      $fastcgi_path_info;
```

和 `/show.php/article/0001` 请求, `SCRIPT_FILENAME` 参数等于 `/path/to/php/show.php`, 并且 `PATH_INFO` 参数等于 `/article/0001`。

3.9.1.24 fastcgi_param

SQL

Syntax: `fastcgi_param parameter value [if_not_empty];`

Default: `-`

Context: `http, server, location`

设置应传递给 FastCGI 服务器的 `parameter` (参数)。该值可以包含文本、变量及其组合。当且仅当在当前级别上没有定义 `fastcgi_param` 指令时, 这些指令才从前一级继承。

以下示例展示了 PHP 的最小要求配置:

Nginx

```
fastcgi_param SCRIPT_FILENAME
/home/www/scripts/php$fastcgi_script_name;
fastcgi_param QUERY_STRING    $query_string;
```

`SCRIPT_FILENAME` 参数在 PHP 中用于确定脚本名称, `QUERY_STRING` 参数用于传递请求参数。

对于处理 POST 请求的脚本, 还需要以下三个参数:

Nginx

```
fastcgi_param REQUEST_METHOD $request_method;
fastcgi_param CONTENT_TYPE   $content_type;
fastcgi_param CONTENT_LENGTH $content_length;
```

如果 PHP 使用了 `--enable-force-cgi-redirect` 配置参数构建, 则还应该使用值 `200` 传递 `REDIRECT_STATUS` 参数:

Nginx

```
fastcgi_param REDIRECT_STATUS 200;
```

如果该指令是通过 `if_not_empty` 指定的, 那么只有当它的值不为空时, 这个参数才会被传递给服务器:

Nginx

```
fastcgi_param HTTPS          $https if_not_empty;
```

3.9.1.25 fastcgi_buffering

```
SQL
Syntax: fastcgi_buffering on | off;
Default: fastcgi_buffering on;
Context: http, server, location
```

启用或禁用来自 FastCGI 服务器的响应缓冲。

当启用缓冲时，njet 会尽可能快地收到接收来自 FastCGI 服务器的响应，并将其保存到由 `fastcgi_buffer_size` 和 `fastcgi_buffers` 指令设置的缓冲区中。如果内存放不下整个响应，响应的一部分可以保存到磁盘上的临时文件中。写入临时文件由 `fastcgi_max_temp_file_size` 和 `fastcgi_temp_file_write_size` 指令控制。

当缓冲被禁用时，njet 在收到响应时立即同步传递给客户端，不会尝试从 FastCGI 服务器读取整个响应。njet 一次可以从服务器接收的最大数据量由 `fastcgi_buffer_size` 指令设置。

通过在 `X-Accel-Buffering` 响应头字段中通过 `yes` 或 `no` 也可以启用或禁用缓冲。可以使用 `fastcgi_ignore_headers` 指令禁用此功能

3.9.1.26 fastcgi_buffer_size

```
SQL
Syntax: fastcgi_buffer_size size;
Default: fastcgi_buffer_size 4k|8k;
Context: http, server, location
```

设置读取 FastCGI 服务器收到的响应的第一部分的缓冲区的大小（大小）。该部分通常包含一个小的响应头。默认情况下，缓冲区大小等于一个内存页。为 4K 或 8K，因平台而异。但是，它可以设置得更小。

3.9.1.27 fastcgi_buffers

```
SQL
Syntax: fastcgi_buffers number size;
Default: fastcgi_buffers 8 4k|8k;
Context: http, server, location
```

设置单个连接从 FastCGI 服务器读取响应的缓冲区的 number（数量）和 size（大小）。默认情况下，缓冲区大小等于一个内存页。为 4K 或 8K，因平台而异。

3.9.1.28 fastcgi_max_temp_file_size

Plaintext

Syntax: `fastcgi_max_temp_file_size size;`

Default: `fastcgi_max_temp_file_size 1024m;`

Context: `http, server, location`

当启用缓冲来自 FastCGI 服务器的响应时并且整个响应不适合由 `{fastcgi_buffer_size}({#fastcgi_buffer_size})` 和 `fastcgi_buffers` 指令设置的缓冲时，响应的一部分可以保存到临时文件中。该指令用于设置临时文件的最大大小（`size`）。一次写入临时文件的数据大小由 `fastcgi_temp_file_write_size` 指令设置。

零值将禁用临时文件响应缓冲。

此限制不适用于将要缓存或存储在磁盘上的响应。

3.9.1.29 fastcgi_temp_file_write_size

SQL

Syntax: `fastcgi_temp_file_write_size size;`

Default: `fastcgi_temp_file_write_size 8k|16k;`

Context: `http, server, location`

设置当开启缓冲 FastCGI 服务器响应到临时文件时，限制写入临时文件的数据 `size`（大小）。默认情况下，大小受 `fastcgi_buffer_size` 和 `fastcgi_buffers` 指令设置的两个缓冲区限制。临时文件的最大大小由 `fastcgi_max_temp_file_size` 指令设置。

3.9.1.30 fastcgi_cache_lock

SQL

Syntax: `fastcgi_cache_lock on | off;`

Default: `fastcgi_cache_lock off;`

Context: `http, server, location`

当启用时，同一时间只允许一个请求通过将请求传递给 FastCGI 服务器来填充 `fastcgi_cache_key` 指令标识的新缓存元素。同一缓存元素的其他请求将等待响应出现在缓存中，或等待此元素的缓存锁释放，直到 `fastcgi_cache_lock_timeout` 指令设置的时间。

3.9.1.31 fastcgi_cache_lock_timeout

SQL

Syntax: `fastcgi_cache_lock_timeout time;`

```
Default: fastcgi_cache_lock_timeout 5s;  
Context: http, server, location
```

设置 `fastcgi_cache_lock` 的超时时间。当时间到期时，请求将被传递给 FastCGI 服务器，但是，响应不会被缓存。

3.9.1.32 fastcgi_cache_lock_age

```
SQL  
Syntax: fastcgi_cache_lock_age time;  
Default: fastcgi_cache_lock_age 5s;  
Context: http, server, location
```

如果传递给 FastCGI 服务器的最后一个请求填充新缓存元素没能在指定的 `time` 内完成，则可能会有其他另一个请求被传递给 FastCGI 服务器。

3.9.1.33 fastcgi_limit_rate

```
SQL  
Syntax: fastcgi_limit_rate rate;  
Default: fastcgi_limit_rate 0;  
Context: http, server, location
```

限制读取 FastCGI 服务器响应的速度。`rate` 以每秒字节数为单位。零值则禁用速率限制。该限制是针对每个请求设置的，因此如果 `njet` 同时打开两个连接到 FastCFI 服务器的连接，则整体速率将是指定限制的两倍。该限制仅在启用缓冲来自 FastCGI 服务器的响应时才起作用。

3.9.1.34 fastcgi_store

```
SQL  
Syntax: fastcgi_store on | off | string;  
Default: fastcgi_store off;  
Context: http, server, location
```

启用将文件保存到磁盘。`on` 参数将文件保存为与指令 `alias` 或 `root` 相对应的路径。`off` 参数禁用保存文件。另外，可以使用带变量的字符串显式设置文件名：

```
Nginx  
fastcgi_store /data/www$original_uri;
```

文件的修改时间根据收到的 `Last-Modified` 响应头字段设置。首先将响应写入临时文件，然后重命名该文件。建议，对于任何给定位置，保存的文件和由

`fastcgi_temp_path` 指令设置的保存临时文件的目录都放在同一个文件系统上。避免文件放在不同的文件系统上，跨文件系统进行复制

该指令可用于创建静态不可更改文件的本地副本，例如：

```
Nginx
location /images/ {
    root                /data/www;
    error_page          404 = /fetch$uri;
}

location /fetch/ {
    internal;

    fastcgi_pass        backend:9000;
    ...

    fastcgi_store        on;
    fastcgi_store_access user:rw group:rw all:r;
    fastcgi_temp_path    /data/temp;

    alias                /data/www/;
}
```

3.9.1.35 fastcgi_store_access

```
SQL
Syntax: fastcgi_store_access users:permissions ...;
Default: fastcgi_store_access user:rw;
Context: http, server, location
```

为新创建的文件和目录设置访问权限，例如：

```
Nginx
fastcgi_store_access user:rw group:rw all:r;
```

如果指定了任何组或所有访问权限，则可以省略用户权限

```
Nginx
fastcgi_store_access group:rw all:r;
```

3.9.1.36 fastcgi_temp_path

SQL

Syntax: `fastcgi_temp_path path [level1 [level2 [level3]]];`

Default: `fastcgi_temp_path fastcgi_temp;`

Context: `http, server, location`

定义一个目录，用于存储从 FastCGI 服务器接收到的数据的临时文件。指定目录下最多可有三级子目录。例如以下配置

Nginx

```
fastcgi_temp_path /spool/nginx/fastcgi_temp 1 2;
```

临时文件如下所示：

Nginx

```
/spool/nginx/fastcgi_temp/7/45/00000123457
```

3.10 配额限制

3.10.1 limit_conn

3.10.1.1 功能概述

用于限制客户端连接的数量。该模块可以在不同的级别上实现连接限制，例如：

- 每个客户端 IP 地址的连接数量限制
- 每个请求的并发连接数量限制
- 每个请求的总连接数量限制

3.10.1.2 配置示例

Bash

```
http {
    limit_conn_zone $binary_remote_addr zone=addr:10m;

    ...

    server {

        ...

        location /download/ {
```

```
        limit_conn addr 1;
    }
```

3.10.1.3 指令介绍

3.10.1.3.1 limit_conn

```
SQL
Syntax: limit_conn zone number;
Default:—
Context:http, server, location
```

设置给定键值的共享内存区域和最大允许连接数。当超过此限制时，服务器将返回错误响应请求。例如：

```
Nginx
limit_conn_zone $binary_remote_addr zone=addr:10m;

server {
    location /download/ {
        limit_conn addr 1;
    }
}
```

同一时间只允许一个 IP 地址一个连接。

在 HTTP/2 和 SPDY 中，每个并发请求都被视为一个单独的连接。

可以有多个 `limit_conn` 指令。例如，以下配置将限制每个客户端 IP 连接到服务器的数量，同时限制连接到虚拟服务器的总数：

```
Nginx
limit_conn_zone $binary_remote_addr zone=perip:10m;
limit_conn_zone $server_name zone=perserver:10m;

server {
    ...
    limit_conn perip 10;
    limit_conn perserver 100;
}
```

当且仅当在当前级别上没有 `limit_conn` 指令时，这些指令才从前一级继承。

3.10.1.3.2 limit_conn_dry_run

```
SQL
Syntax:limit_conn_dry_run on | off;
Default:limit_conn_dry_run off;
Context:http, server, location
```

启用干运行模式。在此模式下，连接数没有限制，但在共享内存区域中，超额连接的数量仍会像平常一样进行计算。

3.10.1.3.3 limit_conn_log_level

```
Go
Syntax:limit_conn_log_level info | notice | warn | error;
Default:limit_conn_log_level error;
Context:http, server, location
```

当服务器限制连接数时，设置所需的日志记录级别。

3.10.1.3.4 limit_conn_status

```
SQL
Syntax:limit_conn_status code;
Default:limit_conn_status 503;
Context:http, server, location
```

设置响应拒绝请求返回的状态码。

3.10.1.3.5 limit_conn_zone

```
SQL
Syntax:limit_conn_zone key zone=name:size;
Default:--
Context:http
```

为共享内存区域设置参数，该区域将保留各种键的状态。特别是，该状态包含当前的连接数。**key** 可以包含文本、变量及其组合。不包括有空键值的请求。

用法示例：

```
Nginx
limit_conn_zone $binary_remote_addr zone=addr:10m;
```

在这里，客户端 IP 地址作为 **key**。请注意，不是 **\$remote_addr**，而是使用 **\$binary_remote_addr** 变量。**\$remote_addr** 变量的大小可以为 7 到 15 个字节不等。存储状态在 32 位平台上占用 32 或 64 字节的内存，在 64 位平台

上总是占用 64 字节。对于 IPv4 地址，`$binary_remote_addr` 变量的大小始终为 4 个字节，对于 IPv6 地址则为 16 个字节。存储状态在 32 位平台上始终占用 32 或 64 个字节，在 64 位平台上占用 64 个字节。一兆字节的区域可以保持大约 32000 个 32 字节的状态或大约 16000 个 64 字节的状态。如果区域存储耗尽，服务器会将错误返回给所有其余的请求

3.10.2 limit_req

3.10.2.1 功能概述

用于控制客户端在一定时间内发送请求的速率，以避免服务器过载。

该模块使用令牌桶算法来限制请求速率，它会记录每个客户端的请求时间和数量，如果客户端在规定时间内发送请求的速率超过了限制，那么该模块将拒绝客户端的请求

3.10.2.2 配置示例

```
SQL
http {
    limit_req_zone $binary_remote_addr zone=one:10m
    rate=1r/s;

    ...

    server {

        ...

        location /search/ {
            limit_req zone=one burst=5;
        }
    }
}
```

3.10.2.3 指令介绍

3.10.2.3.1 limit_req

```
TypeScript
Syntax: limit_req zone=name [burst=number] [nodelay |
delay=number];
Default: -
```

Context:http, server, location

设置共享内存区域和请求的最大突发大小。如果请求速率超过为某个区域配置的速率，则它们的处理会延迟，从而使请求以定义的速率处理。过多的请求被延迟，直到它们的数量超过最大突发大小，在这种情况下请求被终止并出现[错误](#)。默认情况下，最大突发大小等于零。例如：

```
Nginx
limit_req_zone $binary_remote_addr zone=one:10m rate=1r/s;

server {
    location /search/ {
        limit_req zone=one burst=5;
    }
}
```

平均每秒不超过 1 个请求，并且突发不超过 5 个请求。

如果在限制期间延迟请求过多，则不需要使用参数 `nodelay`：

```
Nginx
limit_req zone=one burst=5 nodelay;
```

可以存在多个 `limit_req` 指令。例如，以下配置将限制来自单个 IP 地址请求的处理速率，同时限制虚拟服务器的请求处理速率：

```
Nginx
limit_req_zone $binary_remote_addr zone=perip:10m
rate=1r/s;
limit_req_zone $server_name zone=perserver:10m rate=10r/s;

server {
    ...
    limit_req zone=perip burst=5 nodelay;
    limit_req zone=perserver burst=10;
}
```

当且仅当在当前级别上没有 `limit_req` 指令时，这些指令才从上一级继承。

3.10.2.3.2 limit_req_dry_run

```
SQL
Syntax:limit_req_dry_run on | off;
Default:limit_req_dry_run off;
Context:http, server, location
```

启用干运行模式。在此模式下，请求处理速率不受限制，但在共享内存区域中，多余请求的数量仍会像平常一样计算。

3.10.2.3.3 limit_req_log_level

```
Go
Syntax:limit_req_log_level info | notice | warn | error;
Default:limit_req_log_level error;
Context:http, server, location
```

当服务器由于速率超出而拒绝处理请求或延迟请求处理时，设置所需的日志记录级别。延误情况的记录等级比拒绝情况的记录低一些。例如，如果指定了 `limit_req_log_level notice`，则延迟情况将会在 `info` 级别记录。

3.10.2.3.4 limit_req_status

```
SQL
Syntax:limit_req_status code;
Default:limit_req_status 503;
Context:http, server, location
```

设置响应拒绝请求返回的状态码。

3.10.2.3.5 limit_req_zone

```
SQL
Syntax:limit_req_zone key zone=name:size rate=rate [sync];
Default:–
Context:http
```

为共享内存区域设置参数，该区域将保留各种键的状态。特别是，该状态包含当前的连接数。`key` 可以包含文本、变量及其组合。不包括有空键值的请求。

用法示例：

```
Nginx
limit_req_zone $binary_remote_addr zone=one:10m rate=1r/s;
```

在这里，状态保持在 10 兆字节的区域 `one`，并且该区域的平均请求处理速率不能超过每秒 1 个请求。

客户端 IP 地址作为 key。请注意，不是 `$remote_addr`，而是使用 `$binary_remote_addr` 变量。`$binary_remote_addr` 变量的大小始终为 4 个字节，对于 IPv6 地址则为 16 个字节。存储状态在 32 位平台上始终占用 32 或 64 个字节，在 64 位平台上占用 64 个字节。一兆字节的区域可以保持大约

32000 个 32 字节的状态或大约 16000 个 64 字节的状态或大约 8000 个 128 字节的状态。

如果区域存储耗尽，最近最少使用的状态将被删除。即使在此之后无法创建新状态，该请求也会因错误而终止。

速率以每秒请求数（r/s）指定。如果需要每秒小于一个请求的速率，则按每分钟请求（r/m）指定。例如，每秒半请求是 30r/m。

3.10.3 limit_rate

3.10.3.1 功能概述

用于限制对客户端的响应传输速率

3.10.3.2 配置示例

JSON

```
http{
    upstream backed1{
        zone upstream_http 64k;
        server example.com;
    }

    server {
        listen 80;
        server_name location;

        location / {
            proxy_pass http://backed1;
            limit_rate 1k;
            limit_rate_after 5k;
        }
    }
}
```

3.10.3.3 指令介绍

3.10.3.3.1 limit_rate

SQL

Syntax: limit_rate rate;
Default: limit_rate 0;

```
Context:http, server, location, if in location
```

限制向客户端传输响应的速率。速率以每秒字节数为单位指定。零值禁用速率限制。限制是针对请求设置的，因此如果客户端同时打开两个连接，则总速率将是指定限制的两倍。

参数值可以包含变量。在速率应根据某些条件限制的情况下，这可能非常有用：

```
Bash
map $slow $rate {
    1 4k;
    2 8k;
}
limit_rate $rate;
```

速率限制也可以在代理服务器响应的“X-Accel-Limit-Rate”标头字段中设置。可以使用 `proxy_ignore_headers`, `fastcgi_ignore_headers`, `uwsgi_ignore_headers` 和 `scgi_ignore_headers` 指令禁用此功能。

3.10.3.3.2 limit_rate_after

```
SQL
Syntax:limit_rate_after size;
Default:limit_rate_after 0;
Context:http, server, location, if in location
```

设置初始速率限制的字节数。一旦客户端接收的响应超过此字节数，进一步的传输速率将被限制。参数值可以包含变量。

例如：

```
Bash
location /flv/ {
    flv;
    limit_rate_after 500k;
    limit_rate 50k;
}
```

3.10.4 熔断功能

3.10.4.1 功能概述

熔断机制是一种中间层，用于检测和处理不稳定的服务。当一个服务出现故障时，熔断机制会立即停止向该服务发送请求，并返回一个错误信息，从而避免系

统崩溃。如果服务在一段时间内仍然无法恢复正常运行，熔断机制将完全停止对该服务的请求，直到服务恢复正常。

3.10.4.2 配置示例

```
SQL
upstream backend1{
    server backend1.example.com    max_fails=10
    fail_timeout=7s;;
    server backend2.example.com    max_fails=10
    fail_timeout=7s;;
    server backend3.example.com    max_fails=10
    fail_timeout=7s;;
    server backend4.example.com    max_fails=10
    fail_timeout=7s;;
}
```

3.10.4.3 指令介绍

3.10.4.3.1 max_fails

`max_fails=number`

该指令设置了与服务器通信的失败次数，如果在 `fail_timeout` 参数设置的时间范围内失败次数达到该值，则认为该服务器在 `fail_timeout` 参数设置的时间范围内不可用。默认情况下，失败次数为 1 次。如果将该值设置为 0，则不会统计失败次数。什么被视为失败的尝试是由 `proxy_next_upstream`、`fastcgi_next_upstream`、`uwsgi_next_upstream`、`scgi_next_upstream`、`memcached_next_upstream` 和 `grpc_next_upstream` 指令定义的。

3.10.4.3.2 fail_timeout

`fail_timeout=time`

这个指令用来设置服务器在一段时间内的不可用状态，需要满足两个条件：

- a. 在 `fail_timeout` 参数所设置的时间内，发生指定次数的通信失败尝试，才会被认为不可用；
- b. 在 `fail_timeout` 参数所设置的时间内，服务器会被认为不可用。

该指令默认情况下的时间为 10 秒。

3.11 telemetry 功能

3.11.1 介绍

3.11.1.1 什么是 OpenTelemetry?

OpenTelemetry 合并了 OpenTracing 和 OpenCensus 项目，提供了一组 API 和库来标准化遥测数据的采集和传输。OpenTelemetry 提供了一个安全，厂商中立的工具，这样就可以按照需要将数据发往不同的后端。

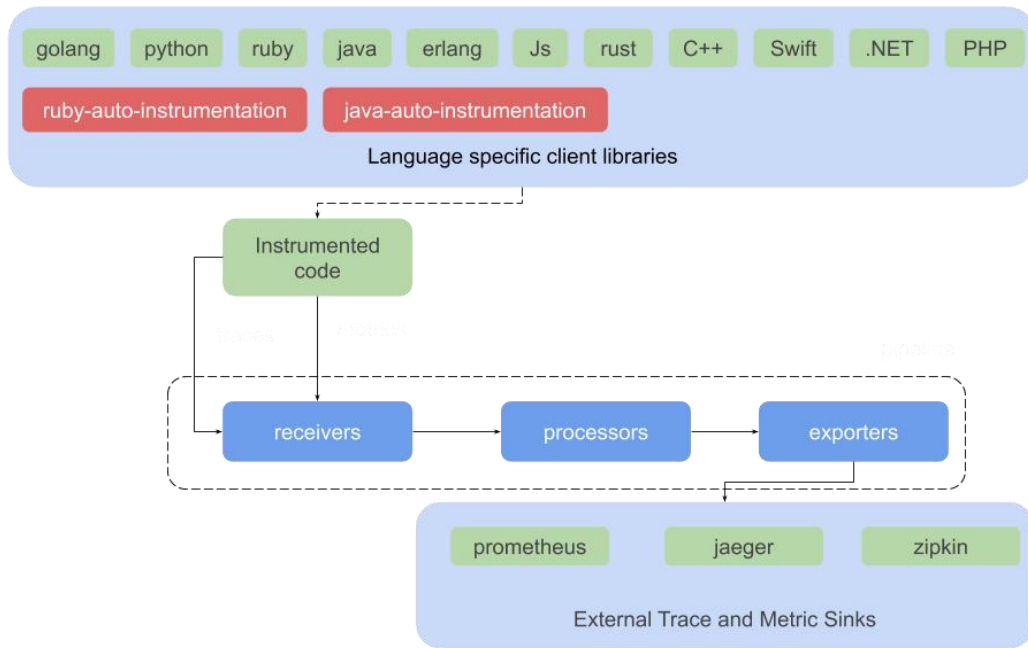
OpenTelemetry 项目由如下组件构成：

- 推动在所有项目中使用一致的规范
- 基于规范的，包含接口和实现的 APIs
- 不同语言的 SDK(APIs 的实现)，如 Java, Python, Go, Erlang 等
- Exporters：可以将数据发往一个选择的后端
- Collectors：厂商中立的实现，用于处理和导出遥测数据

3.11.1.2 术语

- Traces：记录经过分布式系统的请求活动，一个 trace 是 spans 的有向无环图
- Spans：一个 trace 中表示一个命名的，基于时间的操作。Spans 嵌套形成 trace 树。每个 trace 包含一个根 span，描述了端到端的延迟，其子操作也可能拥有一个或多个子 spans。
- Metrics：在运行时捕获的关于服务的原始度量数据。Opentelemetry 定义的 metric instruments(指标工具)如下。Observer 支持通过异步 API 来采集数据，每个采集间隔采集一个数据。
- Context：一个 span 包含一个 span context，它是一个全局唯一的标识，表示每个 span 所属的唯一的请求，以及跨服务边界转移 trace 信息所需的数据。
OpenTelemetry 也支持 correlation context，它可以包含用户定义的属性。correlation context 不是必要的，组件可以选择不携带和存储该信息。
- Context propagation：表示在不同的服务之间传递上下文信息，通常通过 HTTP 首部。Context propagation 是 OpenTelemetry 系统的关键功能之一。除了 tracing 之外，还有一些有趣的用法，如，执行 A/B 测试。OpenTelemetry 支持通过多个协议的 Context propagation 来避免可能发生的问题，但需要注意的是，在自己的应用中最好使用单一的方法。

3.11.1.3 OpenTelemetry 架构



opentelemetry 也是个插件式的架构，针对不同的开发语言会有相应的 Client 组件，叫 **Instrumentation**，也就是在代码中埋点调用的 api/sdk 采集 telemetry 数据

NJet 主要集成了分布式服务 telemetry 模块：

分布式服务追踪模块： `njt_otel_module.so`

3.11.2 分布式服务追踪模块: `njt_otel_module.so`

3.11.2.1 指令介绍

3.11.2.1.1 `opentelemetry`

启用或禁用 OpenTelemetry (默认为启用)。

- **required:** `false`
- **syntax:** `opentelemetry on|off`
- **block:** `http, server, location`

3.11.2.1.2 `opentelemetry_trust_incoming_spans`

启用或禁用使用传入请求的 spans 作为创建请求的父级。(默认值：启用)。

- **required:** `false`

- **syntax:** `opentelemetry_trust_incoming_spans on|off`
- **block:** `http, server, location`

3.11.2.1.3 `opentelemetry_attribute`

向 span 添加自定义属性，可以访问 nginx 变量，如： `opentelemetry_attribute "my.user.agent" "$http_user_agent"`.

- **required:** `false`
- **syntax:** `opentelemetry_attribute <key> <value>`
- **block:** `http, server, location`

3.11.2.1.4 `opentelemetry_config`

Exporters, processors

- **required:** `true`
- **syntax:** `opentelemetry_config /path/to/config.toml`
- **block:** `http`

3.11.2.1.5 `opentelemetry_operation_name`

在启动新的 span 时设置操作名称。

- **required:** `false`
- **syntax:** `opentelemetry_operation_name <name>`
- **block:** `http, server, location`

3.11.2.1.6 `opentelemetry_propagate`

启用分布式跟踪头的传播, e.g. `traceparent`。

当没有给出父跟踪时，将启动新的跟踪。默认的传播器是 W3C。

应用的继承规则与 `proxy_set_header` 相同，这意味着当且仅当在较低的配置级别上没有定义 `proxy_set_header` 指令时，才在当前配置级别应用该指令。

- **required:** `false`
- **syntax:** `opentelemetry_propagate` or `opentelemetry_propagate b3`
- **block:** `http, server, location`

3.11.2.1.7 opentelemetry_capture_headers

允许捕获请求和响应头(默认值: 禁用)。

- **required:** false
- **syntax:** opentelemetry_capture_headers on|off
- **block:** http, server, location

3.11.2.1.8 opentelemetry_sensitive_header_names

对于名称匹配给定正则表达式(不区分大小写)的所有头, 将捕获的头值设置为 [REDACTED]。

- **required:** false
- **syntax:** opentelemetry_sensitive_header_names <regex>
- **block:** http, server, location

3.11.2.1.9 opentelemetry_sensitive_header_values

将捕获的头值设置为 [REDACTED], 用于所有与给定正则表达式匹配的头(不区分大小写)。

- **required:** false
- **syntax:** opentelemetry_sensitive_header_values <regex>
- **block:** http, server, location

3.11.2.1.10 opentelemetry_ignore_paths

不会为匹配给定正则表达式的 URI 创建 span (不区分大小写)。

- **required:** false
- **syntax:** opentelemetry_ignore_paths <regex>
- **block:** http, server, location

3.11.2.2 Njet.conf 配置

```
Bash
```

```
#user root;
worker_processes 1;
daemon off;
#master_process on;
error_log /home/njet/clbtest/logs/error.log debug;
pid /home/njet/clbtest/njet.pid;

#load_module /root/project/telemetry/opentelemetry-cpp-contrib-
main/instrumentation/nginx/build/otel_ngx_module.so;
load_module /home/njet/modules/njt_otel_module.so;

events {
    worker_connections 1024;
}

http {
    opentelemetry_config /home/njet/clbtest/conf/otel-njet.toml;
    opentelemetry off;

    upstream http1{
        server 192.168.40.139:9002;
        #server 192.168.40.136:8082;
    }

    server {
        listen 8081;
        server_name otel_example;

        location = / {
            opentelemetry on;
            opentelemetry_operation_name my_example_backend;
            opentelemetry_propagate;
            proxy_pass http://http1;
        }

        location = /b3 {
            opentelemetry on;
            opentelemetry_operation_name my_other_backend;
            opentelemetry_propagate b3;
            # Adds a custom attribute to the span
            opentelemetry_attribute "req.time" "$msec";
            proxy_pass http://http1;
```

```
}  
}  
}
```

3.11.2.3 Otel module 配置

```
Bash  
exporter = "otlp"  
processor = "batch"  
  
[exporters.otlp]  
#collector server address  
# Alternatively the OTEL_EXPORTER_OTLP_ENDPOINT environment  
variable can also be used.  
host = "192.168.40.136"  
port = 4317  
# Optional: enable SSL, for endpoints that support it  
# use_ssl = true  
# Optional: set a filesystem path to a pem file to be used for SSL  
encryption  
# (when use_ssl = true)  
# ssl_cert_path = "/path/to/cert.pem"  
  
[processors.batch]  
max_queue_size = 2048  
schedule_delay_millis = 5000  
max_export_batch_size = 512  
  
[service]  
# Can also be set by the OTEL_SERVICE_NAME environment variable.  
name = "njet-proxy" # Opentelemetry resource name  
  
[sampler]  
name = "AlwaysOn" # Also: AlwaysOff, TraceIdRatioBased  
ratio = 0.1  
parent_based = false
```

3.11.3 启动 collector 以及 jaeger 服务

通过 docker-compose 启动 collector 以及 jaeger 服务

docker-compose up -f docker-compose.yaml -d

docker-compose.yaml

Bash

version: "2"

services:

Jaeger

jaeger-all-in-one:

image: jaegertracing/all-in-one:latest

restart: always

ports:

- "16686:16686"

- "14268"

- "14250"

Zipkin

zipkin-all-in-one:

image: openzipkin/zipkin:latest

restart: always

ports:

- "9411:9411"

Collector

otel-collector:

image: \${OTELCOL_IMG}

restart: always

command: ["--config=/etc/otel-collector-config.yaml",
"\${OTELCOL_ARGS}"]

volumes:

- ./otel-collector-config.yaml:/etc/otel-collector-
config.yaml

ports:

- "1888:1888" # pprof extension

- "8888:8888" # Prometheus metrics exposed by the collector

- "8889:8889" # Prometheus exporter metrics

- "13133:13133" # health_check extension

- "4317:4317" # OTLP gRPC receiver

- "4318:4318" # OTLP http receiver

- "55679:55679" # zpages extension

depends_on:

- jaeger-all-in-one

- zipkin-all-in-one

```
#demo-client:
#  build:
#    dockerfile: Dockerfile
#    context: ./client
#  restart: always
#  environment:
#    - OTEL_EXPORTER_OTLP_ENDPOINT=otel-collector:4317
#    - DEMO_SERVER_ENDPOINT=http://demo-server:7080/hello
#  depends_on:
#    - demo-server

#demo-server:
#  build:
#    dockerfile: Dockerfile
#    context: ./server
#  restart: always
#  environment:
#    - OTEL_EXPORTER_OTLP_ENDPOINT=otel-collector:4317
#  ports:
#    - "7080"
#  depends_on:
#    - otel-collector

prometheus:
  container_name: prometheus
  image: prom/prometheus:latest
  restart: always
  volumes:
    - ./prometheus.yaml:/etc/prometheus/prometheus.yml
  ports:
    - "9090:9090"
```

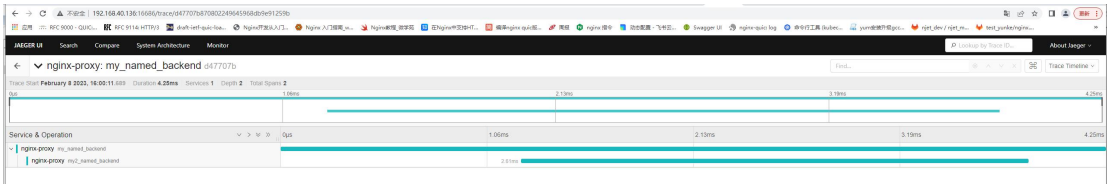
3.11.4 效果

3.11.4.1 分布式服务追踪模块: njt_otel_module.so

Jaeger ui: <http://{jaeger server ip}:16686>

通过 jaeger ui 查看 trace

<http://192.168.40.136:16686>



3.8 Upstream api

3.8.1 简介

该模块使用 restful API，实现动态添加、删除、修改 http 模块和 stream 模块的 upstream server 的功能。

3.8.2 配置说明

请参考标准配置文件章节配置，全量配置即可配置上此功能，但请注意一定要包含以下指令和块：

要开启该功能，需在 `ctrl.conf` 配置文件的 `main` 块中加载该模块：

```
Plaintext
load_module /home/njet/modules/njt_http_upstream_api_module.so;
```

并在 `http` 块下，指定该模块的配置文件：

```
Bash
http {
    access_log /home/njet/logs/access_ctrl.log combined;
    server {
        listen      8081 ;
        location / api {
            api write=on;
        }
    }
}
```

配置项	必须修改	配置说明
api	是	在该location 下，接收upstream api 请求
write	否	off 只支持查询get 方法（只能查询）。 on 支持 get, patch, post, delete 方法（支持修改）。

3.8.3 参考说明

参考 swagger-ui:

使用时亦可参考 swagger 的使用说明，所有 upstream_api 的接口和使用方法皆有介绍。（下方 swagger 地址的 server 即为所部属 njet 机器的 ip）

'server':8081/doc/swagger/

HTTP Upstreams			^
GET	/http/upstreams/	Return status of all HTTP upstream server groups	▼
GET	/http/upstreams/{httpUpstreamName}/	Return status of an HTTP upstream server group	▼
DELETE	/http/upstreams/{httpUpstreamName}/	Reset statistics of an HTTP upstream server group	▼
GET	/http/upstreams/{httpUpstreamName}/servers/	Return configuration of all servers in an HTTP upstream server group	▼
POST	/http/upstreams/{httpUpstreamName}/servers/	Add a server to an HTTP upstream server group	▼
GET	/http/upstreams/{httpUpstreamName}/servers/{httpUpstreamServerId}	Return configuration of a server in an HTTP upstream server group	▼
PATCH	/http/upstreams/{httpUpstreamName}/servers/{httpUpstreamServerId}	Modify a server in an HTTP upstream server group	▼
DELETE	/http/upstreams/{httpUpstreamName}/servers/{httpUpstreamServerId}	Remove a server from an HTTP upstream server group	▼

Stream Upstreams			^
GET	/stream/upstreams/	Return status of all stream upstream server groups	▼
GET	/stream/upstreams/{streamUpstreamName}/	Return status of a stream upstream server group	▼
DELETE	/stream/upstreams/{streamUpstreamName}/	Reset statistics of a stream upstream server group	▼
GET	/stream/upstreams/{streamUpstreamName}/servers/	Return configuration of all servers in a stream upstream server group	▼
POST	/stream/upstreams/{streamUpstreamName}/servers/	Add a server to a stream upstream server group	▼
GET	/stream/upstreams/{streamUpstreamName}/servers/{streamUpstreamServerId}	Return configuration of a server in a stream upstream server group	▼
PATCH	/stream/upstreams/{streamUpstreamName}/servers/{streamUpstreamServerId}	Modify a server in a stream upstream server group	▼
DELETE	/stream/upstreams/{streamUpstreamName}/servers/{streamUpstreamServerId}	Remove a server from a stream upstream server group	▼

3.8.4 Api 实例

3.8.4.1 http 模块

http 模块可以使用 upstream_api 功能进行增删改查的操作，以下将介绍具体使用方法和效果展示。

3.8.4.1.1 添加

添加功能支持 ip 和域名的添加：

ip 添加

```
Bash
curl -X POST -s
http://127.0.0.1:8081/api/7/http/upstreams/backend1/servers/ -d
```

```
'{"server":"192.168.40.101:8080","weight":2,"max_conns":2,"max_fails":1,"fail_timeout":"5s","slow_start":"5s","backup":true,"down":false}'|jq .
{
  "id": 0,
  "server": "192.168.40.101:8080",
  "weight": 2,
  "max_conns": 2,
  "max_fails": 1,
  "fail_timeout": "5s",
  "slow_start": "5s",
  "route": "",
  "backup": true,
  "down": false
}
```

域名添加

```
Bash
curl -X POST -s
http://127.0.0.1:8081/api/7/http/upstreams/beckend2/servers/ -d
'{"server":"www.dev.test.com:8080","weight":1,"max_conns":1,"max_fails":1,"fail_timeout":"1s","slow_start":"1s","backup":false,"down":true}'|jq .
{
  "id": 0,
  "server": "www.dev.test.com:8080",
  "weight": 1,
  "max_conns": 1,
  "max_fails": 1,
  "fail_timeout": "1s",
  "slow_start": "1s",
  "route": "",
  "backup": false,
  "down": true
}
```

3.8.4.1.2 查询

查询功能支持多层级查询，首先是 upstreams 级，能够查找出所有的 upstream 信息，且无法查询出域名 server 的父级节点

```
Bash
curl -s http://127.0.0.1:8081/api/7/http/upstreams/ |jq .
```

```
{
  "backend1": {
    "peers": [
      {
        "id": 0,
        "server": "192.168.40.101:8080",
        "name": "192.168.40.101:8080",
        "backup": true,
        "weight": 2,
        "state": "up",
        "active": 0,
        "max_conns": 2,
        "requests": 0,
        "responses": {
          "1xx": 0,
          "2xx": 0,
          "3xx": 0,
          "4xx": 0,
          "5xx": 0,
          "codes": {},
          "total": 0
        },
        "sent": 0,
        "received": 0,
        "fails": 0,
        "unavail": 0,
        "health_checks": {
          "checks": 0,
          "fails": 0,
          "unhealthy": 0
        },
        "downtime": 0
      }
    ],
    "keepalive": 0,
    "zombies": 0,
    "zone": "cluster1"
  },
  "backend2": {
    "peers": [
      {
        "id": 1,
        "server": "192.168.40.101:8080",
        "name": "www.dev.test.com:8080",
```

```

    "backup": false,
    "weight": 1,
    "state": "down",
    "active": 0,
    "max_conns": 1,
    "requests": 0,
    "responses": {
      "1xx": 0,
      "2xx": 0,
      "3xx": 0,
      "4xx": 0,
      "5xx": 0,
      "codes": {},
      "total": 0
    },
    "sent": 0,
    "received": 0,
    "fails": 0,
    "unavail": 0,
    "health_checks": {
      "checks": 0,
      "fails": 0,
      "unhealthy": 0
    },
    "downtime": 0
  }
],
"keepalive": 0,
"zombies": 0,
"zone": "cluster2"
}
}

```

upstream 级，可以指定查看单个 upstream 的信息

```

Bash
curl -s http://127.0.0.1:8081/api/7/http/upstreams/backend1 |jq .
{
  "peers": [
    {
      "id": 0,
      "server": "192.168.40.101:8080",
      "name": "192.168.40.101:8080",
      "backup": true,

```

```

    "weight": 2,
    "state": "up",
    "active": 0,
    "max_conns": 2,
    "requests": 0,
    "responses": {
      "1xx": 0,
      "2xx": 0,
      "3xx": 0,
      "4xx": 0,
      "5xx": 0,
      "codes": {},
      "total": 0
    },
    "sent": 0,
    "received": 0,
    "fails": 0,
    "unavail": 0,
    "health_checks": {
      "checks": 0,
      "fails": 0,
      "unhealthy": 0
    },
    "downtime": 0
  }
],
"keepalive": 0,
"zombies": 0,
"zone": "cluster1"
}

```

servers 级，可以查看单个 upstream 的所有 server 级节点

```

Bash
curl -s
http://127.0.0.1:8081/api/7/http/upstreams/backend2/servers |jq .
[
  {
    "id": 1,
    "server": "192.168.40.101:8080",
    "weight": 1,
    "max_conns": 1,
    "max_fails": 1,
    "fail_timeout": "1s",

```

```

    "slow_start": "1s",
    "route": "",
    "backup": false,
    "down": true,
    "parent": 0,
    "host": "www.dev.test.com:8080"
  },
  {
    "id": 0,
    "server": "www.dev.test.com:8080",
    "weight": 1,
    "max_conns": 1,
    "max_fails": 1,
    "fail_timeout": "1s",
    "slow_start": "1s",
    "route": "",
    "backup": false,
    "down": true
  }
]

```

id 级，可以查看单个 upstream 的单个 id 级节点

```

Bash
curl -s
http://127.0.0.1:8081/api/7/http/upstreams/backend2/servers/1
|jq .
{
  "id": 1,
  "server": "192.168.40.101:8080",
  "weight": 1,
  "max_conns": 1,
  "max_fails": 1,
  "fail_timeout": "1s",
  "slow_start": "1s",
  "route": "",
  "backup": false,
  "down": true,
  "parent": 0,
  "host": "www.dev.test.com:8080"
}

```

3.8.4.1.3 修改

修改功能可以对 server 节点参数进行修改，其中 ip 格式 server 节点可以修改

server 参数，域名 server 及其子节点 server 无法修改，且 backup 参数无法修改
修改 ip 格式 server

```
Bash
curl -X PATCH -s
http://127.0.0.1:8081/api/7/http/upstreams/backend1/servers/0 -d
'{"server":"192.168.40.114:8080","weight":1,"max_conns":1,"max_fai
ls":2,"fail_timeout":"1s","slow_start":"1s","backup":false,"down
":true}'|jq .
{
  "id": 0,
  "server": "192.168.40.114:8080",
  "weight": 1,
  "max_conns": 1,
  "max_fails": 2,
  "fail_timeout": "1s",
  "slow_start": "1s",
  "route": "",
  "backup": true,
  "down": true
}
```

修改域名格式 server，父节点 server 参数修改

```
Bash
curl -X PATCH -s
http://127.0.0.1:8081/api/7/http/upstreams/backend2/servers/0 -d
'{"server":"www.dev.test2.com:8080","weight":2,"max_conns":2,"max_
fails":2,"fail_timeout":"5s","slow_start":"5s","backup":true,"do
wn":false}'|jq .
{
  "error": {
    "status": 400,
    "text": "server address is immutable",
    "code": "UpstramServerImmutable"
  },
  "request_id": "730e18137d711d23bb93b1c867c21b89",
  "href": "https://njet.org/en/docs/http/njt_http_api_module.html"
}
```

修改域名格式 server，子节点 server 参数修改

```
Bash
curl -X PATCH -s
```

```

http://127.0.0.1:8081/api/7/http/upstreams/backend2/servers/1 -d
'{"server":"192.168.40.114:8080","weight":2,"max_conns":2,"max_fai
ls":2,"fail_timeout":"5s","slow_start":"5s","backup":true,"down"
:false}'|jq .
{
  "error": {
    "status": 400,
    "text": "server address is immutable",
    "code": "UpstramServerImmutable"
  },
  "request_id": "bcda20dbc9276a1e46229e620e5c5813",
  "href": "https://njet.org/en/docs/http/njt_http_api_module.html"
}

```

修改域名格式 server, 只修改父节点基本参数

```

Bash
curl -X PATCH -s
http://127.0.0.1:8081/api/7/http/upstreams/backend2/servers/0 -d
'{"weight":2,"max_conns":2,"max_fails":2,"fail_timeout":"5s","sl
ow_start":"5s","backup":true,"down":false}'|jq .
{
  "id": 0,
  "server": "www.dev.test.com:8080",
  "weight": 2,
  "max_conns": 2,
  "max_fails": 2,
  "fail_timeout": "5s",
  "slow_start": "5s",
  "route": "",
  "backup": false,
  "down": false
}

```

修改域名格式 server, 只修改子节点基本参数

```

Bash
curl -X PATCH -s
http://127.0.0.1:8081/api/7/http/upstreams/backend2/servers/1 -d
'{"weight":1,"max_conns":3,"max_fails":3,"fail_timeout":"5s","sl
ow_start":"5s","backup":true,"down":false}'|jq .
{
  "id": 1,
  "server": "192.168.40.101:8080",

```

```
"weight": 1,
"max_conns": 3,
"max_fails": 3,
"fail_timeout": "5s",
"slow_start": "5s",
"route": "",
"backup": false,
"down": false,
"parent": 0,
"host": "www.dev.test.com:8080"
}
```

3.8.4.1.4 删除

url -X DELETE http://127.0.0.1:8081/api/7/http/upstreams/http1/servers/0 |jq .

删除功能可以针对单个 server 进行删除，且删除域名格式 server 父节点时会兼带子节点全部删除，但是删除域名格式 server 子节点时将无法删除

删除 ip 格式 server

```
Bash
curl -X DELETE -s
http://127.0.0.1:8081/api/7/http/upstreams/backend1/servers/0
|jq .
[]
```

删除域名格式 server 子节点

```
Bash
curl -X DELETE -s
http://127.0.0.1:8081/api/7/http/upstreams/backend2/servers/1
|jq .
{
  "error": {
    "status": 400,
    "text": "server not removeable",
    "code": "UpstreamServerImmutable"
  },
  "request_id": "5399a28db3d4598b821c1c65cb7efa86",
  "href": "https://njet.org/en/docs/http/njt_http_api_module.html"
}
```

删除域名格式 server 父节点

```
Bash
curl -X DELETE -s
http://127.0.0.1:8081/api/7/http/upstreams/backend2/servers/0
|jq .
[]
```

3.8.4.1.5 重置

针对 upstream，清空健康检查等数据

使用查询命令查看 upstream 的数据，可以观察到 requests 参数有一些数据信息

```
Bash
curl -s http://127.0.0.1:8081/api/7/http/upstreams/backend1 |jq .
{
  "peers": [
    {
      "id": 0,
      "server": "192.168.40.101:8080",
      "name": "192.168.40.101:8080",
      "backup": true,
      "weight": 2,
      "state": "up",
      "active": 0,
      "max_conns": 2,
      "requests": 2,
      "responses": {
        "1xx": 0,
        "2xx": 0,
        "3xx": 0,
        "4xx": 0,
        "5xx": 0,
        "codes": {},
        "total": 0
      },
      "sent": 0,
      "received": 0,
      "fails": 0,
      "unavail": 0,
      "health_checks": {
        "checks": 0,
        "fails": 0,
        "unhealthy": 0
      },
    }
  ]
}
```

```
    "downtime": 0
  }
],
"keepalive": 0,
"zombies": 0,
"zone": "cluster1"
}
```

使用清除数据命令进行数据清空

Bash

```
curl -X DELETE -s
http://127.0.0.1:8081/api/7/http/upstreams/backend1 |jq .
```

再次查询 upstream 的数据，可以看到已经被清空

Bash

```
curl -s http://127.0.0.1:8081/api/7/http/upstreams/backend1 |jq .
{
  "peers": [
    {
      "id": 0,
      "server": "192.168.40.101:8080",
      "name": "192.168.40.101:8080",
      "backup": true,
      "weight": 2,
      "state": "up",
      "active": 0,
      "max_conns": 2,
      "requests": 0,
      "responses": {
        "1xx": 0,
        "2xx": 0,
        "3xx": 0,
        "4xx": 0,
        "5xx": 0,
        "codes": {},
        "total": 0
      },
      "sent": 0,
      "received": 0,
      "fails": 0,
      "unavail": 0,
      "health_checks": {
```

```
        "checks": 0,  
        "fails": 0,  
        "unhealthy": 0  
    },  
    "downtime": 0  
}  
],  
"keepalive": 0,  
"zombies": 0,  
"zone": "cluster1"  
}
```

3.8.4.2 stream 模块

stream 模块可以使用 upstream_api 功能进行增删改查的操作，以下将介绍具体使用方法和效果展示。

3.8.4.2.1 添加

添加功能支持 ip 和域名的添加：

ip 添加

```
Bash  
curl -X POST -s  
http://127.0.0.1:8081/api/7/stream/upstreams/backend3/servers/ -d  
'{"server": "192.168.40.101:8080", "weight": 2, "max_conns": 2, "max_fai  
ls": "1", "fail_timeout": "5s", "slow_start": "5s", "backup": true, "down"  
: false}' | jq .  
{  
  "id": 0,  
  "server": "192.168.40.101:8080",  
  "weight": 2,  
  "max_conns": 2,  
  "max_fails": 1,  
  "fail_timeout": "5s",  
  "slow_start": "5s",  
  "route": "",  
  "backup": true,  
  "down": false  
}
```

域名添加

Bash

```
curl -X POST -s
http://127.0.0.1:8081/api/7/stream/upstreams/backend4/servers/ -d
'{"server":"www.dev.test.com:8080","weight":1,"max_conns":1,"max_f
ails":1,"fail_timeout":"1s","slow_start":"1s","backup":false,"do
wn":true}'|jq .
{
  "id": 0,
  "server": "www.dev.test.com:8080",
  "weight": 1,
  "max_conns": 1,
  "max_fails": 1,
  "fail_timeout": "1s",
  "slow_start": "1s",
  "route": "",
  "backup": false,
  "down": true
}
```

3.8.4.2.2 查询

查询功能支持多层级查询，首先是 upstreams 级，能够查找出所有的 upstream 信息，且无法查询出域名 server 的父级节点

Bash

```
curl -s http://127.0.0.1:8081/api/7/stream/upstreams/ |jq .
{
  "backend3": {
    "peers": [
      {
        "id": 0,
        "server": "192.168.40.101:8080",
        "name": "192.168.40.101:8080",
        "backup": true,
        "weight": 2,
        "state": "up",
        "active": 0,
        "max_conns": 2,
        "connecions": 0,
        "sent": 0,
        "received": 0,
        "fails": 0,
        "unavail": 0,
        "health_checks": {
```

```

        "checks": 0,
        "fails": 0,
        "unhealthy": 0
    },
    "downtime": 0
}
],
"zombies": 0,
"zone": "cluster3"
},
"backend4": {
    "peers": [
        {
            "id": 1,
            "server": "192.168.40.101:8080",
            "name": "www.dev.test.com:8080",
            "backup": false,
            "weight": 1,
            "state": "down",
            "active": 0,
            "max_conns": 1,
            "connecions": 0,
            "sent": 0,
            "received": 0,
            "fails": 0,
            "unavail": 0,
            "health_checks": {
                "checks": 0,
                "fails": 0,
                "unhealthy": 0
            },
            "downtime": 0
        }
    ],
    "zombies": 0,
    "zone": "cluster4"
}
}

```

upstream 级，可以指定查看单个 upstream 的信息

```

Bash
curl -s http://127.0.0.1:8081/api/7/stream/upstreams/backend3
|jq .

```

```
{
  "peers": [
    {
      "id": 0,
      "server": "192.168.40.101:8080",
      "name": "192.168.40.101:8080",
      "backup": true,
      "weight": 2,
      "state": "up",
      "active": 0,
      "max_conns": 2,
      "connecions": 0,
      "sent": 0,
      "received": 0,
      "fails": 0,
      "unavail": 0,
      "health_checks": {
        "checks": 0,
        "fails": 0,
        "unhealthy": 0
      },
      "downtime": 0
    }
  ],
  "zombies": 0,
  "zone": "cluster3"
}
```

servers 级，可以查看单个 upstream 的所有 server 级节点

```
Bash
curl -s
http://127.0.0.1:8081/api/7/stream/upstreams/backend4/servers
|jq .
[
  {
    "id": 1,
    "server": "192.168.40.101:8080",
    "weight": 1,
    "max_conns": 1,
    "max_fails": 1,
    "fail_timeout": "1s",
    "slow_start": "1s",
    "backup": false,
```

```
    "down": true,
    "parent": 0,
    "host": "www.dev.test.com:8080"
  },
  {
    "id": 0,
    "server": "www.dev.test.com:8080",
    "weight": 1,
    "max_conns": 1,
    "max_fails": 1,
    "fail_timeout": "1s",
    "slow_start": "1s",
    "backup": false,
    "down": true
  }
]
```

id 级，可以查看单个 upstream 的单个 id 级节点

```
Bash
curl -s
http://127.0.0.1:8081/api/7/stream/upstreams/backend4/servers/1
|jq .
{
  "id": 1,
  "server": "192.168.40.101:8080",
  "weight": 1,
  "max_conns": 1,
  "max_fails": 1,
  "fail_timeout": "1s",
  "slow_start": "1s",
  "backup": false,
  "down": true,
  "parent": 0,
  "host": "www.dev.test.com:8080"
}
```

3.8.4.2.3 修改

修改功能可以对 server 节点参数进行修改，其中 ip 格式 server 节点可以修改 server 参数，域名 server 及其子节点 server 无法修改，且 backup 参数无法修改
修改 ip 格式 server

```
Bash
```

```
curl -X PATCH -s
http://127.0.0.1:8081/api/7/stream/upstreams/backend3/servers/0 -d
'{"server":"192.168.40.114:8080","weight":1,"max_conns":1,"max_fai
ls":2,"fail_timeout":"1s","slow_start":"1s","backup":false,"down
":true}'|jq .
{
  "id": 0,
  "server": "192.168.40.114:8080",
  "weight": 1,
  "max_conns": 1,
  "max_fails": 2,
  "fail_timeout": "1s",
  "slow_start": "1s",
  "backup": true,
  "down": true
}
```

修改域名格式 server, 父节点 server 参数修改

```
Bash
curl -X PATCH -s
http://127.0.0.1:8081/api/7/stream/upstreams/backend4/servers/0 -d
'{"server":"www.dev.test2.com:8080","weight":2,"max_conns":2,"max_
fails":2,"fail_timeout":"5s","slow_start":"5s","backup":true,"do
wn":false}'|jq .
{
  "error": {
    "status": 400,
    "text": "server address is immutable",
    "code": "UpstramServerImmutable"
  },
  "request_id": "730e18137d711d23bb93b1c867c21b89",
  "href": "https://njet.org/en/docs/http/njt_http_api_module.html"
}
```

修改域名格式 server, 子节点 server 参数修改

```
Bash
curl -X PATCH -s
http://127.0.0.1:8081/api/7/stream/upstreams/backend4/servers/1 -d
'{"server":"192.168.40.114:8080","weight":2,"max_conns":2,"max_fai
ls":2,"fail_timeout":"5s","slow_start":"5s","backup":true,"down"
:false}'|jq .
{
```

```
"error": {
  "status": 400,
  "text": "server address is immutable",
  "code": "UpstramServerImmutable"
},
"request_id": "bcda20dbc9276a1e46229e620e5c5813",
"href": "https://njet.org/en/docs/http/njt_http_api_module.html"
}
```

修改域名格式 **server**, 只修改父节点基本参数

Bash

```
curl -X PATCH -s
http://127.0.0.1:8081/api/7/stream/upstreams/backend4/servers/0 -d
'{"weight":2,"max_conns":2,"max_fails":2,"fail_timeout":"5s","slow_start":"5s","backup":true,"down":false}'|jq .
{
  "id": 0,
  "server": "www.dev.test.com:8080",
  "weight": 2,
  "max_conns": 2,
  "max_fails": 2,
  "fail_timeout": "5s",
  "slow_start": "5s",
  "backup": false,
  "down": false
}
```

修改域名格式 **server**, 只修改子节点基本参数

Bash

```
curl -X PATCH -s
http://127.0.0.1:8081/api/7/stream/upstreams/backend4/servers/1 -d
'{"weight":1,"max_conns":3,"max_fails":3,"fail_timeout":"5s","slow_start":"5s","backup":true,"down":false}'|jq .
{
  "id": 1,
  "server": "192.168.40.101:8080",
  "weight": 1,
  "max_conns": 3,
  "max_fails": 3,
  "fail_timeout": "5s",
  "slow_start": "5s",
  "backup": false,
```

```
"down": false,
"parent": 0,
"host": "www.dev.test.com:8080"
}
```

3.8.4.2.4 删除

删除功能可以针对单个 **server** 进行删除，且删除域名格式 **server** 父节点时会兼带子节点全部删除，但是删除域名格式 **server** 子节点时将无法删除

删除 ip 格式 **server**

```
Bash
curl -X DELETE -s
http://127.0.0.1:8081/api/7/stream/upstreams/backend3/servers/0
|jq .
[]
```

删除域名格式 **server** 子节点

```
Bash
curl -X DELETE -s
http://127.0.0.1:8081/api/7/stream/upstreams/backend4/servers/1
|jq .
{
  "error": {
    "status": 400,
    "text": "server not removeable",
    "code": "UpstreamServerImmutable"
  },
  "request_id": "5399a28db3d4598b821c1c65cb7efa86",
  "href": "https://njet.org/en/docs/http/njt_http_api_module.html"
}
```

删除域名格式 **server** 父节点

```
Bash
curl -X DELETE -s
http://127.0.0.1:8081/api/7/stream/upstreams/backend4/servers/0
|jq .
[]
```

3.8.4.2.5 重置

针对 **upstream**，清空健康检查等数据

使用查询命令查看 upstream 的数据，可以观察到 requests 参数有一些数据信息

```
Bash
curl -s http://127.0.0.1:8081/api/7/stream/upstreams/backend3
|jq .
{
  "peers": [
    {
      "id": 0,
      "server": "192.168.40.101:8080",
      "name": "192.168.40.101:8080",
      "backup": true,
      "weight": 2,
      "state": "up",
      "active": 0,
      "max_conns": 2,
      "connecions": 4,
      "sent": 0,
      "received": 0,
      "fails": 0,
      "unavail": 0,
      "health_checks": {
        "checks": 0,
        "fails": 0,
        "unhealthy": 0
      },
      "downtime": 0
    }
  ],
  "keepalive": 0,
  "zombies": 0,
  "zone": "cluster3"
}
```

使用清除数据命令进行数据清空

```
Bash
curl -X DELETE -s
http://127.0.0.1:8081/api/7/stream/upstreams/backend3 |jq .
```

再次查询 upstream 的数据，可以看到已经被清空

```
Bash
curl -s http://127.0.0.1:8081/api/7/stream/upstreams/backend3
```

```

ljq .
{
  "peers": [
    {
      "id": 0,
      "server": "192.168.40.101:8080",
      "name": "192.168.40.101:8080",
      "backup": true,
      "weight": 2,
      "state": "up",
      "active": 0,
      "max_conns": 2,
      "connecions": 0,
      "sent": 0,
      "received": 0,
      "fails": 0,
      "unavail": 0,
      "health_checks": {
        "checks": 0,
        "fails": 0,
        "unhealthy": 0
      },
      "downtime": 0
    }
  ],
  "keepalive": 0,
  "zombies": 0,
  "zone": "cluster3"
}

```

3.8.4.3 持久化

upstream_api 功能可以使用持久化配置来保证增加、删除、修改后数据的持久性，例如重启服务后，之前的配置依然可以使用，且当某个 upstream 配置了 state 持久化后，upstream 就无法再静态添加 server 指令想要使用持久化功能，需要添加以下配置。

```

Bash
upstream http1{
    zone cluster 64k;
    state /home/njet/state/njet_state.file; #state 持久化保存
    文件可以自定义名称
}

```

3.9 Split Clients_2 功能

3.9.1 功能说明

开源 nginx 提供了一个流量配比的模块 [split_clients](#)，但是该模块是根据指定的一个变量值（如客户端 ip, url 中的参数等），然后计算 Hash 值，然后根据计算的值在 $0 - 2^{32}-1$ 的位置来进行流量配比，无法做到根据请求的数量做概率分布。

Split Clients 2 模块提供流量配比，是根据概率进行分布，当访问样本足够大的情况下，与设置的比例基本达到一致，通过该模块的动态配置，可以实现蓝绿发布（配置比例为 0%，及 100%）。

Split Clients 2 模块提供了两种方式进行流量配比：

- 静态配置方式：通过静态配置文件实现不同流量配比，分配的百分比，已经配置好，可以在配置文件中进行修改，但需要重新 reload 才能生效。
- 动态配置方式：通过声明式 API 实现动态修改流量配比

3.9.2 静态配置

Plaintext

语法： `split_clients_2 $variable { ... };`

默认值：

允许配置位置： `http`

配置样例：

Nginx

```
split_clients_2 $backend {  
    10%        backend1;  
    *          backend2;  
}
```

静态配置百分比修改后，需要 reload 才能生效。

- 百分比一行结束后，如果缺少分号，会导致下一行配比不生效。注意不能缺少分号。
- 配比分流 upstream 的数量，限制只能配置两个。

3.9.2.1 静态分流设置值的校验规则

百分比数值非法格式：校验提示如下：

njet: [emerg] percentage should be an integer, invalid percent value "80s%", in /home/dyn_split_clients_2/njet/conf/njet.conf:26

```
[root@master110 conf]# ../sbin/njet -p /home/dyn_split_clients_2/njet -c conf/njet.conf -s reload
njet: [emerg] percentage should be an integer, invalid percent value "80s%", in /home/dyn_split_clients_2/njet/conf/njet.conf:26
[root@master110 conf]# ^C
[root@master110 conf]# ^C
[root@master110 conf]# vi njet.conf
[root@master110 conf]# ../sbin/njet -p /home/dyn_split_clients_2/njet -c conf/njet.conf -s reload
njet: [emerg] percentage should be an integer, invalid percent value "80", in /home/dyn_split_clients_2/njet/conf/njet.conf:26
[root@master110 conf]#
```

3.9.2.2 分号缺少校验规则

无论配置几行，非最后一行 upstream 缺少分号的提示如下

```
[root@master110 conf]# ../sbin/njet -p /home/dyn_split_clients_2/njet -c conf/njet.conf -s reload
njet: [emerg] split_clients_2 config error, semicolon is missing in /home/dyn_split_clients_2/njet/conf/njet.conf:27
[root@master110 conf]#
```

最后一行缺少分号，同 nginx 普通功能，提示 unexpected }.

3.9.2.3 配置 upstream 个数校验规则

只能支持配置两行 upstream，如果多余两行，给与如下提示

```
[root@master110 conf]# ../sbin/njet -p /home/dyn_split_clients_2/njet -c conf/njet.conf -s reload
njet: [emerg] split_clients_2 should be configured with 2 groups, check if there are more than 2 lines in /home/dyn_split_clients_2/njet/conf/njet.conf:29
[root@master110 conf]#
```

3.9.3 当前动态分流比例查询

Bash

```
curl http://127.0.0.1:2012/api/1/config/http_split_clients_2
```

HTTP

HTTP/1.1 200 OK

Server: njet/1.23.1

Date: Fri, 24 Feb 2023 06:52:19 GMT

Content-Type: application/json

Content-Length: 480

Connection: keep-alive

```
{
  "http": {
    "split_clients_2": {
      "backend1": 10,
      "backend2": 90
    }
  }
}
```

3.9.4 动态设置分流比例

HTTP

```
vi test.json
```

```
{
    "http": {
        "split_clients_2": {
            "backend1": 20,
            "backend2": 80
        }
    }
}
```

Bash

```
curl http://127.0.0.1:2012/api/1/config/http_split_clients_2 -X
PUT -d @test.json
```

HTTP

HTTP/1.1 204 No Content

Server: njet/1.23.1

Date: Thu, 23 Feb 2023 11:33:08 GMT

Connection: keep-alive

Bash

```
curl http://127.0.0.1:2012/api/1/config/http_split_clients_2
```

HTTP

HTTP/1.1 200 OK

Server: njet/1.23.1

Date: Fri, 24 Feb 2023 06:52:19 GMT

Content-Type: application/json

Content-Length: 480

Connection: keep-alive

```
{
    "http": {
```

```
        "split_clients_2": {
            "backend1": 20,
            "backend2": 80
        }
    }
}
```

3.9.4.1 比例之和小于 100

静态配置时，比例非*的后端，更新后的比例将根据报文值进行设置，静态配置时比例为*的后端，更新后的比例是 100%减去上一个的比例之后的值。

3.9.4.2 比例之和大于 100

服务端检测到该类型的配置时，配置将视为无效配置，修改不报错，但修改不生效。

3.9.4.3 比例值为非数字格式

服务端检测到该类型的配置时，配置将视为无效配置，修改不报错，但修改不生效。

3.9.4.4 比例值为带小数点的数字格式

服务端检测到该类型的配置时，配置将视为无效配置，修改不报错，但修改不生效。

3.9.4.5 比例值为数字百分比格式

服务端检测到该类型的配置时，配置将视为无效配置，修改不报错，但修改不生效。

3.9.4.6 比例值为空

服务端检测到该类型的配置时，配置将视为无效配置，修改不报错，但修改不生效。

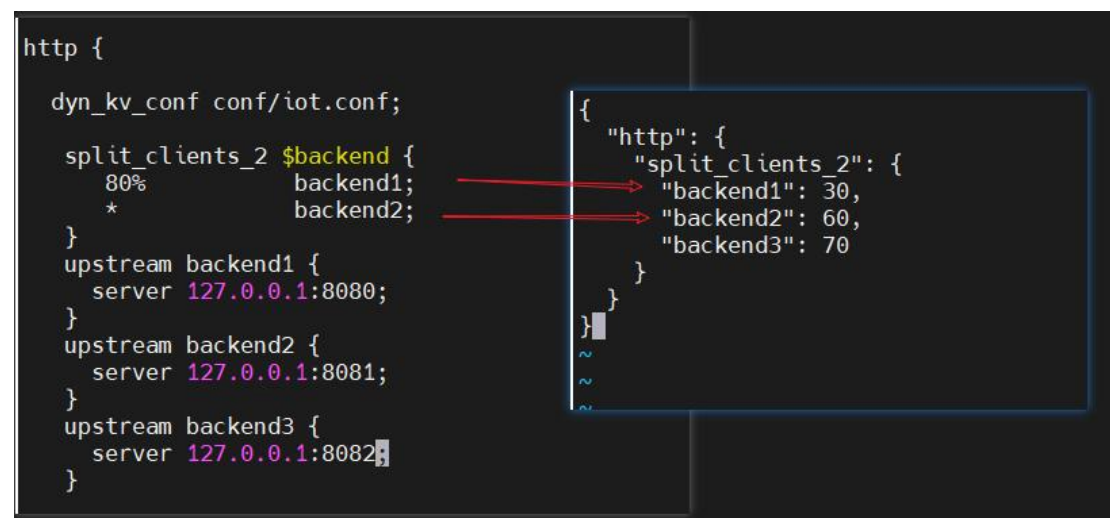
3.9.4.7 多余 2 个 backend

业务规则是先根据 backend 所有的比例之和进行判断，大于 100%，修改不报错，但修改不生效。

静态配置时，比例非*的后端，更新后的比例将根据报文值进行设置，静态配置时比例为*的后端，更新后的比例是 100%减去上一个的比例之后的值。

新增 backend 不做路由分配。如 upstream 顺序有变化，不影响，还是根据原有

upstream 名字，及原静态配置的路由来进行分流映射。



3.9.5 分流是否生效验证方式

可以通过如下脚本实现模拟大样本的流量配比验证。如配比为 50%，请求 100 次，会在 40 多到 50 多区间进行配比，如下。

```
[root@master110 conf]# for i in $(seq 1 100); do curl localhost:80; sleep 1s; done
welcome to 8081 http server
welcome to 8080 http server
welcome to 8080 http server
welcome to 8080 http server
welcome to 8080 http server
welcome to 8080 http server
^C
[root@master110 conf]# rm -rf static.txt; for i in $(seq 1 100); do curl localhost:80 >> static.txt >> /dev/null; done; grep -o "8081" static.txt | wc -l
57
[root@master110 conf]# ^C
[root@master110 conf]#
```

3.10 动态 Access Log

3.10.1 功能说明

通过动态 access_log，无需对配置文件进行编辑及重载，就可以实现对 access log 开关状态的动态控制功能，现提供了动态查询接口和动态修改接口。

3.10.2 动态查询 accesslog 配置

动态查询接口可以对 access log 的开关状态进行查询，可以对所有 location 的 access log 存储位置，名称，以及存储日志的格式进行查询，包括如果将日志存储在 syslog server 上。

```
HTTP
GET http://127.0.0.1:8081/api/1/config/http_log
```

返回值：

HTTP

HTTP/1.1 200 OK

Server: njet/1.23.1

Date: Thu, 09 Mar 2023 06:28:06 GMT

Content-Type: application/json

Content-Length: 1479

Connection: keep-alive

```
{
  "servers": [
    {
      "listens": [
        "127.0.0.1:1081",
        "127.0.0.1:1082",
        "[::]:1082",
        "unix:/home/cx/test.socket"
      ],
      "serverNames": [
        "*.tmlake.com",
        "*.tmlake1.com",
        "*.tmlake2.com"
      ],
      "locations": [
        {
          "location": "/",
          "accessLogOn": true,
          "accessLogs": [
            {
              "path": "logs/access.log",
              "formatName": "combined"
            }
          ]
        },
        {
          "location": "/demo",
          "accessLogOn": true,
          "accessLogs": [
            {
              "path": "logs/access.log",
              "formatName": "combined"
            }
          ]
        }
      ]
    }
  ]
}
```

```

    },
    {
      "location": "= /test/demo",
      "accessLogOn": true,
      "accessLogs": [
        {
          "path":
"syslog:server=192.168.40.140:514,facility=local7,tag=nginx,severity=info",
          "formatName": "test"
        }
      ]
    }
  ],
},
{
  "listens": [
    "0.0.0.0:1080"
  ],
  "serverNames": [
    ""
  ],
  "locations": [
    {
      "location": "/",
      "accessLogOn": true,
      "accessLogs": [
        {
          "path": "logs/access.log",
          "formatName": "combined"
        }
      ]
    }
  ],
},
{
  "location": "= /test/demo",
  "accessLogOn": true,
  "accessLogs": [
    {
      "path": "logs/access.log",
      "formatName": "combined"
    }
  ]
},
{

```

```

        "location": "/demo",
        "accessLogOn": true,
        "accessLogs": [
            {
                "path": "logs/$host.log",
                "formatName": "combined"
            }
        ]
    },
    {
        "location": "/demos",
        "accessLogOn": true,
        "accessLogs": [
            {
                "path": "logs/access.log",
                "formatName": "combined"
            }
        ]
    },
    {
        "location": "/named",
        "accessLogOn": true,
        "accessLogs": [
            {
                "path": "logs/access.log",
                "formatName": "combined"
            }
        ]
    }
]
},
"accessLogFormats": [
    {
        "name": "combined",
        "format": "$remote_addr - $remote_user [$time_local]
\"$request\" $status $body_bytes_sent \"$http_referer\"
\"$http_user_agent\""
    },
    {
        "name": "test",
        "format": "$remote_addr - $remote_user [$time_local]
\"$request\" $status $body_bytes_sent \"$http_referer\"
\"$http_user_agent\""
    }
]
}

```

```
}  
]  
}
```

3.10.3 动态配置 **accesslog** 的本地存储

如果开启 **accesslog**，则必须配置 **access log** 的写入位置与格式。

HTTP

PUT http://127.0.0.1:8081/api/1/config/http_log

Content-Type: application/json

```
{  
  "servers": [  
    {  
      "listens": [  
        "127.0.0.1:1081",  
        "127.0.0.1:1082",  
        "[::]:1082",  
        "unix:/home/cx/test.socket"  
      ],  
      "serverNames": [  
        "*.tmlake.com",  
        "*.tmlake1.com",  
        "*.tmlake2.com"  
      ],  
      "locations": [  
        {  
          "location": "= /test/demo",  
          "accessLogOn": true,  
          "accessLogs": [  
            {  
              "path": "logs/access_new.log",  
              "formatName": "test"  
            }  
          ]  
        }  
      ]  
    }  
  ]  
}
```

返回值：

```
HTTP
HTTP/1.1 204 No Content
Server: njet/1.23.1
Date: Thu, 23 Feb 2023 11:33:08 GMT
Connection: keep-alive
```

3.10.4 动态配置 access log 的 syslog server 存储

如果开启 accesslog, 也可以将 access log 写入位置定义为远端的 syslog server。

```
HTTP
PUT http://127.0.0.1:8081/api/1/config/http_log
Content-Type: application/json

{
  "servers": [
    {
      "listens": [
        "127.0.0.1:1081",
        "127.0.0.1:1082",
        "[::]:1082",
        "unix:/home/cx/test.socket"
      ],
      "serverNames": [
        "*.tmlake.com",
        "*.tmlake1.com",
        "*.tmlake2.com"
      ],
      "locations": [
        {
          "location": "= /test/demo",
          "accessLogOn": true,
          "accessLogs": [
            {
              "path":
"syslog:server=192.168.40.140:514,facility=local7,tag=nginx,severity=info",
              "formatName": "test"
            }
          ]
        }
      ]
    }
  ]
}
```

```
}  
]  
}
```

server=address

定义系统日志服务器的地址。地址可以指定为域名或 IP 地址，使用可选端口，或作为 UNIX 域套接字路径在 "unix:" 前缀后指定。如果未指定端口，则使用 UDP 端口 514。如果一个域名解析为多个 IP 地址，则使用第一个解析地址。

facility=string

设置 [RFC 3164](#) 中定义的 syslog 的 facility。facility 可以是 "kern", "user", "mail", "daemon", "auth", "intern", "lpr", "news", "uucp", "clock", "authpriv", "ftp", "ntp", "audit", "alert", "cron", "local0".. "local7"，默认值为 "local7"。

severity=string

设置 access_log 的系统日志消息的严重性，如 [RFC 3164](#) 中所定义。可能的值与 error_log 指令的第二个参数（级别）相同。默认值为 "info"。

tag=string

设置系统日志消息的标签。默认值为 "nginx"。

nohostname

禁用将 "hostname" 字段添加到系统日志消息标头

3.10.5 动态配置 access log 的 logformat

日志的格式，也可以通过动态进行自定义。如果 format 存在则更新，如果 format 不存在则创建。

```
HTTP  
PUT http://127.0.0.1:8081/api/1/config/http_log  
Content-Type: application/json  
  
{  
  "accessLogFormats": [  
    {  
      "name": "test",  
      "format": "$remote_addr - $remote_user [$time_local]  
\"$request\" $status $body_bytes_sent \"$http_referer\"  
\"$http_user_agent\""  
    }  
  ]  
}
```

3.10.6 accesslog 的验证方式

accesslog 开关状态为 false，即关闭时，有请求，相应路径下的 accesslog 日志文件也不会增加新的日志记录功能；

如果开关状态为 true，即打开状态时，有请求过来，对应的 accesslog 文件里就会增加新的日志记录。

```
[root@master110 conf]# curl localhost:2013/zxlconfapi/1/config/http_log |jq |tee accesslog.json
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100    536    100    536     0     0    9100       0  --:--:-- --:--:-- --:--:--   9241
{
  "servers": [
    {
      "listens": [
        "0.0.0.0:80"
      ],
      "serverNames": [
        "www.crm.com"
      ],
      "locations": [
        {
          "location": "/cust",
          "accessLogOn": false
        },
        {
          "location": "/order",
          "accessLogOn": false
        }
      ]
    },
    {
      "listens": [
        "0.0.0.0:2011"
      ],
      "serverNames": [
        ""
      ],
      "locations": [
        {
          "location": "/",
          "accessLogOn": false
        }
      ]
    },
    {
      "listens": [
        "0.0.0.0:2012"
      ],
      "serverNames": [
        ""
      ],
      "locations": [
        {
          "location": "/",
          "accessLogOn": false
        }
      ]
    }
  ],
  "accessLogFormats": [
    {
      "name": "combined",
      "format": "$remote_addr - $remote_user [$time_local] \"$request\" $status $body_bytes_sent \"$"
    }
  ]
}
```

关闭状态

```

[root@master110 conf]# vi accesslog.json
[root@master110 conf]# curl localhost:2013/zxlconfapi/1/config/http_log -X PUT -d @accesslog.json
[root@master110 conf]# curl localhost:2013/zxlconfapi/1/config/http_log |jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
100    607    100    607    0     0   9688      0 --:--:-- --:--:-- --:--:--   9790
{
  "servers": [
    {
      "listens": [
        "0.0.0.0:80"
      ],
      "serverNames": [
        "www.crm.com"
      ],
      "locations": [
        {
          "location": "/cust",
          "accessLogOn": false
        },
        {
          "location": "/order",
          "accessLogOn": true,
          "accessLogs": [
            {
              "path": "logs/access_order.log",
              "formatName": "combined"
            }
          ]
        }
      ]
    }
  ],
  "listens": [
    "0.0.0.0:2011"
  ],
  "serverNames": [
    ""
  ],
  "locations": [
    {
      "location": "/",
      "accessLogOn": false
    }
  ],
  "listens": [
    "0.0.0.0:2012"
  ],
  "serverNames": [
    ""
  ],
  "locations": [
    {
      "location": "/",
      "accessLogOn": false
    }
  ]
},
  "accessLogFormats": [
    {
      "name": "combined",
      "format": "$remote_addr - $remote_user [$time_local] \"$request\" $status $body_bytes_sent \\"
    }
  ]
}

```

修改为开启状态，并定义存储路径和存储日志名称

```

[root@master110 conf]# cd ../logs
[root@master110 logs]# ll
总用量 28
-rw-r--r--. 1 root root 454 3月 27 10:38 access_ctrl.log
-rw-r--r--. 1 root root 0 3月 27 10:30 access.log
-rw-r--r--. 1 root root 0 3月 27 10:38 access_order.log
-rw-r--r--. 1 root root 1676 3月 27 10:38 ctrl-error.log
-rw-r--r--. 1 root root 2752 3月 27 10:38 error.log
-rw-----. 1 root root 5288 3月 27 10:42 mosquitto.log
-rw-rw-rw-. 1 root root 168 3月 27 10:38 sendmsg_2
-rw-rw-rw-. 1 root root 244 3月 27 10:38 work_iot_0
[root@master110 logs]# cat access_order.log
[root@master110 logs]# curl www.crm.com/order
这是订单服务中心，欢迎您！
[root@master110 logs]# cat access_order.log
192.168.40.110 - - [27/Mar/2023:10:44:50 +0800] "GET /order HTTP/
[root@master110 logs]#

```

日志查看，在指定目录下生成了指定名称的访问日志文件在有请求时，可见，访问日志中增加了相应的记录内容。

3.10.7 附 accesslog 静态配置相关知识

3.10.7.1 默认存储路径

Plain Text
access_log logs/access.log;

3.10.7.2 默认开关状态

Plain Text
默认不配置表示，access log 是开启的状态。

3.10.7.3 默认日志内容格式

Plain Text
默认日志格式 combined

3.10.7.4 访问日志内容格式自定义语法

log_format 指令要写在 http 下

Plain Text

`log_format` 格式名 `escape=json/none/default` 格式体;

3.10.7.5 存储路径自定义指令

可以写在 `http` 下, `server` 下, `location` 下, 子 `location` 下。

Plain Text

`access_log` 相对路径/访问日志名.log 格式名;

3.10.7.6 访问日志开关状态控制指令

Plain Text

`access_log` `off`; #注意无 `on` 参数, 只有 `off`。

3.10.7.7 nginx 输出到 syslog 日志

Bash

```
access_log
syslog:server=192.168.40.111:514,facility=local7,tag=nginx,severity=info combined;
```

3.11 缓存功能

3.11.1 简介

Njet 会缓存来自代理 Web 或应用程序服务器的静态和动态内容, 以加快向客户端的响应速度并减少上游服务器上的负载。启用缓存后, Njet 将响应保存在 Njet 磁盘缓存中, 并使用它们来响应客户端, 而不必每次都对相同内容向上游服务器发起请求。

3.11.2 启用缓存

若要启用缓存, 请在 `Http` 上下文中包含 `proxy_cache_path` 指令。第一个参数是缓存内容的本地文件系统路径, 第二个参数定义用于存储有关缓存项的元数据的共享内存区域的名称和大小。

Nginx

```
http {
    # ...
    proxy_cache_path /etc/njet/cache/ keys_zone=mycache:20m;
```

```
    proxy_cache mycache;
}
```

然后在要缓存 **server** 响应的上下文（**http**、**server** 或 **location**）中包含 **proxy_cache** 指令，并指定由上述指令参数定义的共享内存区域名称。

```
Nginx
http {
    # ...
    proxy_cache_path /etc/njet/cache/ keys_zone=mycache:20m;
    server {
        proxy_cache mycache;
        location / {
            proxy_pass http://backend;
        }
    }
}
```

请注意，参数定义的大小不会限制缓存响应数据的总量。缓存的响应本身与元数据的副本一起存储在文件系统上的特定文件中。要限制缓存的响应数据量，请将参数 **max_size** 配置在 **proxy_cache_path** 指令中。（但请注意，缓存的数据量可能会暂时超过此限制）。

3.11.3 指定要缓存的请求

默认情况下，NJet 会在第一次从代理服务器收到此类响应时缓存对使用 HTTP GET 和 HEAD 方法发出的请求的所有响应。NJet 使用请求字符串作为请求的标识符。如果请求与缓存的响应具有相同的键，NJet 会将缓存的响应发送到客户端。您可以在 **http{}、server{} 或 location{} 中包含各种指令，以控制缓存哪些响应。**

要更改用于计算缓存文件标识，请使用 **proxy_cache_key** 指令。

```
Nginx
proxy_cache_key "$host$request_uri$cookie_user";
```

要定义在缓存响应之前必须发出具有相同键的请求的最小次数，请使用 **proxy_cache_min_uses** 指令：

```
Nginx
proxy_cache_min_uses 5;
```

要使用 GET 和 HEAD 以外的方法缓存对请求的响应，请使用

`proxy_cache_methods` 指令并配置要缓存的方法。

```
Nginx
proxy_cache_methods GET HEAD POST;
```

3.11.4 限制或禁用缓存

默认情况下，响应无限期地保留在缓存中。仅当缓存超过最大配置大小时，才会删除它们，然后按自上次请求它们以来的时间排序最长最少使用的缓存进行删除。您可以通过在 `http{}`、`server{}` 或 `location{}` 上下文中包含指令来设置缓存响应被视为有效的时间和是否使用缓存：

要限制具有特定状态代码的缓存响应被视为有效的时间，请使用 `proxy_cache_valid` 指令：

```
Nginx
proxy_cache_valid 200 302 10m;
proxy_cache_valid 404      1m;
```

在此示例中，包含代码 200 或 302 的响应被视为有效 10 分钟，包含代码的响应的有效期为 1 分钟。要定义具有所有状态代码的响应的有效期，请指定为第一个参数为 `any`：

```
Nginx
proxy_cache_valid any 5m;
```

要定义 NJet 不向客户端发送缓存响应的条件，请使用 `proxy_cache_bypass` 指令。每个参数定义一个条件，并由许多变量组成。如果至少有一个参数不为空且不等于“0”，NJet 不会在缓存中查找响应，而是立即将请求转发到后端服务器。

```
Nginx
proxy_cache_bypass $cookie_nocache $arg_nocache$arg_comment;
```

要定义 NJet 根本不缓存响应的条件，请使用 `proxy_no_cache` 指令，并以与 `proxy_cache_bypass` 指令相同的方式定义参数。

```
Nginx
proxy_no_cache $http_pragma $http_authorization;
```

3.11.5 从缓存中清除内容

NJet 可以从缓存中删除过时的缓存文件。这对于删除过时的缓存内容以防止同时提供

旧版和新版本的网页是必需的。缓存在收到包含自定义 HTTP 标头或 HTTP 方法的特殊“PURGE”请求时被清除。

3.11.5.1 配置缓存清除功能

让我们设置一个配置来标识使用 HTTP PURGE 方法的请求并删除匹配的 URL。

1. 在上下文中，使用 map 创建一个新变量，例如：

```
Nginx
http {
    # ...
    map $request_method $purge_method {
        PURGE 1;
        default 0;
    }
}
```

2. 在配置缓存的 location{} 块中，配置 proxy_cache_purge 指令以指定缓存清除请求的条件。在我们的示例中，它是在上一步中配置的：

```
Nginx
server {
    listen      80;
    server_name www.example.com;
    location / {
        proxy_pass https://localhost:8002;
        proxy_cache mycache;
        proxy_cache_purge $purge_method;
    }
}
```

3.11.5.2 发送 PURGE 请求

配置 proxy_cache_purge 指令后，需要发送特殊的缓存清除请求来清除缓存。您可以使用一系列工具发出清除请求，包括以下示例中的 curl 命令：

```
Bash
$ curl -X PURGE -D - "https://www.example.com/*"
HTTP/1.1 204 No Content
Server: njet/1.23.2
Date: Sat, 19 May 2018 16:33:04 GMT
Connection: keep-alive
```

在此示例中，将清除具有公共 URL 部分（由星号通配符指定）的资源。但是，此类缓存条目不会立即从缓存中完全删除：它们将保留在磁盘上，直到它们因不活动（由 `proxy_cache_path` 指令的参数确定）或缓存清除器（使用 `purge` 参数启用）而被删除，或者客户端尝试访问它们。

3.11.5.3 限制对清除命令的访问

我们建议您限制允许发送缓存清除请求的 IP 地址：

```
Nginx
geo $purge_allowed {
    default      0; # deny from other
    10.0.0.1      1; # allow from 10.0.0.1 address
    192.168.0.0/24 1; # allow from 192.168.0.0/24
}
map $request_method $purge_method {
    PURGE    $purge_allowed;
    default  0;
}
```

在此示例中，Njet 检查请求中是否使用了 PURGE 方法，如果是则分析客户端 IP 地址。如果 IP 地址已列入白名单，则 设置为： 允许清除，否则拒绝清除。

3.11.5.4 从缓存中完全删除文件

要完全删除与星号匹配的缓存文件，请激活 `purge` 功能，该功能将循环访问所有缓存条目并删除与通配符键匹配的条目。在配置中包含 `purge` 参数到 `proxy_cache_path` 指令：

```
Nginx
proxy_cache_path /etc/njet/cache levels=1:2 keys_zone=mycache:20m
purger=on;
```

3.11.5.5 缓存清除配置示例

```
Nginx
http {
    # ...
    proxy_cache_path /etc/njet/cache levels=1:2
keys_zone=mycache:20m purger=on;

    geo $purge_allowed {
        default      0;
```

```

        10.0.0.1        1;
        192.168.0.0/24 1;
    }
    map $request_method $purge_method {
        PURGE    $purge_allowed;
        default 0;
    }
}
server {
    listen      80;
    server_name www.example.com;
    location / {
        proxy_pass      https://localhost:8002;
        proxy_cache      mycache;
        proxy_cache_purge $purge_method;
    }
}
}

```

3.11.6 文件分片缓存

初始缓存填充操作有时需要相当长的时间，尤其是对于大文件。例如，当视频文件开始下载以满足对文件一部分的初始请求时，后续请求必须等待整个文件下载并放入缓存中。

NJet 可以缓存此类分片请求，并使用缓存切片模块逐渐填充缓存，该模块将文件划分为较小的“分片”。每个 **Range** 请求都会选择覆盖所请求范围的特定切片，如果此 **Range** 仍未缓存，则将其放入缓存中。对这些切片的所有其他请求都会从缓存中获取数据。

3.11.6.1 开启分片缓存

1. 使用 **slice** 指令指定切片的大小：

```

Nginx
location / {
    slice 1m;
}

```

选择可加快切片下载速度的切片大小。如果大小太小，则内存使用量可能过多，并且在处理请求时打开了大量文件描述符，而过大会导致下载生成缓存缓慢。

2. 将 **\$slice_range** 变量包含在缓存键中：

```
Nginx
proxy_cache_key $uri$is_args$args$slice_range;
```

3. 状态代码 206 启用响应缓存:

```
Nginx
proxy_cache_valid 200 206 1h;
```

4. 通过在标头字段中设置 `$slice_range` 变量, 允许将 `range` 请求传递到代理服务器

```
Nginx
proxy_set_header Range $slice_range;
```

全部配置如下:

```
Nginx
location / {
    slice            1m;
    proxy_cache      cache;
    proxy_cache_key  $uri$is_args$args$slice_range;
    proxy_set_header Range $slice_range;
    proxy_cache_valid 200 206 1h;
    proxy_pass       http://localhost:8000;
}
```

3.11.7 其他缓存相关指令

3.11.7.1 `proxy_cache_use_stale` 指令

```
SQL
Syntax: proxy_cache_use_stale error | timeout | invalid_header |
updating | http_500 | http_502 | http_503 | http_504 | http_404 |
off ...;
Default: proxy_cache_use_stale off;
Context: http, server, location
```

该指令可以理解为是 `nginx` 缓存的一个托底机制, 如果 `nginx` 在访问被代理服务器过程中出现被代理的服务器无法访问或者访问错误等现象时, `nginx` 服务器可以使用历史缓存响应客户端的请求, 这些数据不一定和被代理服务器上最新的数据相一致, 但对于更新频率不高的后端服务器来说, `nginx` 服务器的该功能在一定程度上能够为客户端提供不间断访问

```

Nginx
http{
    # ...
    server{
        # ...
        location / {
            # ...
            proxy_cache_use_stale error | timeout |
invalid_header | updating | http_500 | http_502 | http_503 |
http_504 | http_404 | off ...;
        }
    }
}

```

- **errore**、**updating** 等参数：可以通过这些参数来设置一些状态，当后端被代理的服务器处于这些状态时，nginx 服务器启用该功能，设置为 **off** 则关闭此功能，此处推荐配置为 **updating**，该参数告知 nginx 在客户端请求的项目的更新正在原服务器中下载时发送旧内容，而不是向服务器转发重复的请求。第一个请求陈旧文件的用户不得不等待文件在原服务器中更新完毕。陈旧的文件会返回给随后的请求直到更新后的文件被全部下载。

3.11.7.2 proxy_cache_lock 指令

```

SQL
Syntax: proxy_cache_lock on | off;
Default: proxy_cache_lock off;
Context: http, server, location

```

开启此功能时，对于相同的请求，同时只允许一个请求发往后端，并根据 **proxy_cache_key** 指令的设置在校对中植入一个新条目。其他请求相同条目的请求将一直等待，直到缓存中出现相应内容，或者锁在 **proxy_cache_lock_timeout** 指令设置的超时后被释放。如果不启用 **proxy_cache_lock**，则所有在校对中找不到文件的请求都会直接与服务器通信。

```

Nginx
http{
    # ...
    server{
        # ...
        location{
            # ...
            proxy_cache_lock on | off;
        }
    }
}

```

```

    }
}
}

```

- on | off: 控制此功能是否开启，此处推荐开启，减小并发压力，提高服务器性能。

3.11.7.3 proxy_cache_background_update 指令

SQL

Syntax: proxy_cache_background_update on | off;

Default: proxy_cache_background_update off;

Context: http, server, location

该指令允许启动后台子请求以更新过期的缓存项，同时用过期的缓存向下游发送响应。

```

Nginx
http{
    # ...
    server{
        # ...
        location{
            # ...
            proxy_cache_background_update on | off;
        }
    }
}

```

- on | off: 控制此功能是否开启，此处推荐开启（请注意，如果需要使用分片下载功能务必将此指令关闭，以免影响分片功能），返回陈旧数据时，也跟源站获取一份新的，下次返回时使用，如果没有过期就直接向下游返回响应。

3.11.7.4 proxy_cache_revalidate 指令

SQL

Syntax: proxy_cache_revalidate on | off;

Default: proxy_cache_revalidate off;

Context: http, server, location

该指令是指 nginx 在反向代理时，如果缓存文件过期了，就需要向源服务器请求资源，这时会使用“If-Modified-Since”和“If-None-Match”头部信息来做验证文件是否需要

更新。

```

Nginx
http{
    # ...
    server{
        # ...
        location{
            # ...
            proxy_cache_revalidate on | off;
        }
    }
}

```

- on | off: 控制此功能是否开启，此处推荐开启，因为 NGINX 在刷新来自服务器的内容时使用 GET 请求。如果客户端的请求项已经被缓存过了，但是在缓存控制头部中定义为过期，那么 NGINX 就会在 GET 请求中包含 If-Modified-Since 字段，发送至服务器端。这时使用这项配置可以节约带宽，因为对于 NGINX 已经缓存过的文件，服务器只会在该文件请求头中 Last-Modified 记录的时间内被修改时才将全部文件一起发送，如果记录时间没有被修改就不会一起发送。

3.11.8 组合配置示例

以下示例配置结合了上述一些缓存选项。

```

Nginx
http {
    # ...
    proxy_cache_path /etc/njet/cache keys_zone=mycache:20m
    loader_threshold=300 loader_files=200 max_size=200m;
    server {
        listen 8080;
        proxy_cache mycache;
        location / {
            proxy_pass http://backend1;
        }
        location /test_cache {
            proxy_pass http://backend2;
            proxy_cache_valid any 1m;
            proxy_cache_min_uses 3;
            proxy_cache_bypass $cookie_nocache
            $arg_nocache$args_comment;
        }
    }
}

```

```
    }  
  }  
}
```

在此示例中，两个 `location` 使用相同的缓存，但方式不同。

由于 `backend1` 响应很少更改，因此不包括缓存控制指令。响应在首次发出请求时缓存，并无限期有效。

相比之下，对 `backend2` 请求的响应经常更改，因此它们被认为仅在 1 分钟内有效，并且在发出 3 次相同的请求之前不会缓存。此外，如果请求符合 `proxy_cache_bypass` 指令定义的条件，Njet 会立即将请求传递给 `backend2`，而无需在缓存中查找相应的响应。

3.12 key value 功能

3.13.1 简介

控制面的 `Sendmsg` 模块提供了可选的配置指令 `dyn_sendmsg_kv`，这个指令将对外提供 KV 值设置及查询的 HTTP API 接口。

3.13.2 启用 KV API

通过在控制面的配置文件中，添加一个 `location`，并在 `location` 中使用 `dyn_sendmsg_kv` 指令开启 KV HTTP API。以下的配置示例中的端口及 URL 可以根据需要进行更改。

```
Nginx  
server {  
    listen 8081;  
    location /kv {  
        dyn_sendmsg_kv;  
    }  
}
```

3.13.1 API 说明

通过配置的 `location` 的 url，使用 POST 方法，设置 key value 值，提交的报文为 JSON 格式。

以下请求请注意替换 IP、端口、URL，与实际的配置需要一致

设置 KV 值

请求:

```
HTTP
POST http://127.0.0.1:8081/kv
Content-Type: application/json

{
  "key": "sc_test_key",
  "value": "here is the test msg"
}
```

返回:

```
HTTP
HTTP/1.1 200 OK
Server: njet/1.23.1
Date: Tue, 07 Mar 2023 08:10:20 GMT
Content-Length: 0
Connection: keep-alive
```

查询 key value 值

请求:

```
HTTP
GET http://127.0.0.1:8081/kv?key=sc_test_key
```

返回:

```
HTTP
HTTP/1.1 200 OK
Server: njet/1.23.1
Date: Tue, 07 Mar 2023 08:14:38 GMT
Content-Type: text/plain
Content-Length: 20
Connection: keep-alive
```

here is the test msg

3.13 正向代理

配置在客户端，客户端所有流量都通过正向代理去访问

3.13.1 指令介绍

3.13.1.1 proxy_connect

Syntax: **proxy_connect**

Default: none

Context: server

启用“CONNECT”HTTP 方法支持。

3.13.1.2 proxy_connect_allow

Syntax: **proxy_connect_allow** all | [port ...] | [port-range ...]

Default: 443 563

Context: server

此指令指定代理 CONNECT 方法可以连接到的端口号或范围的列表。

默认情况下，只启用默认的 https 端口(443)和默认的 snws 端口(563)。

使用此指令将覆盖此默认值，并仅允许连接到列出的端口。

All 值将允许所有端口进行代理。

给定值将允许指定的端口代理。

port-range 将允许指定的端口到代理的范围，例如：

Plaintext

```
proxy_connect_allow 1000-2000 3000-4000; # allow range of port  
from 1000 to 2000, from 3000 to 4000.
```

3.13.1.3 proxy_connect_connect_timeout

Syntax: **proxy_connect_connect_timeout** time

Default: none

Context: server

定义与代理服务器建立连接的超时。

3.13.2 代理配置示例

Example

```

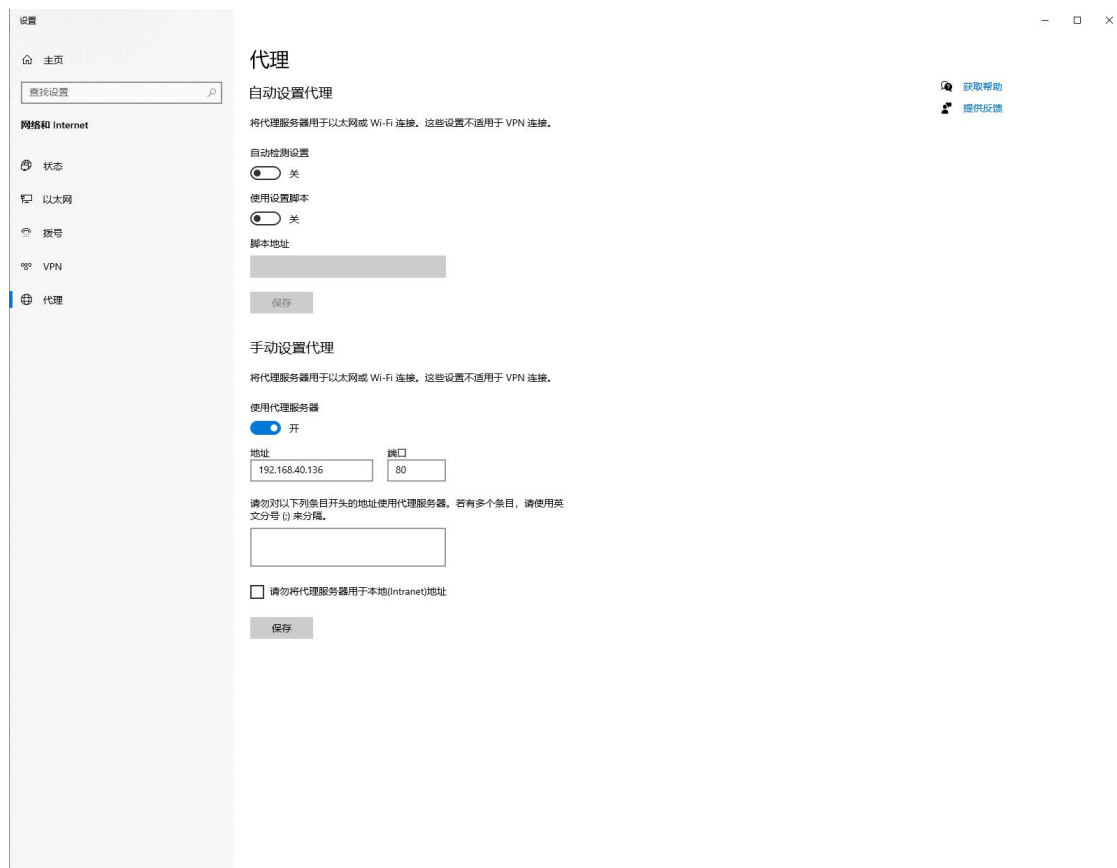
Bash
server {
    listen 80;    //端口设定
    resolver 114.114.114.114; # dns 解析服务器

    proxy_connect;
    proxy_connect_allow      443, 456;
    proxy_connect_connect_timeout 10s;

    location /{
        proxy_pass $scheme://$host$request_uri; #proxy_pass 用
来要代理的网站，
        #scheme 是客户端请求的协议（如 http, https）；
        #host 是客户端请求的域名（如 baidu.com）；
        #request_uri 是客户端访问的 url 地址（如
/baidu?s=12345）。
        #他们拼接成就是 http://baidu.com/baidu?s=12345
    }
}

```

3.13.3 客户端配置（win10 系统为例）



3.14 反向代理

反向代理（Reverse Proxy）是指以代理服务器来接受 internet 上的连接请求，然后将请求转发给内部网络上的服务器，并将从服务器上得到的结果返回给 internet 上请求连接的客户端，此时代理服务器对外就表现为一个反向代理服务器。

3.14.1 指令介绍

3.14.1.1 proxy_pass

设置连接被代理服务器的协议、IP 地址或套接字，也可以是域名或 upstream 定义的服务器组。

```
SQL
Syntax: proxy_pass URL;
Default: -
Context: location, , if in location limit_except
```

3.14.1.2 proxy_hide_header

指定被代理服务器响应数据中不向客户端传递的 HTTP 头字段名称。

```
SQL
Syntax:proxy_hide_header field;
Default:–
Context:http, server, location
```

3.14.1.3 proxy_http_version

设置用于代理的 HTTP 协议版本，若使用 keepalive 或 NTLM 认证，建议指令值设置为 1.1。

```
SQL
Syntax:proxy_http_version 1.0 | 1.1;
Default:proxy_http_version 1.0;
Context:http, server, location
```

3.14.1.4 proxy_redirect

替换被代理服务器返回的响应头中属性字段 location 或 Refresh 的值，并返回给客户端。指令值为 default 时，使用 proxy_pass 指令值的内容进行替换。

```
SQL
Syntax:proxy_redirect default;
        proxy_redirect off;
        proxy_redirect redirect replacement;
Default:proxy_redirect default;
Context:http, server, location
```

3.14.1.5 proxy_pass_header

默认配置下 Nginx 不会将头属性字段 Status 和 X-Accel-... 传递给客户端，可通过该指令开放传递。

```
SQL
Syntax:proxy_pass_header field;
Default:–
Context:http, server, location
```

3.14.1.6 proxy_set_header

在转发给被代理服务器前，修改或添加客户端的请求头属性字段

```
SQL
Syntax:proxy_set_header field value;
```

```
Default:proxy_set_header Host $proxy_host;
        proxy_set_header Connection close;
Context:http, server, location
```

3.14.1.7 proxy_socket_keepalive

设置 Nginx 与被代理服务器的 TCP keepalive 行为的心跳检测机制，默认使用操作系统的 socket 配置。若指令值为 on，则开启 SO_KEEPALIVE 选项进行心跳检测

```
SQL
Syntax:proxy_socket_keepalive on | off;
Default:proxy_socket_keepalive off;
Context:http, server, location
```

3.14.2 配置示例

```
Bash
upstream backend1{
    zone upstream_backend1 64k;
    server 127.0.0.1:8001;
    server 127.0.0.1:8002;
}
upstream backend2{
    zone upstream_backend2 64k;
    server 127.0.0.1:8003;
}

server{
    location /nginx {
        proxy_http_version 1.1;
        proxy_pass http://http1;
        proxy_hide_header Vary;
        proxy_pass_header server;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_redirect http://http1 http://http2;
        proxy_socket_keepalive on;
    }
}
```

3.15 JWT 认证

3.15.1 简介

该模块用于检查 JWT-token 是否有效。

3.15.2 指令说明

配置语法: `auth_jwt $variable | on | off;`
默认值: `auth_jwt off;`
允许配置位置: http、http>server、http>location

启用 JWT 的验证，仅当变量值为 on 时或者配置为 on 时开启验证。

配置语法: `auth_jwt_key value [encoding];`
默认值: 无
允许配置位置: http、http>server、http>location

指定用于验证 JWT 签名的密钥。

编码 可以是 hex | utf8 | base64 | file（默认值为 utf8）。

如果是文件该选项要求该值为有效的文件路径（指向 PEM 编码的公钥）。

配置语法: `auth_jwt_alg any | HS256 | HS384 | HS512 | RS256 | RS384 | RS512 | ES256 | ES384 | ES512;`
默认值: `auth_jwt_alg any;`
允许配置位置: http、http>server、http>location

指定服务器期望在接收的 JWT 的算法。

3.15.3 配置示例

```
Nginx
server {
    auth_jwt_key "0123456789abcdef" hex; # Your key as hex string
    auth_jwt off;
    location /secured-by-cookie/ {
        auth_jwt $cookie_MyCookieName;
    }
    location /secured-by-auth-header/ {
```

```

        auth_jwt on;
    }
    location /secured-by-auth-header-too/ {
        auth_jwt_key "another-secret"; # Your key as utf8 string
        auth_jwt on;
    }
    location /secured-by-rsa-key/ {
        auth_jwt_key /etc/keys/rsa-public.pem file; # Your key from
a PEM file
        auth_jwt on;
    }
}

```

3.16 http2

HTTP/2 是 HTTP 协议的 2.0 版本，该协议通过多路复用、请求优化、HTTP 头压缩等功能提升网络传输速度、优化用户体验。HTTP/2 使用二进制分帧层将传输的数据分割为更小的数据和帧，并对它们进行二进制格式编码处理，以实现在不改变 HTTP 现有语义等标准的基础上提升传输性能，从而降低响应延迟、提高请求吞吐的能力。

3.16.1 配置示例

```

Bash
worker_processes auto;
events {
    worker_connections 1024;
}
pid run/nginx.pid;
error_log logs/error.log info;

http {
    server {
        listen 2443 ssl http2;
        http2_max_field_size 4k;

        location / {
            root /home/njet1/html;
            index index.html;
        }
    }
}

```

```
}
```

3.16.2 不开启 http2 访问

```
[njet1@master110 conf]$ curl https://localhost:2443 -k --http2 -v
* Trying 127.0.0.1:2443...
* Connected to localhost (127.0.0.1) port 2443 (#0)
* ALPN: offers h2
* ALPN: offers http/1.1
* Cipher selection: ALL:!EXPORT:!EXPORT40:!EXPORT56:!aNULL:!LOW:!RC4:@STRENGTH
* [CONN-0-0][CF-SSL] TLSv1.2 (OUT), TLS header, Certificate Status (22):
* [CONN-0-0][CF-SSL] TLSv1.2 (OUT), TLS handshake, Client hello (1):
* [CONN-0-0][CF-SSL] TLSv1.2 (IN), TLS handshake, Server hello (2):
* [CONN-0-0][CF-SSL] TLSv1.2 (IN), TLS handshake, Certificate (11):
* [CONN-0-0][CF-SSL] TLSv1.2 (IN), TLS handshake, Server key exchange (12):
* [CONN-0-0][CF-SSL] TLSv1.2 (IN), TLS handshake, Server finished (14):
* [CONN-0-0][CF-SSL] TLSv1.2 (OUT), TLS handshake, Client key exchange (16):
* [CONN-0-0][CF-SSL] TLSv1.2 (OUT), TLS change cipher, Change cipher spec (1):
* [CONN-0-0][CF-SSL] TLSv1.2 (OUT), TLS handshake, Finished (20):
* [CONN-0-0][CF-SSL] TLSv1.2 (IN), TLS change cipher, Change cipher spec (1):
* [CONN-0-0][CF-SSL] TLSv1.2 (IN), TLS handshake, Finished (20):
* SSL connection using TLSv1.2 / ECDHE-RSA-AES256-GCM-SHA384
* ALPN: server accepted http/1.1
* Server certificate:
* subject: C=zg; ST=jl; O=tmh; OU=nginxdev; CN=cluster2
* start date: Sep  9 06:37:34 2022 GMT
* expire date: Sep  6 06:37:34 2032 GMT
* issuer: C=zg; ST=jl; L=cc; O=tmh; OU=nginxdev; CN=zx111; emailAddress=zhouxlo@tmh.com
* SSL certificate verify result: unable to get local issuer certificate (20), continuing anyway.
> GET / HTTP/1.1
> Host: localhost:2443
> User-Agent: curl/7.87.0
> Accept: /*/*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: njet/1.23.1
< Date: Thu, 02 Feb 2023 08:43:37 GMT
< Content-Type: text/html
< Content-Length: 153
< Last-Modified: Thu, 02 Feb 2023 08:24:35 GMT
< Connection: keep-alive
< ETag: "63db7343-99"
< Accept-Ranges: bytes
<
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>hello world</h1>
  
</body>
</html>
* Connection #0 to host localhost left intact
[njet1@master110 conf]$
```

3.16.3 开启 http2 访问

```
[njet1@master110 conf]$ curl https://192.168.40.110:2443 -k -v
* Trying 192.168.40.110:2443...
* Connected to 192.168.40.110 (192.168.40.110) port 2443 (#0)
* ALPN: offers h2
* ALPN: offers http/1.1
* Cipher selection: ALL:!EXPORT:!EXPORT40:!EXPORT56:!aNULL:!LOW:!RC4:@STRENGTH
* [CONN-0-0][CF-SSL] TLSv1.2 (OUT), TLS header, Certificate Status (22):
* [CONN-0-0][CF-SSL] TLSv1.2 (OUT), TLS handshake, Client hello (1):
* [CONN-0-0][CF-SSL] TLSv1.2 (IN), TLS handshake, Server hello (2):
* [CONN-0-0][CF-SSL] TLSv1.2 (IN), TLS handshake, Certificate (11):
* [CONN-0-0][CF-SSL] TLSv1.2 (IN), TLS handshake, Server key exchange (12):
* [CONN-0-0][CF-SSL] TLSv1.2 (IN), TLS handshake, Server finished (14):
* [CONN-0-0][CF-SSL] TLSv1.2 (OUT), TLS handshake, Client key exchange (16):
* [CONN-0-0][CF-SSL] TLSv1.2 (OUT), TLS change cipher, Change cipher spec (1):
* [CONN-0-0][CF-SSL] TLSv1.2 (OUT), TLS handshake, Finished (20):
* [CONN-0-0][CF-SSL] TLSv1.2 (IN), TLS change cipher, Change cipher spec (1):
* [CONN-0-0][CF-SSL] TLSv1.2 (IN), TLS handshake, Finished (20):
* SSL connection using TLSv1.2 / ECDHE-RSA-AES256-GCM-SHA384
* ALPN: server accepted h2
* Server certificate:
* subject: C=zg; ST=jl; O=tmh; OU=nginxdev; CN=cluster2
* start date: Sep  9 06:37:34 2022 GMT
* expire date: Sep  6 06:37:34 2032 GMT
* issuer: C=zg; ST=jl; L=cc; O=tmh; OU=nginxdev; CN=zxl111; emailAddress=zhouxlo@tmh.com
* SSL certificate verify result: unable to get local issuer certificate (20), continuing anyway.
* Using HTTP2, server supports multiplexing
* Copying HTTP/2 data in stream buffer to connection buffer after upgrade: len=0
* h2h3 [:method: GET]
* h2h3 [:path: /]
* h2h3 [:scheme: https]
* h2h3 [:authority: 192.168.40.110:2443]
* h2h3 [user-agent: curl/7.87.0]
* h2h3 [accept: /*/*]
* Using Stream ID: 1 (easy handle 0x189ceb0)
> GET / HTTP/2
> Host: 192.168.40.110:2443
> user-agent: curl/7.87.0
> accept: /*/*
>
* Connection state changed (MAX_CONCURRENT_STREAMS == 128)!
< HTTP/2 200
< server: njet/1.23.1
< date: Thu, 02 Feb 2023 09:41:09 GMT
< content-type: text/html
< content-length: 153
< last-modified: Thu, 02 Feb 2023 08:24:35 GMT
< etag: "63db7343-99"
< accept-ranges: bytes
<
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>hello world</h1>
  
</body>
</html>
* Connection #0 to host 192.168.40.110 left intact
[njet1@master110 conf]$
```

3.16.4 http2_push uri 功能验证

Plain Text

```
worker_processes auto;
events {
    worker_connections 1024;
}
pid run/nginx.pid;
error_log logs/error.log info;

http {

    server {
        listen 2443 ssl http2;
```

```

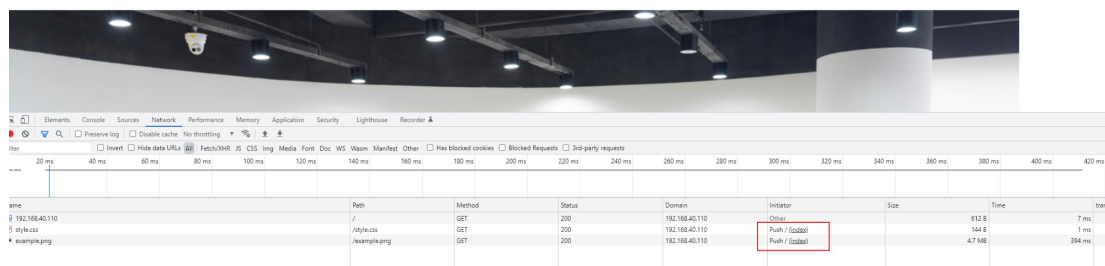
ssl_certificate server.crt;
ssl_certificate_key server.key;
location / {
    root /home/njet1/html;
    index index.html;
    http2_push /style.css;
    http2_push /example.png;
}
}
}
}

```

```

<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>hello world</h1>
  
</body>
</html>

```



Time	Type	Method	Status	Domain	Initiator	Size	Time	Trans
0	192.168.40.110	GET	200	192.168.40.110	Other	912 B	7 ms	
1	style.css	GET	200	192.168.40.110	Push / (css)	144 B	1 ms	
2	example.png	GET	200	192.168.40.110	Push / (img)	4.7 MB	284 ms	

请求一次，被推送的其他资源也同时显示，提示有 push 字样，如图。

3.17 Mirror

3.17.1 简介

mirror 流量复制功能开启后，会将源请求进行备份并转发到指定镜像上游服务组去处理，处理后响应结果 njet 接收到后会丢弃，并不会返回给客户端。流量复制几份，就会转发几次，实现流量收集并放大的作用。

3.17.2 配置示例

```

worker_processes auto;
events {
    worker_connections 1024;
}
http {
    include mime.types;
    default_type application/octet-stream;
    sendfile on;
    keepalive_timeout 65;
    server {
        listen 8181;
        access_log logs/test.log;
        root html/test;
    }
    server {
        listen 8282;
        access_log logs/mir1.log;
        root html/mir1;
    }
    server {
        listen 8383;
        access_log logs/mir2.log;
        root html/mir2;
    }
    upstream backend {
        server 127.0.0.1:8181;
    }
    upstream test_backend1 {
        server 127.0.0.1:8282;
    }
    upstream test_backend2 {
        server 127.0.0.1:8383;
    }
    server {
        listen 80;
        server_name localhost;
        location / {
            mirror /mirror1;
            mirror /mirror2;
            proxy_pass http://backend;
        }
        location = /mirror1 {
            internal;
            proxy_pass http://test_backend1$request_uri;
        }
        location = /mirror2 {
            internal;
            proxy_pass http://test_backend2$request_uri;
        }
    }
}

```

3.17.3 实际效果

```

[njet1@master110 logs]$ njet -p /home/njet1/njet -c conf/njet_mirror.conf -e logs/error_mirror.log -s reload
[njet1@master110 logs]$ cat test.log
[njet1@master110 logs]$ cat mir1.log
[njet1@master110 logs]$ cat mir2.log
[njet1@master110 logs]$ curl localhost:8484
test
[njet1@master110 logs]$ cat mir1.log
127.0.0.1 - - [07/Feb/2023:16:55:57 +0800] "GET / HTTP/1.0" 200 5 "-" "curl/7.87.0"
[njet1@master110 logs]$ cat mir2.log
127.0.0.1 - - [07/Feb/2023:16:55:57 +0800] "GET / HTTP/1.0" 200 5 "-" "curl/7.87.0"
127.0.0.1 - - [07/Feb/2023:16:55:57 +0800] "GET / HTTP/1.0" 200 5 "-" "curl/7.87.0"
[njet1@master110 logs]$ cat test.log
127.0.0.1 - - [07/Feb/2023:16:55:57 +0800] "GET / HTTP/1.0" 200 5 "-" "curl/7.87.0"
[njet1@master110 logs]$

```

3.18 服务发现

3.18.1 简介

http, stream upstream 配置域名格式的 server, 定期查询 dns server 进行域名解析, 实现对 upstream server 的添加, 删除, 修改。

3.18.2 配置说明:

支持 http 和 stream 的 upstream 配置。

要开启该功能, 需在 njet.conf 的 http 或 stream 的 upstream block 修改配置:

```
Shell
user root root;
worker_processes 1;
error_log /home/njet/logs/error.log debug;
events {
    worker_connections 1024;
}

http {
    dyn_kv_conf conf/iot.conf;
    error_log /home/njet/logs/error.log debug;
    access_log /home/njet/logs/access.log combined;

    upstream http1{
        zone upstream_http 64k;
        resolver 192.168.40.101 valid=10s;
        state zyg11.conf;
    }
    upstream http2{
        zone upstream_http 64k;
        resolver 192.168.40.101 valid=10s;
        server www.dev.test.com resolve;
    }
    server {
        listen 8000;
        server_name cluster1;
        location / {
            proxy_pass http://http1;
        }
    }
    server {
```

```

        listen 8899;
        server_name cluster2;
    }
}
stream{

    upstream stream_upstream{
        zone upstream_stream 64k;
        resolver 192.168.40.101 valid=10s;
        state zyg_stream.conf;
    }
    server {
        listen          9000;
        proxy_pass    stream_upstream;
    }
}
helper ctrl /home/njet/modules/njt_helper_ctrl_module.so
/home/njet/conf/njet_ctrl.conf;
helper broker /home/njet/modules/njt_helper_broker_module.so
conf/mqtt.conf;
cluster_name helper;
node_name node1;

```

配置项	必须 修改	配置说明
zone	是	upstream 的 zone 的名字，及大小。例如： zone upstream_http 64k; zone upstream_stream 64k;
resolver	是	dns 服务器的地址。resolver 192.168.40.101 valid=10s;
valid	否	域名失效的周期，也就是查询 dns 的周期。支持 标准的 NJet 时间字段。resolver 192.168.40.101 valid=10s;
resolver_timeout	否	查询 dns 超时时间。支持标准的 NJet 时间字 段。
state	否	state zyg_stream.conf; 通过 upstream api 接

		口添加的 upstream server 信息的保存路径。 reload 之后不会丢失。 不配置的话，不保存。 reload 之后丢失。
resolve	否	配置静态的动态域名。删除后 reload 之后，还会加载。 例如： <pre>server www.dev.test.com resolve;</pre> 注意：server 字段和 state 字段冲突。 不能静态动态混用。

3.18.3 依赖

1. Upstream api 接口， 动态解析的 ip 可以通过 upstream api 查看信息。
2. 需要搭建 dns 服务器，进行域名解析 ip， 的添加，删除，修改

3.19 动态开关功能

3.19.1 简介

该模块使用声明式 API 实现 access log 等模块的部分功能动态开关。其通过控制面 Copilot:njet_ctrl 实现接口，借助 Copilot: broker, sendmsg, kv 等模块构建的消息总线，传递到 worker 中，worker 即数据面通过实现特定模块的动态配置接口从而实现整体的动态控制。

3.19.2 控制面配置说明

依赖模块

```
njt_http_sendmsg_module.so
njt_ctrl_config_api_module.so
```

指令说明

config_req_pool_size

语法: config_req_pool_size number;

默认值: 500

允许配置位置: http->main

配置查询请求 (GET) 等待池的大小, 要求大于查询接口请求最大 qps。

config_api

语法: config_api;

默认值: 无

允许配置位置: http->location

配置开启动态 config api。

配置示例

控制面配置

```

Nginx
load_module
/mnt/f/workspace/c/njet_main/objs/njt_http_sendmsg_module.
so;
load_module
/mnt/f/workspace/c/njet_main/objs/njt_ctrl_config_api_modu
le.so;
error_log stderr info;# 注意修改为具体的文件路径, 此配置为输出
到控制台

events {
    worker_connections 1024;
}
cluster_name helper;
node_name node1;
http {

    dyn_sendmsg_conf conf/iot_ctrl.conf;
    access_log logs/access_ctrl.log combined;
    config_req_pool_size 1000;

    server {
        listen      8081;

        location /api {
            config_api;
        }
    }
}
```

```
}  
}
```

conf/iot_ctrl.conf 文件内容

```
Nginx  
log_type error
```

3.19.2.1 动态黑白名单配置

3.17.3.4.4 接口示例

使用 GET 方法获取当前各 location 的黑白名单配置，使用 PUT 方法更新黑白名单配置。PUT 提交时报文格式与 GET 获取到的报文格式一致，可以只提交需要改动的 location。

修改黑白名单

请求

```
HTTP  
PUT http://127.0.0.1:8081/config/1/config/http_dyn_bwlist  
  
{  
  "servers": [  
    {  
      "listens": [  
        "0.0.0.0:90"  
      ],  
      "serverNames": [  
        "localhost"  
      ],  
      "locations": [  
        {  
          "location": "/"  
        },  
        {  
          "location": "/test_bwlist",  
          "accessIpv4": [  
            {  
              "rule": "deny",  
              "addr": "192.168.40.118",  
              "mask": "255.255.255.255"  
            }  
          ]  
        }  
      ]  
    }  
  ]  
}
```

```
}
  }
]
}
]
}
]
}
```

返回值

JSON

```
HTTP/1.1 204 OK
Server: njet/1.23.1
Date: Thu, 30 Mar 2023 09:59:23 GMT
Content-Type: application/json
Content-Length: 492
Connection: keep-alive
```

查询当前黑白名单

请求

HTTP

```
GET http://127.0.0.1:8081/config/1/config/http_dyn_bwlist
```

返回

JSON

```
connection: keep-alive
content-length: 207
content-type: application/json
date: Thu,30 Mar 2023 08:31:06 GMT  server: njet/1.23.1
```

```
{
  "servers": [
    {
      "listens": [
        "0.0.0.0:90"
      ],
      "serverNames": [
        "localhost"
      ],
      "locations": [
```

```

    {
      "location": "/"
    },
    {
      "location": "/test_bwlist",
      "accessIpv4": [
        {
          "rule": "deny",
          "addr": "192.168.40.118",
          "mask": "255.255.255.255"
        }
      ]
    }
  ]
}
]
}
]
}

```

3.19.2.2 动态 metric

3.19.2.2.1 依赖模块

控制面参见上文控制面配置说明。

数据面需要加载如下模块

```

Plain Text
njt_helper_broker_module
njt_http_vtsc_module.so

```

3.19.2.2.2 配置指令

动态配置无特殊的指令。

3.19.2.2.3 配置示例

主要配置示例

```

Nginx
load_module objs/njt_http_vtsc_module.so;

helper ctrl objs/njt_helper_ctrl_module.so conf/njet_ctrl.conf;
helper broker objs/njt_helper_broker_module.so conf/mqtt.conf;

```

```
worker_processes 1;
error_log logs/error.log info;

events {
    worker_connections 1024;
}

http {
    dyn_kv_conf conf/iot.conf;

    access_log logs/access.log combined;
    variables_hash_max_size 2048;

    proxy_cache_path cache levels=1:2 keys_zone=cache:20m
max_size=1g inactive=30d use_temp_path=off;
    proxy_cache_valid any 10m ;
    proxy_cache_revalidate on;
    proxy_cache_lock on;
    proxy_cache_key $scheme$host$request_uri$slice_range;

    vhost_traffic_status_zone;
    vhost_traffic_status_filter_by_set_key "$request_uri"
"$server_name";

    upstream backgroup {
        zone backcenter 128k;
        server 192.168.30.222:80;
    }

    upstream errgroup {
        zone backcenter 128k;
        server 192.168.40.108:28000;
    }

    server {
        listen 8081;
        server_name server1;
        server_name server11;

        location / {
            add_header x-Cache-Status $upstream_cache_status;
            add_header x-Cache-Key
$scheme$host$request_uri$slice_range;
```

```

        proxy_cache cache;

        proxy_pass      http://backgroup;
    }

    location /a {
        return 200 "welcome to 8081 http server a\n";
    }
}

server {
    listen      8082;
    server_name server2;

    location / {
        proxy_pass      http://errgroup;
    }

    location /c {
        root html;
        index index.html;
    }

    location /d {
        root html;
        index index.html;
    }
}

server {
    listen      8083;
    server_name server3;

    location / {
        root html;
        index index.html;
    }

    location /a {
        root html;
        index index.html;
    }
}

```

```
        location /b {
            root html;
            index index.html;
        }
    }
}
```

3.19.2.2.4 接口示例

使用 GET 方法获取当前 metric 模块的设置，使用 PUT 方法更新 metric 模块的设置。
PUT 时使用的报文格式与 GET 获取到的报文格式一致。

查询当前 metric 模块的设置：

```
HTTP
curl -X GET http://127.0.0.1:8081/api/1/config/http_vts

返回：
{
  "vhost_traffic_status_filter_by_set_key": "\"$request_uri\" \"$server_name\"",
  "servers": [
    {
      "listens": [
        "0.0.0.0:8081"
      ],
      "serverNames": [
        "server1",
        "server11"
      ],
      "locations": [
        {
          "location": "/",
          "vhost_traffic_status": true
        },
        {
          "location": "/a",
          "vhost_traffic_status": true
        }
      ]
    }
  ],
  {
```

```
"listens": [  
  "0.0.0.0:8082"  
],  
"serverNames": [  
  "server2"  
],  
"locations": [  
  {  
    "location": "/",  
    "vhost_traffic_status": true  
  },  
  {  
    "location": "/c",  
    "vhost_traffic_status": true  
  },  
  {  
    "location": "/d",  
    "vhost_traffic_status": true  
  }  
]  
},  
{  
  "listens": [  
    "0.0.0.0:8083"  
  ],  
  "serverNames": [  
    "server3"  
  ],  
  "locations": [  
    {  
      "location": "/",  
      "vhost_traffic_status": true  
    },  
    {  
      "location": "/a",  
      "vhost_traffic_status": true  
    },  
    {  
      "location": "/b",  
      "vhost_traffic_status": true  
    }  
  ]  
}  
]
```

```
}
```

动态配置:

Nginx

```
curl -X GET http://127.0.0.1:8081/api/1/config/http_vts
```

返回:

```
{
  "vhost_traffic_status_filter_by_set_key": "\"$status\" \"$server_name:$request_uri\"",
  "servers": [
    {
      "listens": [
        "0.0.0.0:8081"
      ],
      "serverNames": [
        "server1"
      ],
      "locations": [
        {
          "location": "/a",
          "vhost_traffic_status": false
        }
      ]
    }
  ]
}
```

再次查询当前的配置:

Nginx

```
PUT http://127.0.0.1/api/1/config/http_vts HTTP/1.1
```

```
content-type: application/json
```

```
{
  "vhost_traffic_status_filter_by_set_key": "\"$status\" \"$server_name:$request_uri\"",
  "servers": [
    {
      "listens": [
```

```
    "0.0.0.0:8081"
  ],
  "serverNames": [
    "server1",
    "server11"
  ],
  "locations": [
    {
      "location": "/",
      "vhost_traffic_status": true
    },
    {
      "location": "/a",
      "vhost_traffic_status": false
    }
  ]
},
{
  "listens": [
    "0.0.0.0:8082"
  ],
  "serverNames": [
    "server2"
  ],
  "locations": [
    {
      "location": "/",
      "vhost_traffic_status": true
    },
    {
      "location": "/c",
      "vhost_traffic_status": true
    },
    {
      "location": "/d",
      "vhost_traffic_status": true
    }
  ]
},
{
  "listens": [
    "0.0.0.0:8083"
  ],
  "serverNames": [
```

```

        "server3"
    ],
    "locations": [
        {
            "location": "/",
            "vhost_traffic_status": true
        },
        {
            "location": "/a",
            "vhost_traffic_status": true
        },
        {
            "location": "/b",
            "vhost_traffic_status": true
        }
    ]
}
]
}

```

3.19.2.3 动态 telemetry

3.19.2.3.1 依赖模块

上文控制面配置说明

```

njl_helper_broker_module
njl_ctrl_config_api_module.so;

```

本功能需要模块，因为本功能采用外部 build，所以版本发布的时候提供对应的 so 服务间追踪模块(Otel)依赖:

```

njl_otel_module.so
njl_agent_dyn_otel_module.so

```

3.19.2.3.2 配置指令

无

3.19.2.3.3 配置示例

主要配置示例

注意主配置文件中需要修改 so 路径，log 路径

Nginx

分布式 **telemetry** 动态开关需要加载这两个模块

```
load_module
/root/project/telemetry/njet_opentelemetry/njet_otel_module/build/
njet_otel_module.so;
load_module
/root/project/telemetry/njet_opentelemetry/njet_otel_module/build/
njet_agent_dyn_otel_module.so;
```

```
worker_processes 1;
```

```
error_log stderr info; # 注意修改为具体的文件路径，此配置为输出到控制台
```

```
helper broker
/mnt/f/workspace/c/njet_main/objs/njt_helper_broker_module.so
conf/mqtt.conf;
helper ctrl
/mnt/f/workspace/c/njet_main/objs/njt_helper_ctrl_module.so
conf/ctrl_demo.conf;
```

```
events {
    worker_connections 1024;
}
cluster_name helper;
node_name node1;
```

```
http {
    dyn_kv_conf conf/iot.conf;
    access_log logs/access.log combined; ## 此处为必要配置，日志格式
    可以修改
```

分布式 **telemetry** 模块配置文件

```
opentelemetry_config /home/njet/clbtest/conf/otel.toml;
```

```
upstream demo {
    zone demo 128k;
    server 192.168.40.141:8080;
}

upstream demos {
    zone demos 128k;
    server 192.168.40.141:443;
}

server {
    listen      1081;

    location / {
        # 分布式 telemetry 配置
        opentelemetry on;
        opentelemetry_operation_name my_example_backend;
        opentelemetry_propagate;

        root html;
        index index.html;
        location /demo {
            proxy_pass http://demo;
        }
    }

    location = /test/demo {
        # 分布式 telemetry 配置
        opentelemetry on;
        opentelemetry_operation_name my_example_backend;
        opentelemetry_propagate;

        proxy_pass http://demo;
    }
}

server {
    listen      1080;

    location / {
        root html;
        index index.html;
    }
}
```

```

        location = /test/demo {
            limit_conn addr 100;
            proxy_pass http://demo;
        }

        location /demo {
            add_header test api;
            proxy_pass http://demo;
        }
        location /demos {
            limit_conn addr 100;
            proxy_ssl_certificate nginx.pem;
            proxy_ssl_certificate_key nginx-key.pem;
            proxy_pass https://demos;
        }
    }
}

```

mqtt.conf

```

Nginx
log_type error

```

iot.conf

```

Nginx
log_type error

```

3.19.3.7.4 接口示例

使用 GET 方法获取当前模块的设置，使用 PUT 方法更新设置。PUT 时使用的报文格式与 GET 获取到的报文格式一致。

查询当前模块的设置：

```

Shell
curl http://127.0.0.1:8081/config/1/config/otel|jq .
{
  "servers": [
    {
      "listens": [
        "0.0.0.0:8082"
      ]
    }
  ]
}

```

```

    ],
    "serverNames": [
        ""
    ],
    "locations": [
        {
            "location": "= /",
            "opentelemetry": false
        }
    ]
},
{
    "listens": [
        "0.0.0.0:8000"
    ],
    "serverNames": [
        ""
    ],
    "locations": [
        {
            "location": "= /",
            "opentelemetry": false
        },
        {
            "location": "= /up",
            "opentelemetry": false
        },
        {
            "location": "= /b3",
            "opentelemetry": false
        },
        {
            "location": "= /off",
            "opentelemetry": false
        }
    ]
}
]
}
}

```

动态配置:

```

Shell
curl -S -s -X PUT

```

```
'http://192.168.40.109:8081/config/1/config/otel' -H 'Content-
Type: application/json' --data-raw '{
  "servers": [
    {
      "listens": [
        "0.0.0.0:8082"
      ],
      "serverNames": [
        ""
      ],
      "locations": [
        {
          "location": "= /",
          "opentelemetry": true
        }
      ]
    },
    {
      "listens": [
        "0.0.0.0:8000"
      ],
      "serverNames": [
        ""
      ],
      "locations": [
        {
          "location": "= /",
          "opentelemetry": true
        },
        {
          "location": "= /up",
          "opentelemetry": true
        },
        {
          "location": "= /b3",
          "opentelemetry": true
        },
        {
          "location": "= /off",
          "opentelemetry": false
        }
      ]
    }
  ]
}
```

```
}'
```

再次查询当前的配置：

```
Shell
curl http://127.0.0.1:8081/config/1/config/otel|jq .
{
  "servers": [
    {
      "listens": [
        "0.0.0.0:8082"
      ],
      "serverNames": [
        ""
      ],
      "locations": [
        {
          "location": "= /",
          "opentelemetry": true
        }
      ]
    },
    {
      "listens": [
        "0.0.0.0:8000"
      ],
      "serverNames": [
        ""
      ],
      "locations": [
        {
          "location": "= /",
          "opentelemetry": true
        },
        {
          "location": "= /up",
          "opentelemetry": true
        },
        {
          "location": "= /b3",
          "opentelemetry": true
        },
        {
          "location": "= /off",
```

```
        "opentelemetry": false
    }
}
]
```

3.19.2.4 动态证书

3.19.2.4.1 简介

该模块支持在运行中替换 http server 的服务端证书。

3.19.2.4.2 依赖模块

njt_dyn_ssl_module.so

3.19.2.4.3 配置指令

无

3.19.2.4.4 配置示例

主要配置示例

注意主配置文件中需要修改 so 路径，log 路径，替换 ssl 证书

```
Nginx
load_module
/mnt/f/workspace/c/njet_main/objs/njt_dyn_ssl_module.so;
worker_processes 1;
daemon off;
master_process on;
error_log stderr info;
helper broker
/mnt/f/workspace/c/njet_main/objs/njt_helper_broker_module.so
conf/mqtt.conf;
helper ctrl
/mnt/f/workspace/c/njet_main/objs/njt_helper_ctrl_module.so
conf/ctrl_ssl.conf;

events {
    worker_connections 1024;
```

```

}
cluster_name helper;
node_name node1;
http {
    access_log logs/access.log combined;
    dyn_kv_conf conf/iot.conf;
    upstream demo {
        zone demo 128k;
        server 192.168.40.141:8080;
        keepalive 10240;
    }
    server {
        listen      1080 ssl;
        ssl_certificate_key 40.206-key.pem;
        ssl_certificate    40.206.pem;

        location / {
            add_header test api;
            proxy_set_header Connection "";
            proxy_http_version 1.1;
            proxy_pass http://demo;
        }
    }
}

```

mqtt.conf

```

Nginx
log_type error

```

iot.conf

```

Nginx
log_type error

```

3.19.2.4.5 接口文档

字段详细说明参见：<http://192.168.40.108:9000/#7>

3.19.2.4.6 接口示例

3.19.2.4.6.1 查询 http server ssl 当前配置

该接口可以查看 HTTP 块下所有 server 的证书配置情况

请求

HTTP

GET http://127.0.0.1:8081/config/1/config/http_ssl

返回值

HTTP

HTTP/1.1 200 OK

Server: njet/1.23.1

Date: Fri, 03 Mar 2023 08:42:22 GMT

Content-Type: application/json

Content-Length: 282

Connection: keep-alive

```
{
  "servers": [
    {
      "listens": [
        "0.0.0.0:1080"
      ],
      "serverNames": [
        ""
      ],
      "certificates": [
        {
          "certificate": "./conf/40.206.pem",
          "certificateKey": "./conf/40.206-key.pem"
        }
      ]
    },
    {
      "listens": [
        "0.0.0.0:1090"
      ],
      "serverNames": [
        ""
      ],
      "certificates": [
        {
          "certificate": "$host.pem",
          "certificateKey": "$host-key.pem"
        }
      ]
    }
  ]
}
```

```
}
```

3.19.2.4.6.2 更新 http server ssl 当前证书

请求

注意事项

该接口不是声明式全量配置，多次配置证书会存在内存泄露，所以更新 json 内容应为最小变更

HTTP

PUT http://127.0.0.1:8081/config/1/config/http_ssl

Content-Type: application/json

```
{
  "servers": [
    {
      "listens": [
        "0.0.0.0:1080"
      ],
      "serverNames": [
        ""
      ],
      "certificates": [
        {
          "certificate": "data:-----BEGIN CERTIFICATE-----
\r\nmIIEITCCAwmGAWIBAgIUK6l+HK4KR\hLosx2XqNTaHLUK00wDQYJKoZIhvcNA
QEL\r\nBQAwZTELMAkGA1UEBhMCQ04xEDAOBgNVBAgTB0JlaUppbmcxEDA0BgNVBAc
TB0Jl\r\naUppbmcxDjAMBgNVBAoTBW5naW54MQ8wDQYDVQQLEWZ0bWxha2UxETAPB
gNVBAMT\r\nCG5naW54LWNhMCAXDTEyMDgwOTA5NDEwMFOYDzIxMjIwNzE2MDk0MTA
wWjBiMQsw\r\nCQYDVQQGEwJDTjEQA4GA1UECBMHQmVpSm1uZzEQMA4GA1UEBxMHQ
mVpSm1uZzE0\r\nMAwGA1UEChMFbmdpbngxDzANBgNVBAstBnRtbGFrZTEOMAwGA1U
EAXMFbmdpbngw\r\nnggEiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQC0hs5E8
ZZT+ZU\DivvIRrSL\r\nnokk60E5EA4UR06zawqguKR1Tc1USgfMdZFwR2znCxjFcsk
7W\FSwqA5k5zyePYL8\r\n\01hn3i\GQGczfa9dMZZEtN9EMD6TP0iDI\CawAF
c3U7qE+SYckUHLu\lMX8LshR\r\nndNkGecq41Ck5Sftm3xwy4cmpcCIyMpNumHrjJ
yxkp3S\QGewgZRQypntG9R34EKj\r\nQNFnovsn39TeeG+DBjS5m1+roth0JJ6Dbe
mB47hy\uaEwAQpobN1etx20wCwxR07\r\n7Huuh4kmWXjUol0nvzXM3ZZX2QnHn4j
Ig\ALS706p86z+m4vEHK4hrfWf52xGjMv\r\nAgMBAAGjgckwgcYwDgYDVR0PAAQH\
/BAQDAgWgMB0GA1UdJQQWMBQGCCsGAQUFBwMB\r\nBggRBgEFBQCDAjAMBGNVHRMBA
f8EAJAAMB0GA1UdDgQWBBQz6z5AIFRrP+KD1gBt\r\nw6j5NEMzwDAfBgNVHSMEGDA
WgBQ64+C++PsR6pDZoQ7EORozSJ8d9DBHBgNVHREE\r\nQDA+ggkxOTIuMTY4LiqCD
```

iouY2x1c3RlcjEuY29tgg0qLmNsdXN0ZXIuY29tggwq\r\nLnRtbGFrZS5jb22HBH8
AAAEwDQYJKoZIhvcNAQELBQADggEBADouKA9FFf1NAs2u\r\nnu7yj4cGNG+sC8r1wD
WyGiXGDzvaB7C\ /sPTZiCgh8hI5gxzOSC+Se51kNFYymN9g4\r\nnBPFb\ /\ /SZ+Cx0
60B0t5eFF4PxIW7+d8gGFJr7Yx55QGA9DrvGi\ /LEiwjV3bQaezjN\r\nnoXUkfFgyK
v79AfNFShkUPiHCKlnfyqcCF+GEKxNC1q4wyC1Ut2EGtwfi04YR85s0\r\nn7jQ2BKJ
Osw3LLd8pZSv0XYu\ /I9Do05w3gM5TgjnJlQPq+bmGqJ7bfWbSpFo70cdF\r\nnFLfE
9MLpwhDjVQEJ6D0mZqU1dVdNtwLrUrm0\ /8xkkmac0\ /8BaEYyPy1SNMGJTMGE\r\nn
W8psxto=\r\n-----END CERTIFICATE-----\r\n",

"certificateKey": "data:-----BEGIN RSA PRIVATE KEY-----
\r\nmIIEogIBAAKCAQEAtIbORPGWU\ /mVPw77yEa0i6JJ0jhORAOfEa0s2sKoLikZU
3NV\r\nnEoHzHWRcEds5wsYxXLJ01vxUlqgOZ0c8nj2C\ /P9JYZ94vxkBnM32vXTGWR
LTfRDA\r\nn+kzzogyPwm1gBXN106hPkmHJFBy7v5TF\ /C7IUXTZBnnKuNQp0Un7Zt8
cMuHJqXaI\r\nnMjKTbph64ycsZKd0v0BnsIGUUMqZ7RvUd+BCo0DRZ6L7J9\ /U3nhv
gwY0uZtfq6LY\r\nndCSeg23pge04cv7mhMAEKaGzdXrcdtMASMUTu+x7roeJJ1141K
JdJ781zN2WV9kJ\r\nnx5+IyIPwC0uzuqf0s\ /puLxByuIUX1n+dsRozLwIDAQABAOI
BAFh+VJLbUn0rvwUA\r\nnTtBoSIzCat8NRuB0UUDKWSuLjGHEZ9P0j39ZEFHyJmfib
Tgba4sjJR6h5t1LWHMC\r\nnH2b6hEF86v3d7JTQr0esdy18FtcHMYD304H3Qt7HBZm
pihZh+K\ /b29XH9YfUZfyN\r\nn81ehnzS+8LwJ6+QarHKW35QX\ /ny5+pCw+G97cjd
E7sClZnryd1KseWubAXDLSgmQ\r\nnJcjXMMkwjtd2x\ /w23ZT97WMEcToL1wMw4hF\
/tHtu9u3UHMhzLHgrfrkW4QmACbLF\r\nn8MJR94wzscJrUEviz5fV+3ZTmK+luJGmO
\ /Kk0pNAPGf8QAEbfa\ /qyBTQCQgCdxRo\r\nn8hrDT4ECgYEA5vLsDDWiAR0vvPaRk
Cv0y0UWkdgyBxv2T6pjDJZzMxb0yKZ48E6l\r\nnh0PI9bf0KbI1ZZviiZg20oZ\ /K
eo2wLYa+W00rZhw1jWUh0Ivd\ /5vTPHBmpsXl6+\r\nnRK49aNVkfqlleherhfrYUKH
KKDeg4lKVY3fwAlAQeg\ /y4Ds\ /8k3HUUECgYEAyBu8\r\nns7Ku07sAaJVj+QFfRrR
6u2eJD48HPB4UoeDYMaKAuPTiM5\ /AUjH\ /YdD4E2lGAyJJ\r\nnJVN413yu3Dk9UIA
ivgqCR0epAcnnZ1BwE2qI\ /gfr2GD95o8JQSphze0pC\ /6WS\ /jG\r\nniBR+j7MGWI
kHJHEIUj0rf8eqtVLAywjxz9rPWA8CgYBTkuzgrjfl893Qn9m1NoLr\r\nnXCECvh28
fN3xjlmxpvAh0Nl0EuoI7CzyehEq+1YlJ3Xd9QaAE8tRD8u\ /plxwhOMU\r\nniJea+
OZZ6PQF2wPi0j5pvWb0Z2a378kiyXrniPFI9LwIJrCnV1MY0T36t8a8n+33\r\nnhNu
Ruq97vHHDuy003fiXgQKBgGKzM6cauc+iU\ /hBvzbBk4HnYSXwUm1HKdVgLOMP\r\nn
aPNKaN1RhATchdrE6GcR0FqasTq4fYWYn2ECEalz3idHnFtKCaj87qKAOM\ /\ /n9gj
\r\nn0wAhXhWy+WjwIitvQSB2Gqnc37sHML1MBoTQU4\ /1vn0d93G8JJn5HN0kvQ0oE
0Vn\r\nncp\ /HAoGAGCxpM24rHBK0Hm1ABZd8ZfLW\ /sW4Y7bMsQmrOVuPZcTWc05Hz
kkiGzTc\r\nn8DCDDeGdfJUC9NaM0DnVuMwp8\ /f5uZv4dTS7Hzf8Ie5vR9oQx\ /z73
iakIKUIpDQc\r\nn0\ /sUJSKijOQRT1xZQmkw5+QC1mbjLLS3ZXn9kMt0bbxZRPkcV2
A=\r\n-----END RSA PRIVATE KEY-----\r\n"

}
]
}
]
}

3.20 doc (openapi、gui、manual) 配置模块

3.20.1 概述

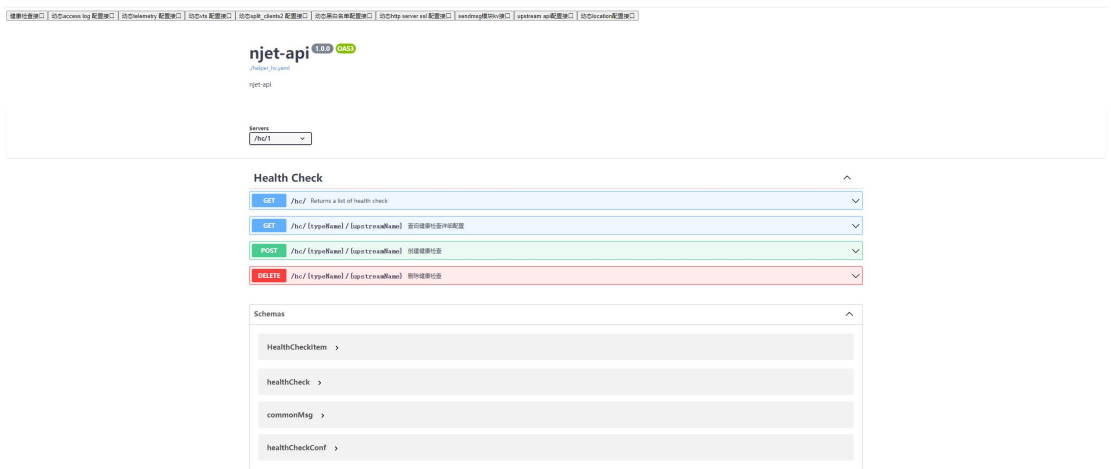
配置后可以打开 Swagger、GUI 等页面。

3.20.1.1 Swagger

url: http://IP:8081/doc/swagger/ (IP 指 njet server 部署所在机器)

可访问各模块 api。

并可通过执行各 api，进行 api 调用。



3.20.1.1.1 使用示例

按照下图示例中，编辑好需要输入的 json 内容。

POST

/hc/{typeName}/{upstreamName}

创建健康检查

创建健康检查

Parameters

Try it out

Name	Description
typeName required (path)	健康检查的类型. Example: HTTP HTTP
upstreamName required (path)	upstream 名称. Example: demo demo

Request body required

application/json

Example Value | Schema

```

{
  "interval": "5s",
  "jitter": "1s",
  "timeout": "5s",
  "port": 80,
  "passes": 1,
  "fails": 1,
  "http": {
    "grpcService": "demo",
    "grpcStatus": 13,
    "uri": "/",
    "header": [
      "string"
    ],
    "body": "string",
    "status": "string"
  },
  "ssl": {
    "enable": false,
    "mtls": false,
    "protocols": "TLSv1 TLSv1.1 TLSv1.2",
    "ciphers": "DEFAULT",
    "name": "upstream name",
    "serverName": false,
    "verify": false,
    "verifyDepth": 1,
    "trustedCertificate": "string",
    "crl": "string"
  }
}

```

点击 execute，使用编辑好的 json 数据调用该 api。

POST

/hc/{typeName}/{upstreamName}

创建健康检查

创建健康检查

Parameters

Cancel

Name	Description
typeName required (path)	健康检查的类型. http
upstreamName required (path)	upstream 名称. backend1

Request body required

application/json

```

{
  "interval": "5s",
  "jitter": "1s",
  "timeout": "5s",
  "port": 80,
  "passes": 1,
  "fails": 1,
  "http": {
    "grpcService": "demo",
    "grpcStatus": 13,
    "uri": "/",
    "header": [
      "string"
    ],
    "body": "string",
    "status": "string"
  },
  "ssl": {
    "enable": false,
    "mtls": false,
    "protocols": "TLSv1 TLSv1.1 TLSv1.2",
    "ciphers": "DEFAULT",
    "name": "upstream name",
    "serverName": false,
    "verify": false,
    "verifyDepth": 1,
    "trustedCertificate": "string",
    "crl": "string"
  }
}

```

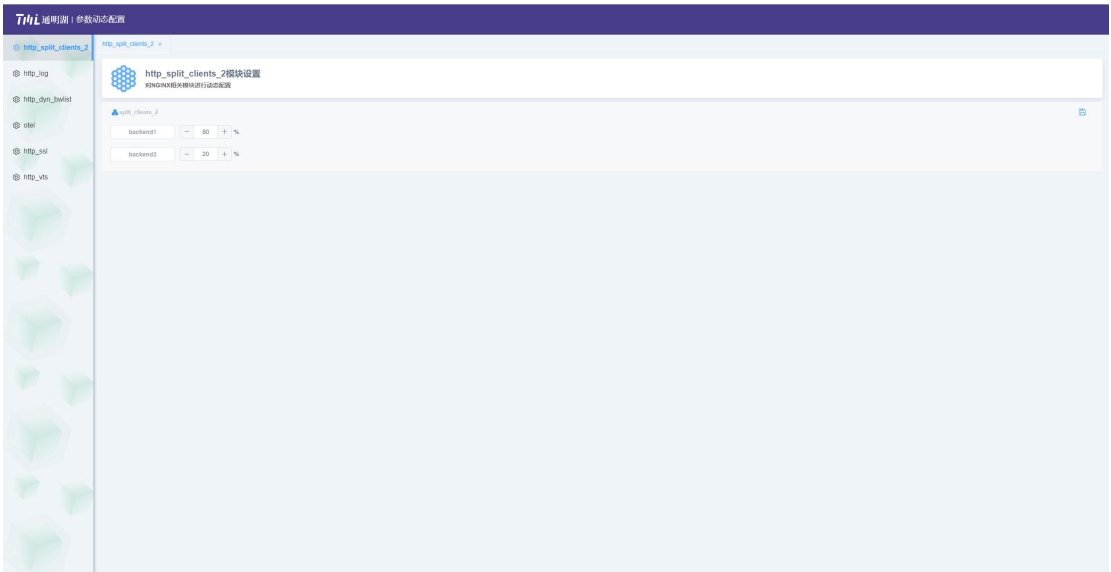
Execute

3.20.1.2 GUI

url: <http://IP:8081/doc/gui/> (IP 指 njet 程序所部署的机器 IP)

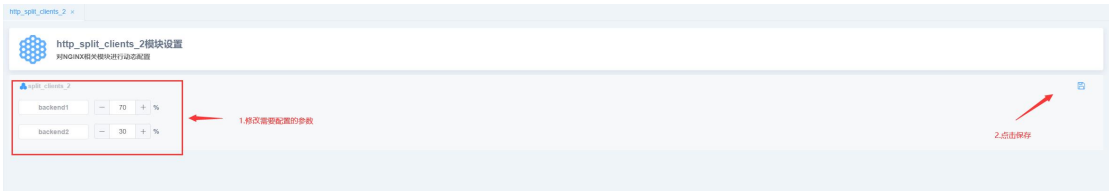
可通过 gui 获取到所有动态模块的配置。

通过修改页面配置，右上角保存，可修改配置。



3.20.1.2.1 使用示例

按照下图示例中，以 http_split_clients_2 为例，修改参数后点击保存，配置即可生效。



3.21 边车

3.21.1 概述

sidecar njet 作为 istio 架构中的数据面，旨在替换原生数据面 sidecar envoy。本文档基于 istio 1.13.3 版本编写，主要针对 sidecar njet 对 istio proxy 功能特性的支持进行了说明。

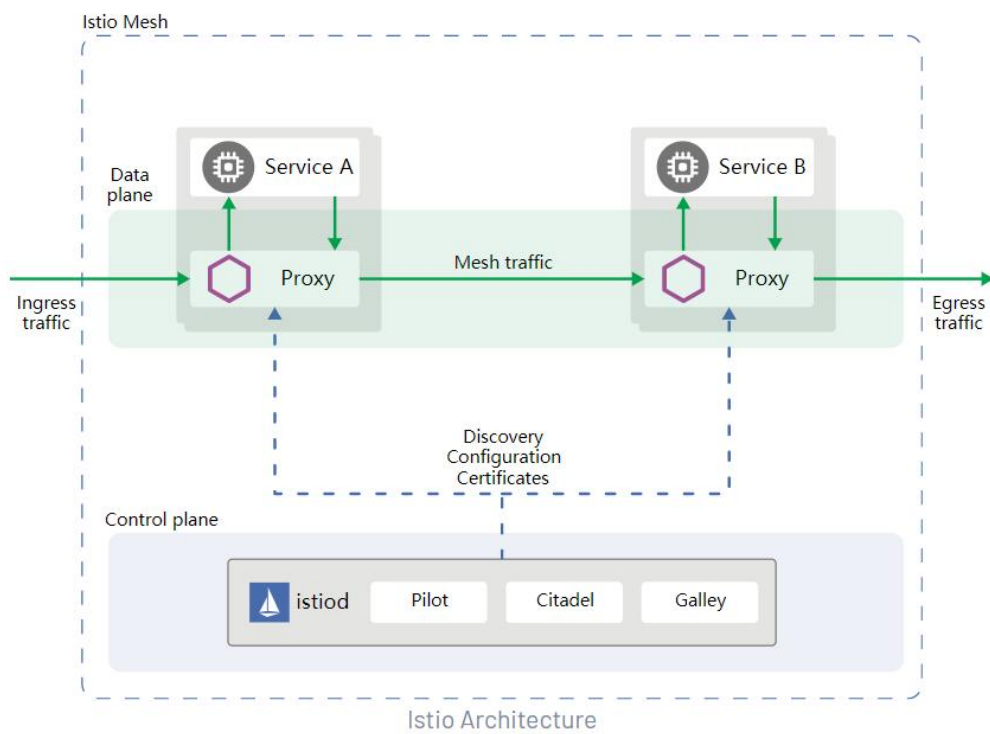
3.21.1.1 istio 架构

Istio 服务网格在逻辑上分为数据平面和控制平面。

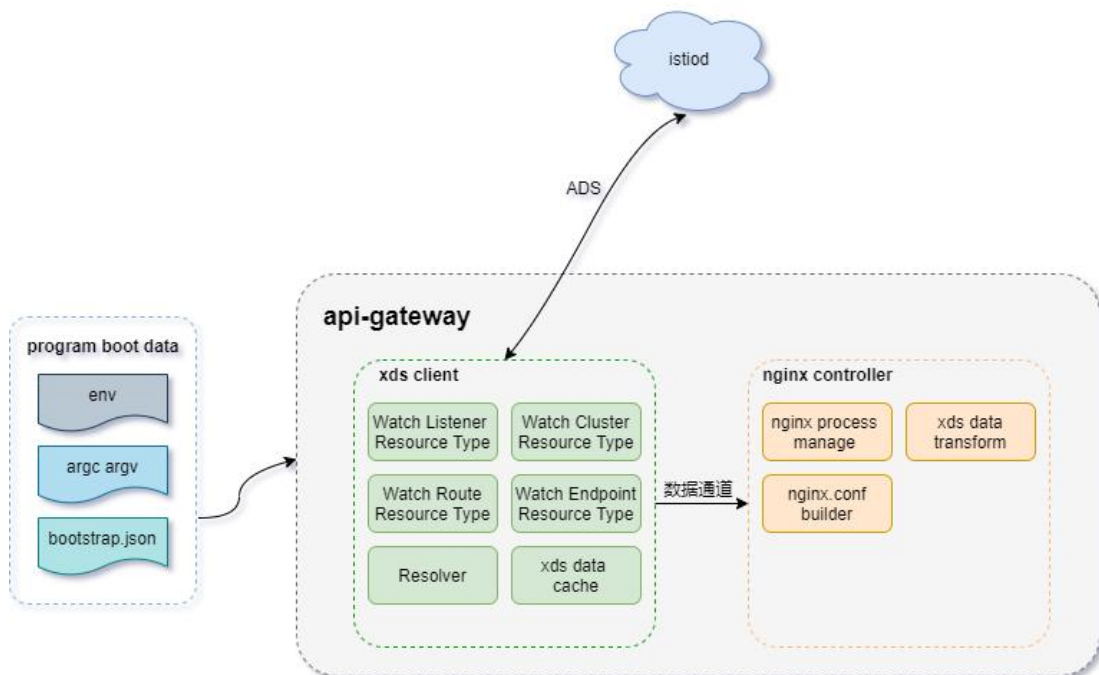
- 数据平面由一组作为 sidecars 部署的智能代理(Envoy)组成。这些代理调解和控制微服务之间的所有网络通信。他们还收集并报告所有网格流量的遥测数据。

- 控制平面对代理进行路由管理和配置。

下图显示了组成每个平面的不同组件：



3.21.1.2 sidecar njet 架构



api-gateway 有两个主要模块：

- xds 客户端

主要与 istiod xds 服务端通信，进行 xds 资源的订阅，基于 grpc-go xds 实现，grpc 支持 SotW ADS

- NJet 控制器
 - 管理 NJet 进程，包括启动，reload
 - xds 资源类型数据进行解析、处理，生成 NJet server、upstream、location 元数据，通过 go templ 技术生成 NJet.conf 文件
 - 集成 lua

3.21.2 镜像构建

通过 git submodule，下载 njet_main 和 lua_module 源码，这两个仓库是 njet proxy 源码仓库。

```
Shell
cd api-gateway
git submodule init
git submodule update
```

执行成功后，源码被下载到 istio-agent/docker/njet_lua_src/

cd 到 istio-agent/docker/njet_lua_src/目录，目录下为 lua 源码与 njet 源码子目录，进入到子目录，切换到对应的分支，确保 njet 源码是您期望的。

```
Shell
cd api-gateway/istio-agent/docker
```

执行

```
Shell
make docker_api_gateway
```

执行上述命令后，sidecar image 开始构建，等待片刻，image 会被构建生成。出现如下提示代表成功。

```

---> 41a58fb4ccf6
Step 19/24 : ENV API_GATEWAY_LOG_SEVERITY_LEVEL=error
---> Running in 611e3ddef179
Removing intermediate container 611e3ddef179
---> 850351843088
Step 20/24 : ENV GRPC_GO_LOG_SEVERITY_LEVEL=info
---> Running in 1553a6c16a9d
Removing intermediate container 1553a6c16a9d
---> 9e9e3d0f080f
Step 21/24 : ENV API_GATEWAY_BOOTSTRAP=$WORKPATH/bootstrap.json
---> Running in d9b1d3a1cdb0
Removing intermediate container d9b1d3a1cdb0
---> 13a1ae03e6d4
Step 22/24 : WORKDIR /
---> Running in 3d2a6901a680
Removing intermediate container 3d2a6901a680
---> 9122c596da76
Step 23/24 : USER 1337:1337
---> Running in a2020973d9b6
Removing intermediate container a2020973d9b6
---> f11097bc860c
Step 24/24 : CMD ["/home/tongminghu/api-gateway/api-gateway"]
---> Running in c18b9c9d52c0
Removing intermediate container c18b9c9d52c0
---> e13f7ab233cf
Successfully built e13f7ab233cf
Successfully tagged tmlake/sidecar-njet-httprounting_ubuntu:v1.0
root@ubuntu:/home/lq/work/tongmingzhiyun/api-gateway/istio-agent/docker#
root@ubuntu:/home/lq/work/tongmingzhiyun/api-gateway/istio-agent/docker#
root@ubuntu:/home/lq/work/tongmingzhiyun/api-gateway/istio-agent/docker#

```

3.21.3 部署 sidecar

使用 sidecar njet 需要定制化部署 istio，自动注入 njet sidecar。定制化部署 istio 需要使用 istio-install 仓库中的文件，此仓库是基于 istio-1.13.3 release 修改的，需要修改 sidecar 镜像名称，修改 sidecar 容器模板。

安装 istio

拉取 istio-install 仓库，修改 default-istio.yaml 文件 values.global.proxy.image 字段，设置为 njet 的镜像名称，必须为包含"/"。执行如下命令安装 istiod：

```

Shell
./istio-1.13.3/bin/istioctl install --manifests=istio-1.13.3/manifests/ -f default-istio.yaml

```

3.21.3.1 注入 sidecar njet

自动注入 njet sidecar(支持 namespace、pod 级别)，在 pod 被创建时被触发。

Pod level

给 pod 打标：sidecar.istio.io/inject: "true"，以 productpage 为例，修改 deploy，增加打标。

Shell

```
kubectl edit deploy productpage-v1
```

```
5 apiVersion: apps/v1
6 kind: Deployment
7 metadata:
8   annotations:
9     deployment.kubernetes.io/revision: "6"
10    kubectl.kubernetes.io/last-applied-configuration: |
11      {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotation
12        etadata":{"labels":{"app":"productpage","version":"v1"},"spec":{"contain
13        volumeMounts":[{"mountPath":"/tmp","name":"tmp"}]}],"serviceName":
14    creationTimestamp: "2022-10-24T06:16:35Z"
15    generation: 6
16    labels:
17      app: productpage
18      version: v1
19    name: productpage-v1
20    namespace: default
21    resourceVersion: "6526320"
22    uid: 2d90c75c-a545-4b01-b691-d861388de033
23 spec:
24   progressDeadlineSeconds: 600
25   replicas: 1
26   revisionHistoryLimit: 10
27   selector:
28     matchLabels:
29       app: productpage
30       version: v1
31   strategy:
32     rollingUpdate:
33       maxSurge: 25%
34       maxUnavailable: 25%
35     type: RollingUpdate
36   template:
37     metadata:
38       creationTimestamp: null
39     labels:
40       app: productpage
41       sidecar.istio.io/inject: "true"
42     spec:
43       containers:
44         - image: docker.io/istio/examples-bookinfo-productpage-v1:1.16.2
45           imagePullPolicy: IfNotPresent
46           name: productpage
47           ports:
```

等待一会

```
root@ubuntu:/home/lq/work/istio# kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-5498c86cf5-6kcpw         1/1     Running   0           100m
productpage-v1-778c664bbd-4ddj9     2/2     Running   0           89m
ratings-v1-b477cf6cf-m4fz4         1/1     Running   0           100m
reviews-v1-79d546878f-bcrqs        1/1     Running   0           6d3h
reviews-v2-548c57f459-tbpj4        1/1     Running   0           6d3h
reviews-v3-6dd79655b9-wmldb        1/1     Running   0           6d3h
```

Namespace level

添加 namespace label,以便后续在此 ns 部署应用时,通知 istio 自动注入 njet sidecar, 下面为 default ns 添加标贴。

Shell

```
kubectl label namespace default istio-injection=enabled
```

查看 ns 标贴

Shell

```
kubectl get ns --show-labels
```

```
root@ubuntu:/home/lq/work/istio# kubectl get ns --show-labels
NAME              STATUS   AGE   LABELS
api-gateway       Active   6d23h  kubernetes.io/metadata.name=api-gateway
default           Active   7d4h   istio-injection=enabled,kubernetes.io/metadata.name=default
istio-system      Active   133m   kubernetes.io/metadata.name=istio-system
kube-node-lease   Active   7d4h   kubernetes.io/metadata.name=kube-node-lease
kube-public       Active   7d4h   kubernetes.io/metadata.name=kube-public
kube-system       Active   7d4h   kubernetes.io/metadata.name=kube-system
kubernetes-dashboard Active   7d4h   addonmanager.kubernetes.io/node=Reconcile,kubernetes.io/metadata.name=kubernetes-dashboard,kubernetes.io/minikube-addons=dashboard
root@ubuntu:/home/lq/work/istio#
```

查看 default ns pod 状态

```
root@ubuntu:/home/lq/work/istio# kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-5498c86cf5-6kcpw        1/1     Running   0           100m
productpage-v1-778c664bbd-4ddj9    2/2     Running   0           89m
ratings-v1-b477cf6cf-m4fz4        1/1     Running   0           100m
reviews-v1-79d546878f-bcrqs        1/1     Running   0           6d3h
reviews-v2-548c57f459-tbpj4        1/1     Running   0           6d3h
reviews-v3-6dd79655b9-wmldb        1/1     Running   0           6d3h
root@ubuntu:/home/lq/work/istio#
```

删除 details pod, 即触发 sidecar 注入

Shell

```
kubectl delete pod details-v1-5498c86cf5-6kcpw
```

查看 default ns pod 状态, details pod 已注入 Njet sidecar

```
root@ubuntu:/home/lq/work/istio# kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-5498c86cf5-m4qwj        2/2     Running   0           8s
productpage-v1-778c664bbd-4ddj9    2/2     Running   0           99m
ratings-v1-b477cf6cf-m4fz4        1/1     Running   0           110m
reviews-v1-79d546878f-bcrqs        1/1     Running   0           6d4h
reviews-v2-548c57f459-tbpj4        1/1     Running   0           6d4h
reviews-v3-6dd79655b9-wmldb        1/1     Running   0           6d4h
root@ubuntu:/home/lq/work/istio#
```

3.21.3.2 卸载 istio

官方使用如下命令卸载 istio,但是部分 cm 和 sa 没有被删除

Shell

```
istioctl x uninstall --purge
```

```

root@ubuntu:/home/lq/work/tongmingzhiyun/istio-install# istioctl x uninstall --purge
All Istio resources will be pruned from the cluster
Proceed? (y/N) y
Removed IstioOperator:istio-system:installed-state.
Removed HorizontalPodAutoscaler:istio-system:istiod.
Removed PodDisruptionBudget:istio-system:istiod.
Removed Deployment:istio-system:istiod.
Removed Service:istio-system:istiod.
Removed ConfigMap:istio-system:istio.
Removed ConfigMap:istio-system:istio-sidecar-injector.
Removed Pod:istio-system:istiod-8488b9bdc7-mxcxj.
Removed ServiceAccount:istio-system:istio-reader-service-account.
Removed ServiceAccount:istio-system:istiod.
Removed ServiceAccount:istio-system:istiod-service-account.
Removed RoleBinding:istio-system:istiod.
Removed RoleBinding:istio-system:istiod-istio-system.
Removed Role:istio-system:istiod.
Removed Role:istio-system:istiod-istio-system.
Removed EnvoyFilter:istio-system:stats-filter-1.11.
Removed EnvoyFilter:istio-system:stats-filter-1.12.
Removed EnvoyFilter:istio-system:stats-filter-1.13.
Removed EnvoyFilter:istio-system:tcp-stats-filter-1.11.
Removed EnvoyFilter:istio-system:tcp-stats-filter-1.12.
Removed EnvoyFilter:istio-system:tcp-stats-filter-1.13.
Removed MutatingWebhookConfiguration::istio-revision-tag-default.
Removed MutatingWebhookConfiguration::istio-sidecar-injector.
Removed ValidatingWebhookConfiguration::istio-validator-istio-system.
Removed ValidatingWebhookConfiguration::istiod-default-validator.
Removed ClusterRole::istio-reader-clusterrole-istio-system.
Removed ClusterRole::istio-reader-istio-system.
Removed ClusterRole::istiod-clusterrole-istio-system.
Removed ClusterRole::istiod-gateway-controller-istio-system.
Removed ClusterRole::istiod-istio-system.
Removed ClusterRoleBinding::istio-reader-clusterrole-istio-system.
Removed ClusterRoleBinding::istio-reader-istio-system.
Removed ClusterRoleBinding::istiod-clusterrole-istio-system.
Removed ClusterRoleBinding::istiod-gateway-controller-istio-system.
Removed ClusterRoleBinding::istiod-istio-system.
Removed CustomResourceDefinition::authorizationpolicies.security.istio.io.
Removed CustomResourceDefinition::destinationrules.networking.istio.io.
Removed CustomResourceDefinition::envoyfilters.networking.istio.io.
Removed CustomResourceDefinition::gateways.networking.istio.io.
Removed CustomResourceDefinition::istiooperators.install.istio.io.
Removed CustomResourceDefinition::peerauthentications.security.istio.io.
Removed CustomResourceDefinition::proxyconfigs.networking.istio.io.
Removed CustomResourceDefinition::requestauthentications.security.istio.io.
Removed CustomResourceDefinition::serviceentries.networking.istio.io.
Removed CustomResourceDefinition::sidecars.networking.istio.io.
Removed CustomResourceDefinition::telemetries.telemetry.istio.io.
Removed CustomResourceDefinition::virtualservices.networking.istio.io.
Removed CustomResourceDefinition::wasmpugins.extensions.istio.io.
Removed CustomResourceDefinition::workloadentries.networking.istio.io.
Removed CustomResourceDefinition::workloadgroups.networking.istio.io.
✓ Uninstall complete

```

删除 istio-system ns，就会删除 istio-system ns 下的所有资源

Shell

```
kubectl delete namespace istio-system
```

3.21.3.3 更新 sidecar njet 镜像

安装 istio，sidecar 成功注入后，当修改了代码重新构建了不同名称的镜像后，需要替换镜像，操作如下：

- 在 k8s 所有节点 load 镜像

Plaintext

docker load -i [xxxxw.tar]

```
[root@mini-master133 /]# ls
bin  dev  home  istio-install  lib  liulu  mnt  proc  run  sidecar-njet-httprouing_ubuntu-20230314.tar  sys  usr
boot  etc  init-config.yaml  kube-flannel.yaml  lib64  media  opt  root  shin  srv  tmp  var
[root@mini-master133 /]# docker load -i sidecar-njet-httprouing_ubuntu-20230314.tar
6a667c68d025: Loading layer [=====] 285.2kB/285.2kB
604b70e6f9ec: Loading layer [=====] 7.168kB/7.168kB
1fd7de440e12: Loading layer [=====] 4.608kB/4.608kB
eb3d024c94f6: Loading layer [=====] 96.23MB/96.23MB
3b6e8a353f72: Loading layer [=====] 3.072kB/3.072kB
cf0baa382f39: Loading layer [=====] 3.584kB/3.584kB
f4296b75f1a8: Loading layer [=====] 46.19MB/46.19MB
83e84e88d32: Loading layer [=====] 134.5MB/134.5MB
Loaded image: tmlake/sidecar-njet-httprouing_ubuntu:v1.0
[root@mini-master133 /]# ls
bin  dev  home  istio-install  lib  liulu  mnt  proc  run  sidecar-njet-httprouing_ubuntu-20230314.tar  sys  usr
boot  etc  init-config.yaml  kube-flannel.yaml  lib64  media  opt  root  shin  srv  tmp  var
[root@mini-master133 /]# docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
tmlake/sidecar-njet-httprouing_ubuntu  v1.0        1227dcf55190     6 hours ago     498MB
tmlake/sidecar-njet-httprouing_ubuntu  v1.0        a7518c5d1202     3 weeks ago     498MB
rancher/mirrored-flannelcni-flannel-cni-plugin  v1.1.2      7a2dcab94698     3 months ago     7.97MB
rancher/mirrored-flannelcni-flannel  v0.20.2     b5c6c9203f83     3 months ago     59.6MB
lqsun666/sidecar-njet-httprouing_ubuntu  v1.0        326db03fb470     4 months ago     476MB
lqsun666/bookinfo-details  v1.2        023110bb0710     5 months ago     149MB
istio/proxyv2  1.13.3      9104024c5e4a     11 months ago     232MB
registry.aliyuncs.com/google_containers/kube-apiserver  v1.23.5     3fc1d62d6587     12 months ago     135MB
registry.aliyuncs.com/google_containers/kube-proxy  v1.23.5     3c53fa8541f9     12 months ago     112MB
registry.aliyuncs.com/google_containers/kube-scheduler  v1.23.5     884d49d6d8c9     12 months ago     53.5MB
registry.aliyuncs.com/google_containers/kube-controller-manager  v1.23.5     b0c9e5e4dbb1     12 months ago     125MB
registry.aliyuncs.com/google_containers/kube-apiserver  v1.23.0     e6bf5ddd4098     15 months ago     135MB
registry.aliyuncs.com/google_containers/kube-scheduler  v1.23.0     56c5af1d00b5     15 months ago     53.5MB
registry.aliyuncs.com/google_containers/kube-controller-manager  v1.23.0     37c6aeb3663b     15 months ago     125MB
registry.aliyuncs.com/google_containers/kube-proxy  v1.23.0     e03484a90585     15 months ago     112MB
registry.aliyuncs.com/google_containers/etcd  3.5.1-0     25f8c7f3da61     16 months ago     293MB
registry.aliyuncs.com/google_containers/coredns  v1.8.6      a4ca41631cc7     17 months ago     46.8MB
registry.aliyuncs.com/google_containers/pause  3.6         6270bb605e12     18 months ago     683kB
istio/examples-bookinfo-reviews-v3  1.16.2      39cff5d782e1     2 years ago     694MB
istio/examples-bookinfo-reviews-v2  1.16.2      39cff5d782e1     2 years ago     694MB
istio/examples-bookinfo-reviews-v1  1.16.2      181be23dc1af     2 years ago     694MB
istio/examples-bookinfo-ratings-v1  1.16.2      99ce598b98cf     2 years ago     161MB
istio/examples-bookinfo-details-v1  1.16.2      edf6b9bea3db     2 years ago     149MB
istio/examples-bookinfo-productpage-v1  1.16.2      7f1e097aad6d     2 years ago     207MB
kennethreitz/httpbin  latest      b138b9264903     4 years ago     534MB
[root@mini-master133 /]#
```

- 查镜像、然后复制该镜像名

Bash

docker images

- istio 通过 ConfigMap 去更新 sidecar 镜像

Shell

```
[root@node151 ~]# kubectl get cm -n istio-system
NAME                                DATA  AGE
istio                               2      121d
istio-ca-root-cert                  1      121d
istio-gateway-deployment-leader     0      121d
istio-gateway-status-leader         0      121d
istio-leader                        0      121d
istio-namespace-controller-election 0      121d
istio-sidecar-injector              2      121d
kube-root-ca.crt                    1      121d
[root@node151 ~]#
```

通过修改名为 istio-sidecar-injector 的 ConfigMap 资源中的配置项，来修改 sidecar 镜像。修改的配置项为 data.values.proxy.image 字段，格式如下：

- 通过如下指令进行编辑，保存；然后删除原有 pod，再重新注入 sidecar（实现新版本功能）

Shell

```
kubectl edit cm -n istio-system istio-sidecar-injector
```

```

    },
    values: |-
    {
      "global": {
        "caAddress": "",
        "caName": "",
        "configCluster": false,
        "configValidation": true,
        "defaultNodeSelector": {},
        "defaultPodDisruptionBudget": {
          "enabled": true
        },
        "defaultResources": {
          "requests": {
            "cpu": "10m"
          }
        },
        "enabled": true,
        "externalIstiod": false,
        "hub": "docker.io/istio",
        "imagePullPolicy": "",
        "imagePullSecrets": [],
        "istioNamespace": "istio-system",
        "istiod": {
          "enableAnalysis": false
        },
        "jwtPolicy": "third-party-jwt",
        "logAsJson": false,
        "logging": {
          "level": "default:info"
        },
        "meshID": "",
        "meshNetworks": {},
        "mountMtlsCerts": false,
        "multiCluster": {
          "clusterName": "",
          "enabled": false
        },
        "namespace": "istio-system",
        "network": "",
        "omitSidecarInjectorConfigMap": false,
        "oneNamespace": false,
        "operatorManageWebhooks": false,
        "pilotCertProvider": "istiod",
        "priorityClassName": "",
        "proxy": {
          "autoInject": "enabled",
          "clusterDomain": "cluster.local",
          "componentLogLevel": "misc:error",
          "enableCoreDump": false,
          "excludeIPRanges": "",
          "excludeInboundPorts": "",
          "excludeOutboundPorts": "",
          "holdApplicationUntilProxyStarts": false,
          "image": "tmllake/sidecar-njet-httprouing ubuntu:v1.0",
          "includeIPRanges": "*",
          "includeInboundPorts": "*",
          "includeOutboundPorts": "",
          "logLevel": "warning",
          "privileged": true,
          "readinessFailureThreshold": 30,
          "readinessInitialDelaySeconds": 1,
          "readinessPeriodSeconds": 2,
          "resources": {
            "limits": {
              "cpu": "2000m",
              "memory": "1024Mi"
            },
            "requests": {
              "cpu": "100m",
              "memory": "128Mi"
            }
          },
          "statusPort": 15020,
          "tracer": "zipkin"
        },
        "proxy_init": {
          "image": "docker.io/istio/proxyv2:1.13.3"
        }
      }
    }
  }
}

```

同时记得把 sidecar 容器的操作权限进行更改：（不然 sidecar 容器内无相关操作权限）

修改 2 个字段为 **true**：privileged、enableCoreDump

```
enableAnalysis": false
},
"jwtPolicy": "third-party-jwt",
"logAsJson": false,
"logging": {
  "level": "default:info"
},
"meshID": "",
"meshNetworks": {},
"mountMtlsCerts": false,
"multiCluster": {
  "clusterName": "",
  "enabled": false
},
"namespace": "istio-system",
"network": "",
"omitSidecarInjectorConfigMap": false,
"oneNamespace": false,
"operatorManageWebhooks": false,
"pilotCertProvider": "istiod",
"priorityClassName": "",
"proxy": {
  "autoInject": "enabled",
  "clusterDomain": "cluster.local",
  "componentLogLevel": "misc:error",
  "enableCoreDump": true,
  "excludeIPRanges": "",
  "excludeInboundPorts": "",
  "excludeOutboundPorts": "",
  "holdApplicationUntilProxyStarts": false,
  "image": "proxyv2",
  "includeIPRanges": "*",
  "includeInboundPorts": "*",
  "includeOutboundPorts": "",
  "logLevel": "warning",
  "privileged": true,
  "readinessFailureThreshold": 30,
  "readinessInitialDelaySeconds": 1,
  "readinessPeriodSeconds": 2,
  "resources": {
    "limits": {
      "cpu": "2000m",
      "memory": "1024Mi"
    },
    "requests": {
      "cpu": "10m",
      "memory": "40Mi"
    }
  },
  "statusPort": 15020,
  "tracer": "zipkin"
```

部署业务容器前，编辑更改 yaml 文件相关字段：（为了保证业务容器具备相关操作的权限）

```
Plaintext
securityContext:      #安全上下文
  privileged: true    #增加此字段&值
  runAsUser: 0        #此处值更改为 0
```

3.21.4 流量管理

3.21.4.1 协议支持

目前支持代理 HTTP1

协议类型	支持情况
HTTP1	支持
HTTP2	不支持
HTTPS	不支持
TCP	不支持
TLS	不支持

3.21.4.2 HTTP 路由

功能描述

通过丰富的路由规则对 HTTP 流量实施细粒度的流量控制。使用路由规则告诉 njet 如何将虚拟服务的流量发送到适当的目的地，路由目的地可以是相同服务的版本，也可以是完全不同的服务。路由目的地支持按权重分流。

高级的路由可以实现灰度发布，发送到灰度版本的流量是基于请求的属性或者是随机选择的，按照请求的百分比进行。

Istio sidecar njet 支持 istio 控制面 HTTP 路由匹配规则，目前支持的路由匹配规则如下表所述：

字段名称	字段类型	Istio njet sidecar 支持情况
uri	支持 exact、prefix、regex	支持 exact、prefix、regex
scheme	支持 exact、prefix、regex	支持 exact
method	支持 exact、prefix、regex	支持 exact
authority	支持 exact、prefix、regex	不支持
headers	支持 exact、prefix、regex	支持 exact
queryParams	exact、regex	支持 exact、regex

ignoreUriCase	uri 大小写忽略 flag(bool), 针对于 exact、prefix	不支持
withoutHeaders	支持 exact、prefix、regex	支持 exact

Istio 控制面 HTTP 路由匹配接口定义参照:

<https://istio.io/v1.13/docs/reference/config/networking/virtual-service/#HTTPMatchRequest>

HTTPMatchRequest

HTTPMatchRequest 不能为空。 **注意：**当指定委托 VirtualService 时，不能设置正则表达式匹配。

字段	类型	描述	必需的
name	string	分配给匹配项的名称。匹配的名称将与父路由的名称连接起来，并将记录在与该路由匹配的请求的访问日志中。	不
uri	String Match	匹配值的 URI 区分大小写，格式如下： exact: "value"精确的字符串匹配 prefix: "value"用于基于前缀的匹配 regex: "value"用于 RE2 样式的基于正则表达式的匹配 (https://github.com/google/re2/wiki/Syntax)。 注意：可以通过标志启用不区分大小写的匹配 ignore_uri_case。	不
scheme	String Match	同上	不
method	String Match	同上	不
authority	String Match	同上	不
headers	map<string, String Match>	标头键必须小写并使用连字符作为分隔符，例如 x-request-id。标头值区分大小写，格式如下： exact: "value"精确的字符串匹配 prefix: "value"用于基于前缀的匹配 regex: "value"用于 RE2 样式的基于正则表达式的匹配 (https://github.com/google/re2/wiki/Syntax)。 如果该值为空并且仅	不

	h>	指定了标头的名称，则会检查标头是否存在。 注意：按键 uri、scheme、method 和 authority 将被忽略。	
port	uint32	指定正在寻址的主机上的端口。许多服务只公开一个端口或使用它们支持的协议标记端口，在这些情况下不需要明确选择端口。	不
sourceLabels	map<string, string>	一个或多个标签，用于限制规则对具有给定标签的源（客户端）工作负载的适用性。如果 VirtualService 具有在顶级字段中指定的网关列表，则它必须包含此字段适用的 gateways 保留网关。mesh	不
gateways	string[]	应应用规则的网关的名称。gatewaysVirtualService 的顶级字段中的网关名称（如果有）被覆盖。网关匹配独立于 sourceLabels。	不
queryParams	map<string, StringMatch>	用于匹配的查询参数。 前任：·对于像“?key=true”这样的查询参数，映射键将是“key”，字符串匹配可以定义为 exact: "true"。·对于像“?key”这样的查询参数，映射键将是“key”，字符串匹配可以定义为 exact: ""。·对于像“?key=123”这样的查询参数，映射键将是“key”，字符串匹配可以定义为 regex: "\d+\$"。 请注意，此配置只会匹配“123”之类的值，而不匹配“a123”或“123a”。 注意： prefix 目前不支持匹配。	不
ignoreUriCase	bool	用于指定 URI 匹配是否不区分大小写的标志。 注意： exact 只有在和 URI 匹配的情况下才会忽略大小写 prefix 。	不
withoutHeaders	map<string, StringMatch>	withoutHeader 与 header 语法相同，但含义相反。如果一个 header 匹配到了 withoutHeader 中的一条匹配规则，则该流量变为不匹配。（与 headers 语法相同，但是有相反的意思，如果在 withoutHeader 中匹配了一个头，则流量变为不匹配。）	不
sourceNamespace	string	源命名空间限制规则对该命名空间中工作负载的适用性。如果 VirtualService 具有在顶级字段中指定的网关列表，则它必须包含此字段适用的 gateways 保留网关。mesh	

[StringMatch](#) 类型属性：

字段	类型	描述
exact	string	精确的字符串匹

		配
prefix	string	前缀匹配
regex	string	正则匹配

HTTPRouteDestination 类型属性

Istio **HTTPRouteDestination** 类型属性配置与 XDS

WeightedCluster(XDS_WeightedCluster)类型属性配置映射关系如下：

字段	类型	描述	必需的	与 XDS 映射关系
destination	Destination	目的地唯一标识请求/连接应转发到的服务实例。	是	生成 cluster name
weight	int32	权重指定要转发到目的地的流量的相对比例。目的地将接收 $\text{weight}/(\text{sum of all weights})$ 请求。如果规则中只有一个目的地，它将接收所有流量。否则，如果 weight 是 0，目的地将不会收到任何流量。	不	clusters[i].weight
headers	Headers	标头操作规则 (istio_Headers 类型属性)	不	

Destination 类型属性：

字段	类型	描述	备注	与 XDS 映射关系
host	string	服务注册中心中的服务名称。		
subset	string	服务子版本名称		
port	PortSelector	指定正在寻址的主机上的端口。如果服务只公开单个端口，则不需要显式地选择端口。		

Headers 类型属性：

Field	Type	Description	Required
request	HeaderOperations	在将请求转发到目标服务之前应用的报头操作规则	No
response	HeaderOperations	在向调用者返回响应之前应用的报头操作规则	No

HeaderOperations 类型属性：

字段	类型	描述	备注	与 XDS 映射关系
set	map<string, string>	用给定的值覆盖由 key 指定的 header		clusters[i].request_headers_to_add 、 clusters[i].response_headers_to_add (使用 header \append 字段)
add	map<string, string>	将给定的值追加到由 key 指定的 header(将创建一个逗号分隔的列表)		clusters[i].request_headers_to_add 、 clusters[i].response_headers_to_add (使用 header \append 字段)
remove	string[]	删除指定的 header		clusters[i].request_headers_to_remove、 clusters[i].response_headers_to_remove



<https://istio.io/v1.13/docs/reference/config/networking/virtual-service/#L4MatchAttributes>

3.21.4.3 Njet HTTP 路由实现

njet 通过 `map+rewrite+split_clients` 实现基于请求内容的高级路由，比如基于请求的 `:header`、`method`、`queryParams`。

3.21.4.3.1 创建一个基于 **header** 的灰度路由

```
YAML
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
```

```

name: reviews
namespace: default
spec:
  hosts:
  - reviews
  http:
  - match:
    - uri:
        exact: /reviews/0
      route:
        - destination:
            host: reviews
            subset: v3
    - match:
      - headers:
          end-user:
            exact: jason
        method:
          exact: get
        uri:
          exact: /reviews/0
      route:
        - destination:
            host: reviews
            subset: v2

```

- 请求中不携带 header->end-user: jason, 请求被路由到 reviews|v3| upstream, 即 reviews v3 版本, v3 会返回 5 颗红色的星星, 在 productpage pod 中的业务容器中通过 curl 访问 reviews:9080 host, 指令如下:

```

Shell
curl http://reviews:9080/reviews/0

```

curl 请求服务时, 主机域名根据实际情况填写, 查看 njet VirtualHOST 配置的域名列表, 即 server 块的 server_name 指令。本测试中名称如下:

```

server_name reviews.default.svc.cluster.local
reviews.default.svc.cluster.local:9080 reviews reviews:9080 reviews.default.svc
reviews.default.svc:9080 reviews.default reviews.default:9080 10.233.213.122
10.233.213.122:9080 ;

```

返回结果如下图所示, 可知, 与期望一致。

```

root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices#
root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices# curl http://reviews:9080/reviews/0
{"id": "0", "reviews": [{"reviewer": "Reviewer1", "text": "An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!", "rating": {"stars": 5, "color": "red"}}, {"reviewer": "Reviewer2", "text": "A mediocre play by Shakespeare. The slapstick humour is refreshing!", "rating": {"stars": 4, "color": "red"}}]}root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices#
root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices#
root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices#

```

- 请求中携带 header->end-user: jason，请求被路由到 reviews|v2| upstream，即 reviews v2 版本，v2 会返回 5 颗黑色的星星，在 productpage pod 中的业务容器通过 curl 访问 reviews:9080 host，指令如下：

Shell

```
curl http://reviews:9080/reviews/0 -H "end-user:jason"
```

返回结果如下图所示，可知，与期望一致。

```

root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices#
root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices# curl http://reviews:9080/reviews/0 -H "end-user:jason"
{"id": "0", "reviews": [{"reviewer": "Reviewer1", "text": "An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!", "rating": {"stars": 5, "color": "black"}}, {"reviewer": "Reviewer2", "text": "A mediocre play by Shakespeare. The slapstick humour is refreshing!", "rating": {"stars": 4, "color": "black"}}]}root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices#
root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices#
root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices#
root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices#

```

3.21.4.3.2 分流

YAML

```

apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: reviews
  namespace: default
spec:
  hosts:
  - reviews
  http:
  - match:
    - method:
        exact: GET
      uri:
        prefix: /productpage
    route:
    - destination:
        host: productpage
        subset: v1
        weight: 80
    - destination:
        host: reviews
        subset: v3
        weight: 20

```

match 指明，当请求方法为 GET，且 uri 有 /productpage 前缀时，请求按权重被路由到 productpage|v1|和 reviews|v3| upstream。

场景：多次请求中（如请求 10 次），8 次会返回 productpage 首页，2 次会返回

404，从而说明按权重分流已生效。

· 在 productpage pod 中的业务容器中通过 curl 访问 reviews:9080 host，指令如下：

```
Shell
curl http://reviews:9080/productpage -v
```

返回结果如下图所示，可知，与期望一致，成功返回 productpage 页面。

```
<!-- empty stars: -->
</font>

</blockquote>
<blockquote>
<p>Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare.</p>
<small>Reviewer2</small>

<font color="red">
<!-- full stars: -->

<span class="glyphicon glyphicon-star"></span>
<span class="glyphicon glyphicon-star"></span>
<span class="glyphicon glyphicon-star"></span>
<span class="glyphicon glyphicon-star"></span>

<!-- empty stars: -->
<span class="glyphicon glyphicon-star-empty"></span>
</font>

</blockquote>

</div>
</div>
</div>

- Latest compiled and minified JavaScript -->
<script src="static/jquery.min.js"></script>
- Latest compiled and minified JavaScript -->
<script src="static/bootstrap/js/bootstrap.min.js"></script>
<script type="text/javascript">
(<#login-modal<#login-modal>).on('shown.bs.modal', function () {
  <#username<#username>).focus();
});
</script>
</body>
```

· 反复执行如下命令，会出现 404（流量到了 reviews|v3|，因为 reviews 服务没有实现/productpage，所以 404 属于正常）。

```
Shell
curl http://reviews:9080/productpage -v
```

由图可知，请求被路由到 reviews v3 版本

```
* Trying 10.233.213.122...
* TCP_NODELAY set
* Expire in 200 ms for 4 (transfer 0x5607ce22ff50)
* Connected to reviews (10.233.213.122) port 9080 (#0)
> GET /productpage HTTP/1.1
> Host: reviews:9080
> User-Agent: curl/7.64.0
> Accept: */*
>
< HTTP/1.1 404 Not Found
< Server: njet/1.23.1
< Date: Wed, 08 Mar 2023 10:12:20 GMT
< Content-Length: 0
< Connection: keep-alive
< X-Powered-By: Servlet/3.1
< Allow: GET,OPTIONS,HEAD
< Content-Language: en-US
< proxy-active-upstream-name: 123
<
* Connection #0 to host reviews left intact
root@productpage-v1-6c594c7b95-qgg8k:/opt/microservices#
```

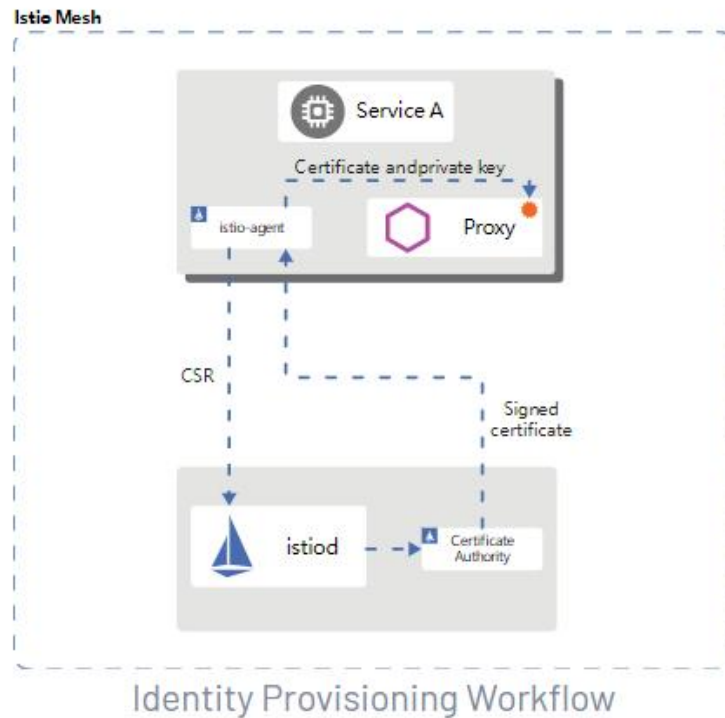
```
TRANSDUCER: The system could not create file /tmp/logs/messages.log because of the following exception: java.security.PrivilegedActionException: java.io.IOException: Permission denied
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*. Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*. Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*. Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*. Please enable FINE/TRACE log level for more details.
TRANSDUCER: The system could not create file /tmp/logs/messages.log because of the following exception: java.security.PrivilegedActionException: java.io.IOException: Permission denied
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*. Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*. Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*. Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*. Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*. Please enable FINE/TRACE log level for more details.
[WARNING] No operation matching request path "/productpage" is found, Relative Path: /productpage, HTTP Method: GET, ContentType: */*, Accept: */*. Please enable FINE/TRACE log level for more details.
```

3.21.5 安全管理

3.21.5.1 证书管理

功能描述

Istio 使用 X.509 证书安全地为每个工作负载提供强身份。与每个 njet(Envoy) proxy 一起运行的 Istio agent 与 istiod 一起工作，实现大规模自动化密钥和证书轮换。下图显示了身份供应流程。



Istio 通过以下流程提供密钥和证书:

- **istiod** 提供 gRPC 服务来接收证书签名请求(csr)。
- 启动时, Istio agent 创建私钥和 CSR, 然后将带有凭证的 CSR 发送给 istiod 进行签名。
- istiod 中的 CA 验证 CSR 中携带的凭据。在成功验证之后, 它签署 CSR 以生成证书。
- 这一步, **njet** 与 **Envoy proxy** 不同, Envoy 支持 SDS 动态获取, Envoy 通过 Envoy SDS API 从同一个容器中的 Istio agent 请求证书和密钥。Istio agent 通过 Envoy SDS API 将从 Istio 接收到的证书和私钥发送给 Envoy。**njet** 的策略是直接在 **njet.conf** 中通过指令配置证书文件及私钥文件, 证书文件及私钥文件会被 Istio agent 写到 **/var/lib/istio/data/**。
- Istio agent 监视证书的过期。对于证书和密钥轮换, 定期重复上述过程。

njet sidecar 目前不支持动态证书加载, 即证书轮换后, **njet** 不会动态加载, 此功能目前未实现

3.21.5.1.1 看证书定期轮换

Istio 默认签发的证书为 24h 有效期, 生成的证书链和根证书及 key 文件被存放到: sidecar 容器的 **/var/lib/istio/data** 目录下, 如下图所示:

/var/lib/istio/data 目录下查看证书:

```
istio-proxy@reviews-v3-697884684f-6jmn:/var/lib/istio/data$ ll
total 12
drwxrwxrwx 2 root root 64 Mar 13 20:02 ./
drwxr-xr-x 3 root root 18 Mar 3 08:02 ../
-rw-r--r-- 1 istio-proxy istio-proxy 2371 Mar 13 20:02 cert-chain.pem
-rw-r--r-- 1 istio-proxy istio-proxy 1679 Mar 13 20:02 key.pem
-rw-r--r-- 1 istio-proxy istio-proxy 1094 Mar 3 20:02 root-cert.pem
istio-proxy@reviews-v3-697884684f-6jmn:/var/lib/istio/data$
```

有效期默认1天, 可更改
有效期10年

1. 可以通过查看文件生成时间, 来确认证书已经轮换, 如下所示:

```
istio-proxy@productpage-v1-7986cd44c7-6wdx:/var/lib/istio/data$ ll
istio-proxy@productpage-v1-7986cd44c7-6wdx:/var/lib/istio/data$ ll
total 12
drwxrwxrwx. 3 root root 82 Feb 21 10:02 ./
drwxr-xr-x. 3 root root 18 Feb 21 10:02 ../
-rw-r--r--. 1 istio-proxy istio-proxy 2397 Feb 21 10:02 cert-chain.pem
-rw-r--r--. 1 istio-proxy istio-proxy 1679 Feb 21 10:02 key.pem
drwxrwxr-x. 4 istio-proxy istio-proxy 110 Nov 16 05:40 nginx-cert/
-rw-r--r--. 1 istio-proxy istio-proxy 1098 Feb 21 10:02 root-cert.pem
istio-proxy@productpage-v1-7986cd44c7-6wdx:/var/lib/istio/data$
istio-proxy@productpage-v1-7986cd44c7-6wdx:/var/lib/istio/data$
```

2. 或者到期后, 通过 openssl 命令查看证书过期时间是否修改, 如下所示:

Shell

```
openssl x509 -noout -text -in /var/lib/istio/data/cert-chain.pem
```

```
istio-proxy@productpage-v1-7986cd44c7-6wdx:/var/lib/istio/data$
istio-proxy@productpage-v1-7986cd44c7-6wdx:/var/lib/istio/data$ openssl x509 -noout -text -in /var/lib/istio/data/cert-chain.pem
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            cf:fe:c2:d8:ff:b4:22:4b:fa:db:ea:b8:62:b3:65:0e
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: O = cluster.local
        Validity
            Not Before: Feb 21 10:00:09 2023 GMT
            Not After : Feb 24 10:02:09 2023 GMT
        Subject: CN = spiffe://cluster.local/ns/default/sa/bookinfo-productpage
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            RSA Public-Key: (2048 bit)
            Modulus:
                00:cd:50:c8:b7:fd:12:7c:64:fa:0b:7c:ca:ae:68:
                14:73:a4:6c:76:61:34:11:35:23:b9:5e:0d:3c:1b:
                5c:72:9f:90:5b:30:90:82:cd:c0:79:af:7d:70:37:
                7e:14:71:31:ca:9b:d2:f7:5b:aa:97:e3:7e:8d:d4:
                62:0d:f6:c7:54:30:74:96:1b:ee:21:be:f4:2a:7d:
                ad:f2:a5:4d:3e:83:9b:5b:45:c9:3a:4d:35:2b:36:
                27:c2:bd:9f:28:9d:d4:1c:3e:22:9f:41:73:c1:b0:
                86:84:74:01:ec:c7:80:aa:6c:9a:30:76:3a:9a:0f:
                5d:00:19:b7:53:c4:67:cd:cd:bb:9b:c6:ed:57:28:
                ee:d0:ab:5d:a6:8e:ee:c5:73:aa:12:d3:d4:d4:20:
                d4:16:56:96:c4:72:66:a8:ae:22:d4:f9:25:ce:d8:
                77:3f:b9:40:62:3a:74:41:23:fc:e3:a9:76:4e:af:
                12:6d:cb:51:e1:0d:ee:07:7e:d0:2a:6c:3a:c4:c5:
                d0:3a:0d:01:4b:37:51:11:02:59:f1:77:e3:32:ea:
                e9:1d:85:87:40:60:1e:9e:45:3f:f1:02:8d:ee:6b:
                a3:a5:25:c6:fa:7a:69:b2:1a:ff:dd:30:e8:5e:44:
                0b:28:26:c4:96:12:46:b5:55:4c:f9:2c:02:67:af:
                50:bd
            Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Key Usage: critical
                Digital Signature, Key Encipherment
            X509v3 Extended Key Usage:
                TLS Web Server Authentication, TLS Web Client Authentication
            X509v3 Basic Constraints: critical
                CA:FALSE
            X509v3 Authority Key Identifier:
                keyid:28:74:62:8E:4A:F3:1C:4A:44:6D:9B:6A:81:20:B1:29:32:53:79:8E
            X509v3 Subject Alternative Name: critical
                URI:spiffe://cluster.local/ns/default/sa/bookinfo-productpage
        Signature Algorithm: sha256WithRSAEncryption
        87:a5:ba:3e:fd:8b:75:a0:2d:fd:26:af:1a:ac:86:65:9b:b6:
        4c:07:d8:b6:2d:1d:ae:69:3a:13:b3:4c:db:f2:74:f1:60:f8:
        25:4c:23:7c:7d:57:0f:06:1c:47:ca:57:61:2c:9f:fe:03:67:
        54:fb:80:22:a7:fc:1c:5d:3c:e7:2b:ba:54:52:30:ed:c2:68:
        7e:87:fb:b3:55:6d:fa:df:c3:0a:a8:a3:a7:60:88:f5:ae:f2:
        da:02:5d:57:a9:76:10:b9:47:ec:c2:37:83:3d:b7:e9:89:67:
        b9:61:41:3b:62:8c:f7:b0:17:87:3e:8a:86:d8:ca:2b:df:17:
        ee:67:66:e2:e6:58:1c:d9:52:ea:dd:81:39:37:a2:46:f9:1c:
        3b:5b:fa:7f:3b:29:2d:62:22:10:a3:5d:4d:c8:66:0e:70:43:
        bb:9e:07:f1:25:21:9c:7c:09:5f:fc:5e:a3:dc:01:3d:3b:6b:
        fb:6c:e1:81:2f:ce:c6:e1:cc:c0:fb:b3:3a:9b:07:cc:1b:ee:
        7b:a5:d6:0e:4d:07:62:e3:96:81:06:15:39:aa:95:db:c0:b3:
        9b:4c:70:fc:21:9c:a4:12:5e:15:ce:7b:09:f3:63:9b:09:d6:
        d0:d6:0d:de:cf:6f:2f:3b:bc:d3:25:f1:b7:85:11:b4:3d:29:
        9a:43:df:ba
istio-proxy@productpage-v1-7986cd44c7-6wdx:/var/lib/istio/data$
```

我修改了证书有效期为3天, 默认为1天

查看 key.pem:

SQL

```
openssl rsa -in /var/lib/istio/data/key.pem -noout -text
```

```
istio-proxy@productpage-v1-5df5d99969-dvgnk:/var/lib/istio/data$ openssl rsa -in ./key.pem -noout -text
RSA Private-Key: (2048 bit, 2 primes)
modulus:
 00:c0:89:38:8c:a5:f6:a1:d2:60:08:50:82:44:a0:
 c0:d8:24:aa:71:3f:50:05:63:89:8f:50:f5:b5:2c:
 50:9f:8c:d7:af:96:42:11:39:10:64:95:bf:46:13:
 bb:3e:d7:2a:c1:ee:57:5d:22:0f:08:fa:ec:99:1a:
 7c:14:2f:7b:f0:db:ac:6d:42:72:23:84:2a:a2:68:
 ef:7f:b8:60:97:f1:8f:65:f0:6a:6e:dc:e2:30:a6:
 19:fc:9f:76:0d:ad:9d:91:d9:54:47:78:de:7b:15:
 ca:23:45:57:8e:21:b8:7b:da:1b:93:d5:5b:d7:f9:
 82:d4:a3:34:2b:9f:16:91:14:9f:0a:9f:7e:0b:ef:
 22:fe:5e:7d:92:f5:57:86:ac:b3:75:72:6a:4c:17:
 21:e3:9f:64:9d:b6:7c:19:0e:4d:5e:86:d0:e1:80:
 72:9c:b4:a2:3a:0a:2e:82:b6:f7:c9:b7:0e:92:d1:
 53:54:2c:17:d0:4c:2f:b4:de:b8:e0:69:cf:4f:11:
 66:82:e1:58:04:ed:78:33:d8:d1:45:70:78:33:f5:
 84:53:87:10:4a:dc:d1:75:d2:5c:17:2f:30:68:a0:
 c8:d0:1a:d9:6f:13:78:0f:b7:1f:ef:2c:d7:b0:d9:
 5c:2f:89:38:0d:5b:ac:17:bf:ff:ff:48:c6:b0:22:
 b2:4b
publicExponent: 65537 (0x10001)
privateExponent:
 00:9b:df:fc:3a:52:3a:e4:33:ef:af:8b:8e:8b:85:
 21:db:5b:83:e9:ff:7a:07:82:d8:61:81:08:0b:cf:
 7e:ce:4d:36:69:5f:5a:c9:9a:cb:5c:c3:a7:81:52:
 84:2b:92:03:79:d2:3e:44:96:48:59:da:dc:25:ef:
 bd:38:92:60:e3:54:d5:b9:f7:2b:74:f9:91:92:bf:
 25:f6:df:0c:c4:b6:ba:83:ab:2a:5f:ea:cf:ec:d6:
 ec:ff:0b:12:86:6c:62:bf:2b:96:01:ef:27:1f:bb:
 3a:55:67:d4:44:ab:a4:32:11:18:76:8d:2b:b8:24:
 b6:f8:f3:d0:4e:e8:93:5a:8c:4f:39:ba:45:c9:32:
 f6:20:e6:8f:84:9d:65:ec:eb:79:11:f8:6a:4d:e8:
 4a:7d:3c:6a:e0:a2:ac:1d:e9:43:e8:0b:ba:83:21:
 fa:2e:c2:22:7f:e9:e4:3c:07:d7:16:a7:14:28:84:
 a9:70:ec:01:b1:65:9c:27:e4:1e:2d:34:cb:3d:1f:
 2e:f8:ea:46:99:75:2c:40:fe:8d:e8:07:f6:ac:0b:
 c4:97:90:90:bc:ae:b5:69:79:e4:74:2d:4b:98:8b:
```

查看 cert-chain.pem 证书:

Bash

```
openssl x509 -noout -text -in /var/lib/istio/data/cert-chain.pem
```

```
istio-proxy@reviews-v3-697884684f-zblmn:/var/lib/istio/data$ openssl x509 -noout -text -in cert-chain.pem
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      9e:ec:5a:ff:6b:cd:a7:4b:e7:e8:8a:62:fd:88:af:42
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: O = cluster.local
    Validity
      Not Before: Mar 30 08:19:59 2023 GMT
      Not After : Mar 31 08:21:59 2023 GMT
    Subject: CN = spiffe://cluster.local/ns/ta/sa/bookinfo-reviews
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      RSA Public-Key: (2048 bit)
      Modulus:
        00:ad:c7:10:08:5e:33:a5:41:f2:70:ef:a7:cd:f4:
        0e:c9:63:f4:7e:d8:9a:27:fe:2e:69:aa:bf:dc:76:
        46:0d:24:0d:dc:c0:22:f9:65:fd:6e:3b:b7:07:36:
        fd:4e:44:b7:31:e7:e7:97:0d:9c:e1:df:64:6d:f0:
        c3:11:8d:85:f5:ec:6b:38:58:a2:7d:89:0a:98:9d:
        e4:3f:69:88:9a:51:59:79:b4:f7:6c:39:b3:27:6a:
        ca:e9:c0:ed:da:c8:39:01:44:7d:88:2d:2e:19:ee:
        bb:3e:8c:f4:7b:1f:46:c9:87:f2:e6:93:66:76:db:
        64:7f:29:c4:0f:05:86:bf:19:39:e0:b7:8f:47:b1:
        b1:79:a4:6f:4e:34:06:c3:01:5c:ee:73:8c:f9:d2:
        c6:b8:64:74:01:75:40:6d:a1:ec:14:a9:0c:37:ce:
        3d:3c:5c:54:75:6d:89:18:ba:bc:31:5c:69:6e:a2:
        1e:3b:41:90:9b:a7:3d:5f:24:85:98:1d:c8:a8:30:
        af:0b:e4:62:31:50:41:59:0a:8d:e2:bb:a6:bb:f7:
        ca:91:5e:92:ed:b6:1c:f5:a5:73:c5:1b:13:62:8d:
        72:53:c5:65:62:de:4f:3e:2b:37:76:b1:6d:bf:20:
        3e:b9:ba:44:6d:7f:51:e0:f9:8a:b4:ae:30:4c:1b:
        1e:19
      Exponent: 65537 (0x10001)
  X509v3 extensions:
    X509v3 Key Usage: critical
      Digital Signature, Key Encipherment
    X509v3 Extended Key Usage:
```

查看 root-cert.pem 证书:

Bash

```
openssl x509 -noout -text -in /var/lib/istio/data/root-cert.pem
```

```
stio-proxy@reviews-v3-697884684f-zblmn:/var/lib/istio/data$ openssl x509 -noout -text -in root-cert.pem
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            49:f8:ce:2f:dd:7b:fa:e0:3b:65:8c:83:55:fe:f4:8c
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: O = cluster.local
        Validity
            Not Before: Jan 16 05:56:44 2023 GMT
            Not After : Jan 13 05:56:44 2033 GMT
        Subject: O = cluster.local
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            RSA Public-Key: (2048 bit)
            Modulus:
                00:a2:d7:b4:30:60:b2:0d:d6:00:30:e8:f6:89:2b:
                c4:b9:80:a9:61:b3:e5:fb:0c:be:21:fd:cd:27:6e:
                b7:e7:47:5e:de:c5:e5:3d:d8:2d:fb:5d:1e:24:17:
                a5:9d:aa:07:58:bb:e0:80:03:93:61:02:1b:4e:63:
                43:dc:83:07:e3:6f:25:0f:7e:e7:8f:1e:ed:7c:62:
                0a:9a:5c:5c:bf:83:80:8e:34:eb:8c:c4:44:71:e4:
                68:de:62:99:8f:72:82:62:65:ea:bf:84:b5:27:cc:
                40:a6:9a:a9:21:8e:af:89:d3:52:bd:07:03:3e:fa:
                12:5f:7a:b8:ad:d4:2b:6e:a4:dd:45:d2:42:07:23:
                02:a8:e2:0b:6f:eb:1e:db:03:ed:55:0a:7e:8e:5e:
                4a:a4:01:a6:cb:ca:0f:99:b7:be:f8:74:f5:53:cb:
                27:95:48:d0:7d:6d:a3:d4:62:c3:60:81:49:f3:90:
                0a:cc:e8:d9:34:e9:0e:a5:a8:fa:7b:6a:db:4a:b5:
                49:74:1f:79:93:ab:c6:8b:c5:92:a4:b9:61:07:08:
                a0:c0:a6:a8:f0:48:df:eb:42:17:6d:9b:61:3b:a1:
                1f:12:a6:19:3c:83:3e:73:7a:03:e2:3f:3e:77:47:
                23:5f:16:fa:a1:d9:92:4c:0d:4a:fe:fb:c6:d5:34:
                2f:95
```

3.21.5.1.2 修改签发证书的有效期时间

通过修改 istio-sidecar-injector k8s ConfigMap 资源中的配置项, 来改变证书的有效期日期。

配置项为: data.config.templates.sidecar.spec.containers 里的 istio-proxy 容器中的 env 变量 **SECRET_TTL** 的 value, 格式为 1h (小时)、10m(分钟)。修改后 delete pod, 重新注入 sidecar, 即可生效。

执行下面指令修改配置(记得修改后保存):

Shell

```
kubectl edit cm -n istio-system istio-sidecar-injector
```

在 (data.config.templates.sidecar.spec.containers 下的 istio-proxy 容器的 env 变量) 字段, 配置文件增加如下几个字段:

修改后 delete pod, 重新注入 sidecar, 即可生效

```

image: "{{ annotation .ObjectMeta `sidecar.istio.io/proxyImage` .Values.global.proxy.image }}"
{{- else }}
image: "{{ .ProxyImage }}"
{{- end }}
ports:
- containerPort: 15090
  protocol: TCP
  name: http-envoy-prom
{{- if .Values.global.proxy.lifecycle }}
lifecycle:
  {{ toYaml .Values.global.proxy.lifecycle | indent 6 }}
{{- else if $holdProxy }}
lifecycle:
  postStart:
    exec:
      command:
      - pilot-agent
      - wait
{{- end }}
env:
{{- if eq (env "PILOT_ENABLE_INBOUND_PASSTHROUGH" "true") "false" }}
- name: REWRITE_PROBE_LEGACY_LOCALHOST_DESTINATION
  value: "true"
{{- end }}
- name: JWT_POLICY
  value: {{ .Values.global.jwtPolicy }}
- name: PILOT_CERT_PROVIDER
  value: {{ .Values.global.pilotCertProvider }}
- name: OUTPUT_CERTS
  value: /var/lib/istio/data
- name: SECRET_TTL
  value: 24h
- name: CA_ADDR
{{- if .Values.global.caAddress }}

```

3.21.5.2 服务到服务的双向认证(service-to-service mTLS)

功能描述

用于 service-to-service 身份验证，客户端与服务端服务彼此认证对端身份信息，验证通过后，按照双方配置的安全通信策略进行数据交互。对于传输层身份认证 (transport authentication)，Istio 提供 mTLS 作为全栈解决方案，无需更改服务代码即可启用。mTLS 方案提供以下特性：

- 为每个服务提供表示其角色的强标识，以支持跨集群和云的互操作性。
- service-to-service 的安全通信。
- 提供一个 key 管理系统，使 key 和证书的生成、分发、轮换自动化。

3.21.5.2.1 客户端 TLS 策略

客户端服务，那些发送请求的服务，是有责任的遵循需要的身份认证机制。你可以通过 DestinationRule CR 中的 **ClientTLSSettings** 去设置与上游服务连接的安全策略。客户端服务根据用户创建的 DestinationRule CR 策略，给予与上游服务通信的约束，例如明文通信或者密文通信，如果密文通信，验证上游服务身份，来判断其是否可信。

在 istio mTLS 方案中，客户端 TLS 策略支持如下模式：

ClientTLSSettings.TLSmode

TLS connection mode

Name	Description
DISABLE	Do not setup a TLS connection to the upstream endpoint.
SIMPLE	Originate a TLS connection to the upstream endpoint.
MUTUAL	Secure connections to the upstream using mutual TLS by presenting client certificates for authentication.
ISTIO_MUTUAL	Secure connections to the upstream using mutual TLS by presenting client certificates for authentication. Compared to Mutual mode, this mode uses certificates generated automatically by Istio for mTLS authentication. When this mode is used, all other fields in ClientTLSSettings should be empty.

istio 通过 k8s DestinationRule CR 配置客户端 TLS 策略。

njet 客户端 TLS 支持情况如下：

njet 客户端 TLS 配置使用 proxy_ssl 相关指令进行配置。

njet 客户端 TLS 默认配置为 ISTIO_MUTUAL 模式。

TLS mode	njet sidecar	描述
DISABLE	支持	明文传输
SIMPLE	非 sidecar 特性	密文传输
MUTUAL	非 sidecar 特性	密文传输
ISTIO_MUTUAL	支持	密文传输

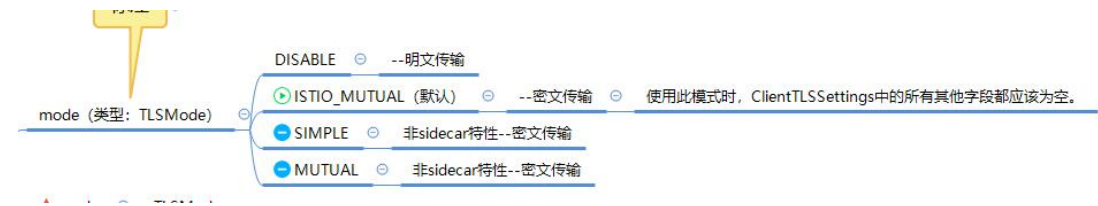
客户端 3 种安全策略级别：

服务级别：(destination-level)

服务子集级别：(subset-level)

端口级别：(port-level)

客户端 4 种策略模式：



详细介绍参照 <https://istio.io/v1.13/docs/reference/config/networking/destination-rule/#ClientTLSSettings>

3.21.5.2.2 服务端 TLS 策略

Istio 通过 PeerAuthentication CR 实现服务端对客户端请求的认证策略配置。

PeerAuthentication 定义流量怎样通往 sidecar，支持服务端口级别的安全认证。在 sidecar 场景，接受请求的服务端为 15006 listener(根据原始目的地 port，转发流量，并且给予端口级别的认证策略)，因为 istio iptables rule 将流量重定向到了 15006，所以所有策略会被应用到此 listener。

在 istio mTLS 方案中，服务端 TLS 策略支持如下模式：

PeerAuthentication.MutualTLS.Mode

Name	Description
UNSET	Inherit from parent, if has one. Otherwise treated as PERMISSIVE.
DISABLE	Connection is not tunneled.
PERMISSIVE	Connection can be either plaintext or mTLS tunnel.
STRICT	Connection is an mTLS tunnel (TLS with client cert must be presented).

istio 通过 k8s PeerAuthentication CR 配置服务端 TLS 策略。

njet 服务端 TLS 支持情况如下：

njet 服务端 TLS 配置在 stream 块中的 server 块使用 njetmesh_port_mode 相关指令进行配置。示例如下所示：

```
server {
    listen 15006 ssl;
    #listen 15006;
    proxy_pass $njetmesh_dest;
    proxy_bind 127.0.0.6;
    njetmesh_dest on;

    #ssl_certificate /var/lib/istio/data/cert-chain.pem;
    #ssl_certificate_key /var/lib/istio/data/key.pem;
    #ssl_client_certificate /var/lib/istio/data/root-cert.pem;
    #ssl_verify_client on;
    #ssl_verify_depth 10;

    njetmesh_port_mode 15006 DISABLE;
    njetmesh_port_mode 200 PERMISSIVE;
    njetmesh_port_mode 9080 DISABLE;

    ssl_certificate /var/lib/istio/data/cert-chain.pem;
    ssl_certificate_key /var/lib/istio/data/key.pem;
    ssl_verify_client on;
    ssl_client_certificate /var/lib/istio/data/root-cert.pem;
    ssl_verify_depth 10;
}
```

或是通过部署 pa 资源配置/修改配置:

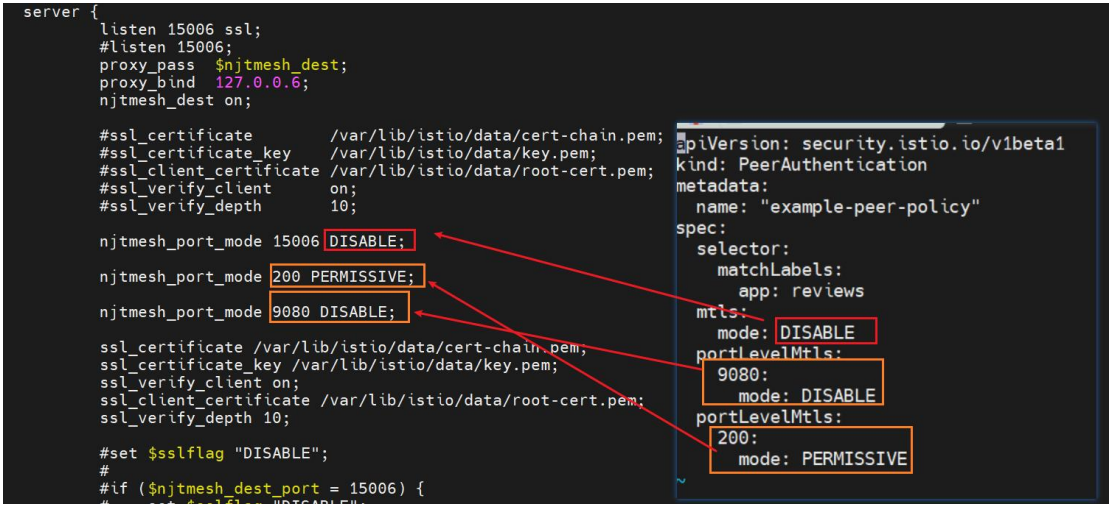
通过下面指令部署 pa:

```
Shell
kubectl apply -f [PeerAuthentication-workload-v-port.yaml]
```

```
[root@mini-master133 pa]# kubectl apply -f PeerAuthentication-workload-v-port.yaml
peerauthentication.security.istio.io/example-peer-policy created
```

通过下面指令修改 pa:

```
Shell
kubectl edit pa reviews
```



njet 服务端 TLS 默认配置为 **PERMISSIVE** 模式。

TLS mode	njet sidecar	描述
UNSET	支持	继承父作用域的模式，如果最高级别(mesh 级别)没有设置，默认使用 PERMISSIVE
DISABLE	支持	只接受明文
PERMISSIVE	支持	可以接受密文和明文
STRICT	支持	严格的 mTLS，不接受明文

服务端策略包括 3 个级别：mesh、ns、workload

istio 安装在哪个空间下，哪个空间就是 mesh 级别：如 istio-system 就是 mesh 级别策略

ex: 以下的 pa 是 mesh 级别的

```
[root@mini-master133 pa]# kubectl get pa -n istio-system
NAME                                MODE    AGE
example-peer-policy-mesh            UNSET   22s
[root@mini-master133 pa]#
```

mesh 影响范围：所有 ns

ns 影响范围：特定 ns 下的所有服务

workload 影响范围：指定的某个服务

服务端策略的 4 种模式：



详细介绍参照 https://istio.io/v1.13/docs/reference/config/security/peer_authentication/

灰度发布：不同 upstream 不同安全策略

使用“3.20.4.2 HTTP 路由”创建一个基于 header 的灰度路由“章节的路由规则。我们修改 reviews|v2| upstream 为明文传输。

通过下面指令修改：

```
Shell
kubectl edit dr reviews
```

```

#
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1alpha3","kind":"DestinationRule","name":"v3"}}
  creationTimestamp: "2022-11-07T09:44:04Z"
  generation: 18
  name: reviews
  namespace: default
  resourceVersion: "25746101"
  uid: 32a0bcc5-d614-48b0-82f5-f179507331be
spec:
  host: reviews
  subsets:
    - labels:
        version: v1
        name: v1
    - labels:
        version: v2
        name: v2
        trafficPolicy:
          tls:
            mode: DISABLE
    - labels:
        version: v3
        name: v3

```

进入 sidecar 容器内查看 njet 配置: `cat /etc/njet/njet.conf`

reviews|v2| upstream 为明文传输

```

location = /internal_location_matches_0_match_0 {
    #log_by_lua_block {
    #balancer.log()
    #}
    set $balancer_ewma_score -1;
    set $proxy_upstream_name "outbound|9080|v2|reviews.default.svc.cluster.local";
    set $proxy_host $proxy_upstream_name;
    set $pass_access_scheme $scheme;

    set $best_http_host $http_host;
    set $pass_server_port $server_port;
    set $pass_port $pass_server_port;

    set $proxy_alternative_upstream_name "";
    set $proxy_active_upstream_name "123";
    set $original_dst_host "original_dst_host";

    proxy_set_header Host $best_http_host;
    proxy_set_header x-njet-original-dst-host $original_dst_host;
    proxy_set_header proxy_active_upstream_name $proxy_active_upstream_name;

    # Allow websocket connections
    #proxy_set_header Upgrade $http_upgrade;
    #proxy_set_header Connection $connection_upgrade;

    proxy_http_version 1.1;

    #configuration.call()
    content_by_lua_block {
        balancer.get_proxy_active_upstream_name()
    }

    header_filter_by_lua_block {
        njet.header.proxy_active_upstream_name = njet.var["proxy_active_upstream_name"]
        local proxy_active_upstream_name = njet.var["proxy_active_upstream_name"]
        njet.log(njet.INFO, "Test proxy_active_upstream_name:" , proxy_active_upstream_name)
    }

    if ($flag = "1") {
        proxy_pass http://outbound|9080|v2|reviews.default.svc.cluster.local$request_uri;
    }
    if ($flag = "0") {
        proxy_pass http://upstream-default-backend;
    }

    #set $uri_key $proxy_active_upstream_name;
    #return 200 'njet.var["proxy_active_upstream_name"]';
}

```

reviews|v3| upstream 为密文传输

```

location = /internal_location_matches_0_default {
    #log_by_lua_block {
    #balancer.log()
    #}
    set $balancer_ewma_score -1;
    set $proxy_upstream_name "outbound|9080|v3|reviews.default.svc.cluster.local";
    set $proxy_host $proxy_upstream_name;
    set $pass_access_scheme $scheme;

    set $best_http_host $http_host;
    set $pass_server_port $server_port;
    set $pass_port $pass_server_port;

    set $proxy_alternative_upstream_name "";
    set $proxy_active_upstream_name "123";
    set $original_dst_host "original_dst_host";

    proxy_set_header Host $best_http_host;
    proxy_set_header x-njet-original-dst-host $original_dst_host;

    proxy_set_header proxy_active_upstream_name $proxy_active_upstream_name;

    # Allow websocket connections
    #proxy_set_header Upgrade $http_upgrade;
    #proxy_set_header Connection $connection_upgrade;

    proxy_http_version 1.1;

    #configuration.call()
    content_by_lua_block {
        balancer.get_proxy_active_upstream_name()
    }

    header_filter_by_lua_block {
        njet.header.proxy_active_upstream_name = njet.var["proxy_active_upstream_name"]
        local proxy_active_upstream_name = njet.var["proxy_active_upstream_name"]
        njet.log(njet.INFO, "test proxy_active_upstream_name:" , proxy_active_upstream_name)
    }

    if ($flag = "1") {
        proxy_pass https://outbound|9080|v3|reviews.default.svc.cluster.local$request_uri;
    }
    if ($flag = "0") {
        proxy_pass http://upstream-default-backend;
    }

    proxy_ssl_certificate /var/lib/istio/data/cert-chain.pem;
    proxy_ssl_certificate_key /var/lib/istio/data/key.pem;
    proxy_ssl_verify on;
    proxy_ssl_trusted_certificate /var/lib/istio/data/root-cert.pem;
    proxy_ssl_verify_depth 10;
    proxy_ssl_name "spiffe://cluster.local/ns/default/sa/bookinfo-reviews";

    #set $uri_key $proxy_active_upstream_name;

    #return 200 'njet.var["proxy_active_upstream_name"]';
}

```

Njet location path 实现如下:

```

map $http_end_user $vs_reviews_default_svc_cluster_local_9080_matches_0_match_0_cond_0 {
    "json" $vs_reviews_default_svc_cluster_local_9080_matches_0_match_0_cond_1;
    default 0;
}

map $request_method $vs_reviews_default_svc_cluster_local_9080_matches_0_match_0_cond_1 {
    "get" 1;
    default 0;
}

map $vs_reviews_default_svc_cluster_local_9080_matches_0_match_0_cond_0 $vs_reviews_default_svc_cluster_local_9080_matches_0 {
    ~^1 /internal_location_matches_0_match_0;
    default /internal_location_matches_0_default;
}

```

```

}

location = /reviews/0 {
    rewrite ^ $vs_reviews_default_svc_cluster_local_9080_matches_0 last;
}

location ~ /details {
    rewrite ^ $vs_reviews_default_svc_cluster_local_9080_matches_1 last;
}

location ~ /productpage {
    rewrite ^ $vs_reviews_default_svc_cluster_local_9080_matches_2 last;
}

location = /ratings/0 {
    rewrite ^ $vs_reviews_default_svc_cluster_local_9080_matches_3 last;
}

```

- 请求中不携带 header->end-user: jason, 请求被路由到 reviews|v3| upstream, 即 reviews v3 版本, v3 会返回红色的星星, 在 productpage pod 中的业务容器中通过 curl 访问 reviews:9080 host, 指令如下:

```

Shell
curl http://reviews:9080/reviews/0

```

返回结果如下图所示, 可知, 与期望一致。

```

root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices# curl http://reviews:9080/reviews/0
{"id": "0", "reviewer": [{"reviewer": "Reviewer1", "text": "An extremely entertaining play by Shakespeare. The slapstick humor is refreshing", "rating": {"stars": 5, "color": "red"}, {"reviewer": "Reviewer2", "text": "Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare.", "rating": {"stars": 4, "color": "red"}}], "productpage": "v1-6c594c7b95-qgq8k"}
root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices#

```

在 reviews v3 pod 中 sidecar 容器抓包

```

istio-proxy@reviews-v3-6f979bc579-pztl:/#
istio-proxy@reviews-v3-6f979bc579-pztl:/# sudo tcpdump -i eth0 port 9080 -n -nn -X -w ./reviews-v3.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
^C36 packets captured
36 packets received by filter
0 packets dropped by kernel
istio-proxy@reviews-v3-6f979bc579-pztl:/# pwd
/
istio-proxy@reviews-v3-6f979bc579-pztl:/# ls

```

```

[roote@node151 ~]$ istio#
[roote@node151 ~]$ kubectl get pod -owide
NAME                                READY    STATUS    RESTARTS   AGE    IP              NODE             NOMINATED NODE    READINESS GATES
details-v1-6bf94bc757-2vwk2         2/2     Running   0           9h     10.234.190.197 node153           <none>             <none>
netchecker-agent-6jcbf               1/1     Running   3 (144d ago) 149d   10.234.190.40  node153           <none>             <none>
netchecker-agent-hostnet-sxxxx       1/1     Running   3 (144d ago) 149d   192.168.40.153 node153           <none>             <none>
netchecker-agent-hostnet-tj69b       1/1     Running   7 (133d ago) 149d   192.168.40.151 node151           <none>             <none>
netchecker-agent-z2pmj               1/1     Running   7 (133d ago) 149d   10.234.96.60  node151           <none>             <none>
productpage-v1-6c594c7b95-qgq8k     2/2     Running   0           9h     10.234.190.196 node153           <none>             <none>
ratings-v1-546f59d6f6-gljkp         2/2     Running   0           9h     10.234.96.131 node151           <none>             <none>
reviews-v1-5bc56fbbd8-qxtzp         2/2     Running   0           9h     10.234.190.194 node153           <none>             <none>
reviews-v2-5cc6595bb6-blxv9         2/2     Running   0           9h     10.234.96.132 node151           <none>             <none>
reviews-v2-ratings-bf9569d7-bm6dj   3/3     Running   0           8h     10.234.190.198 node153           <none>             <none>
reviews-v3-6f979bc579-pztl         2/2     Running   0           9h     10.234.190.195 node153           <none>             <none>
[roote@node151 ~]$ istio#

```

reviews-v3.pcap

文件(F) 编辑(E) 视图(V) 网络(N) 统计(S) 电话(T) 无线(W) 工具(O) 帮助(H)

应用显示过滤器: <<ctrl-/>>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.234.190.196	10.234.190.195	TCP	74	49978 → 9080 [SYN] Seq=0 Win=28800 Len=0 MSS=1440 SACK_PERM=1 TSval=3934749982 TSecr=0 WS=128
2	0.000032	10.234.190.195	10.234.190.196	TCP	66	9080 → 49978 [SYN, ACK] Seq=0 Ack=1 Win=28560 Len=0 MSS=1440 SACK_PERM=1 TSval=3934749982 TSecr=3934749
3	0.000056	10.234.190.196	10.234.190.195	TCP	66	49978 → 9080 [ACK] Seq=1 Ack=1 Win=28800 Len=0 TSval=3934749982 TSecr=3934749982
4	0.000375	10.234.190.196	10.234.190.195	TLV1.2	254	Client Hello
5	0.000391	10.234.190.195	10.234.190.196	TCP	66	9080 → 49978 [ACK] Seq=1 Ack=199 Win=29696 Len=0 TSval=3934749983 TSecr=3934749983
6	0.002015	10.234.190.195	10.234.190.196	TLV1.2	2240	Server Hello, Certificate, Server Key Exchange, Certificate Request, Server Hello Done
7	0.002055	10.234.190.196	10.234.190.195	TCP	66	49978 → 9080 [ACK] Seq=199 Ack=2175 Win=33152 Len=0 TSval=3934749984 TSecr=3934749984
8	0.004408	10.234.190.196	10.234.190.195	TLV1.2	2132	Certificate, Client Key Exchange, Certificate Verify, Change Cipher Spec, Encrypted Handshake Message
9	0.004421	10.234.190.195	10.234.190.196	TCP	66	9080 → 49978 [ACK] Seq=2175 Ack=2265 Win=33792 Len=0 TSval=3934749987 TSecr=3934749987
10	0.005052	10.234.190.195	10.234.190.196	TLV1.2	1236	New Session Ticket, Change Cipher Spec, Encrypted Handshake Message
11	0.005242	10.234.190.196	10.234.190.195	TLV1.2	277	Application Data
12	0.020813	10.234.190.195	10.234.96.131	TCP	74	49038 → 9080 [SYN] Seq=0 Win=28800 Len=0 MSS=1440 SACK_PERM=1 TSval=3934750003 TSecr=0 WS=128
13	0.021134	10.234.96.131	10.234.190.195	TCP	74	9080 → 49038 [SYN, ACK] Seq=0 Ack=1 Win=28560 Len=0 MSS=1440 SACK_PERM=1 TSval=2906976249 TSecr=3934750
14	0.021153	10.234.190.195	10.234.96.131	TCP	66	49038 → 9080 [ACK] Seq=1 Ack=1 Win=28800 Len=0 TSval=3934750004 TSecr=2906976249
15	0.021315	10.234.190.195	10.234.96.131	TLV1.2	1400	Client Hello
16	0.021435	10.234.96.131	10.234.190.195	TCP	66	9080 → 49038 [ACK] Seq=1 Ack=1335 Win=31488 Len=0 TSval=2906976249 TSecr=3934750004
17	0.021857	10.234.96.131	10.234.190.195	TLV1.2	207	Server Hello, Change Cipher Spec, Encrypted Handshake Message
18	0.021864	10.234.190.195	10.234.96.131	TCP	66	49038 → 9080 [ACK] Seq=1335 Ack=142 Win=29952 Len=0 TSval=3934750004 TSecr=2906976249
19	0.022000	10.234.190.195	10.234.96.131	TLV1.2	117	Change Cipher Spec, Encrypted Handshake Message
20	0.022044	10.234.190.195	10.234.96.131	TLV1.2	333	Application Data
21	0.022350	10.234.96.131	10.234.190.195	TCP	66	9080 → 49038 [ACK] Seq=142 Ack=1653 Win=34176 Len=0 TSval=2906976250 TSecr=3934750004
22	0.023703	10.234.96.131	10.234.190.195	TLV1.2	289	Application Data
23	0.023856	10.234.190.195	10.234.96.131	TLV1.2	97	Encrypted Alert
24	0.023909	10.234.190.195	10.234.96.131	TCP	66	49038 → 9080 [FIN, ACK] Seq=1684 Ack=365 Win=30976 Len=0 TSval=3934750006 TSecr=2906976251

由图可知，路由到 reviews v3 服务的请求发送的是密文。

- 请求中携带 header->end-user: jason，请求被路由到 reviews[v2] upstream，即 reviews v2 版本，v2 会返回黑色的星星，在 productpage pod 中的业务容器中通过 curl 访问 reviews:9080 host，指令如下：

Shell

```
curl http://reviews:9080/reviews/0 -H "end-user:jason"
```

返回结果如下图所示，可知，与期望一致。

```
root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices# curl http://reviews:9080/reviews/0 -H "end-user:jason"
{"id": "0", "reviewer": "Reviewer1", "text": "An offm...y entertaining play by Shakespeare. The slapstick humor is refreshing!", "rating": {"stars": 5, "color": "black"}, {"reviewer": "Reviewer2", "text": "Absolutely fun and entertaining th when compared to other plays by Shakespeare.", "rating": {"stars": 4, "color": "black"}}root@productpage-v1-6c594c7b95-qgq8k:/opt/microservices#
```

在 reviews v2 pod 中中 sidecar 容器抓包

```
[root@node151 istio]#
[root@node151 istio]# kubectl get pod -owide
NAME                                READY    STATUS    RESTARTS   AGE    IP              NODE    NOMINATED NODE    READINESS GATES
details-v1-6bf04bc757-2vwk2        1/1     Running   0           9h     10.234.190.197  node153 <none>          <none>
netchecker-agent-6jcbf             1/1     Running   3 (144d ago)  149d   10.234.190.40  node153 <none>          <none>
netchecker-agent-hostnet-skxxx     1/1     Running   3 (144d ago)  149d   192.168.40.153 node153 <none>          <none>
netchecker-agent-hostnet-tj9b8     1/1     Running   7 (133d ago)  149d   192.168.40.151 node151 <none>          <none>
netchecker-agent-z2pmj             1/1     Running   7 (133d ago)  149d   10.234.96.60   node151 <none>          <none>
productpage-v1-6c594c7b95-qgq8k    2/2     Running   0           9h     10.234.190.196  node153 <none>          <none>
ratings-v1-546f59d6f6-gljkp        2/2     Running   0           9h     10.234.96.131  node151 <none>          <none>
reviews-v1-5bc56fbbd8-qxtzp        2/2     Running   0           9h     10.234.190.194  node153 <none>          <none>
reviews-v2-5cc6595bb6-blxv9       2/2     Running   0           9h     10.234.96.132  node151 <none>          <none>
reviews-v2-ratings-bf9569d7-bm6dj 3/3     Running   0           8h     10.234.190.198  node153 <none>          <none>
reviews-v3-6f979bc579-pzltl       2/2     Running   0           9h     10.234.190.195  node153 <none>          <none>
[root@node151 istio]#
```

```
istio-proxy@reviews-v2-5cc6595bb6-blxv9:/# sudo tcpdump port 9080 -i eth0 -n -vv -mm -X
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes

11:35:13.623172 IP (tos 0x0, ttl 62, id 38018, offset 0, flags [DF], proto TCP (6), length 60)
10.234.190.196.37818 > 10.234.96.132.9080: Flags [S], cksum 0xf88e (correct), seq 1924169979, win 28800, options [mss 1440,sackOK,TS val 3935310969 ecr 0,nop,wscale 7], length 0
0x0000: 4500 003c 9482 4000 3e06 731d 0aea bec4 E...<.>S.....
0x0010: 0aea 6084 93ba 2378 72b0 80fc 89f5 a835 ...<#>.....S
0x0020: a002 7080 f88e 0000 0204 05a0 0402 080a ..p.....
0x0030: ea90 1479 0000 0000 0103 0307 .....M.?..y...
11:35:13.623220 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60)
10.234.96.132.9080 > 10.234.190.196.37818: Flags [S.], cksum 0x354b (incorrect -> 0xab6b), seq 2314577972, ack 1924169980, win 28560, options [mss 1440,sackOK,TS val 2907537215 ecr 3935310969
0x0000: 4500 003c 0000 4000 4090 05a0 0aea 6084 E...<.>S.....
0x0010: 0aea bec4 2378 93ba 2378 80fc 89f5 a835 ...<#>.....S
0x0020: a012 6f90 354b 0000 0204 05a0 0402 080a ..o.SK.....
0x0030: ad4d 7f3f ea90 1479 0103 0307 .....M.?..y...
11:35:13.623321 IP (tos 0x0, ttl 62, id 38019, offset 0, flags [DF], proto TCP (6), length 52)
10.234.190.196.37818 > 10.234.96.132.9080: Flags [P.], cksum 0x381e (correct), seq 1, ack 1, win 225, options [nop,nop,TS val 3935310969 ecr 2907537215], length 0
0x0000: 4500 0034 9483 4000 3e06 7324 0aea bec4 E...<.>S$....
0x0010: 0aea 6084 93ba 2378 72b0 80fc 89f5 a835 ...<#>.....S
0x0020: 8010 00e1 381e 0000 0101 080a ea90 1479 ...8.....y
0x0030: ad4d 7f3f .....M.?
11:35:13.623362 IP (tos 0x0, ttl 62, id 38020, offset 0, flags [DF], proto TCP (6), length 251)
10.234.190.196.37818 > 10.234.96.132.9080: Flags [P.], cksum 0x04be (correct), seq 1:200, ack 1, win 225, options [nop,nop,TS val 3935310969 ecr 2907537215], length 199
0x0000: 4500 00fb 9484 4000 3e06 723c 0aea bec4 E...<.>P.....
0x0010: 0aea 6084 93ba 2378 72b0 80fc 89f5 a835 ...<#>.....S
0x0020: 8018 00e1 04be 0000 0101 080a ea90 1479 .....P.....
0x0030: ad4d 7f3f 4745 5420 2f72 6576 6965 7773 /GET /reviews
0x0040: 2f30 2080 5454 302f 3126 310d 04a0 6f73 / HTTP/1.1, host
0x0050: 743a 2072 6576 6965 7773 3a39 3038 306d t:reviews:9080.
0x0060: 0a78 2080 6a65 742d 6f72 6967 696e 616c :x-njet-original
0x0070: 2d64 7574 2d68 6f73 743a 206f 7269 6969 :dst-host:origi
0x0080: 6e61 6c5f 6473 745f 686f 7374 0d0a 7072 na_dst.host.pr
0x0090: 6f78 765f 6163 7460 7605 5f75 7073 7432 :oxy_active.opst
0x00a0: 6561 6d5f 6e61 6d55 3a20 3132 330d 0a43 :e_name:1f3.c
0x00b0: 6f6e 6e65 6374 696f 6e3a 2063 6c6f 7365 connection:close
0x00c0: 0d0a 5573 6572 2d41 6f65 6a74 3a20 6375 :User-Agent:cu
0x00d0: 726c 2f37 2e36 342e 300d 0a41 6363 6570 :/2.564.0.0xosop
0x00e0: 743a 202a 2f2a 0d0a 656a 642d 7573 6572 t:/*.*.end-user
0x00f0: 3a20 6a61 736f 6e6d 0a0d 0a .....:jason
11:35:13.623373 IP (tos 0x0, ttl 64, id 55050, offset 0, flags [DF], proto TCP (6), length 52)
10.234.96.132.9080 > 10.234.190.196.37818: Flags [P.], cksum 0x3543 (incorrect -> 0x3750), seq 1, ack 200, win 232, options [nop,nop,TS val 2907537215 ecr 3935310969], length 0
0x0000: 4500 0034 070a 4000 4090 2e9d 0aea 6084 E...<.>S.....
0x0010: 0aea bec4 2378 93ba 89f5 a835 72b0 81c3 ...<#>.....Sf...
0x0020: 8010 00e8 3543 0000 0101 080a ad4d 7f3f ...5C.....M.?
0x0030: ea90 1479 .....M.?..y...
```

reviews-v2.pcap

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.234.190.196	10.234.96.132	TCP	74	38256 -> 9080 [SYN] Seq=0 Win=28800 Len=0 MSS=1440 SACK_PERM=1 TSval=3935412096 TSecr=0 WS=128
2	0.000061	10.234.96.132	10.234.190.196	TCP	74	9080 -> 38256 [SYN, ACK] Seq=0 Ack=1 Win=28560 Len=0 MSS=1440 SACK_PERM=1 TSval=2907638342 TSecr=3935412096 WS=128
3	0.000252	10.234.190.196	10.234.96.132	TCP	66	38256 -> 9080 [ACK] Seq=1 Ack=1 Win=28800 Len=0 TSval=3935412096 TSecr=2907638342
4	0.000257	10.234.190.196	10.234.96.132	HTTP	265	GET /reviews/0 HTTP/1.1
5	0.000302	10.234.96.132	10.234.190.196	YCP	66	9080 -> 38256 [ACK] Seq=1 Ack=200 Win=29696 Len=0 TSval=2907638342 TSecr=3935412096
6	0.021046	10.234.96.132	10.234.96.131	YCP	74	53398 -> 9080 [SYN] Seq=0 Win=28800 Len=0 MSS=1440 SACK_PERM=1 TSval=2907638363 TSecr=0 WS=128
7	0.021112	10.234.96.131	10.234.96.132	TCP	74	9080 -> 53398 [SYN, ACK] Seq=0 Ack=1 Win=28560 Len=0 MSS=1440 SACK_PERM=1 TSval=2907638363 TSecr=2907638363 WS=128
8	0.021120	10.234.96.132	10.234.96.131	TCP	66	53398 -> 9080 [ACK] Seq=1 Ack=1 Win=28800 Len=0 TSval=2907638363 TSecr=2907638363

由图可知，路由到 reviews v2 服务的请求发送的是明文。

4. 标准配置文件

4.1 njet.conf

```
Bash
helper broker modules/njt_helper_broker_module.so
conf/mqtt.conf;
helper ctrl modules/njt_helper_ctrl_module.so
conf/ctrl.conf;

load_module modules/njt_http_split_clients_2_module.so;
load_module modules/njt_agent_dynlog_module.so;
load_module modules/njt_http_location_module.so;
load_module modules/njt_http_dyn_bwlist_module.so;

load_module modules/njt_otel_module.so;
load_module modules/njt_agent_dyn_otel_module.so;
load_module modules/njt_dyn_ssl_module.so;
load_module modules/njt_http_vtsc_module.so;

cluster_name helper;
node_name node1;

worker_processes auto;
user root;

error_log logs/error.log info;
pid logs/njet.pid;

events {
    worker_connections 1024;
}

http {
    dyn_kv_conf conf/iot-work.conf;
    include mime.types;
    include conf.d/*.conf;
}
```

4.2 ctrl.conf

Bash

```
load_module modules/njt_http_sendmsg_module.so;
load_module modules/njt_ctrl_config_api_module.so;
load_module modules/njt_helper_health_check_module.so;
load_module modules/njt_http_upstream_api_module.so;
load_module modules/njt_http_location_api_module.so;
load_module modules/njt_doc_module.so;
load_module modules/njt_http_vtsd_module.so;
```

```
cluster_name helper;
node_name node1;
```

```
error_log logs/error-ctrl.log info;
```

```
events {
    worker_connections 1024;
}
```

```
http {

    dyn_sendmsg_conf conf/iot-ctrl.conf;
    config_req_pool_size 1000;
    access_log logs/access_ctrl.log combined;
    include mime.types;
```

```
    server {
        listen      8081;

        location /kv {
            dyn_sendmsg_kv;
        }

        location /config {
            config_api;
        }

        location /hc {
            health_check_api;
        }

        location /api {
            api write=on;
        }
    }
}
```

```

        location /doc {
            doc_api;
        }
        location /dyn_loc {
            dyn_location_api;
        }

        location /metrics {
            vhost_traffic_status_display;
            vhost_traffic_status_display_format html;
        }
    }
}

```

4.3 mime.types

Bash

```

types {
    text/html                html htm
    shtml;
    text/css                 css;
    text/xml                 xml;
    image/gif                gif;
    image/jpeg               jpeg jpg;
    application/javascript   js;
    application/atom+xml     atom;
    application/rss+xml      rss;

    text/mathml              mml;
    text/plain               txt;
    text/vnd.sun.j2me.app-descriptor jad;
    text/vnd.wap.wml         wml;
    text/x-component         htc;

    image/avif               avif;
    image/png                png;
    image/svg+xml            svg svgz;
    image/tiff               tif tiff;
    image/vnd.wap.wbmp       wbmp;
    image/webp               webp;

```

image/x-icon	ico;
image/x-jng	jng;
image/x-ms-bmp	bmp;
font/woff	woff;
font/woff2	woff2;
application/java-archive	jar war
ear;	
application/json	json;
application/mac-binhex40	hqx;
application/msword	doc;
application/pdf	pdf;
application/postscript	ps eps
ai;	
application/rtf	rtf;
application/vnd.apple.mpegurl	m3u8;
application/vnd.google-earth.kml+xml	kml;
application/vnd.google-earth.kmz	kmz;
application/vnd.ms-excel	xls;
application/vnd.ms-fontobject	eot;
application/vnd.ms-powerpoint	ppt;
application/vnd.oasis.opendocument.graphics	odg;
application/vnd.oasis.opendocument.presentation	odp;
application/vnd.oasis.opendocument.spreadsheet	ods;
application/vnd.oasis.opendocument.text	odt;
application/vnd.openxmlformats-officedocument.presentationml.presentation	
	pptx;
application/vnd.openxmlformats-officedocument.spreadsheetml.sheet	
	xlsx;
application/vnd.openxmlformats-officedocument.wordprocessingml.document	
	docx;
application/vnd.wap.wmlc	wmlc;
application/wasm	wasm;
application/x-7z-compressed	7z;
application/x-cocoa	cco;
application/x-java-archive-diff	
jardiff;	
application/x-java-jnlp-file	jnlp;
application/x-makeself	run;
application/x-perl	pl pm;

```

        application/x-pilot                prc pdb;
        application/x-rar-compressed        rar;
        application/x-redhat-package-manager rpm;
        application/x-sea                  sea;
        application/x-shockwave-flash      swf;
        application/x-stuffit              sit;
        application/x-tcl                  tcl tk;
        application/x-x509-ca-cert          der pem
crt;
        application/x-xpinstall             xpi;
        application/xhtml+xml              xhtml;
        application/xspf+xml               xspf;
        application/zip                    zip;

        application/octet-stream           bin exe
dll;
        application/octet-stream           deb;
        application/octet-stream           dmg;
        application/octet-stream           iso img;
        application/octet-stream           msi msp
msm;

        audio/midi                        mid midi
kar;
        audio/mpeg                        mp3;
        audio/ogg                         ogg;
        audio/x-m4a                       m4a;
        audio/x-realaudio                  ra;

        video/3gpp                        3gpp 3gp;
        video/mp2t                         ts;
        video/mp4                          mp4;
        video/mpeg                         mpeg mpg;
        video/quicktime                    mov;
        video/webm                         webm;
        video/x-flv                       flv;
        video/x-m4v                       m4v;
        video/x-mng                       mng;
        video/x-ms-asf                     asx asf;
        video/x-ms-wmv                     wmv;
        video/x-msvideo                    avi;
}

```

4.4 **iot-ctrl.conf**

配置为空即可

```
Bash
```

4.5 **iot-work.conf**

配置为空即可

```
Bash
```

4.6 **mqtt.conf**

配置为空即可

```
Bash
```

4.7 **otel.conf**

```
Bash
exporter = "otlp"
processor = "batch"

[exporters.otlp]
#collector server address
# Alternatively the OTEL_EXPORTER_OTLP_ENDPOINT environment
variable can also be used.
host = "192.168.40.136"
port = 4317

[processors.batch]
max_queue_size = 2048
schedule_delay_millis = 5000
max_export_batch_size = 512

[service]
# Can also be set by the OTEL_SERVICE_NAME environment variable.
```

```
name = "njet-proxy" # Opentelemetry resource name

[sampler]
name = "AlwaysOn" # Also: AlwaysOff, TraceIdRatioBased
ratio = 0.1
parent_based = false
```

4.8 重要：部署所需额外配置

提测单里除了有流水线出的包 + 还要有如下三个文件

- 1) 将 `njt_otel_module.so`、`njt_agent_dyn_otel_module.so` 放到与其他动态模块同目录，即放置在 `modules` 目录下。
- 2) `dependlib.tar` 解压后，启动 `njet` 时将该 `dependlib` 目录加入到 `LD_LIBRARY_PATH` 环境变量里
eg:

```
export LD_LIBRARY_PATH=`pwd`/dependlib:$LD_LIBRARY_PATH
vi /etc/profile
source /etc/profile
```

[`dependlib.tar`]

[`njt_otel_module.so`]

[`njt_agent_dyn_otel_module.so`]

5. 常见问题

1. reload 后，需要等待 3 秒左右，动态查询接口，才可用。

```

[root@master110 conf.d]# ../../sbin/njet -p /home/njet_1.2 -c conf/njet.conf -s reload
[root@master110 conf.d]# curl 127.0.0.1:8081/config/1/config/http_log |jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
0    0    0    0    0    0    0  0 0 0 0 0 0 0 0 0 0 0 0 0 0
curl: (7) Failed to connect to 127.0.0.1 port 8081 after 0 ms: Couldn't connect to server
[root@master110 conf.d]# curl 127.0.0.1:8081/config/1/config/http_log |jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
0    0    0    0    0    0    0  0 0 0 0 0 0 0 0 0 0 0 0 0 0
curl: (7) Failed to connect to 127.0.0.1 port 8081 after 0 ms: Couldn't connect to server
[root@master110 conf.d]# curl 127.0.0.1:8081/config/1/config/http_log |jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
0    0    0    0    0    0    0  0 0 0 0 0 0 0 0 0 0 0 0 0 0
curl: (7) Failed to connect to 127.0.0.1 port 8081 after 0 ms: Couldn't connect to server
[root@master110 conf.d]# curl 127.0.0.1:8081/config/1/config/http_log |jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
100 1052 100 1052    0    0 15762    0 0 0 0 0 0 0 0 0 0 0 0 0 0
{
  "servers": [
    {
      "listens": [
        "0.0.0.0:80"
      ],
      "serverNames": [
        "www.crm.com"
      ],
      "locations": [
        {
          "location": "/cust",
          "accessLogOn": true,
          "accessLogs": [
            {
              "path": "logs/accesslog_2345.conf",
              "formatName": "combined"
            }
          ]
        }
      ],
      "location": "/cust"
    }
  ]
}

```

2. 访问不存在的接口，会报 500 的错误。

```

[root@master110 conf.d]# ../../sbin/njet -p /home/njet_1.2 -c conf/njet.conf -s reload
[root@master110 conf.d]# ../../sbin/njet -p /home/njet_1.2 -c conf/njet.conf -s reload
[root@master110 conf.d]# curl 127.0.0.1:8081/config/1/config/split_clients_2 |jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
100    41  100    41    0    0    20      0  0:00:02  0:00:02  --:--:--   20
{
  "code": 500,
  "msg": "rpc timeout"
}
[root@master110 conf.d]# cd ../logs
-bash: cd: ../logs: 没有那个文件或目录
[root@master110 conf.d]# cd ../../logs
[root@master110 logs]# ll
总用量 164
-rw-r--r-- 1 root root 5243 3月 29 13:45 access_ctrl.log
-rw-r--r-- 1 root root 0 3月 29 10:39 access.log
-rw-r--r-- 1 root root 0 3月 29 10:39 accesslog_1.conf
-rw-r--r-- 1 root root 0 3月 29 10:50 accesslog_2345.conf
-rw-r--r-- 1 root root 0 3月 29 10:39 accesslog_2.conf
-rw-r--r-- 1 root root 20779 3月 29 13:45 error-ctrl.log
-rw-r--r-- 1 root root 74459 3月 29 13:45 error.log
-rw-r--r-- 1 root root 0 3月 29 10:39 mosquitto.log
-rw-r--r-- 1 root root 0 3月 29 11:14 new.log
-rw-r--r-- 1 root root 6 3月 29 11:00 njet.pid
-rw-rw-rw- 1 root root 0 3月 29 11:00 sendmsg_1
-rw-rw-rw- 1 root root 0 3月 29 10:39 sendmsg_2
-rw-rw-rw- 1 root root 0 3月 29 10:39 work_iot_0
[root@master110 logs]# tail error.log
2023/03/29 13:45:18 [notice] 23757#0: start copilot process 4075
2023/03/29 13:45:18 [notice] 23757#0: start copilot process 4076
2023/03/29 13:45:18 [notice] 23757#0: signal 29 (SIGIO) received
2023/03/29 13:45:18 [error] 4070#0: Event fired!,try connect again, 000060820010EE40
2023/03/29 13:45:18 [notice] 4070#0: connect to broker failed:4
2023/03/29 13:45:18 [notice] 4076#0: helper ctrl started
2023/03/29 13:45:18 [notice] 4076#0: ctrl reconfiguring
2023/03/29 13:45:19 [error] 4070#0: Event fired!,try connect again, 000060820010EE40
2023/03/29 13:45:19 [notice] 4070#0: connect ok, register io
2023/03/29 13:45:19 [notice] 4070#0: kv module connect ok, register socket:5
[root@master110 logs]# curl 127.0.0.1:8081/config/1/config/split_clients_2 |jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
100    41  100    41    0    0    20      0  0:00:02  0:00:02  --:--:--   20
{
  "code": 500,
  "msg": "rpc timeout"
}
[root@master110 logs]#

```

3. ctrl.conf 中自定义了接口 location, curl 命令可以正常使用, 但是 doc gui 无法使用。

njt_ctrl 标准配置文件: 需要包含如下 location:

- /api 提供 upstream member 维护
- /config 提供声明式 api
- /kv 提供 kv 维护接口
- /dyn_loc 提供动态 location 配置接口
- /hc 主动健康检查接口
- /metrics 指标输出接口
- /doc/swagger openapi 接口
- /doc/gui 简单界面维护

6. 附录

健康检查 API 说明

openapi: 3.0.3

info:

title: njet-api

description: njet-api

version: 1.0.0

servers:

- url: 'http://192.168.40.140/api/1'

paths:

/hc/:

get:

tags:

- Health Check

summary: Returns a list of health check

description: Returns a list of health check.

operationId: getHealthCheckUpstreams

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/HealthCheck'

'500':

description: 服务器内部错误

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

/hc/{typeName}/{upstreamName}:

parameters:

- name: typeName

type: string

in: path

description: 健康检查的类型.

required: true

example: HTTP

enum:

- HTTP

- name: upstreamName

type: string

in: path

description: upstream 名称.

required: true

example: demo

get:

- tags:
 - Health Check
- summary: 查询健康检查详细配置
- description: 查询健康检查详细配置
- operationId: getHealthCheckConf
- responses:
 - '200':
 - description: Success
 - content:
 - application/json:
 - schema:
 - \$ref: '#/components/schemas/HealthCheckConf'
 - '404':
 - description: upstream 或者 type 未找到
 - content:
 - application/json:
 - schema:
 - \$ref: '#/components/schemas/CommonMsg'
 - '500':
 - description: 服务器内部错误
 - content:
 - application/json:
 - schema:
 - \$ref: '#/components/schemas/CommonMsg'

post:

- tags:
 - Health Check
- summary: 创建健康检查
- description: 创建健康检查
- operationId: postHealthCheck
- parameters:
 - in: body
 - name: postHealthCheck
 - description: 创建新的健康检查
 - required: true
 - schema:
 - \$ref: '#/components/schemas/HealthCheckConf'
- responses:
 - '200':
 - description: Success
 - content:
 - application/json:

```

        schema:
          $ref: '#/components/schemas/CommonMsg'
'400':
  description: 请求 body 错误
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/CommonMsg'
'404':
  description: 健康检查类型或 upstream 未找到
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/CommonMsg'
'409':
  description: 健康检查已经定义过
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/CommonMsg'
'500':
  description: 服务器内部错误
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/CommonMsg'

delete:
  tags:
    - Health Check
  summary: 删除健康检查
  description: 删除健康检查
  operationId: deleteHealthCheck
  responses:
    '200':
      description: Success
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/CommonMsg'
    '404':
      description: 健康检查不存在
      content:
        application/json:

```

schema:
\$ref: '#/components/schemas/CommonMsg'

###DEFINITIONS

components:

schemas:

HealthCheckItem:

type: object

properties:

upstream:

type: string

description: name of upstream.

type:

type: string

description: type of health check.

HealthCheck:

title: healthCheck

description: health Check 简易信息

type: array

items:

\$ref: '#/components/schemas/HealthCheckItem'

CommonMsg:

title: commonMsg

description: 公共提示信息

type: object

properties:

code:

type: string

msg:

type: string

HealthCheckConf:

title: healthCheckConf

description: health Check 完整配置

type: object

properties:

interval:

type: string

description: 健康检查间隔

default: 5s

example: 5s

jitter:

type: string

description: 健康检查间隔

default: 0s
example: 1s

timeout:
type: string
description: 健康检查超时时间
default: 5s
example: 5s

port:
type: number
description: 健康检查的端口
example: 80
minimum: 1
maximum: 65535

passes:
type: number
description: 健康检查转变为健康，需要检查连续通过的次数
minimum: 1
default: 1

fails:
type: number
description: 健康检查转变为不健康，需要健康检查连续不通过的次数
minimum: 1
default: 1

http:
type: object
description: HTTP upstream 健康检查相关配置

properties:

- grpcService:
type: string
description: grpc 健康检查请求的 service 名称
default: demo
- grpcStatus:
type: number
default: 13
description: grpc 健康检查标记为通过的状态码
- uri:
type: string
description: http 健康检查请求的 uri
default: /
- header:
type: array
description: http 健康检查校验的 header
- items:
type: string

```

        description:
body:
  type: string
  description: http 健康检查检查 body 的内容是否满足表达式
status:
  type: string
  description: http 健康检查检查返回状态码是否满足
ssl:
  type: object
  description: ssl 客户端相关配置
  properties:
    enable:
      type: boolean
      description: 是否开启 https 健康检查
      default: false
    protocols:
      type: string
      description: 指定健康检查使用的 ssl 协议版本，支持的版本：[SSLv2]
[SSLv3] [TLSv1] [TLSv1.1] [TLSv1.2] [TLSv1.3]
      default: TLSv1 TLSv1.1 TLSv1.2
    ciphers:
      type: string
      description: 对健康检查请求开启 ciphers，ciphers 以 openssl 标准
格式配置。"openssl ciphers" 指令可以查看 ciphers 列表
      default: DEFAULT
    name:
      type: string
      description: 设置健康检查请求时携带的 Host header
      default: upstream name
    serverName:
      type: boolean
      description: 启用或禁用通过 TLS 服务器名称指示扩展（SNI，RFC
6066）传递服务器名称。
      default: false
    verify:
      type: boolean
      description: 启用或禁用对连接的 HTTPS 服务器证书的验证。（需配置
trustedCertificate）
      default: false
    verifyDepth:
      type: number
      description: HTTPS 服务器证书链中设置验证深度。
      default: 1
      minimum: 1

```

trustedCertificate:
 type: string
 description: 指定一个具有 PEM 格式的受信任 CA 证书，用于验证指定 HTTPS 服务器的证书。

crl:
 type: string
 description: 以 PEM 格式指定具有撤销证书（CRL）的文件，用于验证代理 HTTPS 服务器的证书。

certificate:
 type: string
 description: 指定一个带有 PEM 格式证书的文件，用于对 HTTPS 服务器进行身份验证(客户端证书)。

certificateKey:
 type: string
 description: 指定一个带有 PEM 格式的密钥的文件，用于对 HTTPS 服务器进行身份验证(客户端证书 key)。

6.1 动态 access log 配置 API 说明

openapi: 3.0.3

info:

title: dyn config api
 description: 动态 config 开关
 version: 1.0.0

servers:

- url: '/config/1'

paths:

/config/:

get:

tags:

- dyn config module

summary: Returns a list of allow dynamic conf module

description: Returns a list of allow dynamic conf module

operationId: getDynConfModuleList

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/ModuleList'

'500':

description: server error

```
content:
  application/json:
    schema:
      $ref: '#/components/schemas/CommonMsg'
```

/config/http_log:

put:

tags:

- dyn log config

summary: set module conf

description: set module conf

operationId: setDynModuleConf

parameters:

- in: body

name: access log conf

description: access log 配置清单

required: true

schema:

\$ref: '#/components/schemas/MainConf'

responses:

'204':

description: Success

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

get:

tags:

- dyn log config

summary: return dynamic module conf

description: get module conf

operationId: getDynModuleConf

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/MainConf'

'500':

description: server error

content:

```
application/json:
  schema:
    $ref: '#/components/schemas/CommonMsg'
```

###DEFINITIONS

components:

schemas:

CommonMsg:

title: commonMsg

description: 公共提示信息

type: object

properties:

code:

type: string

description: 状态码

msg:

type: string

description: 提示信息

ModuleList:

title: module list

description: 支持动态配置的模块列表

type: array

items:

type: string

MainConf:

title: main Conf

description: 动态 access log main 级别配置

type: object

properties:

servers:

type: array

items:

\$ref: '#/components/schemas/ServerConf'

accessLogFormats:

type: array

items:

\$ref: '#/components/schemas/FormatConf'

FormatConf:

title: FormatConf

description: 动态 access format 配置
type: object
properties:
 name:
 type: string
 description: access log format 名称
 example: test
 format:
 type: string
 description: access log 格式字符串
 example: \$remote_addr - \$remote_user [\$time_local] \"\$request\" \$status
\$body_bytes_sent \"\$http_referer\" \"\$http_user_agent\"
 escape:
 type: string
 description: access log 转义格式
 example: default

ServerConf:

title: ServerConf
description: 动态 access log server 级别配置
type: object
properties:
 listens:
 type: array
 description: server listen 端口列表
 items:
 type: string
 serverNames:
 type: array
 description: server Name 列表.
 items:
 type: string
 locations:
 \$ref: '#/components/schemas/LocationLogConf'

AccessLogConf:

title: AccessLogConf
description: access log 配置
type: object
properties:
 path:
 type: string
 description: access log 写入的位置
 formatName:

type: string
description: access log 格式名称

LocationLogConf:

title: LocationLogConf
description: location 级别 access log 配置开关信息
type: object
properties:
 location:
 type: string
 description: location 名称 .
 example: /demo
 accessLogOn:
 type: boolean
 description: access log 开关状态 .
 example: false
 accessLogs:
 type: array
 items:
 \$ref: '#/components/schemas/AccessLogConf'
 locations:
 \$ref: '#/components/schemas/LocationLogConf'

6.2 动态 telemetry 配置 API 说明

openapi: 3.0.3

info:

title: dyn config api
description: 动态 config 开关
version: 1.0.0

servers:

- url: '/config/1'

paths:

/config/:

get:

tags:

- dyn config module

summary: Returns a list of allow dynamic conf module

description: Returns a list of allow dynamic conf module

operationId: getDynConfModuleList

responses:

'200':

description: Success

```
content:
  application/json:
    schema:
      $ref: '#/components/schemas/ModuleList'
'500':
  description: server error
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/CommonMsg'
```

/config/otel:

```
put:
  tags:
    - dyn telemetry config
  summary: set module conf
  description: set module conf
  operationId: setDynModuleConf
  parameters:
    - in: body
      name: telemetry conf
      description: telemetry 配置清单
      required: true
      schema:
        $ref: '#/components/schemas/MainConf'
  responses:
    '204':
      description: Success
    '500':
      description: server error
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/CommonMsg'
```

get:

```
  tags:
    - dyn telemetry config
  summary: return dynamic module conf
  description: get module conf
  operationId: getDynModuleConf
  responses:
    '200':
      description: Success
      content:
```

```
    application/json:
      schema:
        $ref: '#/components/schemas/MainConf'
  '500':
    description: server error
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/CommonMsg'
```

###DEFINITIONS

```
components:
  schemas:
    CommonMsg:
      title: commonMsg
      description: 公共提示信息
      type: object
      properties:
        code:
          type: string
          description: 状态码
        msg:
          type: string
          description: 提示信息

    ModuleList:
      title: module list
      description: 支持动态配置的模块列表
      type: array
      items:
        type: string

    MainConf:
      title: main Conf
      description: 动态 telemetry main 级别配置
      type: object
      properties:
        servers:
          type: array
          items:
            $ref: '#/components/schemas/ServerConf'
```

ServerConf:

title: ServerConf
description: 动态 telemetry server 级别配置
type: object
properties:
 listens:
 type: array
 description: server listen 端口列表
 items:
 type: string

 serverNames:
 type: array
 description: server Name 列表.
 items:
 type: string

 locations:
 \$ref: '#/components/schemas/LocationLogConf'

LocationLogConf:

title: LocationLogConf
description: location 级别 telemetry 配置开关信息
type: object
properties:
 location:
 type: string
 description: location 名称 .
 example: /demo
 opentelemetry:
 type: boolean
 description: telemetry 开关状态 .
 example: false

 locations:
 \$ref: '#/components/schemas/LocationLogConf'

6.3 动态 vts 配置 API 说明

openapi: 3.0.3

info:

title: dyn config api
description: 动态 config 开关
version: 1.0.0

```
servers:
  - url: '/config/1'
paths:
  /config/:
    get:
      tags:
        - dyn config module
      summary: Returns a list of allow dynamic conf module
      description: Returns a list of allow dynamic conf module
      operationId: getDynConfModuleList
      responses:
        '200':
          description: Success
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ModuleList'
        '500':
          description: server error
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/CommonMsg'
```

```
/config/http_vts:
  put:
    tags:
      - dyn vts config
    summary: set module conf
    description: set module conf
    operationId: setDynModuleConf
    parameters:
      - in: body
        name: vts conf
        description: vts 配置清单
        required: true
        schema:
          $ref: '#/components/schemas/MainConf'
    responses:
      '204':
        description: Success
      '500':
        description: server error
```

```
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/CommonMsg'
get:
  tags:
    - dyn vts config
  summary: return dynamic module conf
  description: get module conf
  operationId: getDynModuleConf
  responses:
    '200':
      description: Success
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/MainConf'
    '500':
      description: server error
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/CommonMsg'
```

###DEFINITIONS

```
components:
  schemas:
    CommonMsg:
      title: commonMsg
      description: 公共提示信息
      type: object
      properties:
        code:
          type: string
          description: 状态码
        msg:
          type: string
          description: 提示信息

    ModuleList:
      title: module list
      description: 支持动态配置的模块列表
```

type: array
items:
type: string

MainConf:

title: main Conf
description: 动态 vts main 级别配置
type: object
properties:
vhost_traffic_status_filter_by_set_key:
type: string
description: vts filter set .
example: \"\$request_uri\" \"\$server_name\"
servers:
type: array
items:
\$ref: '#/components/schemas/ServerConf'

ServerConf:

title: ServerConf
description: 动态 vts server 级别配置
type: object
properties:
listens:
type: array
description: server listen 端口列表
items:
type: string

serverNames:
type: array
description: server Name 列表.
items:
type: string
locations:
\$ref: '#/components/schemas/LocationConf'

LocationConf:

title: LocationConf
description: location 级别 vts 配置开关信息
type: object
properties:
location:
type: string

description: location 名称 .
example: /demo
vhost_traffic_status:
type: boolean
description: vts 开关状态 .
example: false
locations:
\$ref: '#/components/schemas/LocationConf'

6.4 动态 split_clients2 配置 API 说明

openapi: 3.0.3

info:

title: dyn config api
description: 动态 config 开关
version: 1.0.0

servers:

- url: '/config/1'

paths:

/config/:

get:

tags:

- dyn config module

summary: Returns a list of allow dynamic conf module

description: Returns a list of allow dynamic conf module

operationId: getDynConfModuleList

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/ModuleList'

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

/config/http_split_clients_2:

put:

tags:

```

    - dyn split clients 2 config
summary: set module conf
description: set module conf
operationId: setDynModuleConf
requestBody:
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/SplitConf'
      required: true
responses:
  '204':
    description: Success
  '500':
    description: server error
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/CommonMsg'
get:
  tags:
    - dyn split clients 2 config
summary: return dynamic module conf
description: get module conf
operationId: getDynModuleConf
responses:
  '200':
    description: Success
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/SplitConf'
  '500':
    description: server error
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/CommonMsg'
components:
  schemas:
    CommonMsg:
      title: commonMsg
      description: 公共提示信息
      type: object
      properties:

```

code:
 type: string
 description: 状态码
msg:
 type: string
 description: 提示信息

ModuleList:
 title: module list
 description: 支持动态配置的模块列表
 type: array
 items:
 type: string

SplitConf:
 title: Split Clients 2 Conf
 description: 动态 Split Clients 2 配置
 type: object
 properties:
 http:
 type: object
 properties:
 split_clients_2:
 type: object
 properties:
 \${backend_name1}:
 type: integer
 example: 20
 minimum: 0
 maximum: 100
 \${backend_name2}:
 type: integer
 example: 80
 minimum: 0
 maximum: 100

6.5 动态黑白名单配置 API 说明

openapi: 3.0.3

info:

title: dyn config api
 description: 动态 config 开关
 version: 1.0.0

servers:

```

- url: '/config/1'
paths:
  /config/:
    get:
      tags:
        - dyn config module
      summary: Returns a list of allow dynamic conf module
      description: Returns a list of allow dynamic conf module
      operationId: getDynConfModuleList
      responses:
        '200':
          description: Success
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ModuleList'
        '500':
          description: server error
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/CommonMsg'

  /config/http_dyn_bwlist:
    put:
      tags:
        - dyn black white list config
      summary: set module conf
      description: set module conf
      operationId: setDynModuleConf
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/MainConf'
        required: true
      responses:
        '204':
          description: Success
        '500':
          description: server error
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/CommonMsg'

```

```
get:
  tags:
    - dyn black white list config
  summary: return dynamic module conf
  description: get module conf
  operationId: getDynModuleConf
  responses:
    '200':
      description: Success
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/MainConf'
    '500':
      description: server error
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/CommonMsg'
```

```
components:
  schemas:
    CommonMsg:
      title: commonMsg
      description: 公共提示信息
      type: object
      properties:
        code:
          type: string
          description: 状态码
        msg:
          type: string
          description: 提示信息
```

```
ModuleList:
  title: module list
  description: 支持动态配置的模块列表
  type: array
  items:
    type: string
```

```
MainConf:
  title: main Conf
  description: 动态 main 级别配置
  type: object
  properties:
```

servers:
 type: array
 items:
 \$ref: '#/components/schemas/ServerConf'

ServerConf:
 title: ServerConf
 description: 动态 server 级别配置
 type: object
 properties:
 listens:
 type: array
 description: server listen 端口列表
 items:
 type: string

 serverNames:
 type: array
 description: server Name 列表.
 items:
 type: string

 locations:
 \$ref: '#/components/schemas/LocationConf'

LocationConf:
 title: LocationConf
 description: location 级别 vts 配置开关信息
 type: object
 properties:
 location:
 type: string
 description: location 名称 .
 example: /demo
 accessIpv4:
 type: object
 description: access 黑白名单
 properties:
 rule:
 type: string
 enum: [allow, deny]
 addr:
 type: string
 example: "192.168.1.0"
 mask:

```
    type: string
    example: "255.255.255.0"
  locations:
    $ref: '#/components/schemas/LocationConf'
```

6.6 动态 http server ssl 配置 API 说明

openapi: 3.0.3

info:

```
  title: dyn server ssl api
  description: 动态 server ssl 配置
  version: 1.0.0
```

servers:

```
  - url: '/config/1'
```

paths:

/config/:

get:

tags:

```
  - dyn config module
```

summary: Returns a list of allow dynamic conf module

description: Returns a list of allow dynamic conf module

operationId: getDynConfModuleList

responses:

'200':

description: Success

content:

application/json:

schema:

\$ref: '#/components/schemas/ModuleList'

'500':

description: server error

content:

application/json:

schema:

\$ref: '#/components/schemas/CommonMsg'

/config/http_ssl:

put:

tags:

```
  - dyn http server ssl config
```

summary: set http server ssl conf

description: set http server ssl conf

```
operationId: setDynHttpServerSsl
parameters:
  - in: body
    name: http server ssl conf
    description: http server ssl conf 单条配置
    required: true
    schema:
      $ref: '#/components/schemas/MainConf'
responses:
  '204':
    description: Success
  '500':
    description: server error
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/CommonMsg'
get:
  tags:
    - dyn http server ssl config
  summary: return http server ssl config
  description: get http server ssl config
  operationId: getDynHttpServerSsl
  responses:
    '200':
      description: Success
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/MainConf'
    '500':
      description: server error
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/CommonMsg'
```

###DEFINITIONS

```
components:
  schemas:
    CommonMsg:
      title: commonMsg
```

description: 公共提示信息
type: object
properties:
 code:
 type: string
 description: 状态码
 msg:
 type: string
 description: 提示信息

ModuleList:
 title: module list
 description: 支持动态配置的模块列表
 type: array
 items:
 type: string

MainConf:
 title: main Conf
 description: 动态 http server ssl main 级别配置
 type: object
 properties:
 servers:
 type: array
 items:
 \$ref: '#/components/schemas/ServerConf'

ServerConf:
 title: ServerConf
 description: 动态 http server ssl server 级别配置
 type: object
 properties:
 listens:
 type: array
 description: server listen 端口列表
 items:
 type: string
 serverNames:
 type: array
 description: server Name 列表.
 items:
 type: string
 certificates:
 type: array

```
$ref: '#/components/schemas/CertGroup'
```

CertGroup:

title: CertGroupConf

description: 证书组配置信息

```
type: object
```

properties:

certificate:

type: string

description: 证书文件内容 base64(注意需要配置'data:')

example: "data:-----BEGIN CERTIFICATE-----"

\r\nMIIETCCAwmgAwIBAgIUUK6l+HK4KR\hL0sx2XqNTaHLUkO0wDQYJKoZIhvcNA
 QEL\r\nBQAwwZTELMAKGA1UEBhMCQ04xEDAOBgNVBAgTB0JlaUppbmcmcEDAOb
 gNVBAcTB0Jl\r\nnaUppbmcmcDjAMBgNVBAoTBW5naW54MQ8wDQYDVQQLEwZ0b
 Wxha2UxETAPBgNVBAMT\r\nCG5naW54LWNhMCAXDTIyMDgwOTA5NDEwMFoY
 DzIxMjlwNzE2MDk0MTAwWjBiMQsw\r\nCQYDVQQGEwJDTEJEMAA4GA1UECBMH
 QmVpSmluZzEQMA4GA1UEBxMHQmVpSmluZzEO\r\nMAwGA1UEChMFbmdpbng
 xDzANBgNVBASTBnRtbGFrZTEOMAAwGA1UEAxMFbmdpbngw\r\nlringgEiMA0GCsQG
 S1b3DQEBAQUAA4IBDwAwggEKAoIBAQC0hs5E8ZZT+ZUVDvvlRrSL\r\nnokk6OE5E
 A4URo6zawqguKRITc1USgfMdZFwR2znCxjFcsk7WVFSWqA5k5zyePYL8\r\nV0lhni3i
 VGGGczfa9dMZZEtN9EMD6TPOiDiVCaWAFc3U7qe+SYckUHLuVIMX8LshR\r\nndNk
 Gecq41Ck5Sftm3xwy4cmpcClyMpNumHrjJyxkp3SVQGewgZRQypntG9R34EKj\r\nnNQ
 NFnovsn39TeeG+DBjS5m1+roth0JJ6DbemB47hy\vaEwAQpobN1etx20wCwxRO7\r\nln7Huuh4kmWXjUol0nvzXM3ZZX2QnHn4jlgVALS7O6p86z+m4vEHK4hRfWf52xGjM
 v\r\nlnAgMBAAAgjckwgcYwDgYDVR0PAQHVBQAQDAgWgMB0GA1UdJQQWMBCGC
 CsGAQUFBwMB\r\nlnBggrBgEFBQcDAjAMBgNVHRMBAf8EAJAAMB0GA1UdDgQWB
 BQz6z5AlFRrP+KD1gBtl\r\nW6j5NEMzwDaFBgNVHSMEGDAGBQ64+C++PsR6pD
 ZoQ7EORozSJ8d9DBHBgNVHREE\r\nnQDA+ggkxOTluMTY4LiqCDiouY2x1c3RIcjEu
 Y29tggoqlmNsdXNOZXluY29tgqw\r\nlnLnRtbGFrZS5jb22HBH8AAAEdQYJKoZIhv
 cNAQELBQADggEBADouKA9ffflNAs2u\r\nnu7yj4cGNg+sC8r1wDWyGiXGDzvaB7C\
 /sPTZiCgh8hl5gxzOSC+Se51kNFYymN9g4\r\nlnBPfBVVSZ+Cxo60BOt5eFF4PxIW7+
 d8gGFJR7Yx55QGA9DrvGiVL*Eiw*jV3bQaezjN\r\nnoXUkfFgyKv79AfNFShkUPiHCklmfy
 qcCF+GEKxNC1q4wyCIUt2EGtwfiO4YR85s0\r\nln7jQ2BKJOsw3LLd8pZsVOXYu\i9D
 oO5w3gM5TgjnlJPpq+bmqgJ7bfWbSpFo70cdF\r\nlnFLfe9MLpwhDjVQJEJ6DOmZqu1
 dVdNtwLrUrm0V8xkkmac0V8BaEyPy1SNMGJTMGE\r\nlnW8psxt0=\r\n-----END
 CERTIFICATE-----\r\n"

certificateKey:

type: string

description: 证书 key 文件内容 base64(注意需要配置'data:')

example: "data:-----BEGIN RSA PRIVATE KEY-----"

```

\r\nMIIeogIBAAKCAQEAtlbORPGWUVmVPw77yEa0i6JJ0jhORAOF EaOs2sKoLikZ
U3Nv\r\nEoHzHWRcEds5wsYxXLJO1vxUlqgOZOc8nj2CV/P9JYZ94vxkBnM32vXTG
WRLTfRDA\r\n+ kzzogyPwmlgBXN1O6hPkmHJFBy7v5TFVC7IUXTZBnnKuNQpOU n
7Zt8cMuHJqXAi\r\nMjKTbph64yCSKd0v0BnsIGUUMqZ7RvUd+BCo0DRZ6L7J9VU3
nhvgwY0uZtfq6LY\r\nndCSeq23pgeO4cv7mhMAEKaGzdXrcdtMAsMUTu+x7roeJJl41

```

KJdJ781zN2WV9kJ\r\nx5+lylPwC0uzuqfOs\vpuLxByuIUX1n+dsRozLwIDAQABoIB
AFh+VJLbUnOrwuuA\r\nTtBoSlzCat8NRuB0UUDKWSuLjGHEZ9POj39ZEFHyJmfib
Tgba4sjJR6h5t1LWHMC\r\nH2b6hEF86v3d7JTQr0esdy18FtcHMYD3O4H3Qt7HBZ
mpihZh+Kv/b29XH9YfUZfyN\r\n81ehnzS+8LwJ6+QarHKW35QX\nty5+pCw+G97cjdE
7sClZnryd1KseWubAXDLSgmQ\r\nJcjXMMkwjtd2x\vw23ZT97WMEcToL1wMw4hFVt
Htu9u3UHMhzLHgrfrkW4QmACbLF\r\n8MJR94wzscJrUEviz5fV+3ZTmK+lujGmOVK
kOpNAPGf8QAeBfaVqyBTQCQgCdxRo\r\n8hrDT4ECgYEA5vLsDDWiARovvPaRkC
v0y0UWkdgyBxv2T6pjDJZzMxb0yKZ48E6\r\nlnh0PI9bfn0Kbl1ZZviiZg20oZVKeo2wLY
a+W00rZhWljWUh0lvdV5vTPHBmpsXI6+\r\nRK49aNVkfqlleherhfrYUkHKKDeg4IKV
Y3fwAIAQegVY4DsV8k3HUuECgYEAyBu8\r\nns7KuO7sAaJVj+QFfRrR6u2eJD48HPB
4UoeDYMaKAuPTiM5VAUjHVYdD4E2IGAyJJ\r\nr\nJVN413yu3Dk9UIAivgqCROepAcn
nZ1BwE2qIVgfr2GD95o8JQSphze0pCV6WSVjG\r\nr\nniBR+j7MGWIkHJHEIujOrf8eqtVL
Aywxz9rPWA8CgYBTkuzgrjfl893Qn9mlNoLr\r\nr\nXCECvh28fN3xjIMxpvAhONI0Euol7
CzyehEq+IYIJ3Xd9QaAE8tRD8u\vpLxwhOMU\r\nr\nniJea+OzZ6PQF2wPi0j5pvWb0Z2a3
78kiyXrniPFI9LwIJrCnV1MY0T36t8a8n+33\r\nr\nnhNuRuq97vHHDuy003fiXgQKBgGKz
M6cauc+iUvHbVzbBk4HnYSXwUm1HKdVgLOMP\r\nr\nnaPNKaN1RhATchdrE6GcR0Fq
asTq4fYWYn2ECEalz3idHnFtKCaj87qKAOMV\r\nn9gj\r\nr\nn0wAhXhWy+WjwlitvQSB2Gq
nc37sHML1MBotQU4V1vn0d93G8JJn5HN0kvQ0oE0Vn\r\nr\nncpVHAoGAGCxpM24rH
BK0HmlABZd8ZfLWVsW4Y7bMsQmrOVuPZcTWcO5HzkkiGzTc\r\nr\nn8DCDDeGdfJUC
9NaM0DnVuMwp8Vf5uZv4dTS7Hzf8le5vR9oQxVz73iakIKUIpDQc\r\nr\nn0VsUJSKijOQR
T1xZQmkw5+QClmbjLLS3ZXn9kMt0bbxZRPkcV2A=\r\nr\n-----END RSA PRIVATE
KEY-----\r\n"

6.7 动态 upstream 配置 API 说明

swagger: '2.0'

info:

version: '8.0'

title: NJET REST API

description: NJET REST

[API](https://njet.org/en/docs/http/nginx_http_api_module.html)

provides access to NJET status information,
on-the-fly configuration of upstream servers and
key-value pairs management for

[http](https://njet.org/en/docs/http/nginx_http_keyval_module.html) and

[stream](https://njet.org/en/docs/stream/nginx_stream_keyval_module.html).

basePath: /api/8

servers:

- url: 'http://192.168.40.140//api/8'

tags:

- name: HTTP Upstreams

- name: Stream Upstreams

- name: Method GET

- name: Method POST

- name: Method PATCH
- name: Method DELETE

schemes:

- http
- https

paths:

/http/upstreams/:

get:

tags:

- HTTP Upstreams
- Method GET

summary: Return status of all HTTP upstream server groups

description: Returns status of each HTTP upstream server group and its servers.

operationId: getHttpUpstreams

produces:

- application/json

parameters:

- name: fields

in: query

type: string

description: Limits which fields of upstream server groups will be output.

If the “<literal>fields</literal>” value is empty,
only names of upstreams will be output.

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxHTTPUpstreamMap'

'404':

description: Unknown version (*UnknownVersion*)

schema:

\$ref: '#/definitions/NginxError'

/http/upstreams/{httpUpstreamName}/:

parameters:

- name: httpUpstreamName

in: path

description: The name of an HTTP upstream server group.

required: true

type: string

get:

tags:

- HTTP Upstreams
- Method GET

summary: Return status of an HTTP upstream server group

description: Returns status of a particular HTTP upstream server group

and its servers.

operationId: getHttpUpstreamName

produces:

- application/json

parameters:

- name: fields

- in: query

- type: string

- description: Limits which fields of the upstream server group will be output.

responses:

'200':

- description: Success

- schema:

- \$ref: '#/definitions/NginxHTTPUpstream'

'400':

- description: Upstream is static (*UpstreamStatic*)

- schema:

- \$ref: '#/definitions/NginxError'

'404':

- description: |

- Unknown version (*UnknownVersion*),

- upstream not found (*UpstreamNotFound*)

- schema:

- \$ref: '#/definitions/NginxError'

delete:

tags:

- HTTP Upstreams

- Method DELETE

summary: Reset statistics of an HTTP upstream server group

description: Resets the statistics for each upstream server

in an upstream server group and queue statistics.

operationId: deleteHttpUpstreamStat

produces:

- application/json

responses:

'204':

- description: Success

'400':

- description: Upstream is static (*UpstreamStatic*)

- schema:

- \$ref: '#/definitions/NginxError'

'404':

- description: |

- Unknown version (*UnknownVersion*),

- upstream not found (*UpstreamNotFound*)

- schema:

```

    $ref: '#/definitions/NginxError'
  '405':
    description: Method disabled (*MethodDisabled*)
    schema:
      $ref: '#/definitions/NginxError'
'/http/upstreams/{httpUpstreamName}/servers/':
  parameters:
    - name: httpUpstreamName
      in: path
      description: The name of an upstream server group.
      required: true
      type: string
  get:
    tags:
      - HTTP Upstreams
      - Method GET
    summary: Return configuration of all servers in an HTTP upstream server
group
  description: Returns configuration of each server
    in a particular HTTP upstream server group.
  operationId: getHttpUpstreamServers
  produces:
    - application/json
  responses:
    '200':
      description: Success
      schema:
        $ref: '#/definitions/NginxHTTPUpstreamConfServerMap'
    '400':
      description: Upstream is static (*UpstreamStatic*)
      schema:
        $ref: '#/definitions/NginxError'
    '404':
      description: |
        Unknown version (*UnknownVersion*),
        upstream not found (*UpstreamNotFound*)
      schema:
        $ref: '#/definitions/NginxError'
  post:
    tags:
      - HTTP Upstreams
      - Method POST
    summary: Add a server to an HTTP upstream server group
    description: Adds a new server to an HTTP upstream server group.
      Server parameters are specified in the JSON format.
    operationId: postHttpUpstreamServer

```

produces:

- application/json

parameters:

- in: body

name: postHttpUpstreamServer

description: Address of a new server and other optional parameters in the JSON format.

The “*ID*”, “*backup*”, and “*service*” parameters cannot be changed.

required: true

schema:

\$ref: '#/definitions/NginxHTTPUpstreamConfServer'

responses:

'201':

description: Created

schema:

\$ref: '#/definitions/NginxHTTPUpstreamConfServer'

'400':

description: |

Upstream is static (*UpstreamStatic*),
invalid “**parameter**” value (*UpstreamConfFormatError*),
missing “*server*” argument (*UpstreamConfFormatError*),
unknown parameter “**name**” (*UpstreamConfFormatError*),
nested object or list (*UpstreamConfFormatError*),
“*error*” while parsing (*UpstreamBadAddress*),
service upstream “*host*” may not have port (*UpstreamBadAddress*),
service upstream “*host*” requires domain name

(*UpstreamBadAddress*),

invalid “*weight*” (*UpstreamBadWeight*),
invalid “*max_conns*” (*UpstreamBadMaxConns*),
invalid “*max_fails*” (*UpstreamBadMaxFails*),
invalid “*fail_timeout*” (*UpstreamBadFailTimeout*),
invalid “*slow_start*” (*UpstreamBadSlowStart*),
reading request body failed *BodyReadError*),
route is too long (*UpstreamBadRoute*),
“*service*” is empty (*UpstreamBadService*),
no resolver defined to resolve (*UpstreamConfNoResolver*),
upstream “**name**” has no backup (*UpstreamNoBackup*),
upstream “**name**” memory exhausted (*UpstreamOutOfMemory*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Unknown version (*UnknownVersion*),
upstream not found (*UpstreamNotFound*)

schema:

```

    $ref: '#/definitions/NginxError'
  '405':
    description: Method disabled (*MethodDisabled*)
    schema:
      $ref: '#/definitions/NginxError'
  '409':
    description: Entry exists (*EntryExists*)
    schema:
      $ref: '#/definitions/NginxError'
  '415':
    description: JSON error (*JsonError*)
    schema:
      $ref: '#/definitions/NginxError'
'/http/upstreams/{httpUpstreamName}/servers/{httpUpstreamServerId}':
  parameters:
    - name: httpUpstreamName
      in: path
      description: The name of the upstream server group.
      required: true
      type: string
    - name: httpUpstreamServerId
      in: path
      description: The ID of the server.
      required: true
      type: string
  get:
    tags:
      - HTTP Upstreams
      - Method GET
    summary: Return configuration of a server in an HTTP upstream server group
    description: Returns configuration of a particular server
      in the HTTP upstream server group.
    operationId: getHttpUpstreamPeer
    produces:
      - application/json
    responses:
      '200':
        description: Success
        schema:
          $ref: '#/definitions/NginxHTTPUpstreamConfServer'
      '400':
        description: |
          Upstream is static (*UpstreamStatic*),
          invalid server ID (*UpstreamBadServerId*)
        schema:
          $ref: '#/definitions/NginxError'

```

'404':

description: |

Server with ID **"id"** does not exist (*UpstreamServerNotFound*),
unknown version (*UnknownVersion*),
upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

patch:

tags:

- HTTP Upstreams
- Method PATCH

summary: Modify a server in an HTTP upstream server group

description: Modifies settings of a particular server
in an HTTP upstream server group.

Server parameters are specified in the JSON format.

operationId: patchHttpUpstreamPeer

produces:

- application/json

parameters:

- in: body

name: patchHttpUpstreamServer

description: Server parameters, specified in the JSON format.

The **"*ID*"**, **"*backup*"**, and **"*service*"** parameters
cannot be changed.

required: true

schema:

\$ref: '#/definitions/NginxHTTPUpstreamConfServer'

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxHTTPUpstreamConfServer'

'400':

description: |

Upstream is static (*UpstreamStatic*),
invalid **"parameter"** value (*UpstreamConfFormatError*),
unknown parameter **"name"** (*UpstreamConfFormatError*),
nested object or list (*UpstreamConfFormatError*),
"*error*" while parsing (*UpstreamBadAddress*),
invalid **"*server*"** argument (*UpstreamBadAddress*),
invalid server ID (*UpstreamBadServerId*),
invalid **"*weight*"** (*UpstreamBadWeight*),
invalid **"*max_conns*"** (*UpstreamBadMaxConns*),
invalid **"*max_fails*"** (*UpstreamBadMaxFails*),
invalid **"*fail_timeout*"** (*UpstreamBadFailTimeout*),
invalid **"*slow_start*"** (*UpstreamBadSlowStart*),

```

        reading request body failed (*BodyReadError*),
        route is too long (*UpstreamBadRoute*),
        “*service*” is empty (*UpstreamBadService*),
        server “ID” address is immutable (*UpstreamServerImmutable*),
        server “*ID*” weight is immutable (*UpstreamServerWeightImmutable*),
        upstream “*name*” memory exhausted (*UpstreamOutOfMemory*)
    schema:
        $ref: '#/definitions/NginxError'
'404':
    description: |
        Server with ID “id” does not exist (*UpstreamServerNotFound*),
        unknown version (*UnknownVersion*),
        upstream not found (*UpstreamNotFound*)
    schema:
        $ref: '#/definitions/NginxError'
'405':
    description: Method disabled (*MethodDisabled*)
    schema:
        $ref: '#/definitions/NginxError'
'415':
    description: JSON error (*JsonError*)
    schema:
        $ref: '#/definitions/NginxError'
delete:
    tags:
        - HTTP Upstreams
        - Method DELETE
    summary: Remove a server from an HTTP upstream server group
    description: Removes a server from an HTTP upstream server group.
    operationId: deleteHttpUpstreamServer
    produces:
        - application/json
    responses:
        '200':
            description: Success
            schema:
                $ref: '#/definitions/NginxHTTPUpstreamConfServerMap'
        '400':
            description: |
                Upstream is static (*UpstreamStatic*),
                invalid server ID (*UpstreamBadServerId*),
                server “id” not removable (*UpstreamServerImmutable*)
            schema:
                $ref: '#/definitions/NginxError'
        '404':
            description: |

```

Server with ID “**id**” does not exist (*UpstreamServerNotFound*),
 unknown version (*UnknownVersion*),
 upstream not found (*UpstreamNotFound*)
 schema:
 \$ref: '#/definitions/NginxError'

'405':
 description: Method disabled (*MethodDisabled*)
 schema:
 \$ref: '#/definitions/NginxError'

/stream/upstreams/:
 get:
 tags:
 - Stream Upstreams
 - Method GET
 summary: Return status of all stream upstream server groups
 description: Returns status of each stream upstream server group
 and its servers.
 operationId: getStreamUpstreams
 produces:
 - application/json
 parameters:
 - name: fields
 in: query
 type: string
 description: Limits which fields of upstream server groups will be output.
 If the “<literal>fields</literal>” value is empty,
 only names of upstreams will be output.

responses:
 '200':
 description: Success
 schema:
 \$ref: '#/definitions/NginxStreamUpstreamMap'

'404':
 description: Unknown version (*UnknownVersion*)
 schema:
 \$ref: '#/definitions/NginxError'

'/stream/upstreams/{streamUpstreamName}':
 parameters:
 - name: streamUpstreamName
 in: path
 description: The name of a stream upstream server group.
 required: true
 type: string
 get:
 tags:
 - Stream Upstreams

- Method GET

summary: Return status of a stream upstream server group

description: Returns status of a particular stream upstream server group and its servers.

operationId: getStreamUpstream

produces:

- application/json

parameters:

- name: fields

in: query

type: string

description: Limits which fields of the upstream server group will be output.

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxStreamUpstream'

'400':

description: Upstream is static (*UpstreamStatic*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Unknown version (*UnknownVersion*),

upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

delete:

tags:

- Stream Upstreams

- Method DELETE

summary: Reset statistics of a stream upstream server group

description: Resets the statistics for each upstream server in an upstream server group.

operationId: deleteStreamUpstreamStat

produces:

- application/json

responses:

'204':

description: Success

'400':

description: Upstream is static (*UpstreamStatic*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

```

        Unknown version (*UnknownVersion*),
        upstream not found (*UpstreamNotFound*)
    schema:
        $ref: '#/definitions/NginxError'
    '405':
        description: Method disabled (*MethodDisabled*)
        schema:
            $ref: '#/definitions/NginxError'
'/stream/upstreams/{streamUpstreamName}/servers/':
    parameters:
        - name: streamUpstreamName
          in: path
          description: The name of an upstream server group.
          required: true
          type: string
    get:
        tags:
            - Stream Upstreams
            - Method GET
        summary: Return configuration of all servers in a stream upstream server
group
    description: Returns configuration of each server
        in a particular stream upstream server group.
    operationId: getStreamUpstreamServers
    produces:
        - application/json
    responses:
        '200':
            description: Success
            schema:
                $ref: '#/definitions/NginxStreamUpstreamConfServerMap'
        '400':
            description: Upstream is static (*UpstreamStatic*)
            schema:
                $ref: '#/definitions/NginxError'
        '404':
            description: |
                Unknown version (*UnknownVersion*),
                upstream not found (*UpstreamNotFound*)
            schema:
                $ref: '#/definitions/NginxError'
    post:
        tags:
            - Stream Upstreams
            - Method POST
        summary: Add a server to a stream upstream server group

```

description: Adds a new server to a stream upstream server group.

Server parameters are specified in the JSON format.

operationId: postStreamUpstreamServer

produces:

- application/json

parameters:

- in: body

name: postStreamUpstreamServer

description: Address of a new server and other optional parameters in the JSON format.

The “*ID*”, “*backup*”, and “*service*” parameters cannot be changed.

required: true

schema:

\$ref: '#/definitions/NginxStreamUpstreamConfServer'

responses:

'201':

description: Created

schema:

\$ref: '#/definitions/NginxStreamUpstreamConfServer'

'400':

description: |

Upstream is static (*UpstreamStatic*),
invalid “parameter” value (*UpstreamConfFormatError*),
missing “server” argument (*UpstreamConfFormatError*),
unknown parameter “name” (*UpstreamConfFormatError*),
nested object or list (*UpstreamConfFormatError*),
“error” while parsing (*UpstreamBadAddress*),
no port in server “host” (*UpstreamBadAddress*),
service upstream “host” may not have port (*UpstreamBadAddress*),
service upstream “host” requires domain name

(*UpstreamBadAddress*),

invalid “weight” (*UpstreamBadWeight*),
invalid “max_conns” (*UpstreamBadMaxConns*),
invalid “max_fails” (*UpstreamBadMaxFails*),
invalid “fail_timeout” (*UpstreamBadFailTimeout*),
invalid “slow_start” (*UpstreamBadSlowStart*),
“service” is empty (*UpstreamBadService*),
no resolver defined to resolve (*UpstreamConfNoResolver*),
upstream “name” has no backup (*UpstreamNoBackup*),
upstream “name” memory exhausted (*UpstreamOutOfMemory*),
reading request body failed (*BodyReadError*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

```

        Unknown version (*UnknownVersion*),
        upstream not found (*UpstreamNotFound*)
    schema:
        $ref: '#/definitions/NginxError'
'405':
    description: Method disabled (*MethodDisabled*)
    schema:
        $ref: '#/definitions/NginxError'
'409':
    description: Entry exists (*EntryExists*)
    schema:
        $ref: '#/definitions/NginxError'
'415':
    description: JSON error (*JsonError*)
    schema:
        $ref: '#/definitions/NginxError'
'/stream/upstreams/{streamUpstreamName}/servers/{streamUpstreamServerId}':
    parameters:
        - name: streamUpstreamName
          in: path
          description: The name of the upstream server group.
          required: true
          type: string
        - name: streamUpstreamServerId
          in: path
          description: The ID of the server.
          required: true
          type: string
    get:
        tags:
            - Stream Upstreams
            - Method GET
        summary: Return configuration of a server in a stream upstream server group
        description: Returns configuration of a particular server
            in the stream upstream server group.
        operationId: getStreamUpstreamServer
        produces:
            - application/json
        responses:
            '200':
                description: Success
                schema:
                    $ref: '#/definitions/NginxStreamUpstreamConfServer'
            '400':
                description: |
                    Upstream is static (*UpstreamStatic*),

```

invalid server ID (*UpstreamBadServerId*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Unknown version (*UnknownVersion*),

upstream not found (*UpstreamNotFound*),

server with ID “**id**” does not exist (*UpstreamServerNotFound*)

schema:

\$ref: '#/definitions/NginxError'

patch:

tags:

- Stream Upstreams
- Method PATCH

summary: Modify a server in a stream upstream server group

description: Modifies settings of a particular server

in a stream upstream server group.

Server parameters are specified in the JSON format.

operationId: patchStreamUpstreamServer

produces:

- application/json

parameters:

- in: body

name: patchStreamUpstreamServer

description: Server parameters, specified in the JSON format.

The “**ID**”, “**backup**”, and “**service**” parameters cannot be changed.

required: true

schema:

\$ref: '#/definitions/NginxStreamUpstreamConfServer'

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxStreamUpstreamConfServer'

'400':

description: |

Upstream is static (*UpstreamStatic*),

invalid “**parameter**” value (*UpstreamConfFormatError*),

unknown parameter “**name**” (*UpstreamConfFormatError*),

nested object or list (*UpstreamConfFormatError*),

“**error**” while parsing (*UpstreamBadAddress*),

invalid “**server**” argument (*UpstreamBadAddress*),

no port in server “**host**” (*UpstreamBadAddress*),

invalid server ID (*UpstreamBadServerId*),

invalid “**weight**” (*UpstreamBadWeight*),

invalid “*max_conns*” (*UpstreamBadMaxConns*),
invalid “*max_fails*” (*UpstreamBadMaxFails*),
invalid “*fail_timeout*” (*UpstreamBadFailTimeout*),
invalid “*slow_start*” (*UpstreamBadSlowStart*),
reading request body failed *BodyReadError*),
“*service*” is empty (*UpstreamBadService*),
server “ID” address is immutable (*UpstreamServerImmutable*),
server “ID” weight is immutable (*UpstreamServerWeightImmutable*),
upstream “*name*” memory exhausted (*UpstreamOutOfMemory*)

schema:

\$ref: '#/definitions/NginxError'

'404':

description: |

Server with ID “id” does not exist (*UpstreamServerNotFound*),
unknown version (*UnknownVersion*),
upstream not found (*UpstreamNotFound*)

schema:

\$ref: '#/definitions/NginxError'

'405':

description: Method disabled (*MethodDisabled*)

schema:

\$ref: '#/definitions/NginxError'

'415':

description: JSON error (*JsonError*)

schema:

\$ref: '#/definitions/NginxError'

delete:

tags:

- Stream Upstreams
- Method DELETE

summary: Remove a server from a stream upstream server group

description: Removes a server from a stream server group.

operationId: deleteStreamUpstreamServer

produces:

- application/json

responses:

'200':

description: Success

schema:

\$ref: '#/definitions/NginxStreamUpstreamConfServerMap'

'400':

description: |

Upstream is static (*UpstreamStatic*),
invalid server ID (*UpstreamBadServerId*),
server “id” not removable (*UpstreamServerImmutable*)

schema:

```

        $ref: '#/definitions/NginxError'
'404':
  description: |
    Server with ID “id” does not exist (*UpstreamServerNotFound*),
    unknown version (*UnknownVersion*),
    upstream not found (*UpstreamNotFound*)
  schema:
    $ref: '#/definitions/NginxError'
'405':
  description: Method disabled (*MethodDisabled*)
  schema:
    $ref: '#/definitions/NginxError'

###
###DEFINITIONS
###
definitions:
  ArrayOfStrings:
    title: Array
    description: |
      An array of strings.
    type: array
    items:
      type: string
  NginxHTTPUpstreamMap:
    title: HTTP Upstreams
    description: |
      Status information of all HTTP
      <a
href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#zone">dynamical
ly configurable</a>
      <a
href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#upstream">grou
ps</a>.
    type: object
    additionalProperties:
      $ref: '#/definitions/NginxHTTPUpstream'
  example:
    trac-backend:
      peers:
        - id: 0
          server: 10.0.0.1:8088
          name: 10.0.0.1:8088
          backup: false
          weight: 5
          state: up
          active: 0

```

ssl:
 handshakes: 620311
 handshakes_failed: 3432
 session_reuses: 36442
 no_common_protocol: 4
 handshake_timeout: 0
 peer_rejected_cert: 0
 verify_failures:
 expired_cert: 2
 revoked_cert: 1
 hostname_mismatch: 2
 other: 1
requests: 667231
header_time: 20
response_time: 36
responses:
 1xx: 0
 2xx: 666310
 3xx: 0
 4xx: 915
 5xx: 6
 codes:
 200: 666310
 404: 915
 503: 6
 total: 667231
sent: 251946292
received: 19222475454
fails: 0
unavail: 0
health_checks:
 checks: 26214
 fails: 0
 unhealthy: 0
 last_passed: true
downtime: 0
downstart: 2022-06-28T11:09:21.602Z
selected: 2022-06-28T15:01:25Z
- id: 1
 server: 10.0.0.1:8089
 name: 10.0.0.1:8089
 backup: true
 weight: 1
 state: unhealthy
 active: 0
 requests: 0

responses:
 1xx: 0
 2xx: 0
 3xx: 0
 4xx: 0
 5xx: 0
 codes: {}
 total: 0
sent: 0
received: 0
fails: 0
unavail: 0
health_checks:
 checks: 26284
 fails: 26284
 unhealthy: 1
 last_passed: false
downtime: 262925617
downstart: 2022-06-28T11:09:21.602Z
selected: 2022-06-28T15:01:25Z
keepalive: 0
zombies: 0
zone: trac-backend
hg-backend:
 peers:
 - id: 0
 server: 10.0.0.1:8088
 name: 10.0.0.1:8088
 backup: false
 weight: 5
 state: up
 active: 0
 ssl:
 handshakes: 620311
 handshakes_failed: 3432
 session_reuses: 36442
 no_common_protocol: 4
 handshake_timeout: 0
 peer_rejected_cert: 0
 verify_failures:
 expired_cert: 2
 revoked_cert: 1
 hostname_mismatch: 2
 other: 1
 requests: 667231
 header_time: 20

response_time: 36
responses:
 1xx: 0
 2xx: 666310
 3xx: 0
 4xx: 915
 5xx: 6
 codes:
 200: 666310
 404: 915
 503: 6
 total: 667231
sent: 251946292
received: 19222475454
fails: 0
unavail: 0
health_checks:
 checks: 26214
 fails: 0
 unhealthy: 0
 last_passed: true
downtime: 0
downstart: 2022-06-28T11:09:21.602Z
selected: 2022-06-28T15:01:25Z
- id: 1
 server: 10.0.0.1:8089
 name: 10.0.0.1:8089
 backup: true
 weight: 1
 state: unhealthy
 active: 0
 requests: 0
 responses:
 1xx: 0
 2xx: 0
 3xx: 0
 4xx: 0
 5xx: 0
 codes: {}
 total: 0
 sent: 0
 received: 0
 fails: 0
 unavail: 0
 health_checks:
 checks: 26284

```

        fails: 26284
        unhealthy: 1
        last_passed: false
        downtime: 262925617
        downstart: 2022-06-28T11:09:21.602Z
        selected: 2022-06-28T15:01:25Z
    keepalive: 0
    zombies: 0
    zone: hg-backend
NginxHTTPUpstream:
  title: HTTP Upstream
  type: object
  properties:
    peers:
      $ref: '#/definitions/NginxHTTPUpstreamPeerMap'
    keepalive:
      type: integer
      description: The current number of idle
        <a
href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#keepalive">keep
alive</a>
        connections.
    zombies:
      type: integer
      description: The current number of servers removed
        from the group but still processing active client requests.
    zone:
      type: string
      description: The name of the shared memory
        <a
href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#zone">zone</a>
        that keeps the group's configuration and run-time state.
    queue:
      type: object
      description: >
        For the requests
        <a
href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#queue">queue</
a>,
        the following data are provided:
    properties:
      size:
        type: integer
        description: The current number of requests in the queue.
      max_size:
        type: integer

```

description: The maximum number of requests that can be in the queue at the same time.

overflows:

type: integer

description: The total number of requests rejected due to the queue

overflow.

example:

upstream_backend:

peers:

- id: 0

server: 10.0.0.1:8088

name: 10.0.0.1:8088

backup: false

weight: 5

state: up

active: 0

ssl:

handshakes: 620311

handshakes_failed: 3432

session_reuses: 36442

no_common_protocol: 4

handshake_timeout: 0

peer_rejected_cert: 0

verify_failures:

expired_cert: 2

revoked_cert: 1

hostname_mismatch: 2

other: 1

max_conns: 20

requests: 667231

header_time: 20

response_time: 36

responses:

1xx: 0

2xx: 666310

3xx: 0

4xx: 915

5xx: 6

codes:

200: 666310

404: 915

503: 6

total: 667231

sent: 251946292

received: 19222475454

fails: 0

```
    unavail: 0
    health_checks:
      checks: 26214
      fails: 0
      unhealthy: 0
      last_passed: true
    downtime: 0
    downstart: 2022-06-28T11:09:21.602Z
    selected: 2022-06-28T15:01:25Z
  - id: 1
    server: 10.0.0.1:8089
    name: 10.0.0.1:8089
    backup: true
    weight: 1
    state: unhealthy
    active: 0
    max_conns: 20
    requests: 0
    responses:
      1xx: 0
      2xx: 0
      3xx: 0
      4xx: 0
      5xx: 0
      codes: {}
      total: 0
    sent: 0
    received: 0
    fails: 0
    unavail: 0
    health_checks:
      checks: 26284
      fails: 26284
      unhealthy: 1
      last_passed: false
    downtime: 262925617
    downstart: 2022-06-28T11:09:21.602Z
    selected: 2022-06-28T15:01:25Z
  keepalive: 0
  zombies: 0
  zone: upstream_backend
NginxHTTPUpstreamPeerMap:
  title: HTTP Upstream Servers
  description: |
    An array of HTTP
  <a
```

[href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#upstream">upstream](https://njet.org/en/docs/http/nginx_http_upstream_module.html#upstream) servers.

type: array

items:

\$ref: '#/definitions/NginxHTTPUpstreamPeer'

NginxHTTPUpstreamPeer:

title: HTTP Upstream Server

type: object

properties:

id:

type: integer

description: The ID of the server.

readOnly: true

server:

type: string

description: An [address](https://njet.org/en/docs/http/nginx_http_upstream_module.html#server)

of the server.

service:

type: string

description: The

[service](https://njet.org/en/docs/http/nginx_http_upstream_module.html#service)

parameter value of the

[server](https://njet.org/en/docs/http/nginx_http_upstream_module.html#server)

directive.

name:

type: string

description: The name of the server specified in the

[server](https://njet.org/en/docs/http/nginx_http_upstream_module.html#server)

directive.

readOnly: true

backup:

type: boolean

description: A boolean value indicating whether the server is a

[backup](https://njet.org/en/docs/http/nginx_http_upstream_module.html#backup)

server.

weight:

type: integer
description: Weight

of the server.

state:

type: string

enum:

- up
- draining
- down
- unavail
- checking
- unhealthy

description: Current state, which may be one of

"<code>up</code>", "<code>draining</code>", "<code>down</code>",
"<code>unavail</code>", "<code>checking</code>",
and "<code>unhealthy</code>".

active:

type: integer

description: The current number of active connections.

readOnly: true

ssl:

type: object

readOnly: true

properties:

handshakes:

type: integer

description: The total number of successful SSL handshakes.

readOnly: true

handshakes_failed:

type: integer

description: The total number of failed SSL handshakes.

readOnly: true

session_reuses:

type: integer

description: The total number of session reuses during SSL handshake.

readOnly: true

no_common_protocol:

type: integer

description: The number of SSL handshakes failed
because of no common protocol.

handshake_timeout:

type: integer

description: The number of SSL handshakes failed
because of a timeout.

peer_rejected_cert:

- type: integer
- description: The number of failed SSL handshakes when njet presented the certificate to the upstream server but it was rejected with a corresponding alert message.

verify_failures:

- type: object
- description: SSL certificate verification errors
- properties:
 - expired_cert:
 - type: integer
 - description: An expired or not yet valid certificate was presented by an upstream server.
 - revoked_cert:
 - type: integer
 - description: A revoked certificate was presented by an upstream server.
 - hostname_mismatch:
 - type: integer
 - description: Server's certificate doesn't match the hostname.
 - other:
 - type: integer
 - description: Other SSL certificate verification errors.

max_conns:

- type: integer
- description: The
 - max_conns
 - limit for the server.

requests:

- type: integer
- description: The total number of client requests forwarded to this server.
- readOnly: true

responses:

- type: object
- readOnly: true
- properties:
 - 1xx:
 - type: integer
 - description: The number of responses with "<code>1xx</code>" status codes.
 - readOnly: true
 - 2xx:
 - type: integer
 - description: The number of responses with "<code>2xx</code>" status

codes.
 readOnly: true
 3xx:
 type: integer
 description: The number of responses with “<code>3xx</code>” status

codes.
 readOnly: true
 4xx:
 type: integer
 description: The number of responses with “<code>4xx</code>” status

codes.
 readOnly: true
 5xx:
 type: integer
 description: The number of responses with “<code>5xx</code>” status

codes.
 readOnly: true
 codes:
 type: object
 description: The number of responses per each status code.
 readOnly: true
 properties:
 codeNumber:
 type: integer
 description: The number of responses with this particular status

code.
 readOnly: true
 total:
 type: integer
 description: The total number of responses obtained from this server.
 readOnly: true
 sent:
 type: integer
 description: The total number of bytes sent to this server.
 readOnly: true
 received:
 type: integer
 description: The total number of bytes received from this server.
 readOnly: true
 fails:
 type: integer
 description: The total number of unsuccessful attempts
 to communicate with the server.
 readOnly: true
 unavail:
 type: integer

description: How many times the server became unavailable for client requests

(state “`unavail`”) due to the number of unsuccessful attempts reaching the

[max_fails](https://njet.org/en/docs/http/nginx_http_upstream_module.html#max_fails) threshold.

readOnly: true

health_checks:

type: object

readOnly: true

properties:

checks:

type: integer

description: The total number of

[health check](https://njet.org/en/docs/http/nginx_http_upstream_hc_module.html#health_check) requests made.

fails:

type: integer

description: The number of failed health checks.

unhealthy:

type: integer

description: How many times the server became unhealthy (state “`unhealthy`”).

last_passed:

type: boolean

description: Boolean indicating if the last health check request was successful

and passed

[tests](https://njet.org/en/docs/http/nginx_http_upstream_hc_module.html#match).

downtime:

type: integer

readOnly: true

description: Total time the server was in the “`unavail`”, “`checking`”, and “`unhealthy`” states.

downstart:

type: string

format: date-time

readOnly: true

description: The time when the server became “`unavail`”, “`checking`”,

or “`<code>unhealthy</code>`”,
in the ISO 8601 format with millisecond resolution.

selected:

type: string
format: date-time
readOnly: true

description: The time when the server was last selected to process a
request,

in the ISO 8601 format with millisecond resolution.

header_time:

type: integer
readOnly: true
description: The average time to get the

`<a`

href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#var_upstream_h
eader_time">response header

from the server.

response_time:

type: integer
readOnly: true
description: The average time to get the

`<a`

href="https://njet.org/en/docs/http/nginx_http_upstream_module.html#var_upstream_re
sponse_time">full response

from the server.

NginxHTTPUpstreamConfServerMap:

title: HTTP Upstream Servers

description: An array of HTTP upstream servers for dynamic configuration.

type: array

items:

\$ref: '#/definitions/NginxHTTPUpstreamConfServer'

example:

- id: 0

server: 10.0.0.1:8088

weight: 1

max_conns: 0

max_fails: 0

fail_timeout: 10s

slow_start: 10s

route: "

backup: false

down: false

- id: 1

server: 10.0.0.1:8089

weight: 4

max_conns: 0

max_fails: 0
fail_timeout: 10s
slow_start: 10s
route: "
backup: true
down: true

NginxHTTPUpstreamConfServer:

title: HTTP Upstream Server

description: |

Dynamically configurable parameters of an HTTP upstream

server:

type: object

properties:

id:

type: integer

description: The ID of the HTTP upstream server.

The ID is assigned automatically and cannot be changed.

readOnly: true

server:

type: string

description: Same as the

addresses

parameter of the HTTP upstream server.

When adding a server, it is possible to specify it as a domain name.

In this case, changes of the IP addresses

that correspond to a domain name will be monitored and automatically

applied to the upstream configuration

without the need of restarting njet.

This requires the

resolver

directive in the "<code>http</code>" block.

See also the

resolve

parameter of the HTTP upstream server.

service:

type: string

description: Same as the

<a

[service](https://njet.org/en/docs/http/nginx_http_upstream_module.html#service)

parameter of the HTTP upstream server.

This parameter cannot be changed.

readOnly: true

weight:

type: integer

description: Same as the

[weight](https://njet.org/en/docs/http/nginx_http_upstream_module.html#weight)

[weight](https://njet.org/en/docs/http/nginx_http_upstream_module.html#weight)

parameter of the HTTP upstream server.

max_conns:

type: integer

description: Same as the

[max_conns](https://njet.org/en/docs/http/nginx_http_upstream_module.html#max_conns)

[max_conns](https://njet.org/en/docs/http/nginx_http_upstream_module.html#max_conns)

parameter of the HTTP upstream server.

max_fails:

type: integer

description: Same as the

[max_fails](https://njet.org/en/docs/http/nginx_http_upstream_module.html#max_fails)

[max_fails](https://njet.org/en/docs/http/nginx_http_upstream_module.html#max_fails)

parameter of the HTTP upstream server.

fail_timeout:

type: string

description: Same as the

[fail_timeout](https://njet.org/en/docs/http/nginx_http_upstream_module.html#fail_timeout)

[fail_timeout](https://njet.org/en/docs/http/nginx_http_upstream_module.html#fail_timeout)

parameter of the HTTP upstream server.

slow_start:

type: string

description: Same as the

[slow_start](https://njet.org/en/docs/http/nginx_http_upstream_module.html#slow_start)

[slow_start](https://njet.org/en/docs/http/nginx_http_upstream_module.html#slow_start)

parameter of the HTTP upstream server.

route:

type: string

description: Same as the

[route](https://njet.org/en/docs/http/nginx_http_upstream_module.html#route)

[route](https://njet.org/en/docs/http/nginx_http_upstream_module.html#route)

parameter of the HTTP upstream server.

backup:
 type: boolean
 description: When `true`, adds a
[backup](https://njet.org/en/docs/http/nginx_http_upstream_module.html#backup)
 server.
 This parameter cannot be changed.
 readOnly: true
 down:
 type: boolean
 description: Same as the
[down](https://njet.org/en/docs/http/nginx_http_upstream_module.html#down)
 parameter of the HTTP upstream server.
 drain:
 type: boolean
 description: Same as the
[drain](https://njet.org/en/docs/http/nginx_http_upstream_module.html#drain)
 parameter of the HTTP upstream server.
 parent:
 type: string
 description: Parent server ID of the resolved server.
 The ID is assigned automatically and cannot be changed.
 readOnly: true
 host:
 type: string
 description: Hostname of the resolved server.
 The hostname is assigned automatically and cannot be changed.
 readOnly: true
 example:
 id: 1
 server: 10.0.0.1:8089
 weight: 4
 max_conns: 0
 max_fails: 0
 fail_timeout: 10s
 slow_start: 10s
 route: "
 backup: true
 down: true
 NginxStreamUpstreamMap:
 title: Stream Upstreams
 description: Status information of stream upstream server groups.

```
type: object
additionalProperties:
  $ref: '#/definitions/NginxStreamUpstream'
example:
  mysql_backends:
    peers:
      - id: 0
        server: 10.0.0.1:12345
        name: 10.0.0.1:12345
        backup: false
        weight: 5
        state: up
        active: 0
        ssl:
          handshakes: 1045
          handshakes_failed: 89
          session_reuses: 321
          no_common_protocol: 4
          handshake_timeout: 0
          peer_rejected_cert: 0
          verify_failures:
            expired_cert: 2
            revoked_cert: 1
            hostname_mismatch: 2
            other: 1
          max_conns: 30
          connecions: 1231
          sent: 251946292
          received: 19222475454
          fails: 0
          unavail: 0
          health_checks:
            checks: 26214
            fails: 0
            unhealthy: 0
            last_passed: true
          downtime: 0
          downstart: 2022-06-28T11:09:21.602Z
          selected: 2022-06-28T15:01:25Z
      - id: 1
        server: 10.0.0.1:12346
        name: 10.0.0.1:12346
        backup: true
        weight: 1
        state: unhealthy
        active: 0
```

max_conns: 30
connections: 0
sent: 0
received: 0
fails: 0
unavail: 0
health_checks:
 checks: 26284
 fails: 26284
 unhealthy: 1
 last_passed: false
downtime: 262925617
downstart: 2022-06-28T11:09:21.602Z
selected: 2022-06-28T15:01:25Z
zombies: 0
zone: mysql_backends
dns:
 peers:
 - id: 0
 server: 10.0.0.1:12347
 name: 10.0.0.1:12347
 backup: false
 weight: 5
 state: up
 active: 0
 ssl:
 handshakes: 5268
 handshakes_failed: 121
 session_reuses: 854
 no_common_protocol: 4
 handshake_timeout: 0
 peer_rejected_cert: 0
 verify_failures:
 expired_cert: 2
 revoked_cert: 1
 hostname_mismatch: 2
 other: 1
 max_conns: 30
 connections: 667231
 sent: 251946292
 received: 19222475454
 fails: 0
 unavail: 0
 health_checks:
 checks: 26214
 fails: 0

```

        unhealthy: 0
        last_passed: true
        downtime: 0
        downstart: 2022-06-28T11:09:21.602Z
        selected: 2022-06-28T15:01:25Z
    - id: 1
      server: 10.0.0.1:12348
      name: 10.0.0.1:12348
      backup: true
      weight: 1
      state: unhealthy
      active: 0
      connections: 0
      max_conns: 30
      sent: 0
      received: 0
      fails: 0
      unavail: 0
      health_checks:
        checks: 26284
        fails: 26284
        unhealthy: 1
        last_passed: false
      downtime: 262925617
      downstart: 2022-06-28T11:09:21.602Z
      selected: 2022-06-28T15:01:25Z
    zombies: 0
    zone: dns
NginxStreamUpstream:
  title: Stream Upstream
  type: object
  properties:
    peers:
      $ref: '#/definitions/NginxStreamUpstreamPeerMap'
    zombies:
      type: integer
      description: The current number of servers removed from the group
        but still processing active client connections.
    zone:
      type: string
      description: The name of the shared memory
      <a
href="https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#zone">zon
e</a>
        that keeps the group's configuration and run-time state.
    example:

```

dns:

peers:

- id: 0

server: 10.0.0.1:12347

name: 10.0.0.1:12347

backup: false

weight: 5

state: up

active: 0

ssl:

handshakes: 200

handshakes_failed: 4

session_reuses: 189

no_common_protocol: 4

handshake_timeout: 0

peer_rejected_cert: 0

verify_failures:

expired_cert: 2

revoked_cert: 1

hostname_mismatch: 2

other: 1

max_conns: 50

connections: 667231

sent: 251946292

received: 19222475454

fails: 0

unavail: 0

health_checks:

checks: 26214

fails: 0

unhealthy: 0

last_passed: true

downtime: 0

downstart: 2022-06-28T11:09:21.602Z

selected: 2022-06-28T15:01:25Z

- id: 1

server: 10.0.0.1:12348

name: 10.0.0.1:12348

backup: true

weight: 1

state: unhealthy

active: 0

max_conns: 50

connections: 0

sent: 0

received: 0

fails: 0
unavail: 0
health_checks:
 checks: 26284
 fails: 26284
 unhealthy: 1
 last_passed: false
downtime: 262925617
downstart: 2022-06-28T11:09:21.602Z
selected: 2022-06-28T15:01:25Z
zombies: 0
zone: dns

NginxStreamUpstreamPeerMap:

 title: Stream Upstream Servers
 description: Array of stream upstream servers.
 type: array
 items:

 \$ref: '#/definitions/NginxStreamUpstreamPeer'

NginxStreamUpstreamPeer:

 title: Stream Upstream Server
 type: object
 properties:
 id:
 type: integer
 description: The ID of the server.
 readOnly: true

 server:

 type: string
 description: An

of the server.

 service:

 type: string
 description: The

parameter value of the

directive.

 name:

 type: string

format: hostname
description: The name of the server specified in the
[server](https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#server)

directive.
readOnly: true
backup:
type: boolean
description: A boolean value indicating whether the server is a
[backup](https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#backup)
server.
weight:
type: integer
description: [Weight](https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#weight)
of the server.
state:
type: string
readOnly: true
enum:

- up
- down
- unavail
- checking
- unhealthy

description: Current state, which may be one of
“`up`”, “`down`”, “`unavail`”,
“`checking`”, or “`unhealthy`”.

active:
type: integer
description: The current number of connections.
readOnly: true
ssl:
type: object
readOnly: true
properties:
handshakes:
type: integer
description: The total number of successful SSL handshakes.
readOnly: true
handshakes_failed:
type: integer

description: The total number of failed SSL handshakes.
 readOnly: true
 session_reuses:
 type: integer
 description: The total number of session reuses during SSL handshake.
 readOnly: true
 no_common_protocol:
 type: integer
 description: The number of SSL handshakes failed
 because of no common protocol.
 handshake_timeout:
 type: integer
 description: The number of SSL handshakes failed
 because of a timeout.
 peer_rejected_cert:
 type: integer
 description: The number of failed SSL handshakes
 when njet presented the certificate to the upstream server
 but it was rejected with a corresponding alert message.
 verify_failures:
 type: object
 description: SSL certificate verification errors
 properties:
 expired_cert:
 type: integer
 description: An expired or not yet valid certificate
 was presented by an upstream server.
 revoked_cert:
 type: integer
 description: A revoked certificate was presented by an upstream
server.
 hostname_mismatch:
 type: integer
 description: Server's certificate doesn't match the hostname.
 other:
 type: integer
 description: Other SSL certificate verification errors.
 max_conns:
 type: integer
 description: The
 [max_conns](https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#max_connections)
 limit for the server.
 connections:
 type: integer

description: The total number of client connections forwarded to this server.
 readOnly: true

connect_time:
 type: integer
 description: The average time to connect to the upstream server.
 readOnly: true

first_byte_time:
 type: integer
 description: The average time to receive the first byte of data.
 readOnly: true

response_time:
 type: integer
 description: The average time to receive the last byte of data.
 readOnly: true

sent:
 type: integer
 description: The total number of bytes sent to this server.
 readOnly: true

received:
 type: integer
 description: The total number of bytes received from this server.
 readOnly: true

fails:
 type: integer
 description: The total number of unsuccessful attempts
 to communicate with the server.
 readOnly: true

unavail:
 type: integer
 description: How many times the server became unavailable for client
 connections
 (state "`unavail`") due to the number of unsuccessful
 attempts reaching the
[<a href="https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#max_fails"](https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#max_fails)
 >max_fails
 threshold.
 readOnly: true

health_checks:
 type: object
 readOnly: true

properties:
 checks:
 type: integer
 description: The total number of
[](#)

href="https://njet.org/en/docs/stream/nginx_stream_upstream_hc_module.html#health_check">health check

requests made.

readOnly: true

fails:

type: integer

description: The number of failed health checks.

readOnly: true

unhealthy:

type: integer

description: How many times the server became unhealthy
(state “<code>unhealthy</code>”).

readOnly: true

last_passed:

type: boolean

description: Boolean indicating whether the last health check request
was successful and passed

tests.

readOnly: true

downtime:

type: integer

description: Total time the server was in the
“<code>unavail</code>”, “<code>checking</code>”,
and “<code>unhealthy</code>” states.

readOnly: true

downstart:

type: string

format: date-time

description: The time when the server became
“<code>unavail</code>”, “<code>checking</code>”,
or “<code>unhealthy</code>”,
in the ISO 8601 format with millisecond resolution.

readOnly: true

selected:

type: string

format: date-time

description: The time when the server was last selected
to process a connection,
in the ISO 8601 format with millisecond resolution.

readOnly: true

NginxStreamUpstreamConfServerMap:

title: Stream Upstream Servers

description: |

An array of stream upstream servers for dynamic configuration.

type: array

items:

\$ref: '#/definitions/NginxStreamUpstreamConfServer'

example:

- id: 0

server: 10.0.0.1:12348

weight: 1

max_conns: 0

max_fails: 1

fail_timeout: 10s

slow_start: 0

backup: false

down: false

- id: 1

server: 10.0.0.1:12349

weight: 1

max_conns: 0

max_fails: 1

fail_timeout: 10s

slow_start: 0

backup: false

down: false

NginxStreamUpstreamConfServer:

title: Stream Upstream Server

description: |

Dynamically configurable parameters of a stream upstream

server:

type: object

properties:

id:

type: integer

description: The ID of the stream upstream server.

The ID is assigned automatically and cannot be changed.

readOnly: true

server:

type: string

description: Same as the

address

parameter of the stream upstream server.

When adding a server, it is possible to specify it as a domain name.

In this case, changes of the IP addresses

that correspond to a domain name will be monitored and automatically

applied to the upstream configuration
without the need of restarting njet.
This requires the

directive in the “`stream`” block.
See also the

parameter of the stream upstream server.

service:

type: string
description: Same as the

parameter of the stream upstream server.

This parameter cannot be changed.

readOnly: true

weight:

type: integer
description: Same as the

parameter of the stream upstream server.

max_conns:

type: integer
description: Same as the

parameter of the stream upstream server.

max_fails:

type: integer
description: Same as the

parameter of the stream upstream server.

fail_timeout:

type: string
description: Same as the

https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#fail_timeout

parameter of the stream upstream server.

slow_start:

type: string

description: Same as the

[slow_start](https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#slow_start)

[slow_start](https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#slow_start)

parameter of the stream upstream server.

backup:

type: boolean

description: When `true`, adds a

[backup](https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#backup)

[backup](https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#backup)

server.

This parameter cannot be changed.

readOnly: true

down:

type: boolean

description: Same as the

[down](https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#down)

[down](https://njet.org/en/docs/stream/nginx_stream_upstream_module.html#down)

parameter of the stream upstream server.

parent:

type: string

description: Parent server ID of the resolved server.

The ID is assigned automatically and cannot be changed.

readOnly: true

host:

type: string

description: Hostname of the resolved server.

The hostname is assigned automatically and cannot be changed.

readOnly: true

example:

id: 0

server: 10.0.0.1:12348

weight: 1

max_conns: 0

max_fails: 1

fail_timeout: 10s

slow_start: 0

backup: false

down: false

```

NginxError:
  title: Error
  description: |
    njet error object.
  type: object
  properties:
    error:
      type: object
      properties:
        status:
          type: integer
          description: HTTP error code.
        text:
          type: string
          description: Error description.
        code:
          type: string
          description: Internal njet error code.
      request_id:
        type: string
        description: The ID of the request, equals the value of the
          <a
href="https://njet.org/en/docs/http/nginx_http_core_module.html#var_request_id">$req
uest_id</a>
          variable.
      href:
        type: string
        description: Link to reference documentation.

```

6.8 动态 location 配置 API 说明

```

openapi: 3.0.3
info:
  title: njet-api
  description: njet-api
  version: 1.0.0
servers:
  - url: '/'
paths:
  /dyn_loc:
    put:
      tags:
        - dyn_loc
      summary: Delete a dynamic location from server
      description: Delete a dynamic location from server.

```

```

    operationId: delLocation
    responses:
      '200':
        description: Success
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/del_location'
  post:
    tags:
      - dyn_loc
    summary: Add a dynamic location to server
    description: Add a dynamic location to server.
    operationId: addLocation
    responses:
      '200':
        description: Success
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/add_location'
###DEFINITIONS
components:
  schemas:
    del_location:
      title: dyn_loc
      description: delete dynamic location from server
      type: object
      properties:
        type:
          type: string
          description: fix value "del".
          example: del
        addr_port:
          type: string
          description: addr and port of server.
          example: 192.168.40.203:8081
        server_name:
          type: string
          description: server name .
          example: cluster.tmlake.com
        location_rule:
          type: string
          description: Regular Expression.
          example: =
        location_name:

```

```

    type: string
    description: the name of location.
    example: /clb
add_location:
  title: dyn_loc
  description: add dynamic location to server
  type: object
  properties:
    type:
      type: string
      example: add
    addr_port:
      type: string
      description: addr and port of server.
      example: 192.168.40.203:8081
    server_name:
      type: string
      description: server name .
      example: cluster.tmlake.com
    location_rule:
      type: string
      description: Regular Expression.
      example: =
    location_name:
      type: string
      description: the name of location.
      example: /clb
    location_body:
      type: string
      description: the body content of location .
      example: api write=off;
    proxy_pass:
      type: string
      description: the proxy_pass info .
      example: $dest_upstream;

```

6.9 动态 sendmsg 模块 kv API 说明

openapi: 3.0.3

info:

title: kv api

description: 维护 kv

version: 1.0.0

servers:

```
- url: '/'
paths:
  /kv:
    post:
      tags:
        - sendmsg kv
      summary: kvstore key set
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/kvdata'
            required: true
      responses:
        '200':
          description: Success
        '500':
          description: Internal Server Error
    get:
      tags:
        - sendmsg kv
      summary: get value from kvstore
      parameters:
        - in: query
          name: key
          schema:
            type: string
            example: 'test_key'
      responses:
        '200':
          description: Success
          content:
            text/plain:
              schema:
                type: string
                example: 'test msg'
        '404':
          description: Not Found
        '500':
          description: Internal Server Error
  components:
    schemas:
      kvdata:
        title: kv post data
        type: object
```

```
properties:
  key:
    type: string
    example: 'test_key'
  value:
    type: string
    example: 'test msg'
```

7. 版权声明

版权所有 © 通明智云(北京)科技有限公司 2020 保留一切权利

本文档的程序或内容受版权法的保护，未经通明智云(北京)科技有限公司的书面许可，不得擅自摘抄、复制本文档资料的全部或部分内容，并不得以任何形式传播。

注意

您购买的产品、服务或特性等应受通明智云(北京)科技有限公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，通明智云(北京)科技有限公司对本文档内容不做任何明示或默示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

通明智云(北京)科技有限公司

地址：北京市北京经济技术开发区科谷一街 8 号院 1 号楼 24 层 2403

网址：<http://www.tmlake.com>